EXPERT INSIGHT

SPRINT book

RAG и генеративный ИИ

Создаем собственные RAG-пайплайны с помощью LlamaIndex, Deep Lake и Pinecon



Дэнис Ротман

packt

RAG-Driven Generative AI

Build custom retrieval augmented generation pipelines with LlamaIndex, Deep Lake, and Pinecone

Denis Rothman

‹packt›

RAG и генеративный ИИ

Создаем собственные RAG-пайплайны
с помощью LlamaIndex, Deep Lake и Pinecon

Дэнис Ротман

SPRINT book 2025

Дэнис Ротман

RAG и генеративный ИИ. Создаем собственные RAG-пайплайны с помощью LlamaIndex, Deep Lake и Pinecon

Перевел с английского Д. Брайт

Научный редактор Д. Бардин

ББК 32.988.02-018
УДК 004.738.5

Ротман Дэнис

Р79 RAG и генеративный ИИ. Создаем собственные RAG-пайплайны с помощью LlamaIndex, Deep Lake и Pinecon. — Астана: «Спринт Бук», 2025. — 320 с.: ил.

ISBN 978-601-12-3149-7

В книге описываются приемы создания эффективных больших языковых моделей, систем компьютерного зрения и генеративного ИИ, показывающих высокую производительность при относительно невысоких затратах. В ней приводится подробное исследование технологии RAG, а также подходов к проектированию мультимодальных пайплайнов ИИ и управлению ими. Связывая вывод с исходными документами, RAG повышает точность и контекстную релевантность результатов, предлагая динамический подход к управлению большими объемами информации.

Узнайте, как построить инфраструктуру RAG, попутно разобравшись с векторными хранилищами, фрагментацией, индексацией и ранжированием. Познакомьтесь с методами оптимизации производительности и приемами более глубокого изучения данных, включая использование адаптивного RAG и обратной связи от человека для уточнения поиска, тонкую настройку RAG, реализацию динамических RAG для поддержки принятия решений в реальном времени и визуализацию сложных данных с помощью графов знаний. Вы также увидите, как на практике объединить такие фреймворки, как LlamaIndex и Deep Lake, векторные базы данных наподобие Pinecone и Chroma и модели, предлагаемые компаниями Hugging Face и OpenAI. Приобретите навыки внедрения интеллектуальных решений, что повысит вашу конкурентоспособность в различных областях: от продакшна до обслуживания клиентов в любом проекте.

16+ (В соответствии с Федеральным законом от 29 декабря 2010 г. № 436-ФЗ.)

.

ISBN 978-601-12-3149-7

© Packt Publishing 2024. First published in the English language under the title 'RAG-Driven Generative AI – (9781836200918)'
© Перевод на русский язык ТОО «Спринт Бук», 2025
© Издание на русском языке, оформление ТОО «Спринт Бук», 2025

Права на издание получены по соглашению с Packt Publishing. Все права защищены. Никакая часть данной книги не может быть воспроизведена в какой бы то ни было форме без письменного разрешения владельцев авторских прав.

Информация, содержащаяся в данной книге, получена из источников, рассматриваемых издательством как надежные. Тем не менее, имея в виду возможные человеческие или технические ошибки, издательство не может гарантировать абсолютную точность и полноту приводимых сведений и не несет ответственности за возможные ошибки, связанные с использованием книги. В книге возможны упоминания организаций, деятельность которых запрещена на территории Российской Федерации, таких как Meta Platforms Inc., Facebook, Instagram и др. Издательство не несет ответственности за доступность материалов, ссылки на которые вы можете найти в этой книге. На момент подготовки книги к изданию все ссылки на интернет-ресурсы были действующими.

Изготовлено в России. Изготовитель: ТОО «Спринт Бук». Место нахождения и фактический адрес: 010000, Казахстан, город Астана, район Алматы, проспект Ракымжан Кошкарбаев, д. 10/1, н. п. 18.

Дата изготовления: 08.2025. Наименование: книжная продукция. Срок годности: не ограничен.

Подписано в печать 07.07.25. Формат 70×100/16. Бумага офсетная. Усл. п. л. 25,800. Тираж 1000. Заказ 0000.

Краткое содержание

https://t.me/it_booooks/2

Об авторе ... 11
О научных редакторах ... 12
Предисловие ... 13
От издательства ... 19

Глава 1. Почему именно генерация с дополненной выборкой (RAG) ... 20
Глава 2. Хранение эмбеддингов RAG с помощью Deep Lake и OpenAI ... 52
Глава 3. Реализация индексированной RAG с помощью LlamaIndex, Deep Lake и OpenAI ... 80
Глава 4. Мультимодальная RAG ... 110
Глава 5. Повышение эффективности RAG за счет обратной связи от эксперта ... 136
Глава 6. Масштабирование банковских данных клиентов в системе RAG с помощью Pinecone ... 161
Глава 7. Создание масштабируемой RAG на основе графа знаний с помощью API Википедии и LlamaIndex ... 199
Глава 8. Создание динамических RAG с помощью Chroma и Hugging Face Llama ... 227
Глава 9. Улучшение моделей ИИ: тонкая настройка данных RAG и обратная связь ... 253
Глава 10. RAG для создания стоковых видео с помощью Pinecone и OpenAI ... 271

Приложение ... 307

Оглавление

Об авторе ... 11

О научных редакторах ... 12

Предисловие ... 13
- Целевая аудитория ... 13
- Что освещается в книге ... 14
- Необходимые знания ... 16
- Файлы с примерами кода ... 17
- Цветные иллюстрации ... 17
- Условные обозначения ... 17

От издательства ... 19
- О научном редакторе русскоязычного издания ... 19

Глава 1. Почему именно генерация с дополненной выборкой (RAG) ... 20
- Что такое RAG ... 21
- Простая, продвинутая и модульная конфигурации RAG ... 23
- RAG или тонкая настройка? ... 23
- Экосистема RAG ... 25
  - Ретривер (D) ... 26
  - Генератор (G) ... 28
  - Модуль оценки (E) ... 28
  - Учитель (T) ... 29
- Простая, продвинутая и модульная RAG в коде ... 29
  - Часть 1. Основы и базовая реализация ... 30
  - Часть 2. Продвинутые техники и оценка ... 35
- Резюме ... 50
- Вопросы ... 50
- Ссылки ... 51
- Дополнительная информация ... 51

Глава 2. Хранение эмбеддингов RAG с помощью Deep Lake и OpenAI ... 52
- От сырых данных к эмбеддингам в векторной базе данных ... 53
- Организация RAG в виде пайплайна ... 55
- Пайплайн генеративного ИИ на базе RAG ... 56

Создание пайплайна RAG ... 57
Настройка среды ... 57
Сбор и подготовка данных ... 62
Создание и сохранение эмбеддингов ... 65
Информация о векторной базе данных ... 68
Генерация дополненного ввода ... 70
Оценка вывода с помощью метрики косинусного сходства ... 75
Резюме ... 77
Вопросы ... 78
Ссылки ... 79
Дополнительная информация ... 79

Глава 3. Реализация индексированной RAG с помощью LlamaIndex, Deep Lake и OpenAI ... 80
В чем польза RAG с индексированием ... 81
Архитектура ... 81
Создание механизма семантического поиска и генеративного агента для сферы БПЛА ... 84
Настройка среды ... 85
Компонент 1: сбор и подготовка документов ... 85
Компонент 2: создание и заполнение векторной базы данных Deep Lake ... 89
Компонент 3: RAG с индексированием ... 94
Механизм индексирования на основе векторов ... 96
Ответ на запрос и его источник ... 97
Оптимизация разбивки текста ... 99
Оценка эффективности ... 99
Механизм индексирования на основе дерева ... 100
Оценка эффективности ... 102
Механизм индексирования на основе списка ... 103
Оценка эффективности ... 105
Механизм индексирования на основе таблицы ключевых слов ... 105
Оценка эффективности ... 107
Резюме ... 108
Вопросы ... 109
Ссылки ... 109
Дополнительная информация ... 109

Глава 4. Мультимодальная RAG ... 110
Что такое мультимодальная RAG ... 111
Создание мультимодальной RAG для сферы БПЛА ... 114
Загрузка датасета для LLM ... 115
Загрузка и визуализация мультимодального датасета ... 117

Навигация по мультимодальному датасету ... 120
Создание мультимодального механизма обработки запросов ... 124
Обобщение мультимодального вывода ... 129
Оценка эффективности ... 130
Резюме ... 134
Вопросы ... 135
Ссылки ... 135
Дополнительная информация ... 135

Глава 5. Повышение эффективности RAG за счет обратной связи от эксперта ... 136
Адаптивная RAG ... 136
Создание гибридной адаптивной RAG на Python ... 140
1. Ретривер ... 140
2. Генератор ... 143
3. Оценка ... 153
Резюме ... 159
Вопросы ... 160
Ссылки ... 160
Дополнительная информация ... 160

Глава 6. Масштабирование банковских данных клиентов в системе RAG с помощью Pinecone ... 161
Масштабирование с помощью Pinecone ... 162
Архитектура ... 162
Компонент 1: сбор и подготовка данных ... 164
Сбор и обработка датасета ... 165
Изучение и анализ данных (EDA) ... 169
Обучение модели ML ... 172
Компонент 2: масштабирование индекса Pinecone ... 176
Сложности управления векторными базами данных ... 177
Настройка среды ... 179
Обработка датасета ... 179
Создание чанков и их преобразование в эмбеддинги ... 181
Создание индекса Pinecone ... 185
Загрузка ... 186
Запрос к индексу Pinecone ... 189
Компонент 3: генеративный ИИ на базе RAG ... 191
RAG с GPT-4o ... 191
Извлечение релевантных текстов ... 194
Дополнение промта ... 194
Генерация на основе дополненного запроса ... 195

Резюме ... 197
Вопросы ... 198
Ссылки ... 198
Дополнительная информация ... 198

Глава 7. Создание масштабируемой RAG на основе графа знаний с помощью API Википедии и LlamaIndex ... 199
Архитектура RAG для семантического поиска на основе графа знаний ... 200
Создание графов из деревьев ... 203
Компонент 1: сбор и подготовка документов ... 206
Извлечение данных и метаданных из Википедии ... 206
Подготовка данных к загрузке ... 210
Компонент 2: создание и заполнение векторной базы данных Deep Lake ... 212
Компонент 3: RAG на основе индексируемого графа знаний ... 212
Генерация индекса графа знаний ... 214
Вывод графа ... 216
Взаимодействие с индексом графа знаний ... 217
Установка пакетов для измерения сходства и определение функций ... 218
Переупорядочивание ... 218
Примеры оценки ... 220
Резюме ... 225
Вопросы ... 225
Ссылки ... 226
Дополнительная информация ... 226

Глава 8. Создание динамических RAG с помощью Chroma и Hugging Face Llama ... 227
Архитектура динамической RAG ... 228
Настройка среды ... 231
Hugging Face ... 231
Chroma ... 233
Запуск отсчета времени сеанса ... 234
Скачивание и подготовка датасета ... 235
Создание эмбеддингов и их загрузка в коллекцию Chroma ... 237
Выбор модели ... 238
Создание эмбеддингов и сохранение завершенного текста ... 239
Вывод эмбеддингов ... 240
Обращение к коллекции ... 241
Создание промта и извлечение ответа ... 245
RAG с Llama ... 247
Удаление коллекции ... 250

Общая продолжительность сеанса ... 250
Резюме ... 251
Вопросы ... 252
Ссылки ... 252
Дополнительная информация ... 252

Глава 9. Улучшение моделей ИИ: тонкая настройка данных RAG и обратная связь ... 253
Схема тонкой настройки статических данных RAG ... 254
Экосистема RAG ... 255
Настройка среды ... 256
Подготовка датасета к тонкой настройке ... 257
Скачивание и визуализация датасета ... 257
Подготовка датасета к тонкой настройке ... 259
Тонкая настройка модели ... 260
Отслеживание тонкой настройки ... 262
Использование настроенной модели OpenAI ... 264
Метрики ... 266
Резюме ... 269
Вопросы ... 270
Ссылки ... 270
Дополнительная информация ... 270

Глава 10. RAG для создания стоковых видео с помощью Pinecone и OpenAI ... 271
Архитектура RAG для создания видео ... 272
Среда для производства видео ... 275
Импорт модулей и библиотек ... 275
GitHub ... 276
OpenAI ... 277
Pinecone ... 278
Компонент 1: генератор и комментатор ... 278
Генерация датасета с помощью ИИ ... 279
Генератор и комментатор ... 287
Компонент 2: администратор векторной базы данных ... 295
Запрос к индексу Pinecone ... 296
Компонент 3: видеоэксперт ... 300
Резюме ... 305
Вопросы ... 305
Ссылки ... 306
Дополнительная информация ... 306

Приложение ... 307

Об авторе

Дэнис Ротман (Denis Rothman) окончил Парижский университет и Университет Париж Дидро. Будучи студентом, он написал и запатентовал одну из первых систем векторного представления слов word2vector и токенизации их фрагментов. Он организовал компанию, занимающуюся внедрением ИИ, и создал один из первых интеллектуальных NLP-ботов, который применялся для обучения языку в винодельческом доме Moët et Chandon (является частью Louis Vuitton Moët Hennesy) и других крупных компаниях. Дэнис быстро стал экспертом по объяснимому ИИ (explainable AI), занимаясь внедрением доступных интерфейсов и информации, проясняющей работу ИИ, в решения, создаваемые для аэрокосмической отрасли, швейной промышленности и пр. Он абсолютно уверен, что полноценно освоить какую-либо тему можно, только обучая других.

О научных редакторах

Альберто Ромеро (Alberto Romero) с юных лет был увлечен технологиями и программным обеспечением с открытым исходным кодом. Это увлечение уходит корнями в 1990-е, когда в возрасте 12 лет он впервые занялся программированием, а к 14 годам уже вникал в ядро Linux. В 2017-м он стал сооснователем стартапа по разработке искусственного интеллекта, где с нуля создавал передовую платформу InsurTech. Альберто разработал и выпустил множество ИИ-продуктов, автоматизирующих и улучшающих процессы принятия решений, которые уже используются тысячами людей. Сейчас он занимает должность советника в передовом стартапе, предоставляющем услуги по защите корпоративных данных и управлению ими с помощью предиктивных моделей ML и генеративного ИИ.

Хочу выразить глубочайшую признательность своей жене Алисии и нашим дочкам Адриане и Каталине за их постоянную поддержку. Их терпение и любовь безграничны, и я поистине счастлив быть рядом с ними.

Шубхам Гарг (Shubham Garg) является старшим научным сотрудником в Amazon и специализируется на разработке *больших языковых моделей* (Large Language Model, LLM) и *визуально-языковых моделей* (Vision-Language Model, VLM). В Amazon и IBM он руководил инновационными проектами, участвуя в разработке функций перевода для виртуального ассистента Alexa, динамическом построении промтов и оптимизации ИИ-инструментов. Шубхам писал доклады для ведущих NLP-конференций, делал обзоры для журналов и имеет зарегистрированный патент.

Тамилсельван Субраманиан (Tamilselvan Subramanian) — опытный специалист в сфере искусственного интеллекта, основавший две компании и специализирующийся на генеративном ИИ для обработки текста и изображений. Он создал различные ИИ-продукты: платформу для спасения исчезающих видов животных, платформу анализа медицинских снимков, платформу для приобретения электромобилей в лизинг, а также построил с нуля корпоративную ИИ-систему. Тамилсельван работал в IBM и Wipro, а также в различных энергетических и финансовых компаниях на территории Евросоюза, Соединенных Штатов и Австралии. Сейчас он занимается разработкой передовых продуктов в сфере компьютерного зрения, обработки текста и генеративного ИИ.

Выражаю особую благодарность своей жене Суганти, сыну Сандживу, а также отцу и матери за их постоянную поддержку и за то, что у меня было свободное время для работы над этой книгой.

Предисловие

Проектировать надежные мультимодальные пайплайны[1] генеративного ИИ и управлять ими нелегко. Книга «RAG и генеративный ИИ» даст вам необходимые ориентиры для построения эффективных LLM, моделей компьютерного зрения и систем генеративного ИИ с соблюдением баланса между производительностью и затратами. Начиная с фундаментальных концепций и заканчивая сложными реализациями, автор подробно описывает, как RAG (Retrieval Augmented Generation — генерация с дополненной выборкой) позволяет управлять ИИ-системами и улучшать их за счет отслеживания каждого результата до лежащего в его основе документа. Такой отслеживаемый процесс позволяет получать обратную связь от людей для постоянных доработок, а также минимизации неточностей, галлюцинаций моделей и искажения их результатов. Вы научитесь с нуля создавать RAG-фреймворки, получив практические знания о работе с векторными базами данных, фрагментации, индексировании и ранжировании. Здесь вы найдете техники для оптимизации быстродействия систем и снижения сопутствующих затрат, повышения точности моделей за счет интеграции обратной связи от человека, оценки целесообразности тонкой настройки. Вы также научитесь повышать точность и скорость извлечения релевантных данных с помощью встраиваемых индексируемых графов знаний.

В издании удачно совмещаются теория и практика: вы найдете здесь много примеров с использованием фреймворков LlamaIndex, Pinecon и Deep Lake, а также генеративных ИИ-платформ наподобие OpenAI и Hugging Face.

Прочитав книгу, вы получите навыки, необходимые для реализации интеллектуальных решений, которые обеспечат вам конкурентные преимущества в различных отраслях, начиная с производства и заканчивая сферой обслуживания клиентов.

Целевая аудитория

Книга прекрасно подойдет для специалистов по данным, ML и MLOps, инженеров, работающих в сфере ИИ, а также архитекторов решений, разработчиков ПО и руководителей проектов, которые создают проекты с использованием LLM или компьютерного зрения и желают научиться применять RAG в реальных

[1] Пайплайн — последовательность действий или процессов, которые выполняются для достижения заданной цели. — *Здесь и далее примеч. пер.*

сценариях. Она также окажется полезной для исследователей и специалистов по обработке естественного языка, работающих с большими языковыми моделями и генерацией текста.

Что освещается в книге

Глава 1 «Почему именно генерация с дополненной выборкой» знакомит вас с основными концепциями RAG, подчеркивает ее адаптируемость под различные типы данных и разбирает сложности интеграции фреймворка RAG в существующие ИИ-платформы. К ее концу вы уже будете хорошо понимать, что такое RAG, приобретете практический опыт создания различных конфигураций для простых, продвинутых и модульных систем RAG с помощью Python и будете готовы к более сложным случаям их применения, рассматриваемым в последующих главах.

Глава 2 «Сохранение эмбеддингов RAG с помощью Deep Lake и OpenAI» освещает сложности создания генеративных ИИ на базе RAG, фокусируясь на эмбеддингах[1] и видах их хранилищ. Здесь мы изучим переход от сырых данных к организованным векторным хранилищам с помощью моделей Activeloop Deep Lake и OpenAI, подробно разобрав процесс создания эмбеддингов и управления ими. В этой главе вы научитесь с нуля создавать масштабируемый мультимодальный пайплайн RAG на Python, разделив экосистему RAG на независимые компоненты. К ее концу вы будете обладать знаниями, необходимыми для обработки масштабных датасетов с использованием продвинутых возможностей извлечения данных, что позволит вам улучшить результаты выдачи генеративных ИИ-моделей с помощью встраиваемых векторов документов.

Глава 3 «Реализация индексированной RAG с помощью LlamaIndex, Deep Lake и OpenAI» посвящена системам RAG на основе индексирования. В ней разбираются темы повышения точности моделей ИИ, а также их скорости и прозрачности с помощью индексирования. Здесь вы узнаете, как можно интегрировать в систему LlamaIndex, Deep Lake и OpenAI, чтобы получить эффективный пайплайн RAG с возможностью трассировки. На основе практических примеров вы научитесь управлять индексируемыми системами извлечения релевантных данных и оптимизировать их. К концу главы вы сможете эффективно использовать различные виды индексирования и поймете, как повысить целостность данных и качество результатов ваших ИИ-моделей.

Глава 4 «Мультимодальная RAG» поднимает планку всех приложений генеративного ИИ, представляя мультимодальный фреймворк RAG, созданный специально для технологических решений в сфере беспилотных летательных аппаратов. Здесь мы разработаем систему генеративного ИИ, которая не только

[1] Эмбеддинг (от англ. embedding) — векторное представление слов.

обрабатывает текстовую информацию, но и включает в себя продвинутые возможности распознавания изображений. Вы научитесь создавать и оптимизировать мультимодальную систему RAG на базе Python, используя инструменты LlamaIndex, Deep Lake и OpenAI для генерации развернутых ответов на запросы с учетом контекста.

Глава 5 «Повышение эффективности RAG за счет обратной связи от эксперта» познакомит вас с инновационным решением — адаптивной RAG, в которой процесс работы генеративного ИИ оптимизируется благодаря обратной связи от человека. Напрямую интегрировав циклы такой обратной связи с помощью Python, мы создадим гибридную адаптивную систему RAG, которая будет непрерывно уточнять данные, повышая релевантность и точность ответов ИИ-модели.

Глава 6 «Масштабирование банковских данных клиентов в системе RAG с помощью Pinecone» научит вас создавать рекомендательную систему для снижения оттока клиентов банка, начиная с получения и анализа данных на примере датасета Kaggle. Здесь вы перейдете к созданию эмбеддинга и обновлению/вставке[1] больших объемов данных с помощью технологий Pinecone и OpenAI, разработав рекомендательную систему с использованием GPT-4o. К концу главы вы научитесь реализовывать продвинутые техники хранения векторов и аналитику на базе ИИ для улучшения стратегий удержания клиентов.

Глава 7 «Создание масштабируемой RAG на основе графа знаний с помощью API Википедии и LlamaIndex» посвящена разработке трех пайплайнов: сбора данных из Википедии, заполнения векторной БД Deep Lake и реализации RAG на основе индексируемого графа знаний. Здесь вы научитесь автоматизировать извлечение и подготовку данных, создавать и опрашивать граф знаний для визуализации сложных связей между этими данными, а также улучшать генерируемые ИИ ответы структурированной информацией. К концу главы вы уже сможете создавать системы RAG на основе графа знаний, предоставляющие точные результаты с учетом контекста, и управлять ими.

Глава 8 «Создание динамических RAG с помощью Chroma и Hugging Face Llama» посвящена разработке динамических RAG с помощью Chroma и технологии Llama от Hugging Face. В ней вы познакомитесь с концепцией ежедневного создания вре́менных коллекций данных, оптимизированных под конкретные совещания или задачи и исключающих проблемы, возникающие в долгосрочных хранилищах данных. Здесь вы создадите на Python программу, которая будет эффективно управлять вре́менными датасетами и опрашивать их, обеспечивая, чтобы каждое совещание или важное решение подкреплялось наиболее актуальной и свежей информацией. К концу главы вы научитесь реализовывать динамические системы RAG, повышающие отзывчивость и точность сред, работающих на основе данных.

[1] Операция upsert.

Глава 9 «Улучшение моделей ИИ: тонкая настройка данных RAG и обратная связь» посвящена техникам тонкой настройки, используемым для систематизации данных RAG. В ней акцент будет сделан на том, как трансформировать обширные, непараметрические, сырые данные в более управляемый параметрический формат с обучаемыми весами, подходящий для непрерывного взаимодействия с ИИ. Здесь вы познакомитесь с процессом подготовки и тонкой настройки датасета, применяя инструменты OpenAI для преобразования данных в промт и заключительные пары[1] для машинного обучения. Кроме того, в этой главе вы научитесь использовать для тонкой настройки мини-модель GPT-4o, оценив ее эффективность и экономичность.

Глава 10 «RAG для создания стоковых видео с помощью Pinecone и OpenAI» посвящена интеграции RAG в процесс производства стоковых видео для расширения творческих возможностей человека автоматизацией на базе ИИ. В ней описывается разработка ИИ-системы, которая создает, комментирует и размечает видеоконтент, используя модели преобразования текста в видео и модели компьютерного зрения от OpenAI наряду с возможностями векторных БД Pinecone. Начиная с генерации видео и его технических комментариев, автор плавно переходит к управлению встраиваемыми видеоданными внутри векторного хранилища Pinecone.

Необходимые знания

Для наиболее полноценного освоения материала книги вам потребуются базовые знания по *обработке естественного языка* и некоторый опыт работы с Python. Кроме того, большинство программ в ней приводятся в виде блокнотов Jupyter. Для их выполнения вам будет достаточно бесплатного аккаунта в сервисе Google Gmail, который позволит запускать блокноты на *виртуальной машине* (virtual machine, VM) Google Collaboratory. Потребуется также сгенерировать токены API для OpenAI, Activeloop и Pinecone.

Для выполнения блокнотов нужно установить следующие модули.

Модуль	**Версия**
`deeplake`	3.9.18 (с Pillow)
`openai`	1.40.3 (требует регулярного обновления)
`transformers`	4.41.2
`numpy`	≥ 1.24.1 (обновлена для соответствия требованиям chex)
`deepspeed`	0.10.1
`bitsandbytes`	0.41.1

[1] Имеются в виду пары «запрос — ответ».

Модуль	Версия
accelerate	0.31.0
tqdm	4.66.1
neural_compressor	2.2.1
onnx	1.14.1
pandas	2.0.3
scipy	1.11.2
beautifulsoup4	4.12.3
requests	2.31.0

Файлы с примерами кода

Весь код для этой книги размещен на GitHub по адресу https://github.com/Denis2054/RAG-Driven-Generative-AI. Кроме того, в нашем богатом каталоге книг и видео есть и другие наборы кода, доступные по ссылке https://github.com/PacktPublishing/. Рекомендуем ознакомиться.

Цветные иллюстрации

По ссылке https://packt.link/gbp/9781836200918 вы можете скачать PDF-файл, содержащий цветные изображения представленных в книге снимков экрана и диаграмм.

Условные обозначения

В книге применяются определенные условные обозначения.

Курсивом выделены новые термины и важные понятия.

Моноширинный шрифт используется для листингов программ, а также внутри абзацев для обозначения таких элементов, как переменные и функции, базы данных, типы данных, переменные среды, операторы и ключевые слова, каталоги, папки, пути, имена файлов и их расширений. Например, «`self` означает текущий экземпляр класса для доступа к его переменным, методам и функциям».

Блок кода обозначается так:

```
# Косинусный коэффициент
score = calculate_cosine_similarity(query, best_matching_record)
print(f"Best Cosine Similarity Score: {score:.3f}")
```

Любой ввод или вывод в командной строке записывается так:

```
Best Cosine Similarity Score: 0.126
```

Шрифт без засечек используется для обозначения элементов интерфейса, URL и адресов электронной почты.

ПРИМЕЧАНИЕ

Так обозначаются предупреждения или важные заметки.

СОВЕТ

Так обозначаются советы и полезные приемы.

От издательства

Мы выражаем огромную благодарность клубу рецензентов ИТ-литературы ReadIT Club за помощь в работе над русскоязычным изданием книги и их вклад в повышение качества переводной литературы.

Ваши замечания, предложения, вопросы отправляйте по адресу comp@sprintbook.kz (издательство SprintBook, компьютерная редакция).

Мы будем рады узнать ваше мнение!

О научном редакторе русскоязычного издания

Дмитрий Бардин — ведущий разработчик, архитектор решений, один из авторов курса «Архитектор ПО» от «Яндекс Практикума». В настоящее время занимается разработкой бэкенда «КиноПоиска» с применением языков Go и Java. В прошлом руководитель службы продуктовой разработки и ресурс-менеджер. Опыт в ИТ — более 15 лет.

1 Почему именно генерация с дополненной выборкой (RAG)

https://t.me/it_boooks/2

Даже самые продвинутые модели генеративного ИИ могут генерировать ответы на основе лишь тех данных, на которых они обучались. Они не способны давать точные ответы на вопросы по теме, выходящей за рамки обучающего датасета. Модели генеративного ИИ просто не знают, что они чего-то не знают. Такой вот каламбур. Это ведет к неточным или некорректным результатам, иногда называемым галлюцинациями или смещением, проще говоря, чепухе.

Поисково-дополненная генерация, или *генерация с дополненной выборкой* (Retrieval Augmented Generation, RAG), — это механизм, который компенсирует названное ограничение путем расширения генеративных моделей техниками извлечения информации. Его работа заключается в извлечении релевантных данных из внешних источников в реальном времени и использовании этих данных для генерации более точных и соответствующих контексту ответов. Модели генеративного ИИ, совмещенные с механизмами поиска[1] RAG, революционизируют эту сферу своей беспрецедентной эффективностью и возможностями. Одно из ключевых преимуществ RAG — ее адаптируемость. Этот механизм можно с легкостью применить к любым типам данных, будь то текст, изображения или аудио. Такая гибкость делает экосистему RAG надежным и эффективным инструментом для расширения возможностей генеративных ИИ.

Однако руководителям проектов уже известно множество платформ, фреймворков и моделей генеративного ИИ, например Hugging Face, Google Vertex AI, OpenAI, LangChain и др. Поэтому дополнительный уровень в виде появляющихся фреймворков и платформ RAG наподобие Pinecone, Chroma, Activeloop, LlamaIndex и пр. только добавляет сложности. Все эти фреймворки генеративного ИИ и RAG зачастую пересекаются, создавая невероятное множество возможных конфигураций. В итоге руководителю проекта бывает непросто найти правильную конфигурацию моделей и ресурсов RAG для конкретного проекта.

---

[1] Ретриверами (от англ. retriever).

Универсального решения здесь нет. Задача действительно серьезная, но, если ее решить, это принесет огромные дивиденды.

Мы начнем эту главу с общего рассмотрения механизма RAG. Затем определим три основные конфигурации этой системы: простейшую, продвинутую и модульную RAG. Помимо этого, сравним RAG с тонкой настройкой и определим, когда и какой из этих подходов использовать. RAG может существовать только внутри экосистемы, которую мы в текущей главе спроектируем и опишем. Данные должны откуда-то поступать и где-то обрабатываться. Для их поиска и извлечения необходима организованная среда, а модели генеративного ИИ имеют свои ограничения на ввод.

Наконец, в этой главе мы разберем и более практический аспект, а именно создадим на Python программу для выполнения простейшей RAG на базе поиска и сопоставления ключевых слов. Кроме того, напишем продвинутую систему RAG с векторным поиском и извлечением по индексу. А в завершение создадим модульную RAG, которая будет включать в себя простейшую и продвинутую модели механизма. К концу главы вы будете не только знать стоящую за RAG теорию, но и обладать опытом создания генеративного ИИ на ее основе. Такой подход позволит вам лучше понять тему и подготовит к прочтению последующих глав.

Что такое RAG

Когда модель генеративного ИИ не может дать точного ответа, говорится, что она галлюцинирует или дает искаженный результат (bias). Если проще, то она выдает чушь. Тем не менее все сводится к невозможности дать адекватный ответ, что может быть вызвано как классическими проблемами с настройкой модели, так и тем, что процесс ее обучения просто не включал запрошенную информацию. Это часто ведет к выдаче случайной череды наиболее вероятных, а не наиболее достоверных результатов.

И начинается RAG там, где генеративный ИИ предоставляет информацию, на которую модель LLM не может ответить точно. Этот принцип в качестве дополнения LLM в 2020 году разработал специалист по ИИ Льюис (Lewis) в сотрудничестве с коллегами. Механизм RAG выполняет оптимизированные задачи по извлечению информации, а система генерации добавляет ее к вводу (запросу пользователя или автоматизированному промту) для выдачи более точного результата. Схематично RAG можно описать, применяя аналогию, изображенную на рис. 1.1.

Представьте, что вы студент в библиотеке и вам нужно составить реферат с помощью RAG. Аналогично ChatGPT или другому ИИ-ассистенту вы умеете читать и писать. Как и любая *большая языковая модель*, вы достаточно образованны для того, чтобы изучить расширенную информацию, обобщить ее и записать нужное. Однако, как и любой ИИ, созданный компаниями Hugging Face, Certex AI или OpenAI, многого по теме вы не знаете.

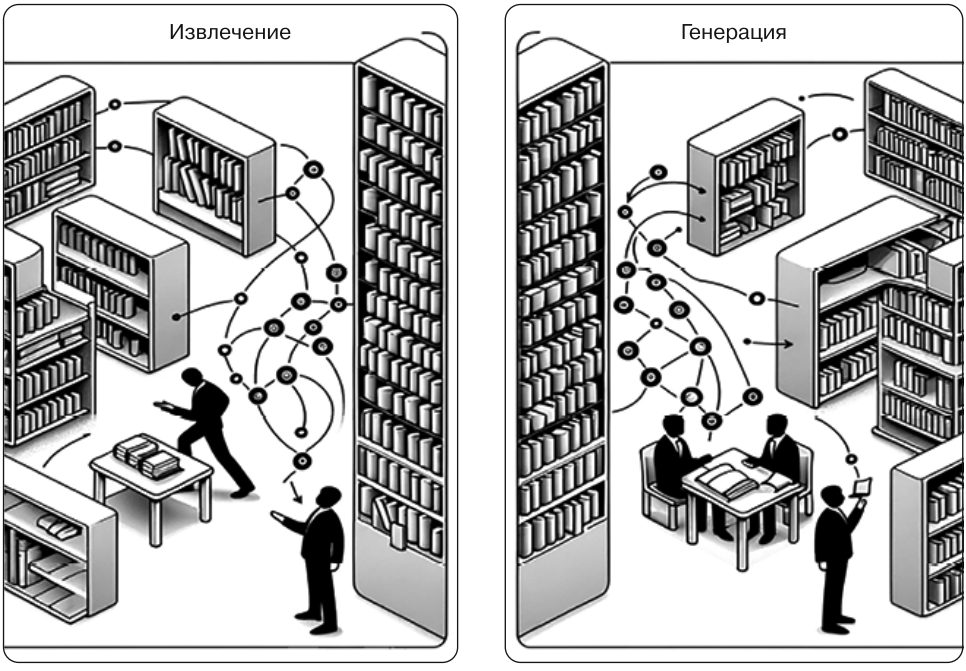


Рис. 1.1. Два новых компонента генеративного ИИ на базе RAG

На этапе *извлечения* вы изучаете библиотеку в поиске книг по интересующей теме (см. рис. 1.1, *слева*). Затем возвращаетесь за свой стол, находите в собранных книгах нужную информацию сами или поручаете поиск помощнику и извлекаете ее. На этапе *генерации* (см. рис. 1.1, *справа*) приступаете к написанию реферата. В описанной схеме вы выступаете в роли генеративного агента-человека, использующего RAG, то есть действуете аналогично генеративному фреймворку ИИ на базе RAG.

В процессе написания реферата вы сталкиваетесь с рядом сложных тем, но у вас нет времени на то, чтобы физически изучить всю эту информацию. Вы, как генеративный человек-агент, оказываетесь в тупике, в каком оказалась бы и модель генеративного ИИ. Тогда можете попробовать написать хоть что-то, как это и делает модель, когда выдает некорректный результат. При этом вы, как и модель ИИ, не поймете, где написанное вами правда, а где — нет, пока кто-нибудь не скорректирует реферат и не присвоит ему оценку.

На этом этапе вы достигли своего предела и решаете вернуться к ИИ-ассистенту на базе RAG, чтобы убедиться в корректности ответов. Но оказываетесь озадачены большим числом доступных LLM-моделей и их конфигураций. Для начала потребуется разобраться, какие ресурсы вам доступны и как вообще организована RAG. Далее мы пройдемся по основным видам конфигурации LLM.

Простая, продвинутая и модульная конфигурации RAG

Фреймворк RAG всегда включает в себя два основных компонента: модуль извлечения информации (ретривер) и генератор результата. Генератором может выступать любая LLM, мультимодальная платформа ИИ либо модель, например GPT-4o, Gemini, Llama или одна из сотен вариаций базовых архитектур. Ретривером же может быть любой из новейших фреймворков, методов и инструментов, например Activeloop, Pinecone, LlamaIndex, LangChain, Chroma и многие другие.

Выбор одного из трех типов RAG (Gao et al., 2024) будет зависеть от задач проекта. Все эти три подхода мы разберем в коде текущей главы.

- *Простая RAG*. Не подразумевает сложного встраивания данных и индексирования. Ее можно эффективно использовать для обработки адекватных объемов данных с помощью ключевых слов, например, для дополнения запроса пользователя и получения желаемого ответа.
- *Продвинутая RAG*. Такие механизмы способны обрабатывать более сложные сценарии с применением векторного поиска и извлечения информации на основе индексирования. Продвинутую RAG можно реализовать множеством разных способов. Этот фреймворк сможет обрабатывать различные типы данных, в том числе мультимодальных, которые бывают структурированными или неструктурированными.
- *Модульная RAG*. Модульные решения расширяют горизонт возможностей, включая сценарии, где задействованы простая RAG, продвинутая RAG, машинное обучение и любой другой алгоритм, необходимый для реализации комплексного проекта.

Но прежде чем идти дальше, нам нужно решить, будем ли мы реализовывать именно RAG или же тонко настраивать (fine-tune) модель.

RAG или тонкая настройка?

RAG не всегда может заменить тонкую настройку, как и тонкая настройка не всегда заменит RAG. Если мы соберем в датасетах RAG слишком много данных, системой станет очень сложно управлять. В то же время мы не можем тонко настроить модель с динамическими, постоянно меняющимися данными, такими как прогнозы погоды, стоимость ценных бумаг, корпоративные новости и всевозможные виды каждодневных событий.

Выбор между созданием RAG и тонкой настройкой модели будет зависеть от конкретной пропорции параметрической и непараметрической информации. Фундаментальное различие между моделью, которая обучалась с нуля или

прошла тонкую настройку, и RAG можно обобщить, опираясь на объем их параметрических и непараметрических знаний.

- *Параметрические.* В экосистеме генеративных ИИ на базе RAG параметрический аспект относится к параметрам (весам) модели, усвоенным из обучающих данных. Это означает, что знания модели хранятся в «выученных» весах и смещениях. Изначальные обучающие данные преобразуются в математическую форму, которую мы называем параметрическим представлением. По сути, модель запоминает то, что выучила из этих данных, но сами данные явно не сохраняет.
- *Непараметрические.* И наоборот, непараметрический аспект экосистемы RAG связан с сохранением явных данных, к которым можно обращаться напрямую. Это означает, что данные остаются доступными и при необходимости их можно запрашивать. В отличие от параметрических моделей, где знания усваиваются косвенно в виде весов, непараметрические данные в RAG позволяют видеть и использовать для вывода фактическую информацию.

Выбор между RAG и тонкой настройкой основывается на количестве статических (параметрических) и динамических (непараметрических) данных, которые модели ИИ нужно обрабатывать. Система, которая излишне опирается на RAG, со временем может стать громоздкой и неудобной в управлении. Если же система чрезмерно опирается на тонкую настройку модели, она будет отличаться неспособностью адаптироваться к ежедневному обновлению информации.

На рис. 1.2 показана схема принятия решения на основе определенного порога. Опираясь на этот порог, руководителю проекта генеративного ИИ на базе RAG нужно оценить потенциал параметрической модели ИИ, прежде чем реализовывать непараметрический (с явными данными) механизм RAG. При этом потенциал компонента RAG также требует внимательной оценки.

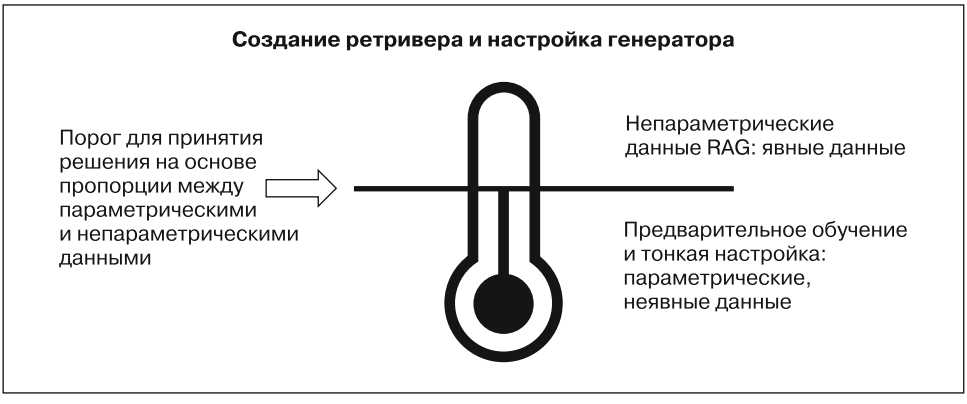


Рис. 1.2. Порог для принятия решения о выборе между расширением датасета RAG и настройкой LLM

Баланс между расширением ретривера и генератора в системе генеративного ИИ на базе RAG зависит от конкретных требований и целей проекта. RAG и тонкая настройка не являются взаимоисключающими.

Для повышения общей эффективности модели можно использовать RAG, дополняя ее тонкой настройкой, которая позволяет улучшить качество как извлечения информации, так и генерации ответа механизмом RAG. Настройкой датасета RAG мы займемся в главе 9.

Сейчас же рассмотрим то множество компонентов, которые включает в себя экосистема генеративного ИИ на базе RAG.

Экосистема RAG

Генеративный ИИ на базе RAG — это фреймворк, который можно реализовать в разных конфигурациях. На рис. 1.3 показана обширная система, внутри которой работает этот фреймворк. Но сколько бы фреймворков извлечения информации или генерации ответов вы ни встречали, их все можно разделить на четыре основные области, сопровождаемые вопросами.

- *Данные*. Откуда берутся данные? Надежны ли они? Достаточно ли этих данных? Связаны ли они с авторскими правами, конфиденциальностью или вопросами безопасности?
- *Хранилище*. Как данные будут храниться до или после их обработки? Какой объем данных будет сохранен?
- *Извлечение*. Как нужные данные будут извлекаться для дополнения ввода пользователя достаточным для генеративной модели объемом информации? Какой вид фреймворка RAG окажется оптимальным для проекта?
- *Генерация*. Какая модель генеративного ИИ впишется в выбранный фреймворк RAG?

Сегменты данных, хранилища и генерации во многом определяются выбранным механизмом RAG. Прежде чем делать этот выбор, необходимо оценить пропорцию параметрических и непараметрических данных в создаваемой экосистеме. На рис. 1.3 показан механизм RAG, включающий перечисленные компоненты без учета реализуемого вида RAG.

- *Ретривер (D)* отвечает за сбор, обработку, хранение и извлечение данных.
- *Генератор (G)* отвечает за дополнение ввода, промт-инжиниринг и генерацию результата.
- *Модуль оценки (E)* обрабатывает вычисление метрик, оценку человеком и обратную связь от него.
- *Учитель (T)* обрабатывает изначально обученную модель и выполняет ее тонкую настройку.

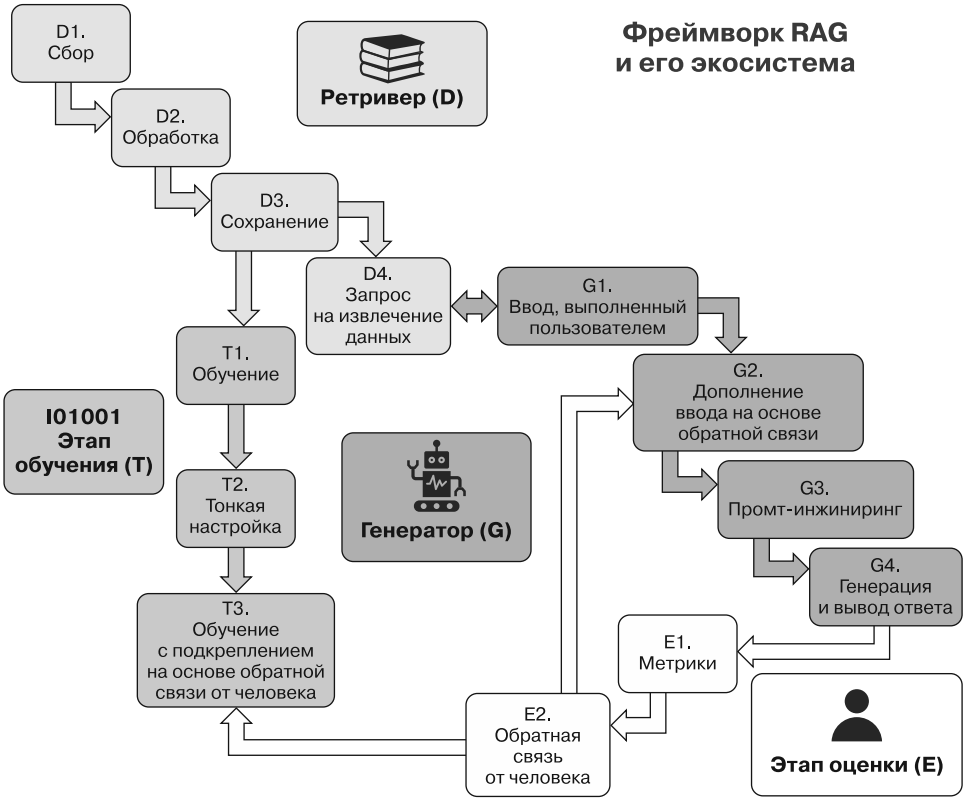


Рис. 1.3. Экосистема генеративного RAG

Каждый из этих четырех компонентов опирается на соответствующие системы, которые формируют общий пайплайн генеративного ИИ на базе RAG. В последующих разделах мы подробнее поговорим о компонентах D, G, E и T. Начнем с ретривера.

Ретривер (D)

Модуль поиска информации в системе RAG собирает, обрабатывает, сохраняет и извлекает данные. В связи с этим стартовой точкой системы выступает процесс поглощения и переработки данных, начинающийся с их сбора.

Сбор (D1)

В современном мире данные ИИ так же разнообразны, как и наши плейлисты медиаконтента. Это может быть что угодно, начиная с фрагмента текста в посте и заканчивая мемом или даже последним эстрадным хитом, транслируемым прямо в наушники. И это далеко не все — файлы могут иметь всевозможные форматы

и размеры. К примеру, PDF, заполненный кучей информации, веб-страницы, простые текстовые файлы, четко организованные файлы JSON, запоминающиеся записи MP3, видео MP4, а также PNG- или JPG-изображения.

Хуже того, значительная часть этих данных не структурирована и встречается в непредсказуемых и сложных формах. К счастью, многие платформы, такие как Pinecone, OpenAI, Chroma и Activeloop, предоставляют готовые инструменты для обработки и хранения таких хаотичных данных.

Обработка (D2)

В схеме мультимодальной обработки на этапе сбора данных D1 различные их типы, такие как текст, изображения и видео, посредством веб-скрапинга извлекаются с сайтов или из других источников. Затем собранные объекты данных преобразуются в однородное представление в виде признаков. Например, данные могут разбиваться на чанки (небольшие части), преобразовываться в векторы (эмбеддинг) и индексироваться для облегчения их поиска и извлечения.

Сохранение (D3)

На этом этапе пайплайна мы уже собрали и начали обрабатывать большой объем разнообразных данных — видео, картинок, текста, да чего угодно. Как теперь поступить с этими данными, чтобы сделать их полезными?

Здесь на помощь приходят такие векторные базы данных, как Deep Lake, Pinecone и Chroma. Можете рассматривать их как умные библиотеки, которые не просто хранят ваши данные, но и преобразуют их в математические сущности — векторы, доступные для вычислительной обработки. Кроме того, в этих хранилищах для ускорения доступа к данным могут применяться различные методы индексирования и прочие техники.

Вместо хранения собранной информации в статических электронных таблицах и файлах мы преобразуем ее в динамическую систему с возможностью поиска, которая может лежать в основе различных инструментов, начиная с чат-ботов и заканчивая механизмами поиска.

Запрос данных (D4)

Процесс извлечения активируется пользовательским или автоматизированным вводом (G1).

Для быстрого извлечения данных после преобразования в подходящий формат мы загружаем их в векторные БД в виде датасетов. Затем, используя комбинацию из поиска по ключевым словам, создания эмбеддингов и индексирования, сможем эффективно эти данные извлекать. К примеру, с помощью косинусной меры сходства можно находить тесно связанные элементы, обеспечивая не только ускоренный поиск, но и выдачу актуальных результатов. После извлечения данных мы дополняем ими ввод.

Генератор (G)

Глядя на рис. 1.3, представляющий фреймворк и экосистему RAG, мы видим, что граница, разделяющая ввод и извлечение, сильно размыта. Ввод G1, будь то автоматизированный или выполненный человеком, взаимодействует с запросом данных (D4), из которого дополняется перед отправкой в генеративную модель.

Иначе говоря, поток генерации начинается с ввода.

Ввод (G1)

Ввод может быть представлен пакетом автоматизированных задач (к примеру, обработка электронных писем) или промтов, переданных человеком через *User Interface* (UI). Эта гибкость позволяет легко интегрировать ИИ в различные профессиональные среды, повышая продуктивность в соответствующих отраслях.

Дополнение ввода обратной связью от человека (G2)

Ввод можно дополнить *обратной связью* (human feedback, HF). Обратная связь позволяет значительно повысить адаптируемость экосистемы RAG, а также обеспечить полный контроль над извлечением данных и вводом для генеративного ИИ. В разделе «Создание гибридной адаптивной RAG на Python» главы 5 мы реализуем механизм дополнения ввода обратной связью от человека.

Промт-инжиниринг (G3)

При подготовке базового и дополненного сообщения, которое нужно будет обработать модели ИИ, ретривер (D) и генератор (G) активно опираются на промт-инжиниринг. В ходе этого процесса вывод ретривера и ввод, выполненный пользователем, объединяются.

Генерация ответа и его вывод (G4)

Выбор модели генеративного ИИ зависит от целей проекта. Llama, Gemini, GPT и прочие их варианты подходят для разных задач. При этом промт должен отвечать характеристикам любой выбранной модели. Фреймворки наподобие LangChain, который мы реализуем позже, помогают упростить интеграцию различных моделей в приложения, предоставляя гибкие интерфейсы и инструменты.

Модуль оценки (E)

При анализе качества модели генеративного ИИ мы зачастую опираемся на математические метрики. Однако они позволяют увидеть лишь часть картины. Здесь важно помнить, что финальной проверкой эффективности ИИ является оценка человеком.

Метрики (E1)

Как и любую систему ИИ, модель невозможно оценить без использования метрик, таких как косинусное сходство (оно же коэффициент Отиаи). Они обеспечивают актуальность и точность извлеченных данных. Количественно измеряя связанность и релевантность точек данных, метрики предоставляют прочную основу для анализа качества и надежности модели.

Обратная связь от человека (E2)

Любая система ИИ, как на базе RAG, так и нет, даже при, казалось бы, достаточном количестве метрик должна проходить этап оценки человеком. Именно он в конечном счете решает, заслуживает ли созданная для людей система похвалы или критики и нужно ли ее принять или отклонить.

Механизм адаптивной RAG вносит человеческую, основанную на реальности обратную связь, которая повышает качество экосистемы генеративного ИИ. Мы реализуем адаптивную RAG в главе 5.

Учитель (T)

Типичная модель генеративного ИИ предварительно обучается на обширных объемах общих данных. Затем эту модель можно настроить (T2), используя данные из конкретной предметной области.

Но мы пойдем еще дальше и в главе 9 интегрируем статические данные RAG в процесс тонкой настройки. Кроме того, включим в этот процесс технику *обучения с подкреплением на основе отзывов людей* (Reinforcement Learning from Human Feedback, RLHF), основанную на получении ценной обратной связи от человека-эксперта.

Теперь мы готовы приступить к написанию простой, продвинутой и модульной RAG на Python.

Простая, продвинутая и модульная RAG в коде

В этом разделе мы познакомимся с простой, продвинутой и модульной системами RAG, используя базовые обучающие примеры. Эта программа будет включать сопоставление по ключевым словам, векторный поиск и извлечение данных по индексу. Применяя GPT-модели от OpenAI, она будет генерировать ответы на основе входящих запросов и извлеченных документов.

Цель этого блокнота — создать диалоговый агент, который будет отвечать на вопросы по RAG в целом. Здесь мы с нуля напишем на Python полноценный ретривер и запустим генератор на базе GPT-4o, разбив все это на две части, включающие восемь секций кода.

Часть 1. Основы и базовая реализация

1. *Настройка среды* для интеграции OpenAI API.
2. *Функция генерации* на основе GPT-4o.
3. *Настройка данных* с помощью списка документов (`db_records`).
4. *Запрос ввода* от пользователя.

Часть 2. Продвинутые техники и оценка

1. *Метрики извлечения* для оценки его результатов.
2. *Простая RAG* на основе поиска по ключевым словам с помощью функции сопоставления.
3. *Продвинутая RAG* с использованием векторного поиска и поиска по индексу.
4. *Модульная RAG*, реализующая гибкие методы извлечения данных.

Для начала откройте `RAG_Overview.ipynb` в репозитории GitHub. Мы начнем с блокнота и познакомимся с базовой реализацией.

Часть 1. Основы и базовая реализация

В этом разделе мы настроим среду, создадим функцию генератора и функцию для вывода форматированного ответа, а также определим механизм создания запроса пользователем.

Первым шагом будет установка рабочей среды.

> **ПРИМЕЧАНИЕ**
>
> Названия разделов приведенной далее реализации блокнота соответствуют структуре кода. То есть вы можете как повторять код из блокнота, так и читать этот самодостаточный раздел.

1. Среда

Основным пакетом для доступа к GPT-4o через API будет OpenAI:

```
!pip install openai==1.40.3
```

Обязательно зафиксируйте установленную версию OpenAI. В экосистеме фреймворков RAG для работы с продвинутыми конфигурациями потребуется установить несколько пакетов. Закончив с настройкой, мы зафиксируем версию этих пакетов, чтобы минимизировать возможные конфликты между библиотеками и реализуемыми нами модулями.

После установки `openai` создайте аккаунт на платформе OpenAI (если у вас его еще нет) и получите ключ API. Прежде чем запускать API, ознакомьтесь с условиями и стоимостью предлагаемых тарифных планов.

Получив ключ API, сохраните его в безопасном месте, например Google Drive, откуда сможете запрашивать этот ключ так:

```
# Ключ API
# Сохраните ключ в файле и впоследствии считывайте его оттуда
# (можете вводить его прямо в блокнот, но тогда его смогут
# увидеть находящиеся рядом люди).
from google.colab import
drive drive.mount('/content/drive')
```

Для хранения ключа вполне подойдет Google Drive или любой другой метод. Вы можете как считывать его из файла, так и прописывать непосредственно в коде:

```
f = open("drive/MyDrive/files/api_key.txt", "r")
API_KEY=f.readline().strip()
f.close()
# Ключ OpenAI
import os
import openai
os.environ['OPENAI_API_KEY'] =API_KEY
openai.api_key = os.getenv("OPENAI_API_KEY")
```

На этом настройка основных ресурсов для проекта закончена. Теперь напишем функцию генерации для модели OpenAI.

2. Генератор

Приведенный далее код импортирует `openai` для генерации контента и `time` для измерения времени выполнения запросов:

```
import openai
from openai import OpenAI
import time
client = OpenAI()
gptmodel="gpt-4o"
start_time = time.time()    # Начало отсчета времени перед запросом
```

Далее мы создадим функцию генерации промта на основе инструкции и ввода, выполненного пользователем:

```
def call_llm_with_full_text(itext):
    # Объединение всех строк в одну
    text_input = '\n'.join(itext)
    prompt = f"Please elaborate on the following content:\n{text_input}"
```

Эта функция будет вызывать `gpt-4o`, чтобы снабдить модель дополнительной информацией:

```
    try:
      response = client.chat.completions.create(
        model=gptmodel,
        messages=[
          {"role": "system", "content": "You are an expert Natural Language
            Processing exercise expert."},
```

```
          {"role": "assistant", "content": "1.You can explain read the input
            and answer in detail"},
          {"role": "user", "content": prompt}
        ],
        temperature=0.1  # Укажите здесь температуру и прочие нужные параметры
      )
    return response.choices[0].message.content.strip()
  except Exception as e:
    return str(e)
```

Заметьте, что для сохранения гибкости модели сообщения инструкций в этом сценарии остаются обобщенными. У нас параметр `temperature` имеет низкое значение `0.1` (высокая точность). Если вы хотите повысить креативность модели, увеличьте этот параметр, например, до `0.7`. Однако в нашем случае лучше настраивать модель на максимально точные ответы.

Добавим также `textwrap`, чтобы ответ модели форматировался в виде аккуратного абзаца:

```
import textwrap
def print_formatted_response(response):
    # Определяем ширину абзаца для переноса текста
    wrapper = textwrap.TextWrapper(width=80)   # Устанавливаем ширину столбцов 80
                                               # и при необходимости корректируем
    wrapped_text = wrapper.fill(text=response)
    # Выводим отформатированный ответ с шапкой и футером
    print("Response:")
    print("---------------")
    print(wrapped_text)
    print("---------------\n")
```

ПРИМЕЧАНИЕ

Генератор готов к использованию. Правда, ввиду вероятностной природы моделей генеративного ИИ он может при каждом вызове выдавать слегка различающиеся результаты.

На этом мы закончили реализацию в программе функции извлечения данных.

3. Данные

Сбор данных может включать получение текста, изображений, аудио и видео. Но в текущем блокноте мы сосредоточимся именно на извлечении данных с помощью простой, продвинутой и модульной конфигураций системы. Сбором и векторизацией данных займемся в главе 2. Поэтому предположим, что нужные нам данные уже были обработаны, то есть собраны, очищены и разбиты на пред-

ложения. Предположим также, что этот процесс включал загрузку предложений в список Python с именем `db_records`.

Такой подход иллюстрирует три особенности экосистемы RAG, представленных на рис. 1.3.

- *Ретривер (D)* состоит из трех подготовительных этапов *обработки данных — сбора D1*, *обработки D2* и *сохранения D3*.
- Получается, что *запрос на извлечение D4* независим от первых трех стадий ретривера (сбора, обработки и сохранения).
- Этап обработки данных зачастую выполняется независимо и до выполнения запроса на извлечение, как мы и будем реализовывать его, начиная с главы 2.

Текущая программа предполагает, что обработка данных произведена и датасет готов:

```
db_records = [
    "Retrieval Augmented Generation (RAG) represents a sophisticated hybrid
     approach in the field of artificial intelligence, particularly within
     the realm of natural language processing (NLP).",
…/…
```

Далее мы можем вывести отформатированную версию этого датасета:

```
import textwrap
paragraph = ' '.join(db_records)
wrapped_text = textwrap.fill(paragraph, width=80)
print(wrapped_text)
```

При выводе предложения из `db_records` объединяются, как показано в отрывке далее, но сам `db_records` остается неизменным:

```
Retrieval Augmented Generation (RAG) represents a sophisticated hybrid approach
in the field of artificial intelligence, particularly within the realm of natural
language processing (NLP)…
```

Теперь программа готова к обработке запроса.

4. Запрос

Запрос на извлечение (D4 на рис. 1.3) выполняется в соответствии с тем, как обрабатывались данные, но представляет собой просто ввод, выполненный пользователем, или автоматизированный ввод от другого ИИ-агента. Мы все мечтаем о пользователях, которые бы вводили в наши системы оптимальные запросы. К сожалению, в реальности ввод, которого программа не ожидает, ведет к ее непредсказуемому поведению. Поэтому системы должны создаваться с учетом подобного неточного ввода.

В текущем разделе мы представим ситуацию, когда сотни пользователей в организации услышали термин «RAG» в контексте LLM и векторных баз данных. Многие из них захотели понять значение этих терминов, чтобы говорить на одном языке с командой, разрабатывающей диалоговый агент в их подразделении. Спустя пару дней они уже смутно помнят эти услышанные ранее термины и для уточнения обращаются к GPT-4o с таким запросом:

```
query = "define a rag store"
```

Здесь мы просто сохраним основной запрос по теме создаваемой программы в виде `query`, рождающегося как результат действий ретривера и генератора. Он будет активировать определенную конфигурацию RAG (простую, продвинутую и модульную), выбор которой будет зависеть от целей конкретного проекта. Программа получает введенный запрос и отправляет его модели GPT-4o для обработки и форматированного вывода:

```
# Вызов функции и вывод результата
llm_response = call_llm_with_full_text(query)
print_formatted_response(llm_response)
```

Вывод, полученный в ответ на этот запрос, многое проясняет. Даже самые мощные модели генеративного ИИ не смогут угадать, что пользователь, который ничего об ИИ не знает, хотел таким запросом выяснить[1]. В этом случае GPT-4o ответит так, как показано в отрывке вывода далее:

```
Response:
---------------
Certainly! The content you've provided appears to be a sequence of characters
that, when combined, form the phrase "define a rag store." Let's break it down
step by step:…
… This is an indefinite article used before words that begin with a consonant
sound. - **rag**: This is a noun that typically refers to a pieceof old, often
torn, cloth. - **store**: This is a noun that refers to a place where goods are
sold. 4. **Contextual Meaning**: - **"Define a rag store"**: This phrase is
asking for an explanation or definition of what a "rag store" is. 5. **Possible
Definition**: - A "rag store" could be a shop or retail establishment that
specializes in selling rags,…
```

Подобный текст может показаться галлюцинацией, но так ли это на самом деле? Пользователь, как и любой человек, начинающий изучение новой темы, написал запрос с благими намерениями. GPT-4o тоже ответил на него со всей ответственностью, опираясь на предоставленный ограниченный контекст и свой вероятностный алгоритм, который может при каждом новом выполнении давать слегка измененный ответ. Тем не менее GPT-4o отнесся к запросу с осторожно-

[1] В английском языке у слова store (как и у rag) есть разные значения, и далее в ответе модели приводится пример, когда она не понимает, о чем речь, и предполагает, что имеется в виду «магазин» (один из вариантов store). В итоге она приходит к тому, что rag store — это магазин продажи поношенной одежды. — *Примеч. пер.*

стью. Ввод, выполненный пользователем, не был ясен модели, поэтому в ответ она запросила дополнительный контекст:

```
…Would you like more information or a different type of elaboration on this
content?…
```

Пользователь озадачен и не знает, что делать, а GPT-4o ожидает дальнейших инструкций. Команде разработчиков этой модели нужно что-то предпринять...

ПРИМЕЧАНИЕ

В основе генеративного ИИ лежат вероятностные алгоритмы, из-за чего их результаты для одного и того же ввода могут каждый раз отличаться друг от друга. То есть модель будет давать похожие, но не одинаковые ответы.

Как раз в таких ситуациях на помощь приходит RAG. Мы не станем менять запрос на протяжении всего блокнота и посмотрим, сможет ли система GPT-4o на базе RAG справиться лучше.

Часть 2. Продвинутые техники и оценка

В части 2 мы познакомимся с простой, продвинутой и модульной RAG. Нашей целью будет изучить все эти три метода, а не обрабатывать сложные документы, чем мы займемся на протяжении остальной части книги.

Начнем с определения метрик, позволяющих оценить точность извлекаемых документов.

1. Метрики оценки извлечения

В качестве первой метрики мы познакомимся с косинусным сходством, позволяющим оценить релевантность текстовых документов. Затем реализуем расширенные метрики оценки сходства, добавив обработку синонимов и предварительную обработку текста. Это позволит повысить точность определения сходства между текстами.

Дополнительные метрики мы изучим в подразделе «Вычисление и вывод метрик» главы 7.

В текущей же главе начнем, как и говорилось, с косинусного сходства.

Косинусное сходство

Косинусное сходство (cosine similarity) основано на измерении косинуса угла между двумя векторами. В нашем случае этими векторами выступают запрос пользователя и каждый документ в корпусе текста.

Сначала программа импортирует нужные нам класс и функцию:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

`TfidfVectorizer` импортирует класс, преобразующий текстовые документы в матрицу признаков *TF-IDF* (term frequency, inverse document frequency). TF-IDF используется для количественной оценки важности слова для конкретного документа из коллекции, позволяя отличать часто встречающиеся типичные слова от имеющих особую значимость для определенных текстов. Производится эта оценка на основе частоты его вхождений в документ и обратной частоты вхождений по всему корпусу. `cosine_similarity` импортирует функцию, которую мы будем применять для вычисления сходства векторов.

Затем `calculate_cosine_similarity(text1, text2)` вычисляет косинусное сходство между запросом (`text1`) и каждой записью датасета.

Эта функция преобразует текст запроса (`text1`) и каждую запись (`text2`) датасета в вектор с помощью векторизатора, после чего вычисляет и возвращает показатель косинусного сходства между двумя векторами:

```
def calculate_cosine_similarity(text1, text2):
    vectorizer = TfidfVectorizer(
        stop_words='english',
        use_idf=True,
        norm='l2',
        ngram_range=(1, 2),  # Используем униграммы и биграммы
        sublinear_tf=True,   # Применяем сублинейное TF-масштабирование
        analyzer='word'      # Можете поэкспериментировать с 'char' или
                             # 'char_wb' для признаков на уровне символов
    )
    tfidf = vectorizer.fit_transform([text1, text2])
    similarity = cosine_similarity(tfidf[0:1], tfidf[1:2])
    return similarity[0][0]
```

Вот ключевые параметры этой функции.

- `stop_words='english` игнорирует распространенные английские слова, чтобы акцентироваться на смысловом содержании.
- `use_idf=True` активирует механизм взвешивания на основе обратной частоты документа.
- `norm='l2'` применяет к каждому выходному вектору нормализацию L2.
- `ngram_range=(1, 2)` учитывает как одиночные слова, так и парные комбинации.
- `sublinear_tf=True` применяет логарифмическое масштабирование частоты термина.
- `analyzer='word'` анализирует текст на уровне слов.

В некоторых случаях механизм вычисления косинусного сходства имеет ограничения, например работая с двусмысленными запросами, так как измеряет сходство строго на основе угла между векторными представлениями текста. Если

пользователь задает неопределенный вопрос наподобие: «Что такое rag?», как мы видели ранее, а база данных программы преимущественно содержит информацию о RAG, которая для ИИ — retrieval-augmented generation, а не «тряпье», то косинусное сходство окажется низким. Показатель получается низким, так как математической модели не хватает понимания контекста для различения возможных значений слова rag. Она вычисляет сходство лишь на основе присутствия и частоты похожих слов в тексте, не понимая намерений пользователя или более широкого контекста запроса. Таким образом, даже если итоговый ответ модели будет технически точным в рамках доступного датасета, косинусное сходство может не отражать этого, если контекст запроса в датасете представлен плохо.

В таком случае можно попробовать метрику расширенного сходства.

Расширенное сходство

Мера расширенного сходства привносит вычисления, реализуемые инструментами обработки естественного языка, позволяя лучше охватить семантическую связь слов. Этот механизм, используя такие библиотеки, как spaCy и NTLK, выполняет предварительную обработку текстов для уменьшения шума, дополняет термины синонимами из WordNet и вычисляет сходство на основе семантической обогащенности полученного словаря. Такой подход помогает повысить точность определения сходства двух текстов за счет учета более широкого контекста в сравнении с типичными методами непосредственного сравнения.

Соответствующий код содержит четыре основные функции.

- `get_synonyms(word)` извлекает из WordNet синонимы указанного слова.
- `preprocess_text(text)` преобразует весь текст в нижний регистр, лемматизирует его (получает корни слов) и удаляет стоп-слова (типичные слова, не несущие смысловой нагрузки) и знаки пунктуации.
- `expand_with_synonyms(words)` расширяет список слов, добавляя к ним синонимы.
- `calculate_enhanced_similarity(text1, text2)` вычисляет косинусное сходство между предварительно обработанными и дополненными синонимами векторами.

Функция `calculate_enhanced_similarity(text1, text2)` получает на входе два текста и в итоге возвращает показатель косинусного сходства между ними после предварительной обработки и дополнения синонимами. Этот показатель количественно отражает текстовое сходство на основе семантического содержимого и дополненных наборов слов.

Код начинается со скачивания и импорта необходимых библиотек, после чего выполняются описанные функции, начиная с `calculate_enhanced_similarity(text1, text2)`:

```
import spacy
import nltk
nltk.download('wordnet')
```

```
from nltk.corpus import wordnet
from collections import Counter
import numpy as np

# Загрузка модели spaCy
nlp = spacy.load("en_core_web_sm")
…
```

Расширенное сходство дает более продвинутую оценку, но даже в этом случае интеграция RAG с генеративным ИИ все равно представляет ряд сложностей.

Вне зависимости от используемой метрики мы столкнемся со следующими ограничениями.

- *Большая разница между длиной запроса и текста*. Если запросы пользователей обычно короткие, то извлекаемые документы часто намного длиннее и богаче, что усложняет прямое вычисление сходства.
- *Креативное извлечение*. Системы могут креативно выбирать более длинные документы, соответствующие запросу пользователя, но дающие низкие значения метрик из-за неожиданного выравнивания содержимого.
- *Потребность в обратной связи от человека*. Зачастую человеческая оценка оказывается важным аспектом точного анализа релевантности и эффективности извлеченного содержимого. Дело в том, что автоматизированные метрики не всегда в достаточной мере отвечают ожиданиям пользователя. Мы будем подробнее говорить об этом значимом аспекте RAG в главе 5.

И нам всегда придется искать правильный баланс между математическими метриками и человеческой оценкой. Ну а теперь пора приступить к созданию образца простой RAG.

2. Простая RAG

Простая RAG на основе поиска по ключевым словам и сопоставления оказывается эффективной в случае четко структурированных документов внутри организации, например юридических или медицинских. Обработку упрощает то, что подобные документы обычно имеют понятные заголовки, подписи к изображениям и прочие детали, помогающие ориентироваться в них. В функции простой RAG мы реализуем поиск по ключевым словам и сопоставление. Для этого используем в коде простой метод извлечения информации.

1. Разделение запроса на отдельные слова.
2. Разделение каждой записи датасета на ключевые слова.
3. Определение длины совпадений.
4. Выбор записи с максимальным показателем.

Генерация при этом будет выполняться так.

1. Дополнение пользовательского ввода результатами извлечения.
2. Обращение к модели генерации, которой в нашем случае будет GPT-4o.
3. Вывод ответа.

Начнем с реализации поиска по ключевым словам и сопоставления.

Поиск и сопоставление ключевых слов

Функция поиска максимального соответствия сначала инициализирует лучший показатель:

```
def find_best_match_keyword_search(query, db_records):
    best_score = 0
    best_record = None
```

Затем запрос разделяется на ключевые слова. Каждая запись также разделяется на слова для поиска общих ключевых слов, затем измеряется длина совпавшего содержимого, после чего находится максимально схожий по количеству совпадений результат:

```
# Разделение запроса на ключевые слова
    query_keywords = set(query.lower().split())

    # Перебор каждой записи в db_records
    for record in db_records:
        # Разделение записи на ключевые слова
        record_keywords = set(record.lower().split())
        # Вычисление количества общих ключевых слов
        common_keywords = query_keywords.intersection(record_keywords)
        current_score = len(common_keywords)
        # Обновление лучшего показателя и его запись,
        # если текущий показатель выше
        if current_score > best_score:
            best_score = current_score
            best_record = record

    return best_score, best_record
```

Теперь вызовем эту функцию, отформатируем ее ответ и выведем его:

```
# Предполагается, что вы определили 'query' и 'db_records'
# в предыдущих ячейках блокнота Colab
best_keyword_score, best_matching_record = find_best_match_keyword_
search(query, db_records)
print(f"Best Keyword Score: {best_keyword_score}")
#print(f"Best Matching Record: {best_matching_record}")
print_formatted_response(best_matching_record)
```

Основным запросом этого блокнота будет `query = "define a rag store"`. С его помощью мы будем определять, выдаст ли каждый из реализуемых нами методов RAG приемлемый результат.

В процессе поиска по ключевым словам была найдена лучшая запись из списка предложений датасета:

```
Best Keyword Score: 3
Response:
--------------
A RAG vector store is a database or dataset that contains vectorized data points.
--------------
```

Пора перейти к метрикам.

Метрики

Мы создали метрики оценки сходства в подразделе «1. Метрики оценки извлечения» текущей главы. Сначала применим метрику косинусного сходства:

```
# Косинусное сходство
score = calculate_cosine_similarity(query, best_matching_record)
print(f"Best Cosine Similarity Score: {score:.3f}")
```

Показатель получился довольно низким, о чем говорится в том же разделе. Запрос пользователя был коротким, а ответ получился более длинным и развернутым.

```
Best Cosine Similarity Score: 0.126
```

Оценка расширенного сходства даст более высокий показатель:

```
# Расширенное сходство
response = best_matching_record
print(query,": ", response)
similarity_score = calculate_enhanced_similarity(query, response)
print(f"Enhanced Similarity:, {similarity_score:.3f}")
```

Более высокий балл объясняется расширенной функциональностью метрики:

```
define a rag store : A RAG vector store is a database or dataset that contains
vectorized data points.
Enhanced Similarity:, 0.642
```

Теперь дополним результатом запроса пользовательский ввод.

Дополненный ввод

Дополненный ввод представляет собой конкатенацию пользовательского ввода и максимально совпавшей с ним записи датасета, найденной в процессе поиска по ключевым словам:

```
augmented_input=query+ ": "+ best_matching_record
```

При необходимости дополненный ввод можно вывести для дополнительного анализа:

```
print_formatted_response(augmented_input)
```

Результат показывает, что дополненный ввод готов:

```
Response:
---------------
define a rag store: A RAG vector store is a database or dataset that contains
vectorized data points.
---------------
```

Можно генерировать запрос.

Генерация

Теперь мы готовы вызвать GPT-4o и вывести отформатированный ответ:

```
llm_response = call_llm_with_full_text(augmented_input)
print_formatted_response(llm_response)
```

Фрагмент ответа показывает, что GPT-4o поняла запрос и дает интересный ответ по делу:

```
Response:
---------------
Certainly! Let's break down and elaborate on the provided content: ### Define
a RAG Store: A **RAG (Retrieval-Augmented Generation) vector store** is a
specialized type of database or dataset that is designed to store and manage
vectorized data points…
```

Простой RAG оказывается вполне достаточно во многих ситуациях. Однако, когда объем документов слишком велик или их содержимое сложное, более качественные результаты позволит получить продвинутая модель RAG. Познакомимся с ней.

3. Продвинутая RAG

С увеличением датасета время выполнения поиска по ключевым словам может сильно затягиваться. Например, если мы работаем с сотнями документов, каждый из которых содержит сотни предложений, станет трудно опираться лишь на поиск по ключевым словам. В этом случае использование индекса позволит сократить вычислительную нагрузку до обработки лишь небольшой части от всех данных.

В текущем разделе мы выйдем за рамки поиска по ключевым словам и увидим, как RAG преобразует текстовые данные в численные представления, повышая эффективность поиска и скорость обработки. В отличие от традиционных

методов, основанных на прямом парсинге текста, RAG сначала преобразует документы и запросы пользователей в векторы, то есть численные представления, которые ускоряют вычисления. Если коротко, то вектор — это список чисел, представляющих различные признаки текста. Простые векторы могут отражать количество вхождений слов (частотность термина, term frequency), а более сложные, называемые эмбеддингами, отражают углубленные лингвистические паттерны.

Реализуем векторный поиск и поиск по индексу.

- *Векторный поиск.* Мы преобразуем каждое предложение датасета в численный вектор. Вычислив косинусное сходство между вектором запроса (запроса пользователя) и этими векторами документов, сможем быстро найти наиболее релевантные из них.
- *Поиск по индексу.* В этом случае все предложения преобразуются в векторы с помощью *TF-IDF* (Term Frequency — Inverse Document Frequency) — статистической меры оценки степени важности слова для документа коллекции. Эти векторы выступают в качестве индексов в матрице, позволяя быстро сравнивать сходство без полного парсинга каждого документа.

Теперь понаблюдаем все это в действии, начиная с векторного поиска.

Векторный поиск

В результате векторного поиска запрос пользователя и документы преобразуются в векторы из численных значений, позволяя применить математические вычисления для *извлечения релевантных данных в условиях обработки больших объемов данных*.

Приведенная далее программа перебирает каждую запись датасета в поиске максимально соответствующего запросу документа путем вычисления косинусного сходства вектора запроса и векторов этих записей.

```
def find_best_match(text_input, records):
    best_score = 0
    best_record = None
    for record in records:
        current_score = calculate_cosine_similarity(text_input, record)
        if current_score > best_score:
            best_score = current_score
            best_record = record
    return best_score, best_record
```

Затем код вызывает функцию векторного поиска и выводит найденную ею лучшую запись:

```
best_similarity_score, best_matching_record = find_best_match(query, db_records)
print_formatted_response(best_matching_record)
```

Вывод получился удовлетворительный:

```
Response:
---------------
A RAG vector store is a database or dataset that contains vectorized data points.
```

Здесь, как и в случае с простой RAG, мы видим лучший найденный ответ. И это показывает, что универсального решения нет. Каждая техника RAG имеет свои достоинства, и далее метрики это подтвердят.

Метрики

Здесь мы применим те же метрики, что и для простой RAG, так как извлекли тот же самый документ:

```
print(f"Best Cosine Similarity Score: {best_similarity_score:.3f}")
```

Вот результат:

```
Best Cosine Similarity Score: 0.126
```

Вычисление расширенного сходства даст нам тот же результат, что и в случае простой RAG:

```
# Расширенное сходство
response = best_matching_record
print(query,": ", response)
similarity_score = calculate_enhanced_similarity(query, best_matching_record)
print(f"Enhanced Similarity:, {similarity_score:.3f}")
```

Пожалуйста:

```
define a rag store : A RAG vector store is a database or dataset that contains
vectorized data points.
Enhanced Similarity:, 0.642
```

Так зачем применять векторный поиск, если он дает тот же результат, что и простая RAG? Просто в небольшом наборе данных все вычисляется довольно просто. Если же мы возьмемся обрабатывать датасеты с миллионами сложных документов, поиск по ключевым словам не отразит все тонкости, доступные поиску по векторам. Теперь дополним запрос пользователя извлеченной информацией.

Дополненный ввод

Мы просто добавляем полученную информацию к запросу пользователя без каких-либо вспомогательных манипуляций и выводим результат:

```
# Вызов функции и вывод результата
augmented_input=query+": "+best_matching_record
print_formatted_response(augmented_input)
```

Здесь мы лишь добавили пробел между запросом пользователя и полученной информацией. Вывод оказался вполне адекватный:

```
Response:
---------------
define a rag store: A RAG vector store is a database or dataset that contains
vectorized data points.
---------------
```

Посмотрим, как на дополненный ввод отреагирует модель генеративного ИИ.

Генерация

Теперь вызовем GPT-4o, используя дополненный ввод, и выведем отформатированный ответ модели:

```
# Вызов функции и вывод результата
augmented_input=query+best_matching_record
llm_response = call_llm_with_full_text(augmented_input)
print_formatted_response(llm_response)
```

Ответ звучит вполне разумно:

```
Response:
---------------
Certainly! Let's break down and elaborate on the provided content: ### Define
a RAG Store: A **RAG (Retrieval-Augmented Generation) vector store** is a
specialized type of database or dataset that is designed to store and manage
vectorized data points…
```

Несмотря на то что векторный поиск значительно ускоряет процесс нахождения релевантных документов за счет последовательного перебора каждой записи, по мере роста датасета его эффективность снижается. Для решения проблемы масштабирования существует поиск по индексу. Посмотрим, как этот метод позволит ускорить извлечение документа.

Поиск на основе индекса

Выполняя поиск по индексу, мы сравниваем вектор запроса пользователя с непосредственным вектором содержимого документа, применяя индексный вектор, представляющий это содержимое.

Сначала импортируем класс и нужную функцию:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

`TfidfVectorizer` импортирует класс, который преобразует текстовые документы в матрицу признаков TF-IDF. Показатель TF-IDF отразит важность слов в до-

кументах на основе частоты их вхождения. Эта функция находит наилучшее совпадение, вычисляя схожесть между запросом и взвешенными векторами матрицы с помощью косинусного сходства:

```
def find_best_match(query, vectorizer, tfidf_matrix):
    query_tfidf = vectorizer.transform([query])
    similarities = cosine_similarity(query_tfidf, tfidf_matrix)
    # Получаем индекс записи с максимальным показателем сходства
    best_index = similarities.argmax()
    best_score = similarities[0, best_index]
    return best_score, best_index
```

Вот основные задачи этой функции.

- *Преобразование запроса*. Преобразовать входящий запрос в вектор TF-IDF с помощью предоставленного векторизатора.
- *Вычисление сходства*. Вычислить косинусное сходство между вектором запроса и всеми векторами в `tfidf_matrix`.
- *Поиск максимального совпадения*. Найти в результатах индекс (`best_index`) записи с наивысшим показателем сходства.
- *Извлечение лучшего показателя*. Извлечь максимальный показатель косинусного сходства (`best_score`).

В выводе отображаются максимальный показатель сходства и индекс соответствующей записи.

Следующий код сначала вызывает векторизатор датасета, после чего находит запись с максимальным сходством по ее индексу:

```
vectorizer, tfidf_matrix = setup_vectorizer(db_records)
best_similarity_score, best_index = find_best_match(query, vectorizer, tfidf_matrix)
best_matching_record = db_records[best_index]
```

И наконец, результат:

```
print_formatted_response(best_matching_record)
```

Система находит самый похожий на запрос пользователя документ:

```
Response:
---------------
A RAG vector store is a database or dataset that contains vectorized data points.
---------------
```

Здесь мы видим, что расплывчатый запрос пользователя привел к выдаче четкого результата еще на этапе извлечения информации, до запуска GPT-4o.

В качестве метрик в этой программе применяются те же, что и в первых двух сценариях. Это нормально, так как найденный документ максимально близок

к запросу пользователя. Мы добавим более сложные документы для RAG, начиная с главы 2. Сейчас же взглянем на признаки, которые определяют то, как слова будут представлены в векторах.

Извлечение признаков

Прежде чем дополнять ввод найденным документом, выполните приведенную далее ячейку. Она снова вызывает функцию `setup_vectorizer(records)`, но на этот раз выводит матрицу, чтобы вы могли увидеть ее формат. Это матрица для слов accurate и additional, содержащихся в одном из предложений (рис. 1.4).

accurate	adapt	additional
0.000000	0.000000	0.000000
0.214779	0.000000	0.000000
0.000000	0.000000	0.000000
0.000000	0.000000	0.000000
0.000000	0.000000	0.236328

Рис. 1.4. Формат матрицы

Теперь дополним ввод.

Дополненный ввод

Здесь мы просто добавим запрос к максимально совпадающей с ним записи, чтобы увидеть, как отреагирует на это GPT-4o:

```
augmented_input=query+": "+best_matching_record
print_formatted_response(augmented_input)
```

Вывод близок к тому, который дал векторный поиск, но в этом случае используется более быстрый метод извлечения:

```
Response:
--------------
define a rag store: A RAG vector store is a database or dataset that contains
vectorized data points.
--------------
```

Подставим дополненный ввод в модель генеративного ИИ.

Генерация

Теперь вызовем GPT-4o с использованием полученного ввода и выведем результат:

```
# Вызов функции и вывод результата
llm_response = call_llm_with_full_text(augmented_input)
print_formatted_response(llm_response)
```

Ответ вполне соответствует тематике изначально туманного запроса пользователя:

```
Response:
---------------
Certainly! Let's break down and elaborate on the given content: --- **Define
a RAG store:** A **RAG vector store** is a **database** or **dataset** that
contains **vectorized data points**. --- ### Detailed Explanation: 1. **RAG
Store**: - **RAG** stands for **Retrieval-Augmented Generation**. It is a
technique used in natural language processing (NLP) where a model retrieves
relevant information from a database or dataset to augment its generation
capabilities…
```

Этот подход хорошо сработал в условиях узкой предметной области внутри закрытой среды организации. В случае же открытой среды пользователю придется поразмыслить, прежде чем задавать модели вопрос.

В текущем разделе мы наблюдали, как матрица TF-IDF предоставляет подготовленные векторы документов, позволяя выполнять ускоренное одновременное сравнение без повторяющегося преобразования векторов. Мы также видели, как векторный поиск и поиск на основе индекса позволяют ускорить извлечение данных. Тем не менее в зависимости от нужных документов в одном проекте нам может потребоваться реализовать модульный механизм, то есть комбинацию из простой и модульной RAG. Посмотрим, как она позволит улучшить нашу систему.

4. Модульная RAG

Какой вид поиска следует использовать при реализации RAG — по ключевым словам, векторный или же по индексу? Каждый из этих подходов имеет свои достоинства, и выбор будет зависеть от нескольких факторов.

- *Поиск по ключевым словам* подходит для простого извлечения данных.
- *Векторный поиск* отлично справляется с семантически богатыми документами.
- *Поиск на основе индекса* обеспечивает высокую скорость обработки больших датасетов.

Однако в проекте прекрасно могут сочетаться все три метода. К примеру, в одном сценарии поиск по ключевым словам перед обработкой документов поможет находить в них четко определенные метки — это могут быть названия PDF-файлов и подписанные изображения. Затем индексный поиск станет группировать документы в индексируемые подмножества. Наконец, модуль извлечения будет находить индексированный датасет, в нем выбирать нужную часть и использовать векторный поиск уже для перебора ограниченного числа документов в поиске наиболее подходящего.

В текущем разделе мы создадим класс `RetrievalComponent`, который будем вызывать на каждом этапе проекта для выполнения нужной задачи. Этот код обобщит

все три созданных нами метода и те, которые мы можем дополнительно включить в `RetrievalComponent` в качестве основных компонентов.

Приведенный далее код инициализирует этот класс с использованием выбора метода поиска и при необходимости подготавливает векторизатор. `self` указывает на текущий экземпляр класса, позволяя обращаться к его переменным, методам и функциям:

```
def __init__(self, method='vector'):
        self.method = method
        if self.method == 'vector' or self.method == 'indexed':
            self.vectorizer = TfidfVectorizer()
            self.tfidf_matrix = None
```

В данном случае активируется векторный поиск.

Метод `fit` создает из записей матрицу TF-IDF и может использоваться для векторного поиска или поиска по индексу:

```
    def fit(self, records):
        if self.method == 'vector' or self.method == 'indexed':
            self.tfidf_matrix = self.vectorizer.fit_transform(records)
```

Метод `retrieve` направляет запрос в подходящий метод поиска:

```
    def retrieve(self, query):
        if self.method == 'keyword':
            return self.keyword_search(query)
        elif self.method == 'vector':
            return self.vector_search(query)
        elif self.method == 'indexed':
            return self.indexed_search(query)
```

Поиск по ключевым словам находит наилучшее совпадение, подсчитывая общее число ключевых слов между запросами и документами:

```
    def keyword_search(self, query):
        best_score = 0
        best_record = None
        query_keywords = set(query.lower().split())
        for index, doc in enumerate(self.documents):
            doc_keywords = set(doc.lower().split())
            common_keywords = query_keywords.intersection(doc_keywords)
            score = len(common_keywords)
            if score > best_score:
                best_score = score
                best_record = self.documents[index]
        return best_record
```

Векторный поиск вычисляет сходство между TF-IDF запроса и матрицей документов, возвращая результат с максимальным совпадением:

```
    def vector_search(self, query):
        query_tfidf = self.vectorizer.transform([query])
        similarities = cosine_similarity(query_tfidf, self.tfidf_matrix)
        best_index = similarities.argmax()
        return db_records[best_index]
```

В поиске по индексу для быстрого извлечения наиболее подходящего документа используется ранее вычисленная матрица TF-IDF:

```
    def indexed_search(self, query):
        # Предполагается, что tfidf_matrix уже вычислена и сохранена
        query_tfidf = self.vectorizer.transform([query])
        similarities = cosine_similarity(query_tfidf, self.tfidf_matrix)
        best_index = similarities.argmax()
        return db_records[best_index]
```

Далее можно задействовать стратегии модульной RAG.

Стратегии модульной RAG. Теперь мы можем вызывать ретривер для любой конфигурации RAG в зависимости от потребности:

```
# Пример использования
# Выбор из 'keyword', 'vector' и 'indexed'
retrieval = RetrievalComponent(method='vector')
retrieval.fit(db_records)
best_matching_record = retrieval.retrieve(query)
print_formatted_response(best_matching_record)
```

В этом случае выполняется векторный поиск.

Приведенный код выбирает лучшую запись, как в подразделе «Векторный поиск» выше, дополняет ввод, вызывает генеративную модель и выводит ее ответ:

```
Response:
---------------
Certainly! Let's break down and elaborate on the content provided: ---
**Define a RAG store:** A **RAG (Retrieval-Augmented Generation) store** is a
specialized type of data storage system designed to support the retrieval and
generation of information...
```

Ранее мы создали программу, демонстрирующую возможности эффективной интеграции в систему RAG различных методов поиска — по ключевым словам, векторного и на основе индекса. Все они имеют свои достоинства и решают конкретные задачи в рамках контекста извлечения данных. Выбор одного из них будет зависеть от размера датасета, типа запроса и требований к качеству ответа, о чем мы будем говорить в последующих главах.

Ну а теперь пора подытожить открытия этой главы и переходить на следующий этап!

Резюме

Механизм RAG для генеративного ИИ опирается на два основных компонента: ретривер и генератор. Ретривер обрабатывает данные и определяет наиболее подходящий метод поиска, например выборку размеченных документов на основе ключевых слов. Полученный результат дополняет пользовательский ввод и передается в генератор, LLM, который обрабатывает дополненный запрос, выдавая итоговый ответ. Мы разобрали три основные конфигурации RAG: простую, анализирующую датасеты на основе ключевых слов и прочих методов начального уровня, продвинутую, которая вносит эмбеддинги и индексы, повышая эффективность поиска, и модульную, совмещающую простые и продвинутые техники, а также другие методы машинного обучения.

Фреймворк RAG опирается на датасеты, которые могут содержать динамические данные, тогда как модель генеративного ИИ оперирует параметрическими данными, представленными в виде весов. Эти два подхода не исключают друг друга. Если датасеты RAG становятся слишком громоздкими, можно прибегнуть к тонкой настройке. Если же тонко настроенные модели не могут гибко реагировать на ежедневно меняющуюся информацию, на выручку приходит RAG. Фреймворки RAG также сильно зависят от экосистемы, которая предоставляет для их систем критическую функциональность. Мы разобрали основные компоненты экосистемы RAG, начиная с ретривера и заканчивая генератором, которые необходимо как учить, так и оценивать. Наконец, мы реализовали элементарные примеры простой, продвинутой и модульной RAG на Python, задействовав сопоставление ключевых слов, векторный поиск и извлечение на основе индекса для дополнения ввода GPT-4o.

Следующим шагом, уже в главе 2, станет преобразование данных в векторы. Мы научимся сохранять векторы в векторных БД для повышения скорости и точности функций извлечения в экосистеме RAG.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. RAG создана для повышения точности моделей генеративного ИИ?
2. Простейшая конфигурация RAG опирается на сложные эмбеддинги данных?
3. Всегда ли тонкая настройка станет лучшим решением, нежели использование RAG?
4. RAG для повышения качества ответов применяет данные из внешних источников в реальном времени?
5. RAG можно применить только к текстовым данным?

6. Процесс извлечения данных в RAG запускается пользователем или автоматизированным вводом?
7. В продвинутых конфигурациях RAG используются и метрики косинусного сходства, и TF-IDF?
8. Экосистема RAG в качестве компонентов включает только сбор данных и генерацию?
9. Могут ли продвинутые конфигурации RAG обрабатывать мультимодальные данные, такие как изображения и аудио?
10. Обратная связь от человека при оценке RAG не имеет значения?

Ссылки

- *Lewis P., Perez E., Piktus A. et al.* Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks: https://arxiv.org/abs/2005.11401.
- *Gao Y., Xiong Y., Gao X. et al.* Retrieval-Augmented Generation for Large Language Models: A Survey: https://arxiv.org/abs/2312.10997.
- OpenAI models: https://platform.openai.com/docs/models.

Дополнительная информация

Чтобы понять, почему в сфере генеративных ИИ на базе RAG важна прозрачность, прочитайте статью https://hai.stanford.edu/news/introducing-foundation-model-transparency-index.

2 Хранение эмбеддингов RAG с помощью Deep Lake и OpenAI

https://t.me/it_boooks/2

При реализации проекта с использованием генеративного ИИ на базе RAG так или иначе в определенный момент возникают сложности, вызванные большими объемами данных. В этом случае мы прибегаем к помощи эмбеддингов, преобразуя громоздкие структурированные или неструктурированные тексты в компактные высокоразмерные векторы, отражающие семантическую суть этих текстов. Такое преобразование позволяет увеличить скорость и эффективность извлечения нужной информации. Тем не менее нам неизбежно придется решать проблемы хранения эмбеддингов, создаваемых в ходе работы с увеличивающимися наборами данных. И здесь у вас может возникнуть вопрос: «Почему бы не использовать вместо эмбеддингов ключевые слова?» Ответ прост: «Несмотря на то что эмбеддинги требуют больше места для хранения, они глубже отражают семантический смысл текстов, обеспечивая более тонкое извлечение информации с учетом контекста в сравнении с негибкими ключевыми словами, которые часто имеют множество совпадений. В итоге мы получаем более точную и соответствующую тематике информацию. Именно поэтому мы прибегаем к использованию векторных хранилищ, обеспечивающих организованное хранение эмбеддингов и быстрый доступ к ним.

Начнем эту главу с того, что научимся переносить сырые данные в хранилища Activeloop платформы Deep Lake с помощью моделей OpenAI для создания эмбеддингов. Для этого потребуется установить и реализовать несколько кросс-платформенных пакетов, в ходе чего мы познакомимся с архитектурой подобных систем. Мы разделим пайплайн RAG на отдельные компоненты, так как это позволит нескольким командам работать над проектом одновременно. После этого определим схему пайплайна генеративного ИИ на базе RAG. Наконец, создадим трехэтапный пайплайн с нуля на Python, используя Activeloop, OpenAI и кастомные функции.

На этом увлекательном пути мы узнаем о сложностях кросс-платформенных сред, связанных с применением различных пакетов и зависимостей. Кроме того, научимся решать такие задачи, как разбивка данных, создание эмбеддингов и их загрузка в векторные базы данных. Мы будем дополнять ввод для модели GPT-4o, используя запросы сопутствующей информации, и получать от нее качественный вывод. К концу главы вы научитесь полноценно сохранять векторные

представления (эмбеддинги) документов в специализированных хранилищах для генеративного ИИ.

Начнем с преобразования сырых данных в эмбеддинги, которые будут храниться в векторной БД.

От сырых данных к эмбеддингам в векторной базе данных

Эмбеддинги — это любые данные (текстовые, изображения или аудио), преобразованные в вещественные числа. То есть они подразумевают преобразование документа в вектор. Такие математические представления документов позволяют вычислять расстояние между документами и извлекать из них схожие данные.

Начинается этот процесс с того, что сырые данные (книги, статьи, посты в блогах, картинки или песни) собираются и очищаются от шумов. После подготовки они передаются в модель наподобие OpenAI `text-embedding-3-small`, которая преобразует их в векторы. К примеру, механизм Activeloop Deep Lake, который мы реализуем в этой главе, будет разбивать текст на чанки (фрагменты), соответствующие конкретному числу символов. В качестве размера чанка может использоваться различное число символов, например 1000. Можно позволить системе оптимизировать чанки. Такое разделенное представление текста упрощает обработку больших объемов данных и создание более детальных эмбеддингов документа (рис. 2.1).

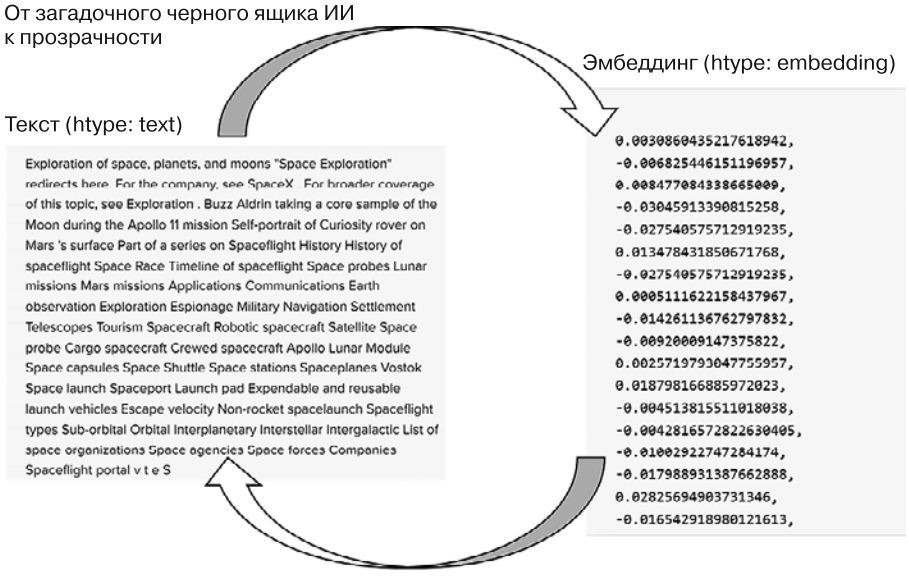


Рис. 2.1. Фрагмент из записи датасета в векторном хранилище Activeloop

С появлением параметрических моделей, в которых информация скрыта в глубинах настраиваемых обучением параметров, формирующих системы по типу «черный ящик», возник острый вопрос обеспечения прозрачности. И разработка RAG в этом смысле стала прорывом (см. рис. 2.1), поскольку информацию, которой подобные механизмы оперируют, легко отследить.

- Слева (текст): во фреймворках RAG каждый фрагмент сгенерированного содержимого можно проследить вплоть до исходных данных, что обеспечивает прозрачность итогового вывода. И генеративная модель OpenAI отвечает, применяя дополненный ввод, выполненный пользователем.
- Справа (эмбеддинги): векторные представления данных видимы и привязаны к тексту, что резко отличает их от параметрических моделей, в которых источники данных закодированы в параметры.

Следующим шагом после создания из текста эмбеддингов будет их удобное сохранение для быстрого извлечения. Для этого используются *векторные хранилища*. Векторное хранилище — это специализированная база данных, созданная для обработки высокоразмерных данных наподобие эмбеддингов. Мы можем создавать датасеты на бессерверных платформах, подобных Activeloop (рис. 2.2). Их создание и последующий доступ через API реализуем в коде в разделе «Создание пайплайна RAG» далее в этой главе.



Рис. 2.2. Управление датасетами с помощью векторных хранилищ

Еще одна особенность векторных баз данных — возможность извлекать информацию с помощью оптимизированных механизмов. Векторные хранилища создаются с использованием мощных методов индексирования, о чем мы поговорим в следующей главе. Такая особенность позволяет механизму RAG быстро находить и получать наиболее релевантные эмбеддинги на этапе генерации, дополнять запрос пользователя и обеспечивать модели больше возможностей для выдачи качественного ответа.

Далее мы узнаем, как построить пайплайн RAG, охватывающий все стадии от сбора и обработки данных до их извлечения и генерации дополненного ввода.

Организация RAG в виде пайплайна

Пайплайн RAG обычно начинается со сбора данных и подготовки, то есть очистки. В ходе этого процесса документы могут разбиваться на чанки (фрагменты), преобразовываться в векторы и сохраняться в датасете векторного хранилища. Впоследствии полученный датасет запрашивается для дополнения запроса пользователя к модели генеративного ИИ, которая дает итоговый ответ. Однако в ходе работы с векторным хранилищем не рекомендуется выполнять всю эту последовательность операций в одной программе. Стоит разделить процесс хотя бы на три компонента:

- сбор и подготовка данных;
- создание эмбеддингов и их загрузка в датасет векторной БД;
- запрос датасета векторов для дополнения запроса к модели генеративного ИИ, дающей итоговый ответ.

Вот основные причины использования подхода из нескольких компонентов.

- *Специализация* позволит каждому участнику команды заниматься тем, что он лучше всего умеет, — собирать и очищать данные, прогонять их через модели создания векторных представлений, управлять векторными базами данных или настраивать модели генеративного ИИ.
- *Масштабируемость* упрощает обновление отдельных компонентов по мере развития технологий, а также их масштабирование с помощью специализированных методов. К примеру, сохранение исходных данных можно вынести на отдельный сервер вне облачной платформы, где полученные эмбеддинги будут сохраняться в датасете векторной БД.
- *Параллельная разработка* позволяет каждой команде продвигаться с комфортной для нее скоростью, не ожидая других. Таким образом можно непрерывно вносить доработки в один компонент, не тормозя процессы в других.
- *Независимость обслуживания* позволяет одной команде работать над одним компонентом, не влияя на другие части системы. Например, если пайплайн RAG уже развернут в продакшене, пользователи смогут продолжать выполнять запросы и получать ответы от генеративного ИИ, пока команда, ответственная за сбор данных, вносит коррективы в свой компонент.
- *Повышение безопасности и конфиденциальности* за счет того, что команда каждого компонента работает отдельно, используя собственные схемы авторизации, доступа и ролей.

Как видите, в реальных продакшен-средах или масштабных проектах редко бывает так, чтобы одна программа или команда управляли всеми процессами от и до. Вот теперь можно приступать к составлению схемы пайплайна RAG, который мы создадим на Python.

Пайплайн генеративного ИИ на базе RAG

Посмотрим, как выглядит реальный пайплайн RAG. Представьте, что мы работаем в команде, которой поставили задачу создать целую систему за пару недель, и нас с ходу озадачивают кучей вопросов наподобие следующих.

- Кто будет собирать и очищать данные?
- Кто будет заниматься настройкой модели OpenAI для создания эмбеддингов?
- Кто будет писать код для создания этих эмбеддингов и управления векторной базой данных?
- Кто будет отвечать за реализацию GPT-4 и обработку ее вывода?

Не прошло и нескольких минут, как все внезапно приуныли. Подобное нагромождение задач кажется невыполнимым — ну серьезно, разве можно представить себе выполнение всего этого в одиночку?

Мы же поступим вот как. Разделимся на три группы, каждая из которых возьмет на себя реализацию отдельной части пайплайна. Схематично это показано на рис. 2.3.

Каждая из трех групп будет отвечать за один компонент (или этап).

- *Сбор и подготовка данных (D1 и D2)*. Одна команда будет собирать и очищать данные.
- *Создание эмбеддингов и их сохранение (D2 и D3)*. Другая команда будет прогонять данные через модель создания эмбеддингов и сохранять эти эмбеддинги в датасете Activeloop Deep Lake.
- *Дополненная генерация (D4, G1–G4 и E1)*. Последняя команда будет выполнять масштабную задачу по генерации контента на основе ввода пользователя и извлеченных данных. Для этого они применяют GPT-4. И хотя этот процесс может показаться очень трудоемким, в реальности он гораздо легче других, так как команде не нужно никого ждать — им нужен лишь компьютер для выполнения расчетов и оценки полученного вывода.

И вот весь этот масштабный проект уже не кажется таким уж страшным. У всех есть четко очерченные задачи, и мы можем работать, не отвлекаясь на другие команды. Это позволит нам продвигаться быстрее без лишних задержек. Такая организация проекта является вариацией механизма RAG, показанного на рис. 1.3.

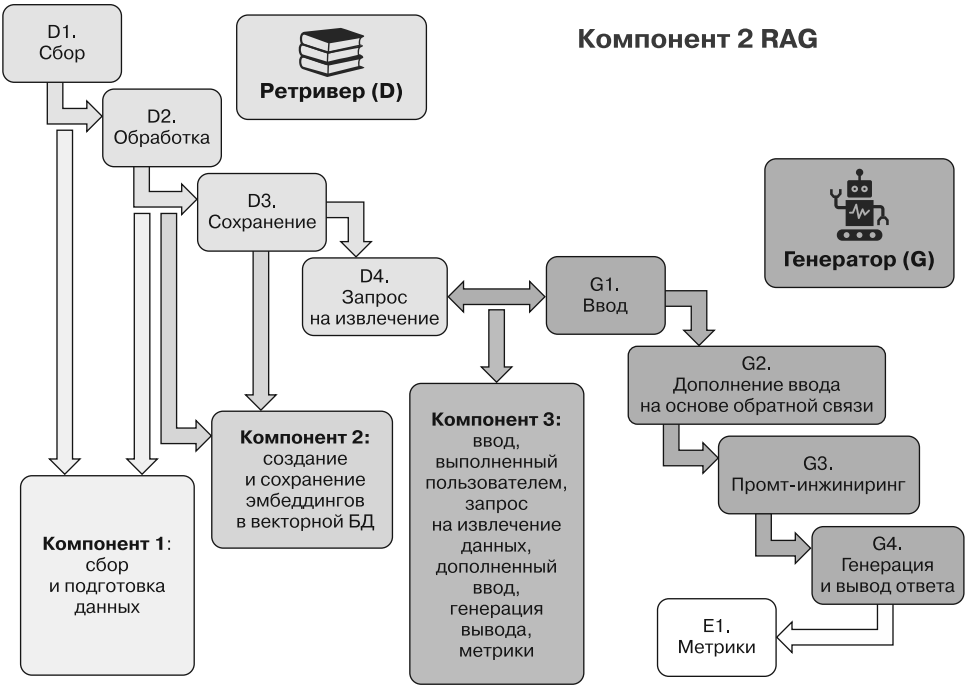


Рис. 2.3. Компоненты пайплайна RAG

Теперь можно приступать к созданию самого пайплайна RAG.

Создание пайплайна RAG

Далее мы создадим пайплайн RAG, реализовав схему из предыдущего раздела, показанную на рис. 2.3. Мы реализуем три компонента, предположив, что команды 1, 2 и 3 работают параллельно.

- Сбор и подготовка данных — команда 1.
- Преобразование данных в эмбеддинги и их сохранение — команда 2.
- Генерация на основе дополненного ввода — команда 3.

Первым этапом будет настройка среды для этих компонентов.

Настройка среды

Давайте проговорим это сразу. Устанавливать кросс-платформенные библиотеки и пакеты со всеми их зависимостями нелегко. Важно учитывать это и быть готовыми к усердной настройке корректной рабочей среды. Каждый пакет имеет свои

зависимости, которые могут иметь конфликтующие версии. И даже если мы эти версии адаптируем, приложение может перестать работать ожидаемым образом. Поэтому не пожалейте времени для установки правильных версий пакетов и их зависимостей.

Мы лишь один раз опишем рабочую среду для всех трех компонентов и будем обращаться к этому описанию по необходимости.

Установка пакетов и библиотек

Для создания пайплайна RAG нам потребуются определенные пакеты, версии которых мы зафиксируем для предотвращения потенциальных конфликтов и проблем с функциями библиотек наподобие таких:

- конфликтов между версиями зависимостей;
- конфликтов, возникающих, когда для работы приложения нужно обновить одну из библиотек. Например, в августе 2024 года для установки Deep Lake требовалась Pillow версии 10.x.x, а в Google Colab была версия 9.x.x. В результате, прежде чем ставить Deep Lake, нужно было деинсталлировать Pillow и установить ее последнюю версию. Несомненно, в Google Colab обновят Pillow, но в условиях быстро меняющегося рынка подобные случаи не редкость;
- жалоб в случае, когда какие-то версии остаются фиксированными слишком долго;
- возможных проблем, если версии остаются фиксированными слишком долго, в связи с чем обнаруженные баги не исправляются.

Получается, если мы фиксируем версии, то приложение может оставаться стабильным какое-то время, но сталкиваться с проблемами. Если же будем обновлять версии слишком быстро, некоторые из других библиотек могут перестать работать. Так что универсального решения тут нет. Здесь мы имеем процесс непрерывного контроля качества.

В этом же разделе зафиксируем версии пакетов нашей программы. А сейчас пройдемся по этапам установки, чтобы создать рабочую среду для пайплайна.

Устанавливаемые компоненты

Начнем с описания компонентов, которые устанавливаются в разделе «Настройка среды» каждого блокнота. Эти компоненты не обязательно устанавливаются во всех блокнотах, и текущий раздел послужит для их инвентаризации.

В первом компоненте пайплайна «1. Сбор и подготовка данных» нужно установить только Beautiful Soup и Requests:

```
!pip install beautifulsoup4==4.12.3
!pip install requests==2.31.0
```

Это объясняет, почему данный компонент пайплайна должен идти отдельно. Его реализация — простая задача для разработчика, которому нравится создавать интерфейсы для взаимодействия с веб-средой. Он также отлично подходит для новичка, который хочет вникнуть в тему сбора и анализа данных.

Два других компонента, которые мы создадим в этом разделе — «2. Создание эмбеддингов данных и их сохранение» и «3. Генерация на основе дополненного ввода», — потребуют больше внимания и установки `requirements01.txt`, о чем говорилось ранее. В дальнейшем мы пройдемся по всем этим этапам шаг за шагом.

Монтирование диска

В этом сценарии программа монтирует Google Drive в Google Colab для безопасного считывания ключа API OpenAI, дающего доступ к моделям OpenAI, а также токена Activeloop для аутентификации доступа к датасетам Activeloop на платформе Deep Lake:

```
# Опция Google Drive для хранения ключей API.
# Сохраните свой ключ в файле и считывайте его (можете ввести его прямо
# в блокнот, но тогда он будет виден тем, кто находится рядом с вами).
from google.colab import drive
drive.mount('/content/drive')
```

Можете сохранить ключи и токены в другом месте. Главное, чтобы оно было безопасным.

Создание подпроцесса для скачивания файлов с GitHub

Здесь мы напишем функцию для скачивания файла `grequests.py` с GitHub. Она будет использовать для скачивания команду `curl` с возможностью при необходимости добавить закрытый токен:

```
import subprocess
url = "https://raw.githubusercontent.com/Denis2054/RAG-Driven-Generative-AI/main/
➥commons/grequests.py"
output_file = "grequests.py"

# Подготовка команды curl с использованием закрытого токена
curl_command = [
    "curl",
    "-o", output_file,
    url
]

# Выполнение команды curl
try:
    subprocess.run(curl_command, check=True)
    print("Download successful.")
except subprocess.CalledProcessError:
    print("Failed to download the file.")
```

Файл `grequests.py` содержит функцию, которая при необходимости может получать закрытый токен или любой другой механизм безопасности, требующий ввода учетных данных при извлечении информации командой `curl`:

```
import subprocess
import os

# При необходимости добавление закрытого токена после имени файла
def download(directory, filename):
    # Базовый URL файлов образов в репозитории GitHub
    base_url = 'https://raw.githubusercontent.com/Denis2054/
      RAG-Driven-Generative-AI/main/'

    # Полный URL файла
    file_url = f"{base_url}{directory}/{filename}"

    # Использование curl для скачивания файла, включая заголовок
    # Authorization закрытого токена
    try:
        # Подготовка команды curl с заголовком Authorization # curl_command = f'curl
        # -H "Authorization: token {private_token}" -o {filename} {file_url}'
        curl_command = f'curl -H -o {filename} {file_url}'

        # Выполнение curl
        subprocess.run(curl_command, check=True, shell=True)
        print(f"Downloaded '{filename}' successfully.")
    except subprocess.CalledProcessError:
        print(f"Failed to download '{filename}'. Check the URL, your
              internet connection, and if the token is correct and has
              appropriate permissions.")
```

Установка необходимых компонентов

Теперь установим компоненты, необходимые для работы с Activeloop Deep Lake и OpenAI:

```
!pip install deeplake==3.9.18
!pip install openai==1.40.3
```

На момент написания книги представленная в Google Colab версия Pillow конфликтует с пакетом `deeplake`. Тем не менее установочный пакет `deeplake` обрабатывает этот конфликт автоматически. Вам лишь нужно перезапустить сеанс и повторить попытку. Именно поэтому `pip install deeplake==3.9.18` идет первой строкой во всех блокнотах, где устанавливается.

После установки этих компонентов нужно отдельно выполнить строку кода, чтобы Activeloop активировал публичный DNS-сервер:

```
# Для Google Colab и Activeloop(библиотека Deeplake)
# Этот код записывает в файл строку "nameserver 8.8.8.8", указывая,
# что DNS-сервер, который система должна использовать, находится
# по адресу 8.8.8.8; это один из публичных DNS-серверов Google
with open('/etc/resolv.conf', 'w') as file:
    file.write("nameserver 8.8.8.8")
```

Аутентификация

Вам потребуется зарегистрироваться в OpenAI для получения ключа API: https://openai.com/. Не забудьте перед его использованием выбрать тарифный план. Для начала активируем этот ключ:

```
# Извлечение и установка ключа OpenAI API
f = open("drive/MyDrive/files/api_key.txt", "r")
API_KEY=f.readline().strip()
f.close()

# Ключ OpenAI API
import os
import openai
os.environ['OPENAI_API_KEY'] =API_KEY
openai.api_key = os.getenv("OPENAI_API_KEY")
```

Затем активируем токен Activeloop API для Deep Lake:

```
# Извлечение и установка токена Activeloop API
f = open("drive/MyDrive/files/activeloop.txt", "r")
API_token=f.readline().strip()
f.close()
ACTIVELOOP_TOKEN=API_token
os.environ['ACTIVELOOP_TOKEN'] =ACTIVELOOP_TOKEN
```

Для получения этого токена зарегистрируйтесь на платформе Activeloop: https://www.activeloop.ai/. Здесь тоже нужно выбрать тарифный план, прежде чем использовать этот токен.

После установки среды можете скрыть ячейки Installing the environment (рис. 2.4), которые мы только что выполнили, чтобы сосредоточить внимание на компонентах пайплайна.

После сворачивания этих ячеек вы по-прежнему сможете их выполнять (рис. 2.5).

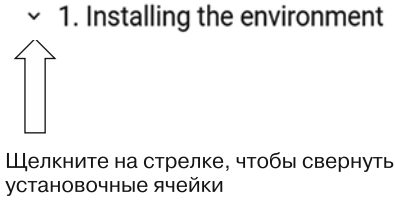


Рис. 2.4. Сворачивание установочных ячеек



Рис. 2.5. Выполнение свернутых ячеек

Теперь можно сосредоточиться на отдельных компонентах пайплайна. Начнем со сбора и подготовки данных.

Сбор и подготовка данных

Начинается пайплайн со сбора и подготовки данных. Как показано на рис. 2.6, этим займется команда 1.

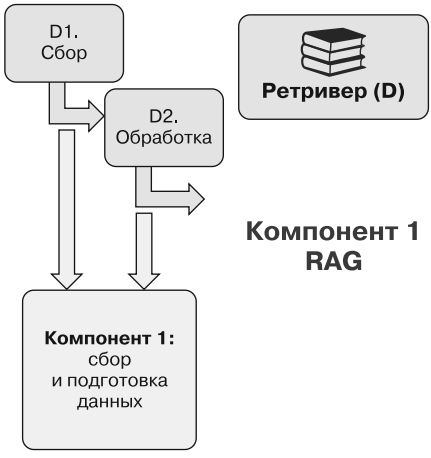


Рис. 2.6. Компонент 1 пайплайна RAG: сбор и подготовка данных

Мы же немного поможем ребятам из команды 1. Наша задача четко определена, поэтому мы можем спокойно погрузиться в реализацию компонента. Извлечем и обработаем десять статей Википедии, дающих полноценное описание различных аспектов изучения космоса.

- *Исследование космоса*. Краткая история, а также обзор технологий, миссий и планов по изучению космоса (https://en.wikipedia.org/wiki/Space_exploration).
- *Программа «Аполлон»*. Рассказ о программе NASA, в ходе которой была произведена первая высадка людей на Луну, а также ее исторически важных миссиях (https://en.wikipedia.org/wiki/Apollo_program).
- *Космический телескоп «Хаббл»*. Информация об одном из наиболее значимых телескопов за всю историю, который сыграл решающую роль в совершении многих астрономических открытий (https://en.wikipedia.org/wiki/Hubble_Space_Telescope).
- *Марсоход*. Описание марсоходов, которые отправлялись на Марс для изучения его поверхности и климата (https://en.wikipedia.org/wiki/Mars_rover).
- *Международная космическая станция (МКС)*. Подробности об МКС, международном сотрудничестве и ее роли в исследовании космоса (https://en.wikipedia.org/wiki/International_Space_Station).
- *SpaceX*. Статья об истории, достижениях и цели SpaceX — одной из наиболее влиятельных частных космических компаний (https://en.wikipedia.org/wiki/SpaceX).
- *«Джуно» (космический аппарат)*. Информация о космическом зонде NASA, который вращается по орбите Юпитера, изучая его структуру и спутники (https://en.wikipedia.org/wiki/Juno_(spacecraft)).
- *Программа «Вояджер»*. Статья, описывающая миссии «Вояджер», включая внесенный ими вклад в наше понимание внешней области Солнечной системы и межзвездного пространства (https://en.wikipedia.org/wiki/Voyager_program).
- *«Галилео» (космический аппарат)*. Обзор миссии по изучению Юпитера и его спутников, в ходе которой были собраны ценные данные об этом газовом гиганте и его планетарной системе (https://en.wikipedia.org/wiki/Galileo_(spacecraft)).

- *Космический телескоп «Кеплер»*. Информация о космическом телескопе, созданном для обнаружения экзопланет земного типа, вращающихся вокруг других звезд (https://en.wikipedia.org/wiki/Kepler_Space_Telescope).

Эти статьи охватывают обширный спектр тем по изучению космоса, начиная с программ, реализованных в прошлом, и заканчивая современными технологическими прорывами и миссиями.

Теперь откройте `1-Data_collection_preparation.ipynb` в репозитории GitHub. Начнем со сбора данных.

Сбор данных

Нам понадобятся только `import requests` для HTTP-запросов, `from bs4 import BeautifulSoup` для парсинга HTML и модуль регулярных выражений `import re`:

```
import requests
from bs4 import BeautifulSoup
import re
```

Далее выбираем нужные URL:

```
# URL-адреса статей Википедии
urls = [
    "https://en.wikipedia.org/wiki/Space_exploration",
    "https://en.wikipedia.org/wiki/Apollo_program",
    "https://en.wikipedia.org/wiki/Hubble_Space_Telescope",
    "https://en.wikipedia.org/wiki/Mars_over",
    "https://en.wikipedia.org/wiki/International_Space_Station",
    "https://en.wikipedia.org/wiki/SpaceX",
    "https://en.wikipedia.org/wiki/Juno_(spacecraft)",
    "https://en.wikipedia.org/wiki/Voyager_program",
    "https://en.wikipedia.org/wiki/Galileo_(spacecraft)",
    "https://en.wikipedia.org/wiki/Kepler_Space_Telescope"
]
```

Список статей прописан в коде, его также можно сохранить в базе данных, файле или другом формате, например JSON. Далее перейдем к подготовке собранных данных.

Подготовка данных

Для начала напишем функцию очистки. Она будет удалять из заданной строки текста ссылки вида [1] и [2], используя регулярные выражения, и возвращать очищенный текст:

```
def clean_text(content):
    # Удаление ссылок, которые обычно присутствуют в виде [1], [2] и т. д.
    content = re.sub(r'\[\d+\]', '', content)
    return content
```

Теперь напишем классическую функцию получения и очистки, которая будет возвращать красивый и чистый текст, извлекая из документов нужное нам содержимое:

```
def fetch_and_clean(url):
    # Получение содержимого по URL-адресу
    response = requests.get(url)
    soup = BeautifulSoup(response.content, 'html.parser')

    # Поиск основного содержания статьи; игнорируем дополнительные
    # информационные блоки и заголовки
    content = soup.find('div', {'class': 'mw-parser-output'})

    # Удаление раздела библиографии, который обычно имеет заголовок
    # наподобие References или Bibliography
    for section_title in ['References', 'Bibliography', 'External links',
        'See also']: section = content.find('span', id=section_title)
        if section:
            # Удаление всего содержимого, начиная с этого раздела
            # и до конца документа
            for sib in section.parent.find_next_siblings():
                sib.decompose()
            section.parent.decompose()

    # Извлечение и очистка текста
    text = content.get_text(separator=' ', strip=True)
    text = clean_text(text)
    return text
```

В итоге мы запишем очищенное содержание в файл `llm.txt` для команды, отвечающей за создание и сохранение эмбеддингов:

```
# Файл для записи очищенного текста
with open('llm.txt', 'w', encoding='utf-8') as file:
    for url in urls:
        clean_article_text = fetch_and_clean(url)
        file.write(clean_article_text + '\n')

print("Content written to llm.txt")
```

Вывод подтверждает, что текст был успешно записан:

```
Content written to llm.txt
```

Эту программу можно изменить, чтобы она сохраняла данные в других форматах и местах в соответствии с задачами конкретного проекта. Полученный файл можно проверить, прежде чем переходить к извлечению и обработке следующего пакета данных:

```
# Открываем файл и считываем первые 20 строк
with open('llm.txt', 'r', encoding='utf-8') as file:
    lines = file.readlines()
    # Выводим первые 20 строк
    for line in lines[:20]:
        print(line.strip())
```

Вывод показывает первые строки документа, которые будут обработаны:

```
Exploration of space, planets, and moons "Space Exploration" redirects here. For
the company, see SpaceX . For broader coverage of this topic, see Exploration .
Buzz Aldrin taking a core sample of the Moon during the Apollo 11 mission…
```

Реализацию этого компонента пайплайна можно поручить команде, которой нравится заниматься поиском документов в Интернете или внутри организации. В итоге ее участники наберутся опыта по определению наилучших документов для проекта, что является основой любого фреймворка RAG.

Теперь команда 2 может заняться преобразованием данных в эмбеддинги и их сохранением в хранилище.

Создание и сохранение эмбеддингов

Команда 2 отвечает за реализацию второго компонента пайплайна (рис. 2.7). Она будет получать пакеты подготовленных данных для дальнейшей обработки. Ее участникам уже не нужно озадачиваться извлечением информации, так как это сделала команда 1, собрав и подготовив всю необходимую информацию.

Теперь поможем ребятам из команды 2. Откройте `2-Embeddings_vector_store.ipynb` в GitHub. Мы будем превращать подготовленные командой 1 данные в векторные представления и сохранять их.

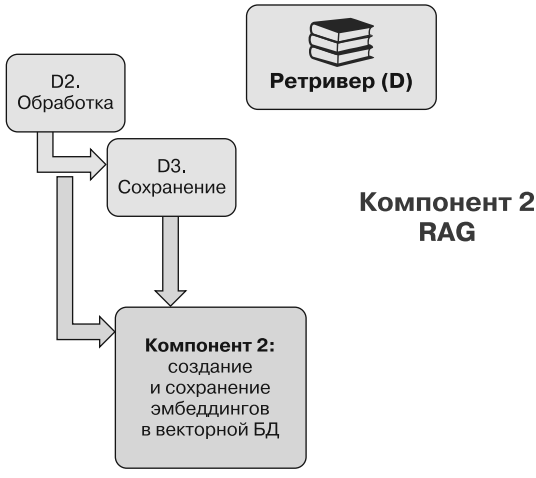


Рис. 2.7. Компонент 2 пайплайна RAG: создание эмбеддингов данных и их сохранение

Получение пакетов подготовленных документов

Для начала скачайте с сервера пакет документов, подготовленный командой 1. Он станет первым в непрерывном потоке входящих документов. В данном случае мы предполагаем, что это файл с информацией об исследовании космоса:

```
from grequests import download
source_text = "llm.txt"

directory = "Chapter02"
filename = "llm.txt"
download(directory, filename)
```

Обратите внимание на то, что `source_text = "llm.txt"` будет использоваться функцией, добавляющей данные в векторную БД. Зная, что команда 1 уже подготовила информацию, мы ограничимся краткой проверкой документа:

```
# Открываем файл и считываем первые 20 строк
with open('llm.txt', 'r', encoding='utf-8') as file:
    lines = file.readlines()
    # Выводим первые 20 строк
    for line in lines[:20]:
        print(line.strip())
```

Вывод подтверждает, что файл в порядке:

```
Exploration of space, planets, and moons "Space Exploration" redirects here.
```

Теперь разобьем данные на чанки, размер которых определим количеством символов. В нашем случае это будет `CHUNK_SIZE = 1000`, но размер можно выбирать и с помощью других техник. В главе 7 мы научимся дополнительно оптимизировать размер чанков, используя автоматизацию.

Разбивка на чанки необходима для упрощения обработки данных — выбора фрагмента текста, преобразования его в вектор и загрузки в датасет. Это также упрощает запрос информации из полученного датасета эмбеддингов. Следующий код разбивает документ, завершая процесс подготовки:

```
with open(source_text, 'r') as f:
    text = f.read()

CHUNK_SIZE = 1000
chunked_text = [text[i:i+CHUNK_SIZE] for i in range(0,len(text), CHUNK_SIZE)]
```

Теперь можно приступать к созданию собственной базы данных векторов или их добавлению в существующую.

Начало работы с векторным хранилищем

Для начала нужно определить путь к векторному хранилищу Activeloop вне зависимости от того, существует ли наш датасет:

```
vector_store_path = "hub://denis76/space_exploration_v1"
```

> **ПРИМЕЧАНИЕ**
>
> Не забудьте заменить hub://denis76/space_exploration_v1 названием вашей организации и датасета.

Далее напишем функцию, которая будет загружать векторную базу данных или автоматически создавать ее в случае отсутствия:

```
from deeplake.core.vectorstore.deeplake_vectorstore import VectorStore
import deeplake.util

try:
    # Попытка загрузки векторной базы данных
    vector_store = VectorStore(path=vector_store_path)
    print("Vector store exists")
except FileNotFoundError:
    print("Vector store does not exist. You can create it.")
    # Код для создания векторной базы данных
    create_vector_store=True
```

Вывод подтверждает создание векторной БД:

```
Your Deep Lake dataset has been successfully created!
Vector store exists
```

Теперь нужно создать функцию генерации эмбеддингов.

Функция генерации эмбеддингов

Эта функция будет преобразовывать чанки данных в векторы, позволяя в дальнейшем использовать векторный поиск. В своей программе мы применим для преобразования документов `"text-embedding-3-small"`.

У OpenAI есть и другие модели создания эмбеддингов: https://platform.openai.com/docs/models/embeddings. В подразделе «Создание эмбеддингов» главы 6 приводится альтернативный код для подобных моделей. В любом случае рекомендуется оценивать эти модели, прежде чем выбирать конкретную для продакшена. Проверьте указанные OpenAI характеристики каждой модели, обратив особое внимание на их размер и возможности. Мы выбрали `text-embedding-3-small`, так как она выделяется своей эффективностью и скоростью:

```
def embedding_function(texts, model="text-embedding-3-small"):
    if isinstance(texts, str):
        texts = [texts]
    texts = [t.replace("\n", " ") for t in texts]
    return [data.embedding for data in openai.embeddings.create(input = texts,
        model=model).data]
```

Модель для создания эмбеддингов из текста `text-embedding-3-small` обычно оперирует эмбеддингами с ограниченным числом измерений, балансируя между достаточным количеством деталей, вычислительными затратами и занимаемым местом. Перед выполнением кода обязательно ознакомьтесь с тарифным планом: https://platform.openai.com/docs/guides/embeddings/embedding-models. Теперь можно приступать к заполнению нашего векторного хранилища.

Добавление данных в векторное хранилище

Мы установили флаг добавления данных на `True`:

```
add_to_vector_store=True
if add_to_vector_store == True:
    with open(source_text, 'r') as f:
        text = f.read()
        CHUNK_SIZE = 1000
        chunked_text = [text[i:i+1000] for i in range(0, len(text), CHUNK_SIZE)]

vector_store.add(text = chunked_text,
              embedding_function = embedding_function,
              embedding_data = chunked_text,
              metadata = [{"source": source_text}]*len(chunked_text))
```

Исходный текст, `source_text = "llm.txt"`, был преобразован в эмбеддинги и сохранен. Вывод демонстрирует структуру датасета, указывая, что он был загружен:

```
Creating 839 embeddings in 2 batches of size 500:: 100%|██████████| 2/2
[01:44<00:00, 52.04s/it]
Dataset(path='hub://denis76/space_exploration_v1', tensors=['text', 'metadata',
'embedding', 'id'])

 tensor       htype         shape      dtype  compression
 -------     -------       -------    -------  -------
  text        text        (839, 1)      str     None
metadata      json        (839, 1)      str     None
embedding   embedding   (839, 1536)   float32   None
    id        text       (839, 1)       str     None
```

Заметьте, что датасет содержит четыре тензора:

- `embedding` — каждый чанк данных преобразован в вектор;
- `id` — представляет строку символов и является уникальным;
- `metadata` — содержит источник данных, в нашем случае файл `llm.txt`;
- `text` — содержимое чанка текста из датасета.

Как мы увидим в главе 4, структура датасета для каждого проекта может быть своя. Кроме того, в любой момент можно визуализировать датасет, чтобы проверить его структуру. Следующий код отобразит сводку, которую мы видели ранее:

```
# Вывод сводки по векторной базе данных
print(vector_store.summary())
```

При желании также можно визуализировать информацию хранилища.

Информация о векторной базе данных

Ссылка из Activeloop API https://docs.deeplake.ai/en/latest/ предоставит всю информацию, необходимую для управления датасетами, которые мы сможем легко визуализировать после авторизации по адресу https://app.activeloop.ai/datasets/mydatasets/.

Можно также загружать датасет с помощью всего одной строки кода:

```
ds = deeplake.load(vector_store_path)
```

В выводе будет указан путь для визуализации датасетов, их запроса и просмотра онлайн:

```
This dataset can be visualized in Jupyter Notebook by ds.visualize() or at
https://app.activeloop.ai/denis76/space_exploration_v1 hub://denis76/space_
exploration_v1 loaded successfully.
```

Кроме того, после авторизации в Activeloop вы будете иметь прямой доступ к датасетам. На рис. 2.8 показано, что вы можете использовать специальные инструменты для их просмотра, создания запросов и многого другого.

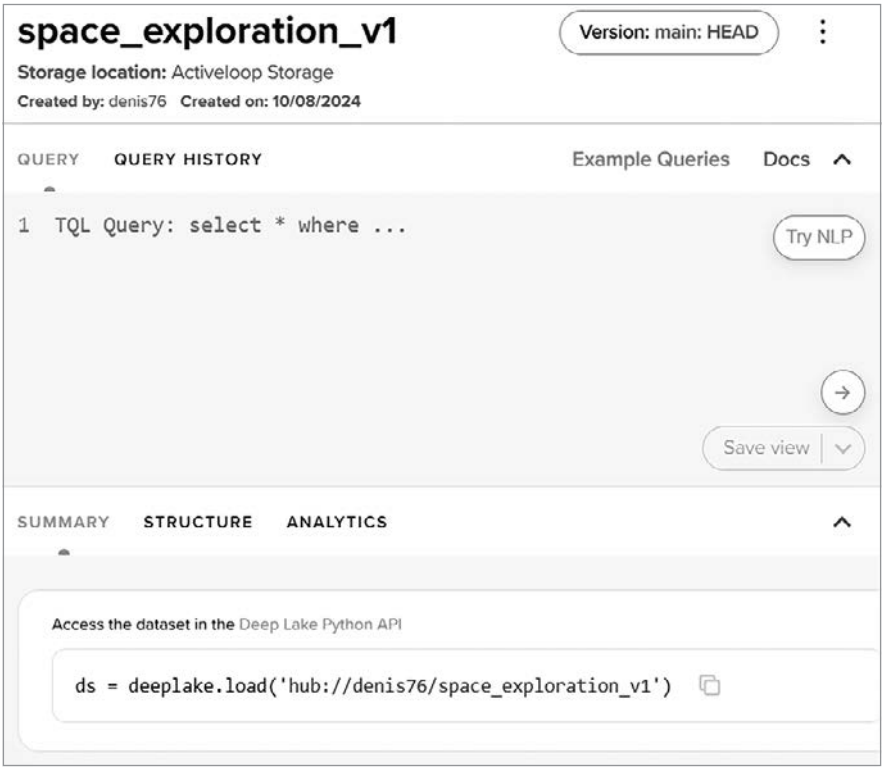


Рис. 2.8. Создание запросов и просмотр датасета на платформе Deep Lake

Среди множества доступных функций есть такая, которая выводит примерный размер датасета:

```
# Примерный размер датасета в байтах
ds_size=ds.size_approx()
```

Узнав размер, можно преобразовать его в мегабайты или гигабайты:

```
# Преобразование байтов в мегабайты и ограничение точности до пяти десятичных знаков
ds_size_mb = ds_size / 1048576
print(f"Dataset size in megabytes: {ds_size_mb:.5f} MB")

# Преобразование байтов в гигабайты и ограничение точности
# до пяти десятичных знаков
ds_size_gb = ds_size / 1073741824
print(f"Dataset size in gigabytes: {ds_size_gb:.5f} GB")
```

Вывод покажет размер датасета в мегабайтах и гигабайтах:

```
Dataset size in megabytes: 55.31311 MB
Dataset size in gigabytes: 0.05402 GB
```

Похоже, что порученный команде 2 компонент пайплайна настроен. Пора перейти к генерации дополненного ввода.

Генерация дополненного ввода

Вот мы и дошли до третьего компонента пайплайна. Здесь будем задействовать извлеченные данные для дополнения ввода пользователя. В этом компоненте происходят обработка запроса пользователя, обращение к векторному хранилищу, дополнение запроса и вызов `gpt-4-turbo`. Все это схематично показано на рис. 2.9.

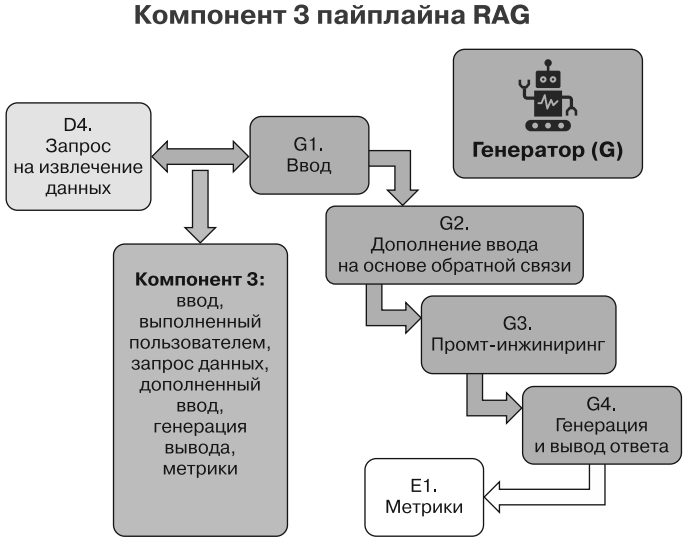


Рис. 2.9. Компонент 3 пайплайна RAG: генерация дополненного ввода

На рис. 2.9 показано, что третий компонент пайплайна по праву заслуживает быть названным генерацией с дополненной выборкой (RAG). Тем не менее

он бы оказался невозможен, если бы не предварительная работа первых двух команд, которые предоставили всю необходимую информацию для генерации дополненного ввода.

Что ж, давайте посмотрим, как свою работу выполняет команда 3. Откройте `3-Augmented_Generation.ipynb` на GitHub. Раздел «Настройка среды» (в оригинальном блокноте *Installing the environment*[1]) блокнота описан в одноименном подразделе текущей главы. Мы выбираем векторную базу данных (замените путь к ней на свой):

```
vector_store_path = "hub://denis76/space_exploration_v1"
```

Затем загружаем датасет:

```
from deeplake.core.vectorstore.deeplake_vectorstore import VectorStore
import deeplake.util
ds = deeplake.load(vector_store_path)
```

Далее выводим сообщение, подтверждающее, что векторная база данных существует. Команда 2 уже все подготовила, так что нас ничто не задерживает:

```
vector_store = VectorStore(path=vector_store_path)
```

Вывод подтверждает, что датасет существует и загружен:

```
Deep Lake Dataset in hub://denis76/space_exploration_v1 already exists, loading
from the storage
```

Предполагается, что во время работы над компонентом 2 пайплайна, который мы реализовали в подразделе «Создание и сохранение эмбеддингов», `vector_store` уже был создан и заполнен, а значит, к нему можно делать запросы. Теперь перейдем к обработке пользовательского ввода.

Ввод и выполнение запроса

Нам потребуется функция генерации эмбеддингов для превращения ввода, выполненного пользователем, в векторное представление:

```
def embedding_function(texts, model="text-embedding-3-small"):
    if isinstance(texts, str):
        texts = [texts]
    texts = [t.replace("\n", " ") for t in texts]
    return [data.embedding for data in openai.embeddings.create(input =
            texts, model=model).data]
```

Обратите внимание, что применена та же модель, что и на этапе создания эмбеддингов. Таким образом мы обеспечиваем полную совместимость между вводом пользователя и векторным датасетом: `text-embedding-ada-002`.

---

[1] В книге названия разделов блокнотов на GitHub приводятся на русском языке, чтобы было удобнее сопоставить их с разделами книги. — *Примеч. пер.*

Теперь можно либо задействовать интерактивный ввод для получения запросов, либо обрабатывать запросы пользователя пакетами. В нашем случае мы обрабатываем уже введенный запрос пользователя, который можно получить, например, из пользовательского интерфейса.

Сначала просим пользователя ввести или определить запрос:

```
def get_user_prompt():
    # Запрос ввода от пользователя
    return input("Enter your search query: ")

# Получение поискового запроса пользователя
# user_prompt = get_user_prompt()
user_prompt="Tell me about space exploration on the Moon and Mars."
```

Далее подставляем ввод, выполненный пользователем, в запрос поиска и сохраняем вывод в `search_results`:

```
search_results = vector_store.search(embedding_data=user_prompt,
  embedding_function=embedding_function)
```

Ввод, сделанный пользователем, и результаты поиска, сохраненные в `search_results`, форматируются для вывода на экран. Для начала выведем запрос пользователя:

```
print(user_prompt)
```

Зададим также нужную ширину вывода для его форматирования:

```
# Функция для вписывания текста в указанную ширину
def wrap_text(text, width=80):
    lines = []
    while len(text) > width:
        split_index = text.rfind(' ', 0, width)
        if split_index == -1:
            split_index = width
        lines.append(text[:split_index])
        text = text[split_index:].strip()
    lines.append(text)
    return '\n'.join(lines)
```

Но мы будем выбирать для вывода только верхний результат:

```
import textwrap

# Предполагается, что результаты поиска выстроены по порядку,
# начиная с наиболее соответствующего
top_score = search_results['score'][0]
top_text = search_results['text'][0].strip()
top_metadata = search_results['metadata'][0]['source']

# Вывод верхнего результата поиска
print("Top Search Result:")
print(f"Score: {top_score}")
print(f"Source: {top_metadata}")
print("Text:")
print(wrap_text(top_text))
```

Судя по выводу, найдено вполне удачное совпадение:

```
Top Search Result:
Score: 0.6016581654548645
Source: llm.txt
Text:
Exploration of space, planets, and moons "Space Exploration" redirects here. For
the company, see SpaceX . For broader coverage of this topic, see Exploration .
Buzz Aldrin taking a core sample of the Moon during the Apollo 11 mission Self-
portrait of Curiosity rover on Mars 's surface Part of a series on…
```

Теперь мы готовы к дополнению ввода полученной информацией.

Дополнение ввода

Следующий код добавляет верхний извлеченный текст к вводу, выполненному пользователем:

```
augmented_input=user_prompt+" "+top_text
print(augmented_input)
```

А вот полученный дополненный ввод:

```
Tell me about space exploration on the Moon and Mars. Exploration of space,
planets …
```

Теперь GPT-4o может обработать дополненный ввод и сгенерировать итоговый ответ:

```
from openai import OpenAI
client = OpenAI()
import time

gpt_model = "gpt-4o"
start_time = time.time()    # Начало отсчета перед отправкой запроса
```

Заметьте, что мы замеряем время выполнения операции. Далее напишем вызов к генеративному ИИ, добавив в сообщение роли:

```
def call_gpt4_with_full_text(itext):
    # Объединение всех строк в одну
    text_input = '\n'.join(itext)
    prompt = f"Please summarize or elaborate on the following content:\n{text_input}"

    try:
        response = client.chat.completions.create(
            model=gpt_model,
            messages=[
                {"role": "system", "content": "You are a space exploration expert."},
                {"role": "assistant", "content": "You can read the input
                    and answer in detail."},
                {"role": "user", "content": prompt}
            ],
```

```
            temperature=0.1    # Тонкая настройка параметров при необходимости
        )
        return response.choices[0].message.content.strip()
    except Exception as e:
        return str(e)
```

Генеративная модель вызывается с использованием дополненного ввода. При этом вычисляется время ее ответа, которое выводится вместе с ним:

```
gpt4_response = call_gpt4_with_full_text(augmented_input)

response_time = time.time() - start_time    # Измеряем время ответа
print(f"Response Time: {response_time:.2f} seconds")    # Выводим время ответа

print(gpt_model, "Response:", gpt4_response)
```

Заметьте, что необработанный вывод показан вместе с временем обработки:

```
Response Time: 8.44 seconds
gpt-4o Response: Space exploration on the Moon and Mars has been a significant
focus of human spaceflight and robotic missions. Here's a detailed summary…
```

Теперь отформатируем вывод с помощью `textwrap` и выведем результат. Здесь сначала `print_formatted_response(response)` проверяет, содержит ли ответ признаки Markdown. Если да, ответ форматируется. Если же нет, применяется стандартное форматирование вывода по указанной ширине:

```
import textwrap
import re
from IPython.display import display, Markdown, HTML
import markdown

def print_formatted_response(response):
    # Проверка присутствия Markdown: заголовков, жирного шрифта, списков и пр.
    markdown_patterns = [
        r"^#+\s",           # Заголовки
        r"^\*+",            # Пункты списка
        r"\*\*",            # Жирный
        r"_",               # Курсив
        r"\[.+\]\(.+\)",    # Ссылки
        r"-\s",             # Тире в списках
        r"\`\`\`"           # Блоки кода
    ]

    # В случае обнаружения какого-либо паттерна предполагается,
    # что ответ представлен в формате Markdown
    if any(re.search(pattern, response, re.MULTILINE)
      for pattern in markdown_patterns):
        # Markdown обнаружен, преобразуем текст в HTML
        # для более качественного отображения
        html_output = markdown.markdown(response)
        display(HTML(html_output)) # Используем display(HTML())
                                   # для отображения HTML в Colab
```

```
    else:
        # Markdown не обнаружен, просто форматируем текст по ширине и выводим
        wrapper = textwrap.TextWrapper(width=80)
        wrapped_text = wrapper.fill(text=response)

        print("Text Response:")
        print("--------------------")
        print(wrapped_text)
        print("--------------------\n")

print_formatted_response(gpt4_response)
```

Вывод получился вполне адекватный:

```
Moon Exploration
Historical Missions:
1. Apollo Missions: NASA's Apollo program, particularly Apollo 11, marked the
first manned Moon landing in 1969. Astronauts like Buzz Aldrin collected core
samples and conducted experiments.
2. Lunar Missions: Various missions have been conducted to explore the Moon,
including robotic landers and orbiters from different countries.
Scientific Goals:
3. Geological Studies: Understanding the Moon's composition, structure, and
history.
4. Resource Utilization: Investigating the potential for mining resources like
Helium-3 and water ice.
Future Plans:
1. Artemis Program: NASA's initiative to return humans to the Moon and establish
a sustainable presence by the late 2020s.
2. International Collaboration: Partnerships with other space agencies and private
companies to build lunar bases and conduct scientific research.

Mars Exploration
Robotic Missions:
1. Rovers: NASA's rovers like Curiosity and Perseverance have been exploring Mars'
surface, analyzing soil and rock samples, and searching for signs of past life.
2. Orbiters: Various orbiters have been mapping Mars' surface and studying its
atmosphere…
```

Теперь внесем метрику для оценки качества вывода.

Оценка вывода с помощью метрики косинусного сходства

В этом разделе мы реализуем метрику косинусного сходства для оценки соответствия пользовательского ввода выводу модели ИИ. Измерим также сходство дополненного ввода, сделанного пользователем, с выводом генеративного ИИ. Сначала определим функцию вычисления косинусного сходства:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
def calculate_cosine_similarity(text1, text2):
    vectorizer = TfidfVectorizer()
    tfidf = vectorizer.fit_transform([text1, text2])
    similarity = cosine_similarity(tfidf[0:1], tfidf[1:2])
    return similarity[0][0]
```

Теперь вычислим показатель сходства между запросом пользователя и ответом GPT-4:

```
similarity_score = calculate_cosine_similarity(user_prompt, gpt4_response)

print(f"Cosine Similarity Score: {similarity_score:.3f}")
```

Показатель низкий, хотя для человека вывод модели выглядел вполне приемлемым:

```
Cosine Similarity Score: 0.396
```

Похоже, либо мы что-то упустили, либо нужно задействовать другую метрику.

Для начала попробуем вычислить косинусное сходство между дополненным вводом и ответом GPT-4:

```
# Пример использования вместе с имеющимися функциями
similarity_score = calculate_cosine_similarity(augmented_input, gpt4_response)

print(f"Cosine Similarity Score: {similarity_score:.3f}")
```

Здесь показатель получше:

```
Cosine Similarity Score: 0.857
```

Можно ли применить другой метод? Метрика косинусного сходства при использовании TF-IDF сильно зависит от точных совпадений со словарем и учитывает важные языковые признаки, такие как семантическое значение, синонимы или контекст. Поэтому данный метод может давать низкие показатели сходства для текстов, семантически схожих, но различающихся выбором слов.

В противоположность этому техника вычисления сходства с помощью библиотеки Sentence Transformers подразумевает обработку эмбеддингов, более глубоко отражающих семантические связи между словами и фразами. Такой подход эффективнее распознает контекстное и тематическое сходство текстов. Попробуем применить его.

Сначала установим библиотеку `sentence-transformers`:

```
!pip install sentence-transformers
```

Будьте внимательны при установке этой библиотеки, так как она может вызвать конфликт с ранее установленными компонентами пайплайна RAG. В зависимости от целей проекта приведенный далее код может стать отдельным компонентом пайплайна.

ПРИМЕЧАНИЕ

Начиная с августа 2024 года использование токена для доступа к Hagging Face опционально. Если для Hugging Face требуется токен, зарегистрируйтесь в Hugging Face, чтобы получить токен API, ознакомьтесь с условиями и настройте ключ в соответствии с инструкциями.

На этот раз мы используем архитектуру MiniLM и выполним задачу с помощью `all-MiniLM-L6-v2`. Эта модель доступна на платформе Hugging Face Model Hub, она — часть библиотеки `sentence-transformers`, являющейся расширением Hugging Face Transformers. Мы выбрали эту архитектуру, так как она предоставляет компактную и эффективную модель с высокими качеством и скоростью генерации содержательных эмбеддингов предложений. Реализуем ее с помощью следующей функции:

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')

def calculate_cosine_similarity_with_embeddings(text1, text2):
    embeddings1 = model.encode(text1)
    embeddings2 = model.encode(text2)
    similarity = cosine_similarity([embeddings1], [embeddings2])
    return similarity[0][0]
```

Теперь можно вызвать эту функцию для вычисления сходства между дополненным вводом пользователя и ответом GPT-4:

```
similarity_score =
    calculate_cosine_similarity_with_embeddings(augmented_input, gpt4_response)
print(f"Cosine Similarity Score: {similarity_score:.3f}")
```

Более высокий показатель подтверждает, что библиотека Sentence Transformers эффективнее отражает семантическое сходство между текстами:

```
Cosine Similarity Score: 0.739
```

Выбор метрики зависит от конкретных задач каждого этапа проекта. В главе 3 мы познакомимся с продвинутыми метриками, реализовав RAG на основе поиска по индексу. Пока же мы успешно завершили создание трех компонентов пайплайна RAG. Предлагаем подвести итоги и двигаться дальше.

Резюме

В этой главе мы проработали сложные моменты использования генеративного ИИ на базе RAG, сосредоточившись на роли эмбеддингов документов при обработке больших датасетов. Мы узнали, как преобразовывать сырые

тексты в эмбеддинги и сохранять их в векторных БД. Векторные БД, такие как Activeloop, в отличие от параметрических моделей генеративного ИИ, предоставляют инструменты API и визуальные интерфейсы, которые позволяют в любой момент просмотреть исходный текст эмбеддинга.

Мы разобрали концепцию пайплайна RAG, описав процесс интеграции эмбеддингов OpenAI в векторные БД Activeloop. Мы разбили пайплайн на отдельные компоненты, которые могут различаться от проекта к проекту. Это позволяет нескольким командам работать одновременно и независимо друг от друга. Такой подход ускоряет общий процесс разработки и позволяет сосредоточиться на отдельных аспектах, таких как сбор данных, создание эмбеддингов и формирование запроса для генерации дополненного ввода.

После этого мы реализовали пайплайн RAG, начав с обозначения необходимости использования кросс-платформенных пакетов и внимательного планирования архитектуры системы. Далее написали на Python с нуля ряд функций, применили модель OpenAI для генерации эмбеддингов, задействовали векторное хранилище Activeloop для организации и хранения этих эмбеддингов в виде датасета, а также задействовали модель генеративного ИИ GPT-4o. Написанная нами программа охватила создание трех компонентов пайплайна с помощью Python, включая описание практических шагов по настройке среды, обработке зависимостей и различных нюансов реализации, таких как разбивка данных на чанки и их интеграция в векторную БД.

На этом пути вы стали хорошо понимать процесс преобразования документов в эмбеддинги, хранящиеся в векторных БД, и их использования для улучшения вывода моделей генеративного ИИ. Теперь вы готовы применить полученные знания в реальных приложениях на базе ИИ, используя грамотно выстроенный процесс и работу команд внутри организации. Векторные БД позволяют повысить качество извлечения документов, когда требуется добиться высокой точности их соответствия запросу. Вдобавок к этому индексирование документов дает возможность повысить скорость и релевантность выдачи. В следующей главе мы пойдем еще дальше и познакомимся с продвинутыми техниками извлечения информации и дополнения ввода пользователя.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. В механизме RAG текст преобразуется в эмбеддинги для ускорения извлечения информации?
2. Оказывается ли поиск по ключевым словам эффективнее использования эмбеддингов при извлечении подробного семантически схожего содержимого?
3. Рекомендуется ли разделять пайплайны RAG на отдельные компоненты?

4. Пайплайн RAG состоит из двух основных компонентов?
5. База данных Activeloop Deep Lake может обрабатывать и создание эмбеддингов, и их хранение?
6. В этой главе модель `text-embedding-3-small` от OpenAI использовалась для генерации эмбеддингов?
7. В системе на базе RAG эмбеддинги видимы и их создание можно легко проследить вплоть до исходных данных?
8. Пайплайн RAG выполняется непрерывно без разделения на отдельные компоненты?
9. Нужно ли для создания и сохранения эмбеддингов разделять большие тексты на меньшие части?
10. Метрики косинусного сходства используются для оценки релевантности извлеченной информации?

Ссылки

- Документация по эмбеддингам OpenAI Ada: https://platform.openai.com/docs/guides/embeddings/embedding-models.
- Документация OpenAI GPT по генерации контента: https://platform.openai.com/docs/models/gpt-4-turbo-and-gpt-4.
- Документация Activeloop API: https://docs.deeplake.ai/en/latest/.
- Ссылка на модель MiniLM: https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2.

Дополнительная информация

- Документация OpenAI по эмбеддингам: https://platform.openai.com/docs/guides/embeddings.
- Документация Activeloop: https://docs.activeloop.ai/.

3 Реализация индексированной RAG с помощью LlamaIndex, Deep Lake и OpenAI

Индексирование повышает точность и скорость обработки, а также преобразует механизм поисково-дополненного генеративного ИИ, привнося в него прозрачность. Это позволяет отслеживать точное местоположение и содержание данных, на основе которых был сгенерирован ответ модели RAG. Такое улучшение механизма не только уменьшает смещение и галлюцинации модели, но и позволяет исключить проблемы с авторскими правами и целостностью данных. Кроме того, это дает возможность очищать вводимые данные, подстраивать конфигурацию системы и переключать компоненты, такие как ПО векторной базы данных и генеративные модели, добиваясь более качественных результатов.

Эту главу мы начнем с оформления архитектуры пайплайна индексированной RAG, которая позволит повысить скорость и точность обработки, а также обеспечит отслеживаемость исходных данных. Здесь вы увидите, как легко интегрировать LlamaIndex, Deep Lake и OpenAI, не создавая все их функции самостоятельно. Таким образом мы заложим твердый фундамент для дальнейшего развития системы. После этого вы познакомитесь с основными видами индексирования, которые будут применяться в наших программах: индексированием с использованием векторов, дерева, списка и ключевых слов. Затем реализуем предметно-ориентированный агент LLM с применением RAG для сферы беспилотников, с которым пользователи смогут взаимодействовать. Технология беспилотных летательных аппаратов (БПЛА) уже применяется во многих областях: для обнаружения пожаров, сбора информации о дорожном трафике, съемки спортивных событий и пр. Собственно, поэтому и было решено использовать ее в нашем примере. Цель текущей главы — подготовить для LLM датасет, который в следующей главе будет расширен мультимодальными данными. Помимо этого, здесь вы познакомитесь с использованием основных методов индексирования в коде.

К концу главы вы научитесь грамотно управлять индексированной RAG с использованием векторных БД, датасетов и LLM, а также узнаете, как оптимизировать системы извлечения данных и обеспечивать их полную прозрачность. Здесь вы увидите, как наш набор инструментов, включающий в себя LlamaIndex, Deep Lake и OpenAI, не только упрощает многие технические нюансы, но и освобождает ваше драгоценное время, которое можно потратить на наработку и оттачивание аналитических навыков, необходимых для лучшего понимания генеративного ИИ на базе RAG.

В чем польза RAG с индексированием

Поиск по индексу поднимает продвинутый генеративный ИИ на базе RAG на новый уровень. Он повышает скорость извлечения при обработке больших объемов данных, превращая их неструктурированные фрагменты (чанки) в организованные индексированные узлы, которые позволяют прослеживать вывод вплоть до исходного документа и его местоположения.

Для начала разберем отличия поиска по индексу от поиска на основе сходства векторов, проанализировав архитектуру RAG, использующую индексы.

Архитектура

В RAG поиск по индексу происходит быстрее, чем по векторам, так как обеспечивает непосредственный доступ к индексированным данным, в то время как при векторном происходит последовательное сравнение эмбеддингов во всех записях. Мы реализовали механизм поиска на основе векторного сходства в главе 2, вы видите его на рис. 3.1. Тогда мы:

- собрали и подготовили данные в первом компоненте пайплайна — *сбор и подготовка данных*;
- преобразовали эти данные и сохранили их в векторной БД во втором компоненте — *эмбеддинги и векторная БД*;
- наконец, в третьем компоненте выполнили запросы на извлечение данных и запустили модель генеративного ИИ — обработали ввод, выполненный пользователем, получили данные на основе поиска схожих векторов, дополнили результатом этого поиска запрос пользователя, сгенерировали ответ и оценили его с помощью различных метрик.

Это гибкий подход, так как он позволяет реализовать каждый компонент разными способами в зависимости от задач проекта.

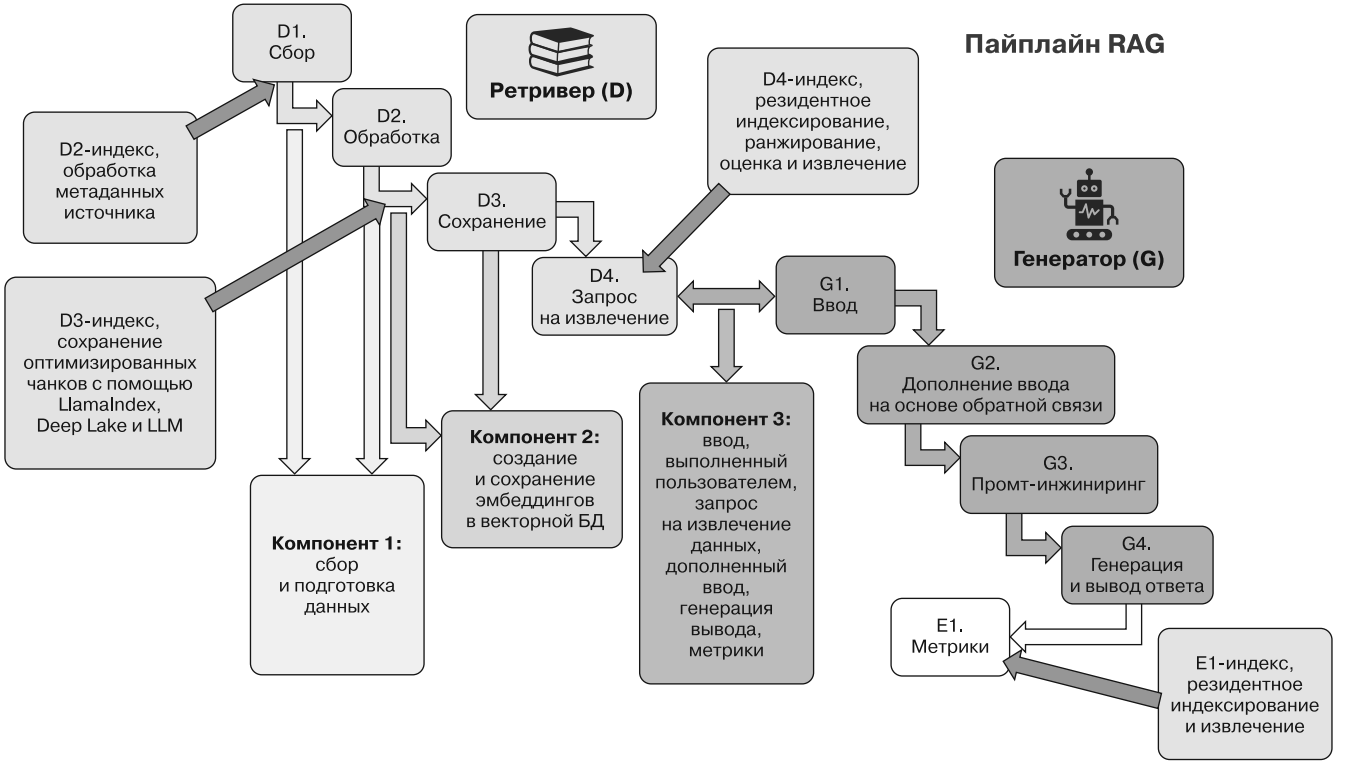


Рис. 3.1. Пайплайн генеративного ИИ на базе RAG с дополнительной функциональностью

Тем не менее поиск по индексу, который быстрее, точнее и прозрачнее, переносит нас в будущее ИИ. Далее мы повторим процесс из главы 2, используя три компонента пайплайна, чтобы максимально подготовить вас к работе в команде, применяющей разделение обязанностей. И поскольку будем работать с теми же компонентами, то просто добавим к ним дополнительные функции (см. рис. 3.1).

- *Компонент 1 и D2-индекс.* Мы, как и прежде, соберем и предварительно обработаем данные, только на сей раз будем подготавливать исходные документы по одному и сохранять их в отдельных файлах. Затем добавим их имена и адреса в метаданные, которые загрузим в векторную БД. Эти метаданные помогут отслеживать ответы модели вплоть до точного файла, который использовал ретривер. Таким образом, мы будем располагать прямой ссылкой, связывающей ответ с данными, на которых он был основан.
- *Компонент 2 и D3-индекс.* Загрузим данные в векторную БД, установив и использовав передовой интегрируемый пакет `llama-index-vector-stores-deeplake`, включающий все необходимое для оптимального начального сценария: разбивку на чанки, создание эмбеддингов, сохранение и даже интеграцию LLM. У нас есть все, чтобы реализовать индексируемую RAG всего в нескольких строках кода. Создав рабочую программу, мы сможем кастомизировать и дополнять ее компоненты, как делали в главе 2, когда явно выбирали модели LLM и размер чанков.
- *Компонент 3 и D4-индекс.* Загрузим данные в датасет, установив и задействовав пакет `llama-index-vector-stores-deeplake`, включающий все необходимое для реализации поиска по индексу и генерации, в том числе автоматическую оценку релевантности и ранжирование. Это очень простой и продуктивный процесс, в котором с помощью LlamaIndex и Deep Lake мы оптимизируем извлечение и обработку информации. Интегрированный ретривер будет эффективно извлекать релевантные данные из хранилища Deep Lake, после чего агент LLM станет грамотно синтезировать полученную информацию и взаимодействовать с ней, генерируя осмысленные ответы или действия. Индексы предназначены для быстрого извлечения, и мы реализуем несколько методов индексирования.
- *Компонент 3 и E1-индекс.* Добавим метрики оценки времени выполнения и точности ответа для оценки результатов.

В предыдущей главе был реализован механизм поиска и извлечения на основе векторов. Мы встроили документы для преобразования данных в высокоразмерные векторы, после чего произвели извлечение, вычислив расстояния между этими векторами. В текущей главе пойдем дальше и создадим векторную БД. При этом мы загрузим данные в датасет, который будет реорганизован для извлечения информации с использованием различных видов индексирования. В табл. 3.1 приводятся различия между методами поиска по индексу и векторам.

Таблица 3.1. Сравнение поиска по сходству векторов и поиска по индексу

Функциональность	Поиск и извлечение на основе сходства векторов	Поиск и извлечение с помощью индексирования на основе векторов, дерева, списка и ключевых слов
Гибкость	Высокая	Средняя (предварительно вычисленная структура)
Скорость	Замедляется с увеличением датасета	Быстрый и оптимизирован для ускоренного извлечения
Масштабируемость	Ограничен обработкой только в реальном времени	Хорошо масштабируется при больших датасетах
Сложность	Простая настройка	Более сложный и требует дополнительного этапа индексирования
Частота обновления	Легко обновляется	При обновлении данных необходимо повторять индексирование

Теперь создадим семантическую RAG на основе поиска по индексу с помощью Deep Lake, LlamaIndex и OpenAI.

Создание механизма семантического поиска и генеративного агента для сферы БПЛА

В этом разделе мы реализуем механизм семантического поиска по индексу и генеративный агент ИИ, используя векторные базы данных Deep Lake, LlamaIndex и OpenAI. Как уже говорилось, технология беспилотников активно применяется для тушения пожаров и контроля дорожного трафика. Так что задачей нашей программы будет продемонстрировать, как БПЛА с помощью техник компьютерного зрения определяют транспортные средства и другие объекты. Мы реализуем архитектуру, приведенную на рис. 3.1, которая была описана в подразделе «Архитектура» этой главы.

ПРИМЕЧАНИЕ

Откройте 2-Deep_Lake_LlamaIndex_OpenAI_indexing.ipynb в репозитории GitHub данной главы. Заголовки разделов в книге соответствуют заголовкам в блокноте, так что можно легко сопоставлять пояснения с кодом.

Начнем с установки среды, после чего реализуем три основных компонента общего пайплайна.

- *Компонент 1: сбор и подготовка данных*. Используя в качестве ресурсов GitHub и Википедию, мы соберем и очистим документы для последующего индексирования.
- *Компонент 2: создание и заполнение векторной базы данных Deep Lake*. Создадим и заполним подготовленными документами хранилище Deep Lake.
- *Компонент 3: RAG с индексированием для обработки запросов и генерирования ответов*. На этом этапе мы оценим время выполнения запроса и точность ответов, используя LLM и метрику косинусного сходства.

По возможности разбейте проект на отдельные пайплайны, чтобы команды могли работать над ними независимо и параллельно. Описанные далее этапы — лишь пример того, как это можно сделать, в зависимости от особенностей проекта можете применить и другие способы. Начнем же мы с установки необходимых компонентов среды.

Настройка среды

Здесь мы будем использовать почти такую же среду, как и в прошлой главе, поэтому сосредоточимся на установке пакетов, интегрирующих LlamaIndex, функциональность векторной БД для Deep Lake и модули OpenAI. Эта интеграция станет значительным шагом к упрощению кросс-платформенных реализаций программы:

```
!pip install llama-index-vector-stores-deeplake==0.1.6
```

Дополнительная функциональность Deep Lake:

```
!pip install deeplake==3.9.8
```

И наконец, LlamaIndex:

```
!pip install llama-index==0.10.64
```

Теперь импортируем из `llama-index` пакеты, включая векторные БД для Deep Lake:

```
from llama_index.core import VectorStoreIndex, SimpleDirectoryReader, Document
from llama_index.vector_stores.deeplake import DeepLakeVectorStore
```

На этом настройка среды закончена. Перейдем к сбору и подготовке документов.

Компонент 1: сбор и подготовка документов

В этом разделе мы соберем и подготовим тематические документы по технологии БПЛА вместе с метаданными, необходимыми для отслеживания этих документов и их источников. Наша цель — получить возможность отслеживать, из какого конкретно чанка извлеченных данных был получен ответ модели, включая

местоположение этих данных. Для начала создадим каталог данных, в который будем загружать документы:

```
!mkdir data
```

Далее с помощью `BeautifulSoup` обработаем разнородный корпус данных по БПЛА:

```
import requests
from bs4 import BeautifulSoup
import re
import os

urls = [
    "https://github.com/VisDrone/VisDrone-Dataset",
    "https://paperswithcode.com/dataset/visdrone",
    "https://openaccess.thecvf.com/content_ECCVW_2018/papers/11133/
      Zhu_VisDrone-DET2018_The_Vision_Meets_Drone_Object_Detection_in_Image_
      Challenge_ECCVW_2018_paper.pdf",
    "https://github.com/VisDrone/VisDrone2018-MOT-toolkit",
    "https://en.wikipedia.org/wiki/Object_detection",
    "https://en.wikipedia.org/wiki/Computer_vision",…
]
```

Этот корпус содержит список сайтов по темам БПЛА, компьютерного зрения и сопутствующих технологий. Но в нем также есть шумные ссылки наподобие https://keras.io/ и https://pytorch.org/, которые *не* содержат интересующей нас информации.

> **ПРИМЕЧАНИЕ**
>
> В реальных проектах не всегда есть удобная возможность работать с идеальными, предметными, структурированными и хорошо отформатированными данными. Поэтому пайплайны RAG должны быть достаточно надежными для извлечения релевантных данных даже при наличии посторонних шумов.

В этом случае мы имеем дело с неструктурированными данными в различных форматах и с разным качеством информации по теме БПЛА. Естественно, оперируя в закрытой среде, можно работать с отдельными людьми или организациями, создавая нужные документы, но в окружающем нас скоротечном цифровом мире необходимо быть готовыми к обработке любого вида данных.

Следующий код получает и очищает данные, как мы делали в главе 2:

```
def clean_text(content):
    # Удаляем ссылки и лишние символы
    content = re.sub(r'\[\d+\]', '', content)    # Удаляем ссылки
    content = re.sub(r'[^\w\s\.]', '', content)  # Удаляем знаки препинания,
                                                 # кроме точек
    return content
```

```
def fetch_and_clean(url):
    try:
        response = requests.get(url)
        response.raise_for_status()  # В случае провала запроса (например, 404)
                                     # вызываем исключение
        soup = BeautifulSoup(response.content, 'html.parser')

        # Отдаем приоритет "mw-parser-output"; в случае его отсутствия
        # используем класс "content"
        content = soup.find('div', {'class': 'mw-parser-output'})
            or soup.find('div', {'id': 'content'})
        if content is None:
            return None

        # Удаляем указанные разделы, включая вложенные
        for section_title in ['References', 'Bibliography', 'External links',
                              'See also', 'Notes']:
            section = content.find('span', id=section_title)
            while section:
                for sib in section.parent.find_next_siblings():
                    sib.decompose()
                section.parent.decompose()
                section = content.find('span', id=section_title)

        # Извлекаем и очищаем текст
        text = content.get_text(separator=' ', strip=True)
        text = clean_text(text)
        return text
    except requests.exceptions.RequestException as e:
        print(f"Error fetching content from {url}: {e}")
        return None    # При ошибке возвращаем None
```

В каждом проекте для исходных данных потребуются определенные имена и пути. Здесь мы внесем дополнительную функцию, чтобы сохранять каждый фрагмент текста с именем его источника, создавая ключевое слово на основе соответствующего URL-адреса:

```
# Каталог для хранения выходных файлов
output_dir = './data/'
os.makedirs(output_dir, exist_ok=True)

# Обработка каждого URL и запись его содержимого в отдельный файл
for url in urls:
    # Обработка расширения .html
    article_name = url.split('/')[-1].replace('.html',")
    # Создание имени файла для статьи
    filename = os.path.join(output_dir, article_name + '.txt')
    clean_article_text = fetch_and_clean(url)
    with open(filename, 'w', encoding='utf-8') as file:
        file.write(clean_article_text)

print(f"Content(ones that were possible) written to files
      in the '{output_dir}' directory.")
```

Судя по выводу, цель достигнута, хотя некоторые документы не удалось декодировать:

```
WARNING:bs4.dammit:Some characters could not be decoded, and were replaced with
REPLACEMENT CHARACTER.
Content(ones that were possible) written to files in the './data/' directory.
```

В зависимости от задач своего проекта вы можете либо попробовать разобраться в причинах неудачной обработки и обеспечить извлечение всех документов, либо оценить, достаточно ли в противном случае у вас будет данных.

Если заглянуть в `./data/`, то мы увидим, что каждая статья находится в отдельном файле (рис. 3.2).

Теперь программа загружает документы с `./data/`:

```
# Загрузка документов
documents = SimpleDirectoryReader("./data/").load_data()
```

Класс LlamaIndex `SimpleDirectoryReader` создан для работы с неструктурированными данными. Он рекурсивно сканирует каталог, определяя и загружая все поддерживаемые типы файлов, такие как `.txt`, `.pdf` и `.docx`. Затем он извлекает содержимое каждого файла и возвращает список объектов документа с текстом и метаданными, включающими имя и путь файла. Выведем первую запись этого списка словарей документов:

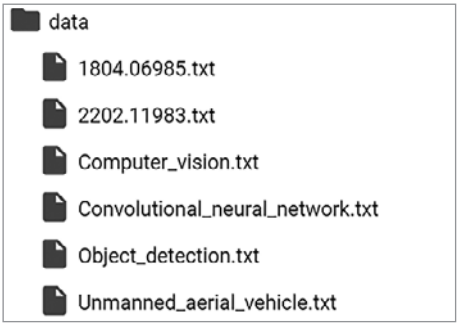


Рис. 3.2. Список подготовленных документов

```
documents[0]
```

Вывод показывает, что парсер каталога предоставил прозрачную информацию об источнике его данных, включая имя документа, в данном случае `1804.06985.txt`:

```
'/content/data/1804.06985.txt', 'file_name': '1804.06985.txt', 'file_type':
'text/plain', 'file_size': 3698, 'creation_date': '2024-05-27', 'last_modified_
date': '2024-05-27'}, excluded_embed_metadata_keys=['file_name', 'file_type',
'file_size', 'creation_date', 'last_modified_date', 'last_accessed_date'],
excluded_llm_metadata_keys=['file_name', 'file_type', 'file_size', 'creation_
date', 'last_modified_date', 'last_accessed_date'], relationships={}, text='High
Energy Physics Theory arXiv1804.06985 hepth Submitted on 19 Apr 2018 Title A
Near Horizon Extreme Binary Black Hole Geometry Authors Jacob Ciafre Maria J.
Rodriguez View a PDF of the paper titled A Near Horizon Extreme Binary Black Hole
Geometry by Jacob Ciafre and Maria J. Rodriguez View PDF Abstract A new solution
of fourdimensional vacuum General Relativity is presented…
```

Текст этого документа содержит шум, который не относится к технологии БПЛА. Но в этом и смысл данной программы, которая должна делать следующее:

- начинать с сырых, неструктурированных, слабо связанных с беспилотниками данных, которые нам доступны;
- симулировать начальную стадию реальных проектов;
- оценивать, насколько хорошо генеративный ИИ на базе индексированной RAG справляется в сложной обстановке.

Теперь создадим и заполним векторную базу данных Deep Lake, обеспечив полную прозрачность.

Компонент 2: создание и заполнение векторной базы данных Deep Lake

Здесь мы создадим векторную БД Deep Lake и заполним ее данными из наших документов. В процессе этого реализуем стандартную конфигурацию тензоров, используя следующее.

- `text (str)`. Текст — это содержимое одного из текстовых файлов, присутствующих в словаре документов. Он цельный и будет разбит на чанки с помощью оптимизированного механизма.
- `metadata(json)`. В этом случае для полной прозрачности и возможности управления метаданные будут содержать имя исходного файла каждого чанка текста. Вы увидите, как обращаться к этой информации в коде.
- `embedding (float32)`. Создание эмбеддингов происходит неразрывно с помощью модели от OpenAI, вызываемой пакетом `LlamaIndex-Deep Lake-OpenAI`.
- `id (str, auto-populated)`. Каждому чанку автоматически присваивается уникальный ID. Вдобавок к нему векторная база данных будет содержать индекс, представляющий число от 0 до *n*, но использовать его семантически нельзя, так как при каждом изменении датасета индексы тоже будут меняться. Тем не менее поле уникального ID останется неизменным, пока мы не решим оптимизировать его с помощью поиска на основе индекса, что будет продемонстрировано позже, в компоненте 3.

Первым делом программа определяет пути к векторной базе данных и датасету:

```
from llama_index.core import StorageContext

vector_store_path = "hub://denis76/drone_v2"
dataset_path = "hub://denis76/drone_v2"
```

Замените эти пути на имена своих учетной записи и датасета:

```
vector_store_path = "hub://[YOUR VECTOR STORE/
```

Далее создадим векторную базу данных, заполним ее и индексируем документы:

```
# overwrite=True означает "перезаписать документ", а False – "дополнить"
vector_store = DeepLakeVectorStore(dataset_path=dataset_path, overwrite=True)
storage_context = StorageContext.from_defaults(vector_store=vector_store)
# Индексируем документы
index = VectorStoreIndex.from_documents(documents, storage_context=storage_context)
)
```

Заметьте, что для создания векторной базы данных и перезаписи существующей `overwrite` устанавливается на `True`. Если `overwrite=False`, датасет будет просто дополнен.

Созданные индексы будут реорганизованы в ходе последующего добавления новых индексов и их переупорядочивания с помощью различных методов. Несмотря на это, в ответе всегда будет указан изначальный источник данных. Следующий вывод подтверждает успешное создание датасета и загрузку данных:

```
Your Deep Lake dataset has been successfully created!
Uploading data to deeplake dataset.
100%|██████████| 41/41 [00:02<00:00, 18.15it/s]
```

В нем также отражается структура датасета после его заполнения:

```
Dataset(path='hub://denis76/drone_v2', tensors=['text', 'metadata', 'embedding',
'id'])
```

Данные хранятся в тензорах с указанием их типа и формы (рис. 3.3).

tensor	htype	shape	dtype	compression
text	text	(81, 1)	str	None
metadata	json	(81, 1)	str	None
embedding	embedding	(81, 1536)	float32	None
id	text	(81, 1)	str	None

Рис. 3.3. Структура датасета

Теперь загрузим датасет в память:

```
import deeplake
ds = deeplake.load(dataset_path)    # Загрузка датасета
```

Щелкнув на указанной в выводе ссылке, можно визуализировать датасет в онлайн-блокноте:

```
/
This dataset can be visualized in Jupyter Notebook by ds.visualize() or at
https://app.activeloop.ai/denis76/drone_v2
```

```
hub://denis76/drone_v2 loaded successfully.
This dataset can be visualized in Jupyter Notebook by ds.visualize() or at
https://app.activeloop.ai/denis76/drone_v1
hub://denis76/drone_v2 loaded successfully.
```

Добавим также код для отображения датасета. Начнем с загрузки данных в DataFrame Pandas:

```
import json
import pandas as pd
import numpy as np

# Предполагается, что 'ds' – это ваш загруженный в Deep Lake датасет

# Создаем словарь для хранения данных
data = {}

# Перебираем тензоры датасета
for tensor_name in ds.tensors:
    tensor_data = ds[tensor_name].numpy()

    # Проверяем, является ли тензор многомерным
    if tensor_data.ndim > 1:
        # Преобразуем многомерные тензоры в одномерные
        data[tensor_name] = [np.array(e).flatten().tolist()
                             for e in tensor_data]
    else:
        # Преобразуем одномерные тензоры в списки и декодируем текст
        if tensor_name == "text":
            data[tensor_name] = [t.tobytes().decode('utf-8') if t else ""
                                 for t in tensor_data]
        else:
            data[tensor_name] = tensor_data.tolist()

# Создаем из словаря DataFrame Pandas
df = pd.DataFrame(data)
```

Далее напишем функцию для вывода записи:

```
# Функция для вывода выбранной записи
def display_record(record_number):
    record = df.iloc[record_number]
    display_data = {
        "ID": record["id"] if "id" in record else "N/A",
        "Metadata": record["metadata"] if "metadata" in record else "N/A",
        "Text": record["text"] if "text" in record else "N/A",
        "Embedding": record["embedding"] if "embedding" in record else "N/A"
    }
```

Наконец, выберем запись и выведем каждое ее поле:

```
# Вызов функции для вывода записи
rec = 0    # Замените номером нужной записи
display_record(rec)
```

id — это уникальная строка кода:

```
ID:
['a89cdb8c-3a85-42ff-9d5f-98f93f414df6']
```

Поле metadata содержит информацию, необходимую для прослеживания истории получения содержимого вплоть до его исходного файла и адреса этого файла, а также все сопутствующие данные, начиная с самого источника и заканчивая векторным представлением. Помимо этого, здесь указывается информация узла, созданного из данных записи, которую далее можно будет использовать для механизма индексирования в компоненте 3:

- file_path — путь к файлу в датасете (/content/data/1804.06985.txt);
- file_name — имя файла (`1804.06985.txt`);
- file_type — тип файла (`text/plain`);
- file_size — размер файла в байтах (`3700`);
- creation_date — дата создания файла (`2024-08-09`);
- last_modified_date — дата последнего изменения файла (`2024-08-09`);
- _node_content — подробное содержимое узла, включая следующие основные элементы:
  - id_ — уникальный идентификатор (`a89cdb8c-3a85-42ff-9d5f-98f93f414df6`);
  - embedding — полученный из текста эмбеддинг (null);
  - metadata — повтор метаданных файла;
  - excluded_embed_metadata_keys — ключи, удаленные из метаданных эмбеддинга (для эмбеддинга не нужны);
  - excluded_llm_metadata_keys — ключи, удаленные из метаданных LLM (для LLM не нужны);
  - relationships — информация о связях с другими узлами;
  - text — фактическое содержимое документа. Это может быть сам текст, его аннотация, конспект или любой иной формат для оптимизации функций поиска;
  - start_char_idx — индекс начального символа текста;
  - end_char_idx — индекс последнего символа текста;
  - text_template — шаблон для вывода текста с метаданными;
  - metadata_template — шаблон для вывода метаданных;
  - metadata_separator — разделитель, используемый при выводе метаданных;
  - class_name — тип узла (например, `TextNode`);

- `_node_type` — тип узла (`` `TextNode` ``);
- `document_id` — идентификатор документа (`` `61e7201d-0359-42b4-9a5f-32c4d67f345e` ``);
- `doc_id` — ID документа, тот же `document_id`;
- `ref_doc_id` — ID ссылочного документа, совпадает с `document_id`.

Поле `text` содержит только чанк данных, а не весь исходный текст:

```
['High Energy Physics Theory arXiv1804.06985 hepth Submitted on 19 Apr 2018
Title A Near Horizon Extreme Binary Black Hole Geometry Authors Jacob Ciafre
Maria J. Rodriguez View a PDF of the paper titled A Near Horizon Extreme Binary…
```

Поле `Embedding` содержит эмбеддинг, полученный из этого текстового содержимого:

```
[-0.0009671939187683165, 0.010151553899049759, -0.010979819111526012,
-0.003061748342588544, -0.00865076668560505, 0.02144993655383587,
-0.01412297785282135, -0.02674516849219799, -0.008693241514265537,
-0.03383851423859596, 0.011404570192098618, 0.015956487506628036,
-0.013691147789359093, 0.008856062777340412,…]
```

Структура и формат датасетов RAG в разных областях и проектах различные. Однако четыре столбца этого датасета передают ценную информацию по теме эволюции ИИ.

- `id` — индекс, который мы будем использовать для организации в датасете чанков текста из столбца `text`. Эти чанки преобразуются в *узлы*, которые могут содержать в столбце метаданных исходный текст, конспект исходного текста и дополнительную информацию, такую как источник данных, задействованный для получения вывода. Мы сформировали этот индекс в компоненте 2 текущего блокнота, когда создавали векторную базу данных. Тем не менее, как мы увидим в главе 4, можно генерировать индексы в памяти существующей базы данных, которая индексов не содержит.
- `metadata` — метаданные были автоматически сгенерированы в компоненте 1, когда `SimpleDirectoryReader` загружал исходные документы в объекты документов, а также когда создавалась векторная база данных. В главе 2 у нас был лишь один исходный файл данных. Здесь же мы сохранили данные в несколько файлов, по одному для каждого источника (URL).
- `text` — текст был автоматически разбит на чанки функцией создания векторной БД, которую мы использовали в компоненте 2, исключив необходимость настраивать размер чанков, как делали в подразделе «Получение пакетов подготовленных документов» главы 2. Напомню, что это бесшовный процесс. Далее в подразделе «Оптимизация разбивки текста» вы увидите, как работает автоматизированное разделение текста на чанки.

- `embedding` — эмбеддинг для каждого чанка данных генерировался с помощью модели векторного представления, не требующей настройки. Можно было выбрать модель эмбеддинга, как мы поступили в подразделе «Создание и сохранение эмбеддингов» главы 2. Тогда мы выбрали модель эмбеддинга и написали функцию. В этой же программе выбор модели и преобразование данных в эмбеддинги лежит на плечах Deep Lake, и нам писать код не нужно.

Как видите, теперь создание эмбеддингов, разбивка текста, индексирование и прочие функции обработки данных возлагаются на платформы и фреймворки, такие как Activeloop Deep Lake, LlamaIndex, OpenAI, LangChain, Hugging Face, Chroma и многие другие. Постепенно изначальное восхищение моделями генеративного ИИ и RAG будет угасать. Они станут более отраслевыми, четко оформленными и типичными компонентами пайплайнов ИИ. Искусственный интеллект развивается, и его можно применять в платформах для обеспечения базовой конфигурации, опирающейся на лучшие практики. В дальнейшем после реализации такой базовой конфигурации можно будет кастомизировать и расширять пайплайны в соответствии с нуждами проектов.

Теперь мы готовы к выполнению RAG на базе индексирования.

Компонент 3: RAG с индексированием

В этом разделе мы реализуем пайплайн RAG с индексированием с помощью LlamaIndex, которая задействует подготовленные нами в Deep Lake данные. Мы извлечем релевантную информацию из разнородных (с шумом) документов по теме БПЛА и синтезируем ответ с помощью моделей LLM от OpenAI. Всего мы создадим четыре механизма индексирования.

- *На основе векторной базы данных*. Создает из документов индексированное векторное хранилище, которое позволяет реализовать эффективный поиск на основе сходства векторов.
- *На основе дерева*. Строит иерархический древовидный индекс, представляющий альтернативную структуру для извлечения данных.
- *На основе списка*. Создает простой индексированный список документов.
- *На основе таблицы ключевых слов*. Выстраивает индексированную таблицу ключевых слов, извлеченных из документов.

Механизм запросов реализуем с помощью LLM как *запрос ответа и источника* — запрос индекса через ввод, выполненный пользователем, извлечение релевантных документов и возвращение синтезированного ответа вместе с информацией об использованном источнике.

В завершение будем измерять качество ответов, применяя *метрику средневзвешенной по времени величины*. Она будет вычисляться на основе показателей

качества ответа LLM и косинусного сходства. При этом ввиду стохастической природы алгоритмов время извлечения содержимого и выполнения запроса от случая к случаю может различаться.

Ввод, сделанный пользователем, и параметры запроса

В этом разделе ввод выступит опорным запросом для оценки четырех механизмов индексирования. Мы будем оценивать каждый ответ, исходя из результатов поиска по индексу, и измерять качество выдачи по соотношению затраченного времени и балла соответствия. Сами же механизмы индексирования и обработки запросов создадим чуть позже.

Начнем с ввода:

```
user_input="How do drones identify vehicles?"
```

Четыре механизма обработки запросов, реализующие LLM (в нашем случае модель OpenAI), будут поочередно вызываться с тремя одинаковыми параметрами:

```
# similarity_top_k
k=3
# temperature
temp=0.1
# num_output
mt=1024
```

Вот их описание.

- `k=3` — механизм обработки запросов должен будет найти три наиболее вероятных ответа. В этом случае `k` будет служить функцией ранжирования, требующей от LLM выбрать верхние документы из результата.
- `temp=0.1` — низкое значение температуры наподобие `0.1` настроит LLM на выдачу точных результатов. Если этот параметр увеличить, скажем, до `0.9`, ответ будет более вольным. Однако здесь мы имеем дело с технологией БПЛА, так что точность важна.
- `mt=1024` — этот параметр ограничит количество токенов выдачи до `1024`.

Далее реализуем метрику оценки косинусного сходства.

Метрика косинусного сходства

Косинусное сходство описывалось в разделе «Оценка вывода с помощью метрики косинусного сходства» главы 2. Если подзабыли, перечитайте его. Здесь же мы создадим функцию для оценки ответов:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
```

```
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')

def calculate_cosine_similarity_with_embeddings(text1, text2):
    embeddings1 = model.encode(text1)
    embeddings2 = model.encode(text2)
    similarity = cosine_similarity([embeddings1], [embeddings2])
    return similarity[0][0]
```

В этой функции используются библиотеки `sklearn` и `SentenceTransformer` от Hugging Face. Сначала программа создает механизм обработки запроса к векторной базе данных.

Механизм индексирования на основе векторов

`VectorStoreIndex` — это вид индекса в LlamaIndex, который для представления информации и ее извлечения из документов использует эмбеддинги. Эмбеддинги документов, имеющих высокое смысловое сходство, расположены близко друг к другу в векторном пространстве, о чем мы говорили в предыдущей главе. Но на сей раз `VectorStoreIndex` не будет автоматически использовать существующую векторную базу данных Deep Lake. Этот механизм станет создавать новый векторный индекс в памяти, заново преобразовывать документы в векторы и генерировать обновленную структуру индексов. В главе 4 мы разовьем этот подход, реализуя датасет без индексов или эмбеддингов.

> **ПРИМЕЧАНИЕ**
>
> Нет универсального ответа на вопрос, какой метод индексирования больше подойдет для вашего проекта. Лучший способ выбрать — это протестировать представленные в текущей главе механизмы индексирования на основе векторов, дерева, списка и ключевых слов.

Сначала создадим индекс векторной БД:

```
from llama_index.core import VectorStoreIndex
vector_store_index = VectorStoreIndex.from_documents(documents)
```

Затем отобразим его:

```
print(type(vector_store_index))
```

Полученный вывод подтверждает, что механизм индексирования создан:

```
<class 'llama_index.core.indices.vector_store.base.VectorStoreIndex'>
```

Теперь нам нужен механизм обработки запросов для извлечения документов и их синтеза с помощью LLM — в нашем случае модели OpenAI (устанавливается командой `!pip install llama-index-vector-stores-deeplake==0.1.2`):

```
vector_query_engine = vector_store_index.as_query_engine(similarity_top_k=k,
temperature=temp, num_output=mt)
```

Параметры этого механизма мы определили в подразделе «Ввод, сделанный пользователем, и параметры запроса». Теперь можно сделать запрос к датасету и сгенерировать ответ.

Ответ на запрос и его источник

Определим функцию, которая будет обрабатывать запрос и возвращать информацию о содержимом ответа:

```
import pandas as pd
import textwrap

def index_query(input_query):
    response = vector_query_engine.query(input_query)

    # Не обязательно: вывод форматированного представления ответа
    # (удалите, если вам в выводе это не нужно)
    print(textwrap.fill(str(response), 100))

    node_data = []
    for node_with_score in response.source_nodes:
        node = node_with_score.node
        node_info = {
            'Node ID': node.id_,
            'Score': node_with_score.score,
            'Text': node.text
        }
        node_data.append(node_info)

    df = pd.DataFrame(node_data)

    # Вместо вывода вернуть DataFrame и объект ответа
    return df, response,
```

`index_query(input_query)` выполняет запрос, используя обработку векторов, и выдает результат в структурированной форме. Эта функция получает запрос и извлекает релевантную информацию с помощью механизма обработки запросов в виде DataFrame: `Node ID`, `Score`, `File Path`, `Filename` и `Text`.

Теперь вызовем запрос:

```
import time
# Запуск отсчета
start_time = time.time()
```

```
df, response = index_query(user_input)
# Остановка отсчета
end_time = time.time()
# Вычисление и вывод времени выполнения
elapsed_time = end_time - start_time
print(f"Query execution time: {elapsed_time:.4f} seconds")

# Вывод DataFrame в формате Markdown
print(df.to_markdown(index=False, numalign="left", stralign="left"))
```

Мы будем измерять время, необходимое механизму для извлечения релевантных данных, и генерировать синтезированный ответ с помощью LLM (в данном случае модели OpenAI). Сначала в выводе семантического поиска отображается ответ LLM:

```
Drones can automatically identify vehicles across different cameras with
different viewpoints and hardware specifications using reidentification methods.
```

Затем идет время выполнения запроса:

```
Query execution time: 0.8831 seconds
```

И далее информация узла. Здесь выводится показатель релевантности каждого узла, соответствующего одному из `k=3` документов, вместе с фрагментами их текста (рис. 3.4).

```
| Node ID                              | Score    | Text
|:-------------------------------------|:---------|:----------------------------------------
| 4befdb13-305d-42db-a616-5d9932c17ac8 | 0.833274 | ['These activities can be carried out with
| bdefbbba-1b1a-4aea-812c-a2f6acb4dfb9 | 0.828414 | ['Degree of autonomy  edit  Drones could
| 381255c4-3bdb-455a-be73-d99cc6b46537 | 0.826526 | ['UAVs with generally nonlethal payloads
```

Рис. 3.4. Вывод информации узла

ID узла гарантирует полную прозрачность, и по нему можно проследить исходный документ даже после реорганизации индексов датасета механизмом индексирования. К примеру, узнать источник первого узла можно так:

```
nodeid=response.source_nodes[0].node_id
nodeid
```

В выводе мы увидим ID этого узла:

```
4befdb13-305d-42db-a616-5d9932c17ac8
```

Можно углубиться и извлечь весь текст содержащего документ узла, который был синтезирован LLM:

```
response.source_nodes[0].get_text()
```

Вот этот текст:

```
['These activities can be carried out with different approaches that include
photogrammetry SfM thermography multispectral images 3D field scanning NDVI
maps etc. Agriculture forestry and environmental studies edit Main article
Agricultural drone As global demand for food production grows exponentially
resources are depleted farmland is…
```

Есть также возможность заглядывать в узлы и извлекать размер их чанков.

Оптимизация разбивки текста

Мы можем устанавливать размер чанков заранее или поручать его выбор Llama-Index. В данном случае код определяет этот размер автоматически:

```
for node_with_score in response.source_nodes:
    node = node_with_score.node   # Извлекаем объект Node из NodeWithScore

    chunk_size = len(node.text)
    print(f"Node ID: {node.id_}, Chunk Size: {chunk_size} characters")
```

Преимущество автоматического определения размера в том, что он может меняться. Например, показанный здесь размер выходных узлов находится где-то в диапазоне от 1800 до 4500 символов:

```
Node ID: 83a135c6-dddd-402e-9423-d282e6524160, Chunk Size: 4417 characters
Node ID: 7b7b55fe-0354-45bc-98da-0a715ceaaab0, Chunk Size: 1806 characters
Node ID: 18528a16-ce77-46a9-bbc6-5e8f05418d95, Chunk Size: 3258 characters
```

Функция определения размера чанка не обрезает содержимое линейно, а оптимизирует его для семантического поиска.

Оценка эффективности

Мы также реализуем механизм оценки эффективности на основе точности соответствия запросу и затраченного времени. Соответствующая функция будет вычислять и выводить показатель релевантности запроса вместе с временем его выполнения. Общая эффективность будет оцениваться как результат деления взвешенного среднего показателя релевантности извлеченной информации на время выполнения запроса. Чем выше этот результат, тем лучше.

Сначала вычислим сумму показателей и их среднее значение, которое затем разделим на время выполнения запроса:

```
import numpy as np

def info_metrics(response):
  # Вычисляем производительность (обработка показателей None)
```

```
scores = [node.score for node in response.source_nodes if node.score is not None]
if scores:    # Проверяем наличие валидных показателей
    weights = np.exp(scores) / np.sum(np.exp(scores))
    perf = np.average(scores, weights=weights) / elapsed_time
else:
    perf = 0  # Или других предустановленных значений, если все показатели None
```

В результате получим соотношение взвешенного среднего показателя и прошедшего времени:

```
perf = np.average(scores, weights=weights) / elapsed_time
```

Теперь можно вызвать написанную функцию:

```
info_metrics(response)
```

Вывод покажет примерную оценку качества ответа:

```
Average score: 0.8374
Query execution time: 1.3266 seconds
Performance metric: 0.6312
```

Полученный показатель эффективности не является абсолютным значением. Это лишь индикатор, который можно использовать для сравнения конкретного вывода с результатом применения других механизмов индексирования. Причем ввиду стохастической природы алгоритмов ML он может различаться от случая к случаю. Кроме того, качество вывода зависит также от субъективного восприятия пользователя. В любом случае эта метрика поможет нам сравнить эффективность механизмов обработки запросов в текущей главе.

Ранее мы получили вполне удовлетворительный показатель, несмотря на то что загрузили в датасет разнородные данные, включая не связанные с темой документы. Интегрированная функциональность LlamaIndex, Deep Lake и OpenAI, объединяющая механизм извлечения и генерации ответа, оказалась довольно эффективной.

Механизм индексирования на основе дерева

Такой механизм LlamaIndex создает иерархическую структуру для эффективного запроса текстовых документов и управления ими. Тем не менее здесь мы имеем дело не с классической иерархической структурой. Древовидный индекс оптимизирует эту структуру, ее содержимое и порядок узлов (рис. 3.5).

В древовидном индексе документы организуются в виде дерева, где на верхних уровнях находятся узлы с общей информацией, а на нижних — с подробностями. Каждый узел описывает текст, с которым он связан. Такой вид индексирования эффективен для больших датасетов и частых запросов к объемным коллекциям

документов, так как разбивает их на удобные оптимизированные чанки. Подобная древовидная оптимизация позволяет быстро извлекать информацию, обходя релевантные узлы без лишней траты времени.

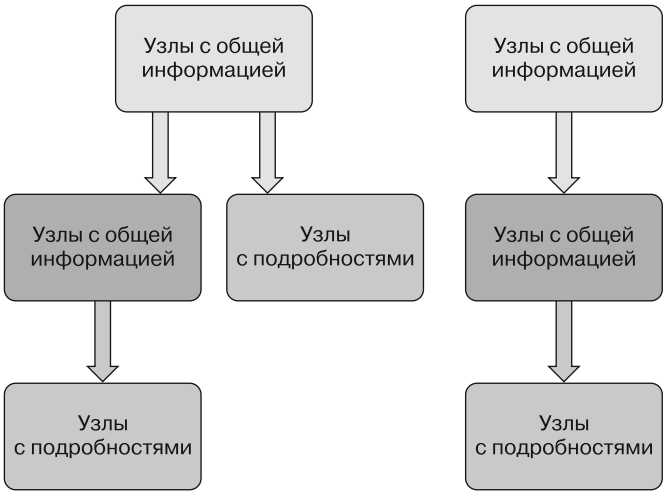


Рис. 3.5. Оптимизированный древовидный индекс

Организацию этой части пайплайна и корректировку параметров, таких как глубина дерева и методы обобщения информации о тексте, можно поручать отдельному члену команды. В зависимости от проекта и рабочей нагрузки этот процесс может быть включен в компонент 2 пайплайна, когда мы создаем и заполняем векторную базу данных. Или же дерево индексов можно создавать в памяти в начале каждого сеанса. Реализация подобных гибких древовидных структур и механизмов индексирования в целом станет интересным и полезным направлением деятельности команды, создающей генеративный ИИ на базе RAG.

В этой модели индексирования LLM (в нашем случае OpenAI) действует так, будто при выборе наиболее подходящих узлов отвечает на вопрос с несколькими вариантами ответа. Она анализирует запрос, сравнивает его с обобщенной информацией потомков текущего узла и решает, по какому пути идти, чтобы найти наиболее релевантные данные.

Теперь модели LLM можно использовать для генерации эмбеддингов, ранжирования документов и создания диалоговых агентов. На рынке сегодня доступны всевозможные языковые модели от таких поставщиков, как OpenAI, Cohere, AI21 Labs и Hugging Face. С момента своего зарождения LLM прошли путь развития от этапа, когда их воспринимали как магию, до современности, когда они стали повсеместными, взаимодополняющими, многофункциональными и интегрированными компонентами более масштабных пайплайнов ИИ.

Теперь буквально в двух строках кода создадим древовидный индекс:

```
from llama_index.core import TreeIndex
tree_index = TreeIndex.from_documents(documents)
```

Далее проверим только что созданный класс:

```
print(type(tree_index))
```

Вывод подтверждает, что мы находимся в классе `TreeIndex`:

```
<class 'llama_index.core.indices.tree.base.TreeIndex'>
```

Теперь можно создать механизм обработки запросов:

```
tree_query_engine = tree_index.as_query_engine(similarity_top_k=k,
temperature=temp, num_output=mt)
```

Параметры LLM мы определили в подразделе «Ввод, сделанный пользователем, и параметры запроса» выше. Следующий код активирует запрос, измеряет время его выполнения и обрабатывает ответ:

```
import time
import textwrap
# Начало отсчета
start_time = time.time()
response = tree_query_engine.query(user_input)
# Конец отсчета
end_time = time.time()
# Вычисление и вывод времени выполнения
elapsed_time = end_time - start_time
print(f"Query execution time: {elapsed_time:.4f} seconds")

print(textwrap.fill(str(response), 100))
```

И время выполнения запроса, и ответ получились удовлетворительными:

```
Query execution time: 4.3360 seconds
Drones identify vehicles using computer vision technology related to object
detection. This technology involves detecting instances of semantic objects of
a certain class, such as vehicles, in digital images and videos. Drones can be
equipped with object detection algorithms, such as YOLOv3 models trained on
datasets like COCO, to detect vehicles in real-time by analyzing the visual data
captured by the drone's cameras.
```

Теперь оценим результат с помощью метрики эффективности.

Оценка эффективности

Для получения этой метрики мы вычислим показатель косинусного сходства, определенный в подразделе «Метрика косинусного сходства» ранее, между вводом, выполненным пользователем, и ответом пайплайна RAG:

```
similarity_score = calculate_cosine_similarity_with_embeddings(user_input,
  str(response))
print(f"Cosine Similarity Score: {similarity_score:.3f}")
print(f"Query execution time: {elapsed_time:.4f} seconds")
performance=similarity_score/elapsed_time
print(f"Performance metric: {performance:.4f}")
```

Вывод показывает, что, несмотря на удовлетворительное качество ответа, выполнялся запрос долго, в связи с чем общий балл эффективности получился низким:

```
Cosine Similarity Score: 0.731
Query execution time: 4.3360 seconds
Performance metric: 0.1686
```

Естественно, время выполнения зависит от сервера (его мощности) и данных (их шумности). Как уже неоднократно говорилось, ввиду стохастичности используемых алгоритмов время выполнения может различаться от случая к случаю. Кроме того, увеличение объема датасета будет влиять на время выполнения всех видов механизмов индексирования.

Механизм запросов на основе древовидного индекса в этом случае может как выиграть, так и проиграть. Чтобы это выяснить, его нужно реализовать.

Механизм индексирования на основе списка

Не стоит думать, что `ListIndex` — это просто список узлов. Данный механизм обрабатывает ввод, выполненный пользователем, и каждый документ в виде промта для LLM. Далее LLM оценивает семантическое сходство между документами и запросом, что упрощает ранжирование и выбор наиболее релевантных узлов. После этого LlamaIndex отфильтровывает документы на основе присвоенных LLM показателей сходства и может выполнять следующий этап задачи, синтезируя информацию из нескольких узлов и документов.

Очевидно, что процесс выбора документов с помощью LLM не опирается ни на какие правила. Здесь ничто не предопределено, и отбор происходит на основе промта, когда запрос пользователя сопоставляется с коллекцией документов. LLM оценивает каждый документ списка *независимо*, присваивая показатель сходства на основе предположительной релевантности запросу. Данный показатель не имеет отношения к другим документам. Это лишь способ измерить, насколько с точки зрения LLM текущий документ отвечает на заданный вопрос. После этого по нашему желанию извлекаются `k` верхних документов, как в описанной далее функции.

Аналогично древовидному индексу списковый индекс создается всего двумя строками кода:

```
from llama_index.core import ListIndex
list_index = ListIndex.from_documents(documents)
```

Проверим созданный класс:

```
print(type(list_index))
```

Вывод подтверждает, что мы находимся в классе `list`:

```
<class 'llama_index.core.indices.list.base.SummaryIndex'>
```

Списковый индекс — это `SummaryIndex`, который показывает, что скрыто выполняется серьезная оптимизация, а именно создаются сводки по документам. Теперь с помощью фреймворка LlamaIndex можно использовать полученный индекс в качестве механизма запросов:

```
list_query_engine = list_index.as_query_engine(similarity_top_k=k,
  temperature=temp, num_output=mt)
```

Параметры LLM остаются неизменными, чтобы можно было объективно сравнить разные методы индексирования. Далее выполним запрос, сформулируем ответ на него и выведем результат:

```
# Начало отсчета
start_time = time.time()
response = list_query_engine.query(user_input)
# Конец отсчета
end_time = time.time()
# Вычисление и вывод времени выполнения
elapsed_time = end_time - start_time
print(f"Query execution time: {elapsed_time:.4f} seconds")

print(textwrap.fill(str(response), 100))
```

В выводе мы видим более длительное время выполнения, но вполне приемлемый ответ:

```
Query execution time: 16.3123 seconds
Drones can identify vehicles through computer vision systems that process image
data captured by cameras mounted on the drones. These systems use techniques
like object recognition and detection to analyze the images and identify
specific objects, such as vehicles, based on predefined models or features.
By processing the visual data in real-time, drones can effectively identify
vehicles in their surroundings.
```

Время выполнения увеличилось, так как механизм обработки запроса в этом случае перебирает список, а не оптимизированное дерево. Тем не менее выводы

из этого делать не стоит, так как каждый проект или даже его задача опирается на свои требования. Теперь перейдем к оценке.

Оценка эффективности

Как и в случае древовидного индекса, здесь мы используем метрику косинусного сходства:

```
similarity_score = calculate_cosine_similarity_with_embeddings(user_input,
  str(response))
print(f"Cosine Similarity Score: {similarity_score:.3f}")
print(f"Query execution time: {elapsed_time:.4f} seconds")
performance=similarity_score/elapsed_time
print(f"Performance metric: {performance:.4f}")
```

Из-за длительного времени выполнения общий показатель эффективности получился меньше, чем у древовидного индекса.

```
Cosine Similarity Score: 0.775
Query execution time: 16.3123 seconds
Performance metric: 0.0475
```

И снова не забываем, что в связи со стохастической природой используемых алгоритмов время выполнения может варьироваться.

Если оглянуться на показатель эффективности каждого метода индексирования, то мы заметим, что пока самым быстрым оказался векторный индекс. Но с выводами опять же спешить не стоит. Для каждого проекта могут получаться свои, порой неожиданные результаты, что будет зависеть от типа и сложности обрабатываемых данных. Далее мы реализуем индексирование по ключевым словам.

Механизм индексирования на основе таблицы ключевых слов

`KeywordTableIndex` — это один из видов индексирования в LlamaIndex. Он основан на извлечении из документов ключевых слов и их организации в табличную структуру. Эта структура упрощает выполнение запроса и нахождение релевантной информации на основе конкретных слов или тем. Опять же не стоит рассматривать эту функцию как простой список извлеченных ключевых слов. Полученные ключевые слова организуются в таблицу, где каждое из них связывается с ID, указывающим на соответствующие узлы.

Этот индекс мы также создадим двумя строками кода:

```
from llama_index.core import KeywordTableIndex
keyword_index = KeywordTableIndex.from_documents(documents)
```

Далее извлечем данные и создадим DataFrame, чтобы увидеть структуру индекса:

```
# Извлечение данных для DataFrame
data = []
for keyword, doc_ids in keyword_index.index_struct.table.items():
    for doc_id in doc_ids:
        data.append({"Keyword": keyword, "Document ID": doc_id})

# Создание DataFrame
df = pd.DataFrame(data)
df
```

Вывод показывает, что каждое ключевое слово связано с ID, который в зависимости от выбранной LlamaIndex оптимизации индекса содержит документ или его сводную информацию (рис. 3.6).

	Ключевое слово	ID документа
0	black	48696b4b-978e-46b1-a5b3-f451d5265f42
1	asymptotically flat	48696b4b-978e-46b1-a5b3-f451d5265f42
2	high energy physics	48696b4b-978e-46b1-a5b3-f451d5265f42
3	entropy	48696b4b-978e-46b1-a5b3-f451d5265f42
4	extreme binary black hole geometry	48696b4b-978e-46b1-a5b3-f451d5265f42
...	...	...

Рис. 3.6. Ключевые слова, связанные с ID документов в DataFrame

Теперь определим созданный индекс ключевых слов в виде механизма обработки запросов:

```
keyword_query_engine = keyword_index.as_query_engine(similarity_top_k=k,
  temperature=temp, num_output=mt)
```

Выполним его, чтобы оценить, насколько качественный ответ он даст и как быстро:

```
import time

# Начало отсчета
start_time = time.time()

# Выполнение запроса (с помощью метода .query())
response = keyword_query_engine.query(user_input)
```

```
# Конец отсчета
end_time = time.time()

# Вычисление и вывод времени выполнения
elapsed_time = end_time - start_time
print(f"Query execution time: {elapsed_time:.4f} seconds")

print(textwrap.fill(str(response), 100))
```

Результат удовлетворительный, как и время выполнения запроса:

```
Query execution time: 2.4282 seconds
Drones can identify vehicles through various means such as visual recognition
using onboard cameras, sensors, and image processing algorithms. They can
also utilize technologies like artificial intelligence and machine learning
to analyze and classify vehicles based on their shapes, sizes, and movement
patterns. Additionally, drones can be equipped with specialized software for
object detection and tracking to identify vehicles accurately.
```

Теперь измерим его общую эффективность.

Оценка эффективности

Приведенный далее код измеряет тот же показатель, что и в предыдущих двух случаях — с древовидным и списковым индексами:

```
similarity_score = calculate_cosine_similarity_with_embeddings(user_input,
  str(response))
print(f"Cosine Similarity Score: {similarity_score:.3f}")
print(f"Query execution time: {elapsed_time:.4f} seconds")
performance=similarity_score/elapsed_time
print(f"Performance metric: {performance:.4f}")
```

Показатель эффективности получился приемлемым:

```
Cosine Similarity Score: 0.801
Query execution time: 2.4282 seconds
Performance metric: 0.3299
```

Но и здесь с выводами спешить не стоит. Все виды индексирования показали удовлетворительные результаты. Однако каждый проект будет отличаться собственной сложностью датасета и вычислительными возможностями. Кроме того, время выполнения в связи с изменчивой природой используемых алгоритмов может различаться от случая к случаю.

Вот мы и познакомились с некоторыми основными методами индексирования и извлечения данных. Пора подытожить пройденную главу и переходить к стратегиям мультимодального извлечения данных и генерации ответа.

Резюме

В этой главе мы познакомились с принципами работы RAG с поиском по индексу и внесли в эту систему кардинальное улучшение — *возможность полного отслеживания*. Документы преобразуются в узлы, которые содержат чанки данных, позволяющих проследить источник запроса вплоть до исходных данных. Индексы также повышают скорость извлечения, что очень важно при обработке больших датасетов. Еще одним значительным улучшением стала интеграция технологий LlamaIndex, Deep Lake и OpenAI, которая переносит нас в следующую эпоху ИИ. Самые продвинутые модели ИИ, такие как GPT-4o от OpenAI, Hugging Face и Cohere, становятся неотъемлемыми компонентами пайплайна генеративного ИИ на базе RAG, как видеокарта в компьютере.

Мы начали с описания архитектуры пайплайна генеративного ИИ на базе индексированной RAG и на практике увидели, как эти замысловатые технологии можно совместить для создания продвинутых систем индексирования и извлечения данных. Такая сложная схема ИИ ведет к тому, что при организации их разработки для создания каждого компонента пайплайна мы назначаем отдельные команды. Это позволяет им работать параллельно, реализуя масштабируемые проекты, оперирующие большими объемами данных. Мы также увидели, что каждый сгенерированный ответ можно проследить обратно до его источника. Такая прозрачность позволяет дополнительно убеждаться в точности соответствия извлеченной информации запросу. Мы познакомились с продвинутой системой RAG, реализовав ее на примере технологии БПЛА.

В главе мы изучили важные инструменты и методы, необходимые для создания этих систем, включая векторные базы данных, датасеты, разбивку текста, создание эмбеддингов, генерацию узлов, ранжирование и индексирование. Реализовали фреймворк LlamaIndex, векторные базы данных Deep Lake и модели OpenAI. Кроме того, мы написали на Python программу, которая собирает информацию и добавляет к ней важные метаданные, позволяющие определить источник каждого чанка данных в датасете. Мы подчеркнули решающую роль индексирования (векторного, на основе списка и ключевых слов) в повышении контролируемости приложений генеративного ИИ с помощью точных корректировок и улучшений.

После этого мы разобрали RAG с индексированием, тщательно изучив сопутствующие блокноты кода Python, которые познакомили нас с настройкой векторных баз данных, выполнением продвинутых запросов и обеспечением прозрачности генерируемых ИИ ответов. Мы использовали метрики оценки эффективности на основе качества выдачи модели и времени обработки запроса. Применение LLM для изучения технологии БПЛА показало, какие новые навыки необходимы для создания основательных пайплайнов ИИ. Кроме того, мы узнали, что технология разработки БПЛА подразумевает применение компьютерного зрения, а значит, и мультимодальных узлов.

В следующей главе мы добавим в свои датасеты мультимодальные данные и расширим нашу RAG.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Повышают ли индексы точность и скорость работы генеративного ИИ на базе генерации с дополненной выборкой?
2. Могут ли индексы обеспечить возможность отслеживания источника вывода RAG?
3. В больших датасетах поиск по индексу медленнее векторного поиска?
4. Легко ли LlamaIndex интегрируется с Deep Lake и OpenAI?
5. Индексы на основе дерева, списка, векторов и ключевых слов — это единственные виды индексов?
6. Опирается ли механизм извлечения данных по индексу ключевых слов на понимание семантики?
7. Может ли LlamaIndex автоматически обрабатывать разбивку текста на чанки и создание эмбеддингов?
8. Необходимо ли добавление метаданных для обеспечения возможности отслеживания источника сгенерированных RAG результатов?
9. Легко ли в механизме поиска по индексу применять обновления в реальном времени?
10. Метрика косинусного сходства в этой главе использовалась для оценки точности запроса?

Ссылки

- LlamaIndex: https://docs.llamaindex.ai/en/stable/.
- Activeloop Deep Lake: https://docs.activeloop.ai/.
- OpenAI: https://platform.openai.com/docs/overview.

Дополнительная информация

- High-Level Concepts (RAG), LlamaIndex: https://docs.llamaindex.ai/en/stable/getting_started/concepts/.

4 Мультимодальная RAG

https://t.me/it_boooks/2

В этой главе поднимем генеративный ИИ на следующий уровень, задействуя модульную RAG. Мы создадим систему, которая для обработки разных типов данных и задач будет использовать разные компоненты, или модули. Например, один модуль будет отвечать за обработку текстовой информации с помощью LLM, как мы делали прежде, а другой — управлять данными изображений, находя и размечая на них объекты. Представьте себе применение этой технологии для БПЛА, которые стали активно использоваться в различных отраслях, предоставляя широкие возможности для аэрофотосъемки, эффективного мониторинга сельскохозяйственных угодий, а также поисковых и спасательных операций. Кроме того, в этих устройствах применяют компьютерное зрение и алгоритмы для анализа получаемых изображений и выявления на них всевозможных объектов, таких как пешеходы, легковые машины, грузовики и т. п. Мы же в этом контексте можем задействовать LLM-агент, который будет извлекать тематическую информацию, дополнять ею запрос пользователя и давать итоговый ответ.

В этой главе мы создадим мультимодальную программу RAG для генерации ответов на запросы о технологии БПЛА с помощью текстовых данных и изображений, извлекаемых из разных источников. Сначала определим основные аспекты модульной RAG, мультимодальных данных, механизма извлечения информации из нескольких источников, модульной генерации ответа и дополненного вывода. Затем построим с помощью Python модульную систему генеративного ИИ по теме БПЛА, используя LlamaIndex, Deep Lake и OpenAI.

Наша система будет оперировать двумя датасетами: содержащим текстовую информацию на тему дронов, который мы собрали в предыдущей главе, и содержащим размеченные изображения, снятые БПЛА. С помощью Deep Lake мы будем обрабатывать мультимодальные данные, с помощью LlamaIndex — индексировать их и извлекать, а с помощью большой языковой модели от OpenAI — дополнять запросы и давать на них ответы. Таким образом мы добавим мультимодальный дополненный вывод, включающий текст и изображения. В завершение же реализуем метрику оценки эффективности для текстовых ответов и добавим метрику оценки распознавания изображений с помощью GPT-4o — мощной *мульти-*

модальной LLM (MMLLM) от OpenAI. К концу главы вы научитесь создавать модульный рабочий поток RAG с использованием передовых техник мультимодальной обработки и извлечения информации из нескольких источников.

Что такое мультимодальная RAG

Мультимодальные данные совмещают в себе разные формы информации: текст, изображения, аудио и видео, — расширяя их анализ и интерпретацию. В контексте RAG мультимодальность реализуется посредством разных модулей, отвечающих за обработку различных типов данных и задач. То есть модули в такой системе специализированные. Например, один отвечает за текст, а другой — за изображения, демонстрируя возможность сложной интеграции, позволяющей использовать при генерации ответа разнородные данные.

Программа, которую мы создадим в этой главе, также будет задействовать в качестве источников два датасета. Одним из них станет набор данных по технологии беспилотников, сформированный в предыдущей главе, а вторым — мультимодальный датасет Deep Lake под названием VisDrone, который содержит тысячи размеченных изображений, снятых с помощью дронов.

Мы выбрали в качестве примера именно БПЛА, так как эта технология все больше внедряется в различные сферы жизни, обеспечивая широкие возможности для аэрофотосъемки, мониторинга сельхозугодий и эффективных поисково-спасательных операций. Дроны облегчают наблюдение за дикой природой, коммерческие перевозки и позволяют безопасно инспектировать инфраструктуру. Кроме того, беспилотники эффективны для исследования окружающей среды, управления дорожным трафиком и тушения пожаров. Эти устройства используют также силовые структуры для ведения наблюдения, радикально трансформируя различные сферы жизни за счет повышения доступности, безопасности и экономичности.

На рис. 4.1 показан рабочий поток, который мы реализуем в текущей главе. В его основе лежит система RAG, проиллюстрированная на рис. 1.3. В предыдущих главах мы реализовали функциональность создания эмбеддингов и индексирования, а в этой займемся извлечением данных и генерацией ответа. В нашей системе грань между извлечением и генерацией будет размыта, так как в блокноте для этой главы генератор будет активно использоваться и для извлечения информации (в том числе ее оценки и ранжирования), и для генерации.

В этой главе мы создадим учебную модульную систему RAG, чтобы ответить на вопросы по технологии БПЛА. Опираться в этом процессе будем на функциональность, реализованную в предыдущих главах, в частности, Deep Lake для создания векторов (см. главу 2) и LlamaIndex для индексирования (см. главу 3). Так что можете вернуться к этим материалам, чтобы освежить их в памяти.

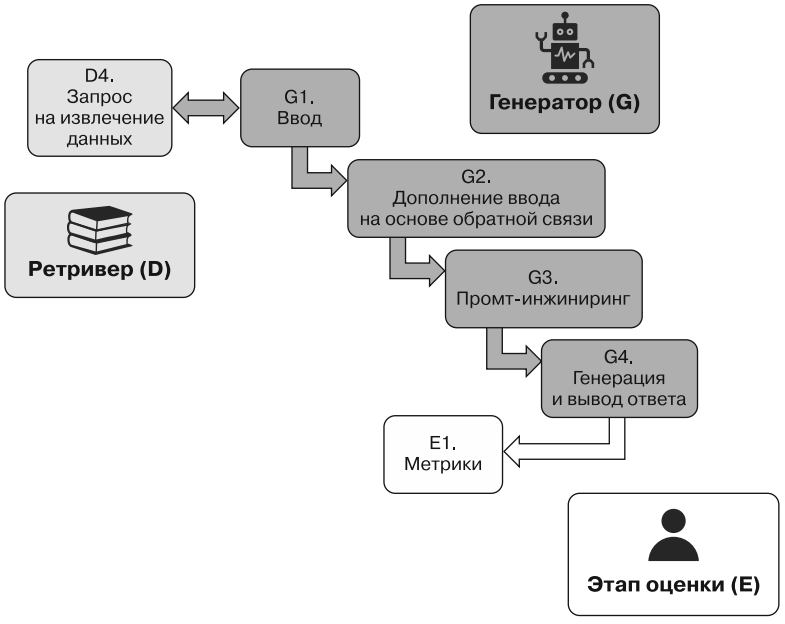


Рис. 4.1. Мультимодальная система RAG

Для начала подробнее разберем схему мультимодальной RAG (см. рис. 4.1), извлекающую информацию из нескольких источников. В списке далее курсивом выделены названия разделов и для каждого этапа приводится его обозначение на рисунке.

- *D4. Загрузка датасета для LLM* с текстовой информацией о дронах из главы 3.
- *D4. Инициализация механизма запросов к LLM* на основе индексированной векторной базы данных LlamaIndex посредством `VectorStoreIndex` и настройка созданного индекса для обработки запросов. Этот пункт пересекается с *G4*, так как модель OpenAI GPT будет выступать и в качестве ретривера, и в качестве генератора.
- *G1.* Определение *ввода, выполненного пользователем, для модульной RAG*, который будет передаваться LLM (для текстового датасета) и мультимодальному механизму обработки запросов (для датасета VisDrone).

  После загрузки текстового датасета, создания механизма обработки запросов и определения базового запроса пользователя для текстового и мультимодального наборов данных следует генерация ответа на основе текстового датасета, созданного в главе 2.
- При выполнении запроса к текстовому датасету этапы *G1*, *G2* и *G4* пересекаются в едином процессе LlamaIndex, обрабатывающем извлечение данных и генерацию ответа. Полученный ответ на время сеанса сохраняется в виде `llm_response`.

Далее следует загрузка в память и выполнение запроса к мультимодальному датасету `VisDrone`.

- *D4*. Мультимодальный процесс начинается с *загрузки и визуализации мультимодального датасета*. Затем путем *навигации по структуре мультимодального датасета* программа *выбирает изображение* и *добавляет к нему ограничивающие рамки*.

После этого к `VisDrone` применяется та же последовательность действий, что и к текстовому датасету.

- *D4. Создание мультимодального механизма обработки запросов* с помощью LlamaIndex происходит путем индексирования векторной базы данных `VisDrone` посредством `VectorStoreIndex` с последующей настройкой созданного индекса для использования при обработке запросов. Этот процесс пересекается с этапом *G4*, так как модель GPT от OpenAI обрабатывает как извлечение данных, так и генерацию ответа.
- *G1*. Ввод от пользователя для механизма мультимодального поиска будет таким же, как и *ввод, выполненный пользователем, для мультимодальной RAG*, поскольку он станет применяться как для механизма LLM (в отношении текстового датасета), так и для механизма мультимодальной обработки запросов (для датасета VisDrone).

На этой стадии мультимодальный датасет `VisDrone` загружен и механизм поиска готов к применению. Задача ввода, выполненного пользователем, *G1* заключается в том, чтобы механизм обработки запросов LlamaIndex извлек из VisDrone релевантные документы с помощью LLM — в нашем случае модели OpenAI. Затем функции извлечения отследят полученный ответ до его источника в мультимодальном датасете, чтобы найти изображение из исходного узла. По сути, мы задействуем механизм обработки запросов для нахождения изображения на основе текстового ответа.

- *G1*, *G2* и *G4* при выполнении запроса к мультимодальному датасету VisDrone пересекаются в едином потоке обработки LlamaIndex.
- Обработка ответа *G4*, заключающаяся в поиске исходного узла и извлечении его изображения, возвращает нас к *D4*, то есть к выбору и обработке изображения.

К этому моменту у нас уже есть текстовый ответ и извлеченное изображение. Теперь можно создать общую сводку и измерить показатель эффективности после вывода времени, затраченного на каждый этап.

- *G4*. Генерируем *мультимодальную сводку*, которая будет включать совмещенный вывод, содержащий ответ LLM, и дополненный вывод, содержащий изображение из мультимодального ответа.

- *E.* Наконец, вычисляем *эффективность LLM* и *эффективность мультимодальной обработки*. В завершение обобщаем эти показатели в едином *показателе эффективности мультимодальной RAG*.

Исходя из такой модульной системы RAG, можно сделать два вывода.

- Создаваемая в этой главе система представляет лишь один из множества способов применения генеративного ИИ на базе RAG в реальных проектах, каждый из которых будет иметь собственные задачи и архитектуру.
- Быстрая эволюция от генеративного ИИ к его сложному аналогу на базе RAG требует реализации тесно интегрированных кросс-платформенных компонентов, таких как использованные в этой главе LlamaIndex, Deep Lake и OpenAI. Эти платформы интегрируются и со многими другими фреймворками, например, Pinecone и LangChain, о которых речь пойдет в главе 6.

Перейдем к созданию описанной модульной RAG с помощью Python.

Создание мультимодальной RAG для сферы БПЛА

В последующих разделах мы с нуля поэтапно создадим генеративный ИИ на базе модульной RAG, используя Python. В ходе этого процесса реализуем:

- LLM под управлением LlamaIndex для обработки и интерпретации текста по теме БПЛА;
- мультимодальные датасеты Deep Lake, содержащие размеченные изображения, снятые дронами;
- функции для вывода этих изображений и определения на них объектов с помощью ограничительных рамок (bounding box);
- систему, способную отвечать на вопросы по технологии БПЛА с использованием текста и изображений;
- метрики эффективности для измерения точности мультимодальных ответов, включая анализ изображений с помощью GPT-4o.

Вам также потребуется датасет из главы 2, поскольку в этом разделе мы будем его загружать. Если вы его не создавали, то можете прочесть главу и без выполнения блокнота, поскольку она самодостаточна и содержит весь необходимый код с пояснениями. Приступим!

Откройте блокнот `Multimodal_Modular_RAG_Drones.ipynb` в репозитории GitHub по адресу https://github.com/Denis2054/RAG-Driven-Generative-AI/tree/main/Chapter04. Здесь нам потребуются пакеты, перечисленные в подразделе «Настройка среды»

в предыдущей главе. Каждый из последующих разделов будет содержать инструкции по созданию мультимодального блокнота, начиная с модуля LLM. Разберем их пошагово.

Загрузка датасета для LLM

Начнем с загрузки датасета, созданного в главе 3. Не забудьте подставить путь к своему:

```
import deeplake
dataset_path_llm = "hub://denis76/drone_v2"
ds_llm = deeplake.load(dataset_path_llm)
```

Вывод подтвердит, что датасет успешно загружен, и будет содержать соответствующую ссылку:

```
This dataset can be visualized in Jupyter Notebook by ds.visualize() or at
https://app.activeloop.ai/denis76/drone_v2
hub://denis76/drone_v2 loaded successfully.
```

Теперь создадим словарь для хранения данных, которые будут загружены в DataFrame для последующего вывода:

```
import json
import pandas as pd
import numpy as np

# Создаем словарь для хранения данных
data_llm = {}

# Перебираем тензоры датасета
for tensor_name in ds_llm.tensors:
    tensor_data = ds_llm[tensor_name].numpy()

    # Проверяем, является ли тензор многомерным
    if tensor_data.ndim > 1:
        # Преобразуем многомерные тензоры в одномерные
        data_llm[tensor_name] = [np.array(e).flatten().tolist()
                                 for e in tensor_data]
    else:
        # Преобразуем одномерные тензоры в списки и декодируем текст
        if tensor_name == "text":
            data_llm[tensor_name] = [t.tobytes().decode('utf-8')
                                     if t else "" for t in tensor_data]
        else:
            data_llm[tensor_name] = tensor_data.tolist()

# Создаем из словаря DataFrame
df_llm = pd.DataFrame(data_llm)
df_llm
```

В выводе (рис. 4.2) мы видим текстовый датасет и его структуру: `embedding` (векторы), `id` (уникальный строковый идентификатор), `metadata` (в нашем случае источник данных) и `text` с фактическим содержанием.

	embedding	id	metadata	text
0	[-0.0009671939187683165, 0.010151553899049759,...	[a89cdb8c-3a85-42ff-9d5f-98f93f414df6]	[{'file_path': '/content/data/1804.06985.txt',...	[High Energy Physics Theory arXiv:1804.06985 h...
1	[-0.015508202835917473, 0.017525529488921165, ...	[e9748376-1181-46a7-a376-d03b3baa82a5]	[{'file_path': '/content/data/2202.11983.txt',...	[Computer Science Computer Vision and Pattern...

Рис. 4.2. Вывод структуры текстового датасета и его содержимого

Далее инициализируем механизм обработки запросов к LLM.

Инициализация механизма для обработки запросов к LLM

Как и в главе 3, мы проиндексируем векторную базу данных документов по технологии БПЛА (`documents_llm`) из датасета (`ds`). Метод `GPTVectorStoreIndex.from_documents()` создает индекс, обеспечивающий ускоренное извлечение документов на основе векторного сходства:

```
from llama_index.core import VectorStoreIndex
vector_store_index_llm = VectorStoreIndex.from_documents(documents_llm)
```

По аналогии с той же главой 3 метод `as_query_engine()` настраивает этот индекс в качестве механизма обработки запросов с конкретными параметрами определения сходства и глубины извлечения, чтобы система отвечала на ввод, выполненный пользователем, находя наиболее релевантные документы:

```
vector_query_engine_llm = vector_store_index_llm.as_query_engine(similarity_top_k=2,
    temperature=0.1, num_output=1024)
```

Далее добавим пользовательский ввод.

Ввод пользователя для мультимодальной RAG

В контексте модульной системы RAG пользовательский ввод применяется для формулирования запроса, который задействует и текстовый механизм обработки, и механизм обработки изображений. Так мы позволим системе генерировать полноценный и точный ответ на основе нескольких источников информации:

```
user_input="How do drones identify a truck?"
```

В этом контексте ввод, выполненный пользователем, выступает своеобразной *основой*, или опорным запросом, применяемым для проверки способностей системы. Он позволит установить базовый ориентир, помогающий понять, насколько хорошо она может обрабатывать запросы и отвечать на них с помощью доступных

ресурсов, например, текста и изображений из разных датасетов. В нашем примере основа является эмпирической и будет служить для оценки системы относительно этого базового ориентира.

Запрос к текстовому датасету

Далее выполним запрос к векторной базе данных, как делали в главе 3:

```
import time
import textwrap
#Начало отсчета.
start_time = time.time()
llm_response = vector_query_engine_llm.query(user_input)
# Конец отсчета
end_time = time.time()
# Вычисление и вывод времени выполнения
elapsed_time = end_time - start_time
print(f"Query execution time: {elapsed_time:.4f} seconds")
print(textwrap.fill(str(llm_response), 100))
```

Время выполнения вполне удовлетворительное:

```
Query execution time: 1.5489 seconds
```

Как и содержимое вывода:

```
Drones can identify a truck using visual detection and tracking methods, which
may involve deep neural networks for performance benchmarking.
```

Теперь займемся загрузкой мультимодального датасета по теме БПЛА.

Загрузка и визуализация мультимодального датасета

Мы загрузим существующий набор данных `VisDrone`, доступный на платформе Deep Lake: https://datasets.activeloop.ai/docs/ml/datasets/visdrone-dataset/. В этом случае *не* будем создавать векторную базу данных, а просто загрузим датасет в память:

```
import deeplake

dataset_path = 'hub://activeloop/visdrone-det-train'
# Возвращает датасет с Deep Lake, не загружая его данные локально
ds = deeplake.load(dataset_path)
```

В выводе будет присутствовать ссылка на онлайн-датасет, который можно изучить с помощью SQL или посредством команд обработки естественного языка, используя инструменты Deep Lake:

```
Opening dataset in read-only mode as you don't have write permissions.
This dataset can be visualized in Jupyter Notebook by ds.visualize() or at
https://app.activeloop.ai/activeloop/visdrone-det-train
hub://activeloop/visdrone-det-train loaded successfully.
```

Выведем сводку датасета для ознакомления с ним:

```
ds.summary()
```

Вывод содержит ценную информацию по структуре датасета:

```
Dataset(path='hub://activeloop/visdrone-det-train', read_only=True,
tensors=['boxes', 'images', 'labels'])

tensor    htype            shape              dtype     compression
------    -----            -----              -----     -----------
boxes     bbox          (6471, 1:914, 4)      float32         None
images    image         (6471, 360:1500,
                         480:2000, 3)          uint8           jpeg
labels    class_label   (6471, 1:914)         uint32          None
```

Он содержит изображения, рамки для обозначения присутствующих на них объектов и метки, описывающие как сами изображения, так и рамки. Визуализируем датасет с помощью кода:

```
ds.visualize()
```

В выводе видим изображения и их рамки (рис. 4.3).



Рис. 4.3. Вывод изображений с ограничивающими рамками

Теперь выведем содержимое датасета в виде DataFrame и посмотрим, что представляют собой изображения:

```
import pandas as pd

# Создаем пустой DataFrame с определенной структурой
df = pd.DataFrame(columns=['image', 'boxes', 'labels'])

# Перебираем образцы данных с помощью enumerate
for i, sample in enumerate(ds):

    # Данные изображения (выбираем либо их путь, либо сжатое представление)
    # Сохранение пути к изображению
    # df.loc[i, 'image'] = sample.images.path
```

```
    # Сохранение сжатых данных изображения
    df.loc[i, 'image'] = sample.images.tobytes()

    # Данные ограничивающей рамки (в виде списка списков)
    boxes_list = sample.boxes.numpy(aslist=True)
    df.loc[i, 'boxes'] = [box.tolist() for box in boxes_list]

    # Данные меток (в списке)
    label_data = sample.labels.data()
    df.loc[i, 'labels'] = label_data['text']

df
```

На рис. 4.4 показан вывод содержимого датасета.

	image	boxes	labels
0	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[1221.0, 84.0, 16.0, 33.0], [1235.0, 71.0, 18...	[pedestrian, pedestrian, tricycle, pedestrian,...
1	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[351.0, 936.0, 305.0, 114.0], [0.0, 818.0, 22...	[car, car, car, car, car, car, car, car, car, ...
2	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[699.0, 716.0, 26.0, 54.0], [600.0, 604.0, 22...	[truck, car, van, car, car, car, car, car, car...
3	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[417.0, 77.0, 57.0, 54.0], [387.0, 109.0, 31....	[ignored regions, ignored regions, car, car, c...
4	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[791.0, 617.0, 97.0, 115.0], [803.0, 539.0, 7...	[car, car, car, car, car, car, car, car, car, ...
...	...	...	...
6466	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[683.0, 710.0, 7.0, 6.0], [681.0, 717.0, 5.0,...	[car, car, car, car, awning-tricycle, car, car...
6467	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[1761.0, 676.0, 207.0, 155.0], [1539.0, 648.0...	[car, car, car, car, car, van, car, car, car, ...
6468	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[0.0, 501.0, 105.0, 93.0], [412.0, 594.0, 172...	[car, van, pedestrian, pedestrian, pedestrian,...
6469	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[200.0, 604.0, 112.0, 64.0], [311.0, 560.0, 1...	[van, truck, van, car, car, van, others, car, ...
6470	b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...	[[60.0, 144.0, 16.0, 21.0], [51.0, 120.0, 15.0...	[people, pedestrian, others, people, people, m...

6471 rows × 3 columns

Рис. 4.4. Фрагмент датасета VisDrone

В датасете 6471 строка изображений и три столбца.

- В столбце `image` находятся изображения. Судя по последовательности байтов `b'\xff\xd8\xff\xe0\x00\x10JFIF\x00\x01\x01\x00...'`, их формат — JPEG. В частности, байты `b'\xff\xd8\xff\xe0'` указывают на начало файла JPEG.
- Столбец `boxes` содержит координаты и размеры рамок, которые обычно представлены в формате `[x, y, width, height]`.
- В столбце `labels` перечислены метки каждой рамки из столбца `boxes`.

Выведем список меток изображений:

```
labels_list = ds.labels.info['class_names']
labels_list
```

Список меток показывает, что конкретно содержится в датасете:

```
['ignored regions',
 'pedestrian',
 'people',
 'bicycle',
 'car',
 'van',
 'truck',
 'tricycle',
 'awning-tricycle',
 'bus',
 'motor',
 'others']
```

На этом загрузка датасета завершена, перейдем к навигации по нему.

Навигация по мультимодальному датасету

В этом разделе мы выберем и выведем изображение, используя соответствующий столбец, после чего добавим на него рамки, связанные с определенной меткой. Сначала выбираем изображение.

Выбор и вывод изображения

Возьмем первое изображение датасета:

```
# Выбор изображения
ind=0
# Получение первого изображения и возвращение массива numpy
image = ds.images[ind].numpy()
```

Теперь выведем его без рамки:

```
import deeplake
from IPython.display import display
from PIL import Image
import cv2    # Импорт OpenCV

image = ds.images[0].numpy()

# Преобразование из BGR в RGB (если требуется)
image_rgb = cv2.cvtColor(image, cv2.COLOR_BGR2RGB)

# Создание и вывод изображения PIL
img = Image.fromarray(image_rgb)
display(img)
```

На выбранном изображении присутствуют грузовики, пешеходы и прочие типы объектов (рис. 4.5).



Рис. 4.5. Вывод изображения

Теперь добавим к объектам рамки.

Добавление рамок и сохранение изображения

Мы вывели первое изображение. После этого программа запрашивает все связанные с ним метки:

```
labels = ds.labels[ind].data()   # Запрос меток на выбранном изображении
print(labels)
```

В выводе присутствуют массив `value`, содержащий численные индексы меток, и `text`, где хранится соответствующий этим меткам текст:

```
{'value': array([1, 1, 7, 1, 1, 1, 1, 6, 6, 6, 6, 6, 6, 6, 6, 6, 6, 1, 6, 6,
6, 6, 1, 1, 1, 1, 1, 1, 6, 6, 3, 6, 6, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
6, 6, 6], dtype=uint32), 'text': ['pedestrian', 'pedestrian', 'tricycle',
'pedestrian', 'pedestrian', 'pedestrian', 'pedestrian', 'truck', 'truck', 'truck',
'truck', 'truck', 'truck', 'truck', 'truck', 'truck', 'truck', 'pedestrian',
'truck', 'truck', 'truck', 'truck', 'pedestrian', 'pedestrian', 'pedestrian',
'pedestrian', 'pedestrian', 'pedestrian', 'truck', 'truck', 'bicycle', 'truck',
'truck', 'pedestrian', 'pedestrian', 'pedestrian', 'pedestrian', 'pedestrian',
'pedestrian', 'pedestrian', 'pedestrian', 'pedestrian', 'pedestrian',
'pedestrian', 'pedestrian', 'truck', 'truck', 'truck']}
```

Выведем эти значения и их текст в двух столбцах:

```
values = labels['value']
text_labels = labels['text']

# Определяем максимальную длину текстовой метки для форматирования
max_text_length = max(len(label) for label in text_labels)
```

```
# Выводим заголовок столбца
print(f"{'Index':<10}{'Label':<{max_text_length + 2}}")
print("-" * (10 + max_text_length + 2))   # Добавляем разделительную линию

# Выводим индексы и метки в двух столбцах
for index, label in zip(values, text_labels):
    print(f"{index:<10}{label:<{max_text_length + 2}}")
```

Полученный вывод отчетливо демонстрирует содержимое присутствующих на изображении меток:

```
Index     Label
----------------------
1         pedestrian
1         pedestrian
7         tricycle
1         pedestrian
1         pedestrian
1         pedestrian
1         pedestrian
6         truck
6         truck   ...
```

Теперь можно сгруппировать и вывести все описывающие изображение метки:

```
# Имена классов на выбранном изображении
ds.labels[ind].info['class_names']
```

Здесь мы видим все классы объектов, которые присутствуют на фото:

```
['ignored regions',
 'pedestrian',
 'people',
 'bicycle',
 'car',
 'van',
 'truck',
 'tricycle',
 'awning-tricycle',
 'bus',
 'motor',
 'others']
```

Иногда количество классов меток на изображении слишком велико для восприятия человеческим глазом.

Теперь добавим рамки. Сначала создадим функцию для их добавления, вывода и сохранения изображения:

```
def display_image_with_bboxes(image_data, bboxes, labels, label_name, ind=0):
    # Вывод изображения с рамками для конкретной метки

    image_bytes = io.BytesIO(image_data)
    img = Image.open(image_bytes)

    # Извлечение имен классов для выбранного изображения
    class_names = ds.labels[ind].info['class_names']

    # Вывод на основе конкретной метки
    # (или вывод всего, если имена классов отсутствуют)
    if class_names is not None:
        try:
            label_index = class_names.index(label_name)
            relevant_indices = np.where(labels == label_index)[0]
        except ValueError:
            print(f"Warning: Label '{label_name}' not found. Displaying all boxes.")
            relevant_indices = range(len(labels))
    else:
        relevant_indices = []   # Метки не обнаружены, значит, рамки не выводим

    # Рисуем рамки
    draw = ImageDraw.Draw(img)
    for idx, box in enumerate(bboxes):   # Перечисляем bboxes
        if idx in relevant_indices:   # Проверяем соответствие рамки
            x1, y1, w, h = box
            x2, y2 = x1 + w, y1 + h
            draw.rectangle([x1, y1, x2, y2], outline="red", width=2)
            draw.text((x1, y1), label_name, fill="red")
    # Сохраняем изображение
    save_path="boxed_image.jpg"
    img.save(save_path)
    display(img)
```

Есть возможность добавлять рамки для конкретной метки, и в данном случае мы выбрали метку `"truck"`:

```
import io
from PIL import ImageDraw
# Получаем метки и данные выбранного изображения
labels = ds.labels[ind].data()['value']
image_data = ds.images[ind].tobytes()
bboxes = ds.boxes[ind].numpy()
ibox="truck" # class in image
# Выводим изображение с рамками для выбранной метки
display_image_with_bboxes(image_data, bboxes, labels, label_name=ibox)
```

Теперь на выбранном изображении грузовики обведены рамками (рис. 4.6).

Пришло время задействовать механизм обработки запросов и получить от него ответ.



Рис. 4.6. Изображение с рамками

Создание мультимодального механизма обработки запросов

В этом разделе создадим запрос к датасету VisDrone и извлечем из него изображение, подходящее под ввод, выполненный пользователем, который мы указали в подразделе «Ввод пользователя для мультимодальной RAG» текущего блокнота. Для этого сделаем следующее.

1. Создадим векторный индекс для каждой строки DataFrame `df`, содержащего изображения, данные о рамках объектов и метки из датасета VisDrone.
2. Создадим механизм обработки запросов, который будет запрашивать текстовые данные из датасета, извлекать информацию о подходящем изображении и выводить ответ в виде текста.
3. Спарсим узлы ответа в поиске ключевых слов, связанных с вводом пользователя.
4. Спарсим узлы ответа в поиске исходного изображения.
5. Добавим рамки с исходного изображения в доработанное.
6. Сохраним изображение.

Создание векторного индекса и механизма обработки запросов

Приведенный далее код сначала создает документ, на основе которого будут сгенерированы индексы векторной базы данных, содержащей мультимодальный датасет. DataFrame `df`, созданный в подразделе «Загрузка и визуализация

мультимодального датасета», не содержит уникальных индексов или эмбеддингов. Мы сами создадим их в памяти с помощью LlamaIndex.

Первым делом код присваивает уникальные ID содержимому DataFrame:

```
# DataFrame именуется 'df'
# Создаем уникальные ID на основе индексов строк
df['doc_id'] = df.index.astype(str)
```

Код, приведенный ранее, добавляет в DataFrame столбец `doc_id`, где каждой записи присваивает уникальный идентификатор, преобразуя их индексы в строки. Здесь инициализируется пустой список `documents`, который мы используем для создания векторного индекса:

```
# Создаем документы (извлекаем текст, связанный с каждой меткой изображения)
documents = []
```

Теперь метод `iterrows()` перебирает каждую строку DataFrame, генерируя последовательность пар «индекс — строка»:

```
for _, row in df.iterrows():
    text_labels = row['labels']    # Теперь каждая метка стала строкой
    text = " ".join(text_labels)   # Соединяем текстовые метки в одну строку
    document = Document(text=text, doc_id=row['doc_id'])
    documents.append(document)
```

Далее добавляется `documents` со всеми записями в датасете, и DataFrame создан:

```
# DataFrame назван 'df'.
# Создаем уникальные ID на основе индексов строк
df['doc_id'] = df.index.astype(str)

# Создаем документы (извлекаем сопутствующий текст
# для каждой метки изображения)
documents = []
for _, row in df.iterrows():
    text_labels = row['labels']    # Теперь каждая метка стала строкой
    text = " ".join(text_labels)   # Объединяем текстовые метки в одну строку
    document = Document(text=text, doc_id=row['doc_id'])
    documents.append(document)
```

Документы готовы для индексирования с помощью `GPTVectorStoreIndex`:

```
from llama_index.core import GPTVectorStoreIndex
vector_store_index = GPTVectorStoreIndex.from_documents(documents)
```

Затем датасет легко дополняется индексами, которые можно визуализировать в словаре:

```
vector_store_index.index_struct
```

Вывод показывает, что индексы были успешно добавлены в датасет:

```
IndexDict(index_id='4ec313b4-9a1a-41df-a3d8-a4fe5ff6022c', summary=None,
nodes_dict={'5e547c1d-0d65-4de6-b33e-a101665751e6': '5e547c1d-0d65-4de6-b33e-
a101665751e6', '05f73182-37ed-4567-a855-4ff9e8ae5b8c': '05f73182-37ed-4567-a855-
4ff9e8ae5b8c'
```

Сейчас мы готовы выполнить запрос к мультимодальному набору данных.

Выполнение запроса к датасету VisDrone

Настроим `vector_store_index` в качестве механизма обработки запросов, как делали в разделе «Механизм запроса индекса векторной базы данных» главы 3:

```
vector_query_engine = vector_store_index.as_query_engine(similarity_top_k=1,
temperature=0.1, num_output=1024)
```

Выполним также запрос к датасету, содержащему фотографии с беспилотников, как делали в главе 3 для датасета LLM:

```
import time
start_time = time.time()
response = vector_query_engine.query(user_input)
# Конец отсчета
end_time = time.time()
# Вычисляем и выводим время выполнения
elapsed_time = end_time - start_time
print(f"Query execution time: {elapsed_time:.4f} seconds")
```

Время выполнения запроса вполне адекватное:

```
Query execution time: 1.8461 seconds
```

Теперь проверим текстовый ответ:

```
print(textwrap.fill(str(response), 100))
```

Вывод получился логичный, а значит, удовлетворительный.

В БПЛА используются различные камеры и датчики наподобие LiDAR и GPS для определения и отслеживания таких объектов, как грузовики.

Обработка ответа

Перейдем к парсингу узлов ответа, чтобы найти уникальные слова и выбрать одно из них:

```
from itertools import groupby

def get_unique_words(text):
    text = text.lower().strip()
    words = text.split()
```

```
    unique_words = [word for word, _ in groupby(sorted(words))]
    return unique_words

for node in response.source_nodes:
    print(node.node_id)
    # Извлекаем из текста узла уникальные слова
    node_text = node.get_text()
    unique_words = get_unique_words(node_text)
    print("Unique Words in Node Text:", unique_words)
```

Мы нашли уникальное слово (`'truck'`) и его уникальный индекс. Он приведет нас прямиком к изображению исходного узла, на основе которого был сгенерирован ответ:

```
1af106df-c5a6-4f48-ac17-f953dffd2402
Unique Words in Node Text: ['truck']
```

В зависимости от спецификаций конкретного проекта можно выбрать больше слов и по-разному написать эту функцию.

Далее займемся поиском изображения в исходных узлах, как делали в отношении датасета для LLM в подразделе «Ответ на запрос и его источник» в предыдущей главе. Мультимодальные векторные базы данных и фреймворки обработки запросов довольно гибкие. Так что, научившись извлекать данные из датасетов для LLM и мультимодальных датасетов, мы будем готовы к решению любых возможных задач.

Ну а теперь выберем и обработаем информацию, связанную с изображением.

Выбор и обработка изображения из исходного узла

Прежде чем выполнять функцию извлечения и вывода изображения, мы удалим то изображение, которое вывели в подразделе «Добавление рамок и сохранение изображения» ранее, чтобы гарантированно работать именно с новым:

```
# Удаление ранее сохраненного изображения
!rm /content/boxed_image.jpg
```

Можно переходить к поиску исходного изображения, применению рамок и выводу полученного изображения с последующим его сохранением с помощью уже знакомой функции:

```
display_image_with_bboxes(image_data, bboxes, labels, label_name=ibox)
```

Следующий код перебирает исходные узлы в поиске ключевого слова `"truck"`, применяет к найденному изображению рамки, после чего выводит его и сохраняет:

```
import io
from PIL import Image
```

```
def process_and_display(response, df, ds, unique_words):
    """Processes nodes, finds corresponding images in dataset,
       and displays them with bounding boxes.

    Args:
        response: The response object containing source nodes.
        df: The DataFrame with doc_id information.
        ds: The dataset containing images, labels, and boxes.
        unique_words: The list of unique words for filtering.
    """

…
            if i == row_index:
                image_bytes = io.BytesIO(sample.images.tobytes())
                img = Image.open(image_bytes)

                labels = ds.labels[i].data()['value']
                image_data = ds.images[i].tobytes()
                bboxes = ds.boxes[i].numpy()
                ibox = unique_words[0]    # Класс на изображении

                display_image_with_bboxes(image_data, bboxes, labels,
                                          label_name=ibox)

# Предполагается, что вы подготовили объекты 'response',
# 'df', 'ds' и 'unique_words':
process_and_display(response, df, ds, unique_words)
```

Вывод получился адекватный (рис. 4.7).



Рис. 4.7. Удовлетворительный вывод

Обобщение мультимодального вывода

Мы шаг за шагом создали мультимодальную программу и теперь обобщим результаты. Для этого создадим функцию, которая будет выводить исходное изображение из ответа, полученного на запрос пользователя, затем выведем сам запрос пользователя и результат LLM, а также итоговое изображение с рамками.

Начнем с функции для вывода исходного изображения, сохраненного мультимодальным механизмом извлечения:

```
# 1. Ввод пользователя = user_input
print(user_input)
# 2. Ответ LLM
print(textwrap.fill(str(llm_response), 100))
# 3. Мультимодальный ответ
image_path = "/content/boxed_image.jpg"
display_source_image(image_path)
```

Теперь можно вывести запрос пользователя, ответ LLM и мультимодальный ответ. Начинается вывод с текстовой информации (запроса пользователя и ответа LLM):

```
How do drones identify a truck?
Drones can identify a truck using visual detection and tracking methods, which
may involve deep neural networks for performance benchmarking.
```

Затем выводится изображение, на котором в данном случае рамками обведены грузовики (рис. 4.8).



Рис. 4.8. Вывод изображения с рамками

Добавив изображение к классическому ответу LLM, мы дополнили вывод. Это позволяет обогатить систему генеративного ИИ за счет добавления информации как к вводу, так и к выводу. Тем не менее, как и в любой программе ИИ, для разработки метрики оценки эффективности модели потребуется качественная функциональность распознавания изображений.

Оценка эффективности

Для оценки эффективности мультимодальной RAG потребуется измерить качество извлечения текста и изображения. В отношении текста это проделать легко, а вот с изображениями посложнее. Анализ изображения из мультимодального ответа отличается от анализа текста. Мы выполнили поиск с помощью мультимодального механизма обработки запросов на основе ключевого слова, после чего спарсили ответ для вывода исходного изображения. Однако для оценки последнего потребуется создать новый механизм. Но начнем с анализа ответа LLM.

Оценка эффективности LLM

LlamaIndex органично вызывал модель OpenAI с помощью ее механизма обработки запросов, например GPT-4, и предоставлял ответ с текстовым содержимым. Для текстовых ответов мы будем использовать ту же метрику косинусного сходства, что в разделах «Оценка вывода с помощью метрики косинусного сходства» главы 2 и «Механизм запроса индекса векторной базы данных» главы 3.

Приведенная далее функция оценивает сходство двух текстов, в данном случае ввода и вывода, используя библиотеки `sklearn` и `sentence_transformers`:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')

def calculate_cosine_similarity_with_embeddings(text1, text2):
    embeddings1 = model.encode(text1)
    embeddings2 = model.encode(text2)
    similarity = cosine_similarity([embeddings1], [embeddings2])
    return similarity[0][0]
```

Теперь можно вычислить сходство между нашим базовым вводом пользователя и изначальным ответом LLM:

```
llm_similarity_score = calculate_cosine_similarity_with_embeddings(user_input,
  str(llm_response))
print(user_input)
print(llm_response)
print(f"Cosine Similarity Score: {llm_similarity_score:.3f}")
```

В выводе мы наблюдаем пользовательский ввод, текстовый ответ на него и показатель косинусного сходства между этими текстами:

```
How do drones identify a truck?
How do drones identify a truck?
Drones can identify a truck using visual detection and tracking methods, which
may involve deep neural networks for performance benchmarking.
Cosine Similarity Score: 0.691
```

Результат удовлетворительный. Но теперь нам нужно придумать способ оценки эффективности мультимодального ответа.

Оценка мультимодального ответа

Для оценки точности соответствия возвращенного изображения запросу нельзя просто опираться на метки датасета. В небольших наборах данных можно сверить изображение вручную, но при больших масштабах системы необходимо этот процесс автоматизировать. В текущем разделе мы используем функции компьютерного зрения модели GPT-4o для анализа изображения, его парсинга в поиске нужных объектов и вывода описания этого изображения. После этого мы также с помощью метрики косинусного сходства сравним полученное текстовое описание с метками, которые изображение должно содержать, и выведем описание этого изображения. Затем с помощью метрики косинусного сходства оценим соответствие полученного описания меткам, которые на нем должны быть. GPT-4o — это мультимодальная модель генеративного ИИ.

Сначала закодируем изображение в формат base64, чтобы упростить передачу его данных в GPT-4o. Кодировка base64 преобразует двоичные данные, такие как изображения, в символы ASCII, которые являются стандартными текстовыми символами. Выполнить это преобразование важно, так как оно позволит передавать данные изображения через протоколы наподобие HTTP, созданные для оптимальной обработки именно текстовых данных. Это также исключит возможные проблемы, связанные с передачей двоичных данных, например, их повреждение или ошибки интерпретации.

Код, приведенный далее, кодирует исходное изображение с помощью модуля Python `base64`:

```
import base64

IMAGE_PATH = "/content/boxed_image.jpg"

# Открываем файл изображения и кодируем его в строку base64
def encode_image(image_path):
    with open(image_path, "rb") as image_file:
        return base64.b64encode(image_file.read()).decode("utf-8")

base64_image = encode_image(IMAGE_PATH)
```

Создадим клиент OpenAI и установим в качестве модели GPT-4o:

```
from openai import OpenAI
# Установка ключа API клиента
client = OpenAI(api_key=openai.api_key)
MODEL="gpt-4o"
```

Уникальное слово будет получено в результате запроса LLM к мультимодальному датасету:

```
u_word=unique_words[0]
print(u_word)
```

Теперь можно отправить изображение в GPT-4o:

```
response = client.chat.completions.create(
    model=MODEL,
    messages=[
        {"role": "system", "content": f"You are a helpful assistant that
          analyzes images that contain {u_word}."},
        {"role": "user", "content": [
            {"type": "text", "text": f"Analyze the following image, tell me
              if there is one {u_word} or more in the bounding boxes and
              analyze them:"},
            {"type": "image_url", "image_url": {
              "url": f"{base64_image}"}
            }
        ]}
    ],
    temperature=0.0,
)
response_image = response.choices[0].message.content
print(response_image)
```

Мы прописали роли `system` и `user` для анализа изображений в поиске нашей целевой метки `u_word`, которой в данном случае является `truck`. Затем отправили изображение из исходного узла в модель, получив удовлетворительное описание:

```
The image contains two trucks within the bounding boxes. Here is the analysis of
each truck:

1. **First Truck (Top Bounding Box)**:
   - The truck appears to be a flatbed truck.
   - It is loaded with various materials, possibly construction or industrial
supplies.
   - The truck is parked in an area with other construction materials and
equipment.

2. **Second Truck (Bottom Bounding Box)**:
   - This truck also appears to be a flatbed truck.
   - It is carrying different types of materials, similar to the first truck.
```

```
   - The truck is situated in a similar environment, surrounded by construction
materials and equipment.

Both trucks are in a construction or industrial area, likely used for
transporting materials and equipment.
```

Теперь можно отправить ответ в функцию оценки косинусного сходства, добавив в начале `"s"` для соответствия множественному числу грузовиков (trucks) в ответе:

```
resp=u_word+"s"
multimodal_similarity_score = calculate_cosine_similarity_with_embeddings(resp,
str(response_image))
print(f"Cosine Similarity Score: {multimodal_similarity_score:.3f}")
```

Полученный вывод хорошо описывает изображение, выходя за понятие `truck`, что ограничивает показатель сходства с запросом:

```
Cosine Similarity Score: 0.505
```

Человек в этом случае может подтвердить соответствие изображения ответу LLM. Однако даже при высоком показателе сходства проблема сохранится. Комплексные изображения сложно анализировать в деталях и точно, хотя доработки в этом направлении постоянно ведутся. Теперь вычислим общий показатель эффективности системы.

Оценка эффективности мультимодальной RAG

Чтобы вычислить общий показатель эффективности системы, разделим сумму показателей точности ответа LLM и двух показателей точности мультимодального ответа на 2:

```
score=(llm_similarity_score+multimodal_similarity_score)/2
print(f"Multimodal, Modular Score: {score:.3f}")
```

Результат показывает, что, несмотря на удовлетворительный результат, с позиции наблюдателя релевантность сложного изображения анализировать попрежнему непросто:

```
Multimodal, Modular Score: 0.598
```

Но этот момент можно улучшить, так как наблюдатель видит, что изображение соответствует запросу. Именно поэтому ведущие агенты ИИ, такие как ChatGPT, Gemini и Bing Copilot, всегда включают процесс обратной связи, который позволяет одобрять или отклонять ответ.

Давайте подытожим пройденную главу и перейдем к дальнейшей доработке RAG с помощью обратной связи от человека.

Резюме

Эта глава познакомила нас с миром мультимодальных RAG, в которых с помощью различных модулей обрабатываются разные виды данных (текст/изображения) и задачи. Мы задействовали функциональность платформ LlamaIndex, Deep Lake и OpenAI, которую освоили в предыдущих главах. Датасет VisDrone с Deep Lake расширил наше знакомство с технологией БПЛА возможностью анализа изображений и определения на них объектов. Он содержал фотографии, метки и информацию об ограничивающих рамках для объектов изображения. При использовании беспилотников приходится обрабатывать мультимодальные данные, что подталкивает нас к наработке навыков, которые пригодятся в различных областях, таких как слежение за жизнью дикой природы, оптимизация коммерческих перевозок и др.

Мы создали мультимодальную систему генеративного ИИ на базе RAG. На первом шаге определили базовый запрос для механизма LLM и мультимодального механизма извлечения информации. Сначала мы обратились к текстовому датасету Deep Lake, который реализовали в главе 3. Механизм обработки запросов LlamaIndex извлек информацию, дополнил ею ввод и сгенерировал ответ. Затем мы загрузили датасет VisDrone и проиндексировали его в памяти с помощью LlamaIndex, создав пайплайн извлечения информации на основе индексируемого векторного поиска. Далее снова через LlamaIndex обратились к этому датасету с запросом, но на сей раз с помощью модели GPT-4, и спарсили текст, сгенерированный для ключевого слова. Наконец, мы перебрали исходные узлы ответа в поиске исходного изображения, после чего вывели его, совместив ответ LLM и ответ с изображением в единый дополненный вывод. Оценили также косинусное сходство текстового ответа. Оценить же сходство изображения оказалось сложнее. Поэтому мы сначала выполнили распознание извлеченного изображения с помощью GPT-4o для получения текстового описания его содержимого, после чего оценили соответствие этого содержимого запросу с помощью той же метрики косинусного сходства.

Создание комплексной системы стало хорошей подготовкой для работы над реальными проектами ИИ, которые зачастую требуют обработки мультимодальных неструктурированных данных из разных источников, что ведет к построению модульных систем. Тем не менее прозрачный доступ к источнику данных ответа позволяет эффективно задействовать потенциал таких комплексных архитектур, управлять ими и улучшать их. Далее вы увидите, как можно применить эту прозрачность для улучшения систем ИИ за счет обратной связи от человека. Следующая глава еще глубже погрузит нас в тему прозрачности и точности механизмов искусственного интеллекта.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Мультимодальная RAG обрабатывает разные типы данных, такие как текст и изображения?
2. Беспилотники используются только для мониторинга в сельском хозяйстве и аэрофотосъемки?
3. Датасет VisDrone применялся в этой главе только для получения текстовых данных?
4. Можно ли добавлять на сделанные беспилотниками снимки ограничивающие рамки для обозначения объектов, например грузовиков и пешеходов?
5. Модульная система в ответ на запросы извлекает текстовые данные и изображения?
6. Необходимо ли создать векторный индекс для запросов к мультимодальному датасету VisDrone?
7. Извлеченные изображения обрабатываются без добавления меток и ограничивающих рамок?
8. Метрики оценки эффективности мультимодальных RAG основываются только на анализе текстовых ответов?
9. Мультимодальная система, подобная описанной в этой главе, может обрабатывать только данные, связанные с технологией БПЛА?
10. В мультимодальной RAG изображения оценивать так же легко, как текст?

Ссылки

- LlamaIndex: https://docs.llamaindex.ai/en/stable/.
- Activeloop Deep Lake: https://docs.activeloop.ai/.
- OpenAI: https://platform.openai.com/docs/overview.

Дополнительная информация

- *Yasunaga et al.* Retrieval-Augmented Multimodal Language Modeling (2023). https://arxiv.org/pdf/2211.12561.

5 Повышение эффективности RAG за счет обратной связи от эксперта

https://t.me/it_boooks/2

Обратная связь от человека (human feedback, HF) не просто полезна для систем генеративного ИИ — она необходима, особенно в случае моделей на базе RAG. Модель генеративного ИИ во время обучения использует информацию из датасетов, содержащих различные документы. Данные, на которых обучалась модель, составляют фундамент в виде ее параметров, которые можно изменить только в процессе очередного обучения. Тем не менее в сфере извлечения данных из текстовых и мультимодальных датасетов есть информация, которую можно корректировать. Как раз этим и привносится обратная связь от человека. Предоставление обратной связи в ответ на извлекаемые моделью данные позволяет непосредственно влиять на качество ее будущих ответов. Включение в этот процесс делает человека активным участником разработки RAG. Это добавляет в проекты ИИ новое измерение, делая знакомую нам RAG адаптивной.

К этому моменту мы уже изучили и реализовали простую, продвинутую и модульную RAG. Теперь же расширим свой инструментарий ИИ еще и адаптивной. Нам известно, что даже лучшие системы генеративного ИИ, демонстрирующие отличные показатели, не смогут убедить разочарованного пользователя в своей полезности, если фактически это не так. В этой главе мы создадим RAG, включающую цикл обратной связи от человека, что сделает ее адаптивной, так как выбор документов для извлечения будет корректироваться. Встраивание HF в RAG ведет к получению гибридного подхода, так как подразумевает включение человека в автоматизированный процесс генерации. Таким образом, используя HF, мы с нуля создадим гибридную программу RAG на Python, начиная с фундаментальных шагов. К концу главы вы будете уверенно разбираться в механизме работы адаптивной RAG и получите практический опыт создания модели ИИ с применением HF.

Адаптивная RAG

RAG не способна решить все наши проблемы и сложности. Этот механизм, как и любая генеративная модель, может давать не соответствующий запросу и ошибочный вывод. Тем не менее этот компонент зачастую оказывается по-

лезным, поскольку модель генеративного ИИ оперирует в своих ответах той тематической информацией, которую мы ей скормили. Но качество выдачи RAG зависит от точности и релевантности этих внутренних данных, что требует дополнительной проверки. Как раз здесь нас и выручает адаптивная RAG. Адаптивный механизм дополненной генерации включает в себя объективную обратную связь от человека, которая позволяет улучшить общую систему генеративного ИИ.

Основная информация в модели генеративного ИИ параметрическая, то есть хранится в виде весов. Но в контексте RAG эти данные можно визуализировать и управлять ими, как мы видели в главе 2. Тем не менее сложность сохраняется. Например, конечный пользователь может передать неоднозначный запрос или выдача RAG окажется некорректной. В связи с этим для повышения надежности системы настоятельно рекомендуется включать в процесс оценки результата обратную связь (HF).

На рис. 1.3 были показаны весь механизм RAG и его экосистема. Давайте подробнее рассмотрим систему адаптивной RAG и выделим в ней ключевые процессы, отраженные на рис. 5.1.

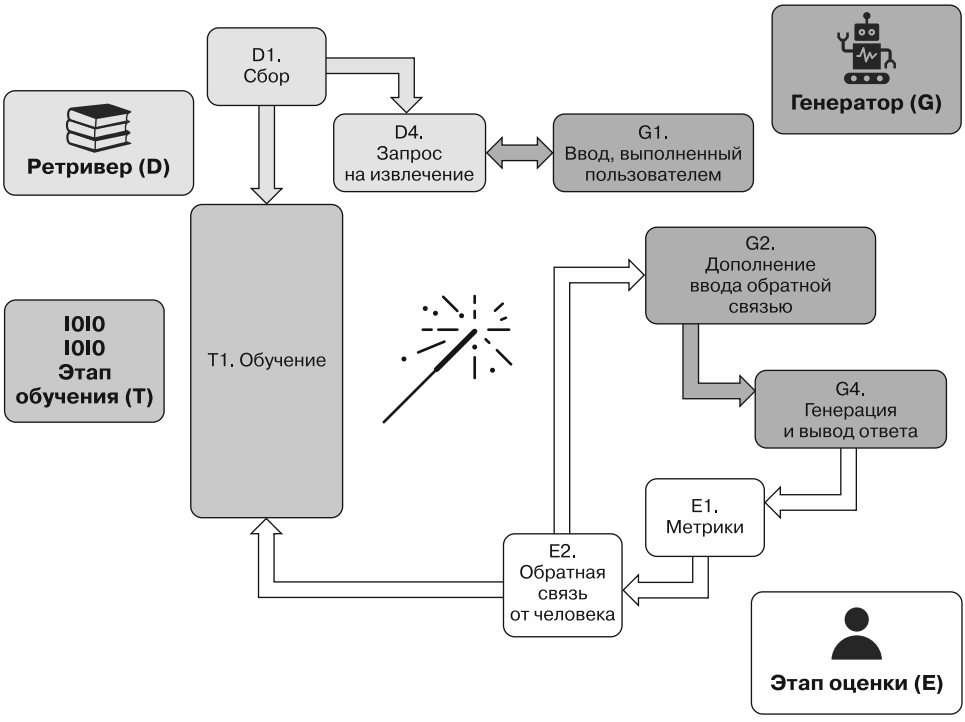


Рис. 5.1. Вариант экосистемы адаптивной RAG

Представленный в этой главе вариант адаптивной RAG включает следующие компоненты (см. рис. 5.1).

Ретривер.

- *D1. Сбор и обработка* статей Википедии по теме LLM путем их скачивания и очистки.
- *D4. Запрос на извлечение* информации из датасета.

Генератор.

- *G1. Ввод* от конечного пользователя.
- *G2. Дополнение ввода обратной связью* и формирование запроса к модели GPT-4o.
- *G4. Генерация ответа и его вывод* моделью ИИ.

Оценка.

- *E1. Метрики* для оценки величины косинусного сходства.
- *E2. Обратная связь от человека* для окончательной оценки системы глазами конечного пользователя и эксперта.

В этой главе разберем адаптивную RAG, создав с помощью Python гибридную программу на платформе Google Colab. Мы напишем ее с нуля, чтобы у вас выработалось отчетливое понимание самого адаптивного процесса, который может меняться в зависимости от задач проекта. Основной же принцип всегда будет один и тот же. Придерживаясь предлагаемого практического подхода, вы научитесь разрабатывать и кастомизировать систему RAG там, где ее готовые решения не отвечают ожиданиям пользователей. Это важно, потому что люди могут быть не удовлетворены ответом модели вне зависимости от показателей оценочных метрик. Мы также рассмотрим добавление шкалы пользовательской оценки для сбора обратной связи по вашей системе генеративного ИИ на базе RAG. В завершение реализуем автоматический механизм ранжирования, который будет решать, как дополнить ввод, выполненный пользователем, для генеративной модели, и попутно предложим практические советы, которые помогут успешно применить RAG в компании.

В качестве опытного подтверждения общей концепции мы создадим систему для гипотетической компании C. Задачей последней будет запуск диалогового агента, объясняющего, что такое ИИ. Замысел в том, чтобы сотрудники этой компании научились понимать базовые термины, а также принципы и способы применения искусственного интеллекта. Ответственный за этот проект инженер по машинному обучению хочет, чтобы будущие пользователи лучше понимали ИИ, реализуя другие проекты на его основе в сфере продаж, производства и доставки.

Сейчас у компании C серьезные проблемы с клиентской поддержкой. В свете растущего числа продуктов и услуг их линейка смартфонов серии C-phone столк-

нулась с техническими сложностями, возникшими у слишком большого числа пользователей. IT-департамент хочет запустить диалоговый агент, который бы обслуживал обращения таких клиентов. Но команды разработчиков пребывают в сомнениях. В связи с этим департамент решил сначала разработать ИИ-агент, который будет объяснять, что такое LLM и как она поможет сервису поддержки клиентов справиться с повышенной нагрузкой.

Для выполнения поставленных компанией задач эта программа будет гибридной и адаптивной.

- *Гибридность*. Реальные сценарии выходят за рамки теоретических фреймворков и конфигураций. Система получается гибридной, так как здесь мы дополняем процесс извлечения данных механизмом обратной связи, которая будет обрабатываться вместе с этими данными в реальном времени. Причем вдобавок к парсингу документов (URL Википедии) на основе ключевого слова станем эти документы размечать, делая это автоматически с возможностью управления и улучшения человеком. Как было показано в предыдущей главе, некоторые документы мы будем заменять обратной связью от эксперта и размечать повторно. В итоге программа будет автоматически извлекать такие экспертные документы вместе с исходными, формируя гибридную (человек — машина) *динамическую* систему RAG.
- *Адаптивность*. Мы добавим механизм ранжирования результатов выдачи пользователями, а также обратную связь от эксперта и автоматическое изменение ранжирования документов. Реализация этого цикла HF приведет нас в самые глубины модульных и гибридных RAG. Адаптивная RAG задействует гибкость системы RAG для адаптации своих ответов на запросы. В данном случае мы хотим, чтобы HF помогала повысить качество выдачи.

В реальных проектах инженеру ML (machine learning — машинного обучения) непременно потребуется выходить за границы предопределенных категорий. Практическая целесообразность вкупе с необходимостью подталкивают к поиску креативных и инновационных решений. Например, в качестве гибридных, динамических и адаптивных аспектов системы инженер может представить любой процесс, который работает с любым видом алгоритмов: классические программные решения, алгоритмы ML для кластеризации или любые другие рабочие функции. В реальном ИИ, если что-то работает, оно реально работает!

Ну а теперь время доказательно подкрепить все сказанное, чтобы показать руководству компании C, как генеративный ИИ на базе адаптивной RAG способен успешно помочь ее командам:

- показать работоспособность ИИ с помощью практического проекта, чтобы руководство могло уверенно вкладывать в него средства и расширять его;
- продемонстрировать, что систему ИИ можно кастомизировать под конкретный проект;

- разработать твердые базовые навыки для решения любой задачи в области ИИ;
- создать механизм управления данными систем ИИ компании;
- заложить основы масштабирования системы, решив проблемы, которые возникнут в рамках реализации доказательного проекта.

За работу!

Создание гибридной адаптивной RAG на Python

Перейдем к созданию конфигурации генеративного ИИ на базе адаптивной RAG. Откройте `Adaptive_RAG.ipynb` на GitHub. Поскольку здесь акцент будет сделан на HF, мы не станем использовать существующий механизм, а создадим свой пайплайн и добавим в него HF.

Как мы уже знаем, программа делится на три отдельные части — *ретривер*, *генератор* и *оценку*, которые в пайплайне реального проекта могут быть отдельными функциями. Постарайтесь разделить их с самого начала, так как это позволит нескольким командам параллельно работать над разными частями механизма RAG.

> **ПРИМЕЧАНИЕ**
>
> Заголовки последующих разделов соответствуют разделам программы в репозитории GitHub, и первым идет ретривер.

1. Ретривер

Начнем с описания первых шагов, необходимых для настройки среды под модель генеративного ИИ на базе RAG. Этот процесс начинается с установки программных компонентов и библиотек, упрощающих извлечение и обработку данных. В частности, мы проговорим процесс скачивания ключевых файлов и установку пакетов, необходимых для эффективного извлечения данных и веб-скрапинга.

1.1. Настройка среды ретривера

Начнем со скачивания `grequests.py` из каталога `commons` репозитория GitHub. Этот репозиторий содержит ресурсы, которые во избежание лишних издержек могут быть общими для нескольких содержащихся в нем программ.

Процесс скачивания стандартный и опирается на запрос:

```
url = "https://raw.githubusercontent.com/Denis2054/RAG-Driven-Generative-AI/
➥main/commons/grequests.py"
output_file = "grequests.py"
```

Поскольку мы создаем генеративный ИИ на базе RAG с нуля, для ретривера потребуются всего два пакета:

- `requests`, HTTP-библиотека для извлечения документов Википедии:

  ```
  !pip install requests==2.32.3
  ```

- `beautifulsoup4` для скрапинга информации с веб-страниц:

  ```
  !pip install beautifulsoup4==4.12.3
  ```

Теперь нам нужен датасет.

1.2. Подготовка датасета

Для этого проекта мы будем извлекать документы Википедии путем их скрапинга через URL-адреса. Каждый документ полученного датасета будет содержать метки, полученные автоматически или присвоенные вручную. Это станет первым этапом индексирования:

```
import requests
from bs4 import BeautifulSoup
import re

# URL-адреса статей Википедии, отображенные в ключевые слова
urls = {
    "prompt engineering": "https://en.wikipedia.org/wiki/Prompt_engineering",
    "artificial intelligence":"https://en.wikipedia.org/wiki/
    ➥Artificial_intelligence",
    "llm": "https://en.wikipedia.org/wiki/Large_language_model", "llms":
    "https://en.wikipedia.org/wiki/Large_language_model"
}
```

Каждому URL-адресу предшествует одна или более меток. Такого механизма будет достаточно для относительно небольшого датасета.

В определенных проектах, включая экспериментальные, этот подход позволяет сделать уверенный шаг от простой RAG (поиск содержимого по ключевым словам) к поиску по датасету с помощью индексов (в данном случае меток). Далее перейдем к обработке данных.

1.3. Обработка данных

Первым делом к извлекаемому документу применяется стандартная функция для скрапинга и очистки текста:

```
def fetch_and_clean(url):
    # Получение содержимого URL-адреса
    response = requests.get(url)
    soup = BeautifulSoup(response.content, 'html.parser')
```

```
    # Поиск основного содержимого статьи;
    # игнорируем дополнительные блоки и заголовки
    content = soup.find('div', {'class': 'mw-parser-output'})

    # Удаление менее релевантных разделов наподобие See also, References и т. д.
    for section_title in ['References', 'Bibliography', 'External links',
                          'See also']:
        section = content.find('span', {'id': section_title})
        if section:
            for sib in section.parent.find_next_siblings():
                sib.decompose()
            section.parent.decompose()

    # Акцент на извлечении и очистке текста только из абзацев
    paragraphs = content.find_all('p')
    cleaned_text = ' '.join(paragraph.get_text(separator=' ', strip=True)
        for paragraph in paragraphs)
    # Удаление ссылочных маркеров наподобие [1], [2] и т. д.
    cleaned_text = re.sub(r'\[\d+\]', '', cleaned_text)

    return cleaned_text
```

Представленный код получает содержимое документа по его URL-адресу, который, в свою очередь, опирается на его метку. Такой простой подход может удовлетворить потребности проекта в зависимости от стоящих задач. Инженеру ML или разработчику всегда нужно следить за тем, чтобы не перегрузить систему затратными и невыгодными функциями. Более того, разметка URL-адресов позволяет направлять пайплайн ретривера на корректные адреса данных, какие бы техники ни использовались (балансировка нагрузки, оптимизация вызова API и пр.). В конечном итоге каждому проекту или подпроекту в зависимости от его задач потребуется одна или несколько техник.

Реализовав функцию получения и очистки данных, можно переходить к процессу извлечения нужной информации на основе ввода, сделанного пользователем.

1.4. Процесс извлечения на основе ввода пользователя

В первую очередь нужно найти ключевое слово в запросе пользователя. Функция `process_query` получает два параметра: `user_input` и `num_words`. Количество извлекаемых слов может ограничиваться как вводом, выполненным пользователем, или установками модели, так и вычислительными затратами или общей эффективностью системы:

```
import textwrap
def process_query(user_input, num_words):
    user_input = user_input.lower()
    # Проверка ввода на наличие указанных ключевых слов
    matched_keyword = next((keyword for keyword in urls if keyword
      in user_input), None)
```

Если обнаруживается совпадение между ключевым словом в запросе пользователя и ключевыми словами, связанными с URL, срабатывают следующие функции для скачивания и очистки данных:

```
if matched_keyword:
    print(f"Fetching data from: {urls[matched_keyword]}")
    cleaned_text = fetch_and_clean(urls[matched_keyword])

    # Ограничиваем вывод до указанного числа ключевых слов из очищенного текста
    words = cleaned_text.split()    # Разделяем текст на слова
    # Объединяем первые n слов в одну строку
    first_n_words = ' '.join(words[:num_words])
```

Параметр `num_words` служит для разбивки текста на чанки. И хотя такой подход может работать при небольших объемах данных, в более сложных сценариях рекомендуется преобразовывать данные в векторы.

Далее очищенный и обрезанный текст форматируется для вывода:

```
    # Ограничиваем ширину вывода первых n слов до 80 символов
    wrapped_text = textwrap.fill(first_n_words, width=80)
    print("\nFirst {} words of the cleaned text:".format(num_words))
    print(wrapped_text)   # Выводим первые n слов в отформатированном абзаце

    # В целях сохранения согласованности при составлении промта
    # для модели GPT-4o используем те же first_n_words
    prompt = f"Summarize the following information
      about {matched_keyword}:\ n{first_n_words}"
    wrapped_prompt = textwrap.fill(prompt, width=80)    # Перенос текста промта
    print("\nPrompt for Generator:", wrapped_prompt)

    # Возвращаем указанное число слов
    return first_n_words
else:
    print("No relevant keywords found. Please enter a query related
           to 'LLM', 'LLMs', or 'Prompt Engineering'.")
    return None
```

Обратите внимание, что эта функция в итоге возвращает первые *n* слов, предоставляя лаконичный и релевантный фрагмент информации на основе запроса пользователя. Такой дизайн позволяет системе эффективно управлять извлечением данных с сохранением участия пользователя.

2. Генератор

Механизм генератора содержит несколько компонентов, часть которых пересекаются с функциями ретривера и пользовательскими интерфейсами в генеративных ИИ на базе RAG.

- *2.1. Выбор выдаваемого адаптивной RAG документа на основе оценок пользователей.* Этот компонент будет опираться на рейтинги, присваиваемые

экспертами в ходе работы с сервисом. В реальном пайплайне эта функциональность может выступать отдельной программой.

- *2.2. Ввод.* На практике управление вводом производится через пользовательский интерфейс (UI). Этот интерфейс и связанный с ним процесс необходимо грамотно выстроить с участием совета пользователей (выбранных сотрудников организации) желательно в контексте совещания, на котором люди смогут полноценно изложить свои предпочтения и потребности.
- *2.3. Симуляция среднего рейтинга.* Вычисление среднего значения оценки пользователей и функциональности.
- *2.4. Проверка ввода перед запуском генератора.* Вывод запроса.
- *2.5. Настройка среды генеративного ИИ.* Установка необходимых компонентов среды генеративного ИИ. В данном случае OpenAI может быть частью другой среды в пайплайне, где работает другая команда, реализуя или развертывая свои компоненты, вне зависимости от функциональности ретривера.
- *2.6. Генерация ответа.* В этом разделе программы модель OpenAI обрабатывает запрос и дает ответ, который подвергается оценке.

Перейдем к описанию системы адаптивной RAG.

2.1. Интеграция HF-RAG для дополнения ввода

Ввиду динамической природы извлечения информации и необходимости контекстного дополнения данных в модели генеративного ИИ нам требуется гибкая система, способная адаптироваться к изменчивому качеству ввода пользователя. Для этого мы реализуем *адаптивную систему RAG*, которая определяет оптимальную стратегию извлечения документов, исходя из рейтинга, выстроенного на базе обратной связи со стороны пользователей. Такая адаптивность выносит систему за рамки простой RAG, делая ее гибридной.

Оценивающие выданный моделью результат люди присваивают ему баллы от 1 до 5, обозначая релевантность и качество возвращенных документов. На основе этих данных срабатывают различные модули, схематично представленные на рис. 5.2.

- *Показатель 1 или 2 балла* означает неспособность системы RAG адекватно дополнять ввод и предполагает, что модели может требоваться тонкая настройка. В этом случае RAG временно деактивируется, пока система не будет доработана. Ввод, выполненный пользователем, станет обрабатываться, но без извлечения релевантных документов.
- *Показатель 3 или 4 балла* ведет к дополнению запроса только экспертной обратной связью, опирающейся на дидактические карточки или фрагменты тематической информации. В этом случае извлечение документов не работает и дополнение ввода происходит на основе данных оценки экспертов.

- *Показатель 5 баллов* ведет к выполнению поиска по ключевым словам, который при необходимости дополняется ранее собранной обратной связью в виде дидактических карточек или фрагментов информации. В данном случае от пользователя не требуется давать обратную связь.

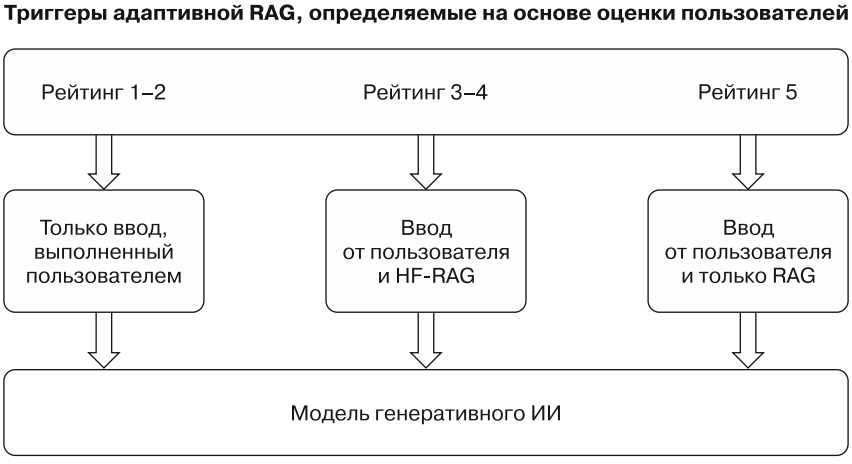


Рис. 5.2. Срабатывание различных вариантов RAG

ПРИМЕЧАНИЕ

Эта программа реализует лишь один из многих сценариев, так что система оценки, уровни показателей и соответствующие им триггеры в разных проектах будут определяться конкретными задачами. Рекомендуется проводить совещания с участием пользователей, чтобы подобрать оптимальную схему реализации адаптивной RAG.

Цель адаптивного подхода — достичь оптимального баланса между автоматизированным извлечением информации и обратной связью. Это позволит обеспечить максимальную релевантность и точность выдачи генеративной модели.

Далее перейдем к вводу.

2.2. Ввод

Пользователю из компании C предлагается ввести запрос:

```
# Запрос пользовательского ввода для парсинга на основе ключевых слов
user_input = input("Enter your query: ").lower()
```

В этом примере и программе мы будем разбирать вопрос: `What is an LLM?` («Что такое LLM?»). После ввода этого вопроса модель его запоминает:

```
Enter your query: What is an LLM?
```

ПРИМЕЧАНИЕ

Наша программа выступает в качестве доказательства концепции и ориентирована на пользователей из компании C, желающих разобраться в теме LLM. В нее можно добавить и другие темы, расширив программу для более широкого спектра задач. Принимать решения о дальнейших шагах рекомендуется по результатам совещания с участием совета пользователей.

На этом подготовка среды закончена, далее мы займемся активацией различных схем RAG.

2.3. Симуляция среднего рейтинга

В рамках нашей программы предположим, что предоставляющие обратную связь пользователи какое-то время оценивали результаты адаптивной RAG на основе функций, описанных в подразделах «3.3. Рейтинг оценок пользователей» и «3.4. Экспертная оценка» далее в этой главе. Пользователи оценили ответы какое-то число раз, что привело к автоматическому обновлению среднего показателя оценок и его сохранению в переменной `ranking`. Показатель `ranking` поможет управляющей команде в ручном или автоматизированном режиме решать, что делать: понижать рейтинг документа, повышать его или полностью блокировать выдачу документов. Можно даже симулировать один из сценариев из подраздела «2.1. Интеграция HF-RAG для дополнения ввода».

Начнем с рейтинга в диапазоне от 1 до 5, который будет деактивировать RAG, демонстрируя нативный ответ генеративной модели:

```
# Выбор показателя от 1 до 5 и выполнение симуляции
ranking=1
```

Затем мы изменим это значение на `ranking=5` для активации RAG без дополнительной обратной связи. Наконец, установим значение `ranking=3`, чтобы активировать дополнительную обратную связь без извлечения документов.

В реальной рабочей среде после проведения совещаний с пользователями и определения ожидаемого поведения системы на основе различных рейтингов будет автоматически активироваться соответствующая функциональность, описанная в подразделах 3.3 и 3.4. Если вы хотите выполнить три сценария из подраздела 2.1, не забудьте инициализировать переменную `text_input`, которую обрабатывает генеративная модель для выдачи ответа:

```
# Инициализация текста для симуляций модели генеративного ИИ
text_input=[]
```

При каждом переключении сценария нужно инициализировать `text_input` заново.

ПРИМЕЧАНИЕ

Ввиду вероятностной природы модели генеративного ИИ ее вывод может от случая к случаю слегка различаться.

Теперь разберем категории рейтинга, описанные в разделе 2.1.

Рейтинг 1 и 2: RAG отключена

Это очень низкая оценка выдачи генеративного ИИ. В таком случае вся функциональность RAG деактивируется, пока команда разработчиков не проанализирует и не доработает систему. Здесь переменная `text_input` равна `user_input`:

```
if ranking>=1 and ranking<3:
  text_input=user_input
```

В итоге модель, в нашем случае GPT-4o из подраздела «2.6. «Генерация ответа» (см. далее), даст следующий вывод:

```
GPT-4 Response:
---------------
It seems like you're asking about "LLM" which stands for "Language Model for
Dialogue Applications" or more commonly referred to as a "Large Language Model."
An LLM is a type of artificial intelligence model designed to understand,
generate, and interact with human language. These models are trained on vast
amounts of text data and use this training to generate text, answer questions,
summarize information, translate languages, and perform other language-related
tasks. They are a subset of machine learning models known as transformers, which
have been revolutionary in the field of natural language processing (NLP).
Examples of LLMs include OpenAI's GPT (Generative Pre-trained Transformer) series
and Google's BERT (Bidirectional Encoder Representations from Transformers).
---------------
```

В данном контексте такой вывод не понравится пользователям компании C, поскольку они не смогут соотнести полученное объяснение с проблемами службы поддержки. Более того, многие пользователи не станут углубляться в ситуацию, так как уже описали команде разработчиков свои потребности и ожидают правильных ответов. Посмотрим, что нам может дать RAG с привлечением экспертной оценки.

Рейтинг 3 и 4: RAG с оценкой экспертов

Это сценарий, когда низкая оценка пользователями автоматической выдачи документов (`ranking=5`) и выдачи без RAG (`ranking=1-2`) привела к запуску механизма экспертной оценки (подробнее — в разделе «3.4. Экспертная оценка»). Этот механизм задействует дидактическую карту, которая предварительно была заполнена советом экспертов и сохранена в виде документа RAG.

Первым делом программа проверяет рейтинг и активирует извлечение с использованием обратной связи:

```
hf=False
if ranking>3 and ranking<5:
  hf=True
```

Программа запросит из датасета экспертной оценки соответствующий документ на основе ключевых слов, эмбеддингов или иных механизмов поиска, вписывающихся в цели проекта. В данном случае мы предположим, что нашли правильную дидактическую карточку, и скачаем ее:

```
if hf==True:
  from grequests import download
  directory = "Chapter05"
  filename = "human_feedback.txt"
  download(directory, filename, private_token)
```

Убедимся в существовании нужного файла, после чего скачаем, очистим и сохраним его содержимое в `content`, также присвоив это содержимое переменной `text_input` для модели GPT-4:

```
if hf==True:
  # Проверяем, существует ли 'human_feedback.txt'
    efile = os.path.exists('human_feedback.txt')

    if efile:
        # Читаем и очищаем содержимое файла
        with open('human_feedback.txt', 'r') as file:
            # Удаляем символы переноса строки и разметку markdown
            content = file.read().replace('\n', ' ').replace('#', '')
            #print(content) # Раскомментируйте для отладки или обслуживания
        text_input=content
        print(text_input)
    else:
      print("File not found")
      hf=False
```

Содержимое файла объясняет, что такое LLM и как она способна помочь компании C повысить качество клиентской поддержки:

```
A Large Language Model (LLM) is an advanced AI system trained on vast amounts of
text data to generate human-like text responses. It understands and generates
language based on the patterns and information it has learned during training.
LLMs are highly effective in various language-based tasks, including answering
questions, making recommendations, and facilitating conversations. They can be
continually updated with new information and trained to understand specific
domains or industries. For the C-phone series customer support, incorporating
```

```
an LLM could significantly enhance service quality and efficiency. The
conversational agent powered by an LLM can provide instant responses to customer
inquiries, reducing wait times and freeing up human agents for more complex
issues. It can be programmed to handle common technical questions about the
C-phone series, troubleshoot problems, guide users through setup processes, and
offer tips for optimizing device performance.
Additionally, it can be used to gather customer feedback, providing valuable
insights into user experiences and product performance. This feedback can then
be used to improve products and services. Furthermore, the LLM can be designed
to escalate issues to human agents when necessary, ensuring that customers
receive the best possible support at all levels. The agent can also provide
personalized recommendations for customers based on their usage patterns and
preferences, enhancing user satisfaction and loyalty.
```

Если теперь проделать операции из подразделов 2.4–2.5, а затем выполнить программу из подраздела 2.6 (см. далее) для генерации ответа на основе этого `text_input`, мы получим удовлетворительный результат:

```
GPT-4 Response:
---------------
A Large Language Model (LLM) is a sophisticated AI system trained on extensive
text data to generate human-like text responses. It understands and generates
language based on patterns and information learned during training. LLMs are
highly effective in various language-based tasks such as answering questions,
making recommendations, and facilitating conversations. They can be continuously
updated with new information and trained to understand specific domains or
industries. For the C-phone series customer support, incorporating an LLM could
significantly enhance service quality and efficiency. The conversational agent
powered by an LLM can provide instant responses to customer inquiries, reducing
wait times and freeing up human agents for more complex issues.
It can be programmed to handle common technical questions about the C-phone
series, troubleshoot problems, guide users through setup processes, and offer
tips for optimizing device performance. Additionally, it can be used to gather
customer feedback, providing valuable insights into user experiences and product
performance. This feedback can then be used to improve products and services.
Furthermore, the LLM can be designed to escalate issues to human agents when
necessary, ensuring that customers receive the best possible support at all
levels. The agent can also provide personalized recommendations for customers
based on their usage patterns and preferences, enhancing user satisfaction and
loyalty.
---------------
```

Этот ответ куда лучше, поскольку дает определение LLM и показывает, как можно улучшить службу поддержки в части проблем с серией C-phone.

В главе 9 мы углубимся в процесс и наладим ежедневную настройку модели для оптимизации, которая позволит уменьшить объем извлекаемых данных. Сейчас же посмотрим, какой результат удастся получить без обратной связи, но с извлечением документов RAG.

Рейтинг 5: RAG без документов из экспертного датасета

Некоторым пользователям не нужны экспертные документы RAG, включающие дидактические карточки или фрагменты тематической информации. Так может быть, например, когда пользователями выступают программисты.

В этом случае для оптимизации затрат API максимальное количество слов выдачи ограничивается 100, но при желании его можно изменить:

```
if ranking>=5:
  max_words=100    # Лимит – размер данных, которые можно добавить к вводу
  rdata=process_query(user_input,max_words)
  print(rdata)    # На случай обслуживания
  if rdata:
        rdata_clean = rdata.replace('\n', ' ').replace('#', '')
        rdata_sentences = rdata_clean.split('. ')
        print(rdata)
  text_input=rdata
  print(text_input)
```

Эта модель дает вполне адекватный ответ, который инженеры могут соотнести со своей задачей:

```
GPT-4 Response:
---------------
A large language model (LLM) is a type of language model known for its capability
to perform general-purpose language generation and other natural language
processing tasks such as classification. LLMs develop these abilities by learning
statistical relationships from text documents through a computationally intensive
training process that includes both self-supervised and semi-supervised learning.
These models can generate text, a form of generative AI, by taking an input text
and repeatedly predicting the next token or word. LLMs are based on artificial
neural networks. As of March 2024, the most advanced and capable LLMs are
constructed using a decoder-only transformer architecture.
---------------
```

По выводу видно, что он основан на данных от марта 2024 года, хотя в документации OpenAI (https://platform.openai.com/docs/models/gpt-4-turbo-and-gpt-4) в качестве крайней точки обучения обозначен декабрь 2023-го.

ПРИМЕЧАНИЕ

В рабочих средах на стороне конечного пользователя подобная ошибка выдачи может проистекать из самих извлеченных данных или вызываться моделью ИИ. Это еще раз подчеркивает важность обратной связи от человека. В данном случае можно понадеяться, что ошибка будет исправлена в извлекаемых документах или самой моделью. Мы же оставили ее для демонстрации того, что обратная связь — это не опция, а обязательный компонент системы.

Подобные дополнения со стороны RAG объясняют потребность в системах ИИ на базе генерации с дополненной выборкой. Тем не менее именно пользователи должны решать, достаточно ли хорош такой тип выводов или нужно внести дополнительную оптимизацию в закрытой среде, например, для компании или внутри нее.

В оставшейся части программы предположим, что `ranking>=5`, после чего перейдем к реализации оценки выдачи. Установим компоненты среды для генерации ответа на основе запроса пользователя и извлеченного документа.

2.4–2.5. Настройка среды генеративного ИИ

> **ПРИМЕЧАНИЕ**
>
> Подраздел «2.4. Проверка ввода перед запуском генератора» подразумевает предварительный вывод запроса пользователя и извлеченного документа до процесса дополнения. После мы перейдем к подразделу «2.5. Настройка среды генеративного ИИ».

Выполните операции этого раздела только раз. Если же вы изменили сценарий из подраздела 2.3, можете пропустить этот и снова выполнить модель генеративного ИИ. Описанная далее установка компонентов находится не в начале блокнота, так как команда проекта может предпочесть выполнить эту программу в другой среде или даже на другом сервере в продакшене.

Желательно максимально разделить функции ретривера и генератора, так как они могут активироваться разными программами и в разное время. Это также позволит одной команде разработки трудиться исключительно над функциональностью ретривера, а другой — над генератором.

Сначала установим OpenAI:

```
!pip install openai==1.40.3
```

Далее извлечем ключ API. Сохраните ключ OpenAI в безопасном месте. Мы в данном случае сохранили его на Google Drive:

```
# Ключ API
# Сохраните ключ в файле и считывайте его (можете вводить ключ в сам блокнот,
# но тогда его будут видеть находящиеся рядом люди)
from google.colab import drive
drive.mount('/content/drive')

f = open("drive/MyDrive/files/api_key.txt", "r")
API_KEY=f.readline().strip()
f.close()
```

```
# Ключ OpenAI
import os
import openai
os.environ['OPENAI_API_KEY'] =API_KEY
openai.api_key = os.getenv("OPENAI_API_KEY")
```

Теперь мы готовы к генерации ответа.

2.6. Генерация ответа

Прежде чем генерировать ответ, мы импортируем и настраиваем все необходимое. Здесь добавили `time` для измерения скорости обработки запроса и выбрали в качестве модели GPT-4o:

```
import openai
from openai import OpenAI
import time

client = OpenAI()
gptmodel="gpt-4o"
start_time = time.time()   # Начинаем отсчет перед выполнением запроса
```

Далее определим для GPT-4o стандартный промт, предоставив модели достаточно информации для ответа и поручив все остальное самой модели и данным RAG:

```
def call_gpt4_with_full_text(itext):
    # Объединяем все строки в одну
    text_input = '\n'.join(itext)
    prompt = f"Please summarize or elaborate on the following content:\n{text_input}"

    try:
      response = client.chat.completions.create(
          model=gptmodel,
          messages=[
             {"role": "system", "content": "You are an expert Natural
               Language Processing exercise expert."},
             {"role": "assistant", "content": "1.You can explain read
               the input and answer in detail"},
             {"role": "user", "content": prompt}
          ],
          temperature=0.1    # Добавляем параметр temperature и др.
        )
      return response.choices[0].message.content.strip()
    except Exception as e:
        return str(e)
```

Затем программа форматирует вывод:

```
import textwrap

def print_formatted_response(response):
    # Определяем ширину для форматирования текста
    # Установим ширину 80 столбцов, при желании можете ее корректировать
```

```
    wrapper = textwrap.TextWrapper(width=80)
    wrapped_text = wrapper.fill(text=response)

    # Выводим отформатированный ответ с шапкой и футером
    print("GPT-4 Response:")
    print("---------------")
    print(wrapped_text)
    print("---------------\n")

# Предполагаем, что 'gpt4_response' содержит ответ на предыдущий вызов GPT-4
print_formatted_response(gpt4_response)
```

В данном случае, как мы видели в подразделе 2.3, ответ удовлетворительный. В сценарии же с `ranking=5`, который разбираем сейчас, ответ будет таким:

```
GPT-4 Response:
---------------
GPT-4 Response:
---------------
### Summary: A large language model (LLM) is a computational model known for
its ability to perform general-purpose language generation and other natural
language processing tasks, such as classification. LLMs acquire these abilities
by learning statistical relationships from vast amounts of text during a
computationally intensive self-supervised and semi-supervised training process.
They can be used for text generation, a form of generative AI, by taking input
text and repeatedly predicting the next token or word. LLMs are artificial
neural networks that use the transformer architecture…
```

Вроде бы хороший ответ, но насколько он точен? Проверим это с помощью оценки.

3. Оценка

В зависимости от спецификаций и задач конкретного проекта можно реализовать любое необходимое число функций для оценки вывода как алгоритмами, так и человеком. Мы в текущем разделе реализуем две автоматические метрики: вычисление времени ответа и косинусное сходство. Затем создадим два интерактивных механизма оценки: рейтинг пользователей и экспертную обратную связь.

3.1. Время ответа

Время ответа вычисляется и выводится посредством вызова API так:

```
import time
…
start_time = time.time()    # Начинаем отсчет перед запросом
…
response_time = time.time() - start_time   # Измеряем время ответа
print(f"Response Time: {response_time:.2f} seconds")  # Выводим время ответа
```

В этом сценарии можно просто вывести время ответа:

```
print(f"Response Time: {response_time:.2f} seconds")  # Выводим время ответа
```

Вывод будет различаться в зависимости от скорости подключения к Интернету и доступных мощностей серверов OpenAI. В данном случае результат такой:

```
Response Time: 7.88 seconds
```

Небыстро, но онлайн-агентам ИИ требуется какое-то время на ответ. Решение об удовлетворительности этого показателя остается за руководством. Теперь перейдем к вычислению меры косинусного сходства.

3.2. Показатель косинусного сходства

Косинусное сходство опирается на измерение косинуса угла между двумя ненулевыми векторами. В контексте анализа текста обычно это векторы значений *TF-IDF* (Term Frequency — Inverse Document Frequency, частота слова — обратная частота документа), отражающих веса слов текста, вычисленные на основе их важности в контексте данного документа и всего корпуса.

Ввод для GPT-4o, представленный `text_input`, и ответ модели, представленный `gpt4_response`, рассматриваются в TF-IDF как два отдельных документа. Функция `vectorizer` преобразует эти документы в векторы, после чего `vectorizer.fit_transform([text1, text2])` соотносит количество совпадений закодированных слов в тексте ввода и ответе, а также их значимости.

Цель — количественно оценить тематическое и лексическое совпадение с помощью следующей функции:

```
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity

def calculate_cosine_similarity(text1, text2):
    vectorizer = TfidfVectorizer()
    tfidf = vectorizer.fit_transform([text1, text2])
    similarity = cosine_similarity(tfidf[0:1], tfidf[1:2])
    return similarity[0][0]

# Пример использования существующих функций
similarity_score = calculate_cosine_similarity(text_input, gpt4_response)

print(f"Cosine Similarity Score: {similarity_score:.3f}")
```

Механизм определения косинусного сходства сначала задействует `TfidfVectorizer`, преобразующий два документа в векторы TF-IDF, после чего функция `cosine_similarity` вычисляет сходство между этими векторами. Результат 1 говорит об идентичности текстов, а 0 — об отсутствии сходства. Вот вывод нашей функции:

```
Cosine Similarity Score: 0.697
```

Этот показатель говорит о высоком сходстве между вводом и выводом модели. Но как оценит этот вывод пользователь? Сейчас выясним.

3.3. Рейтинг оценок пользователей

Интерфейс оценки вывода пользователями позволяет собирать обратную связь. И я уже неоднократно повторял, что этот интерфейс нужно разрабатывать, только хорошо поняв потребности пользователей. В текущем разделе мы предположим, что совет пользователей — это группа разработчиков ПО, тестирующих систему.

Код начинается с параметров интерфейса:

```
# Параметры оценки
counter=20           # Количество оценок пользователей
score_history=30     # Общий балл оценок
threshold=4          # Минимальный рейтинг для привлечения экспертной оценки
```

В этой симуляции параметры показывают, что система вычислила характеристики обратной связи:

- `counter=20` — это количество уже присвоенных пользователями оценок;
- `score_history=30` показывает общий балл 20 оценок;
- `threshold=4` указывает минимальный средний рейтинг, `score_history/counter`, для получения ответа без запроса информации из экспертного датасета.

Теперь проверим этот интерфейс на примере. Приведенный далее код Python определяет функцию `evaluate_response`, оценивающую релевантность и согласованность ответов, сгенерированных языковой моделью наподобие GPT-4. Пользователи оценивают сгенерированный текст по шкале от 1 (плохо) до 5 (отлично), а функция обеспечивает валидность ввода, рекурсивно повторяя вызов в случае его недопустимости. Код вычисляет статистические метрики в виде средних показателей, оценивая эффективность модели на нескольких итерациях.

Функция оценки представляет простой запрос обратной связи для получения значения от 1 до 5:

```
import numpy as np
def evaluate_response(response):
    print("\nGenerated Response:")
    print(response)
    print("\nPlease evaluate the response based on the following criteria:")
    print("1 - Poor, 2 - Fair, 3 - Good, 4 - Very Good, 5 - Excellent")
    score = input("Enter the relevance and coherence score (1-5): ")
    try:
        score = int(score)
        if 1 <= score <= 5:
            return score
        else:
            print("Invalid score. Please enter a number between 1 and 5.")
```

```
            # В случае недопустимого ввода рекурсивный вызов
            return evaluate_response(response)
    except ValueError:
        print("Invalid input. Please enter a numerical value.")
        # В случае недопустимого ввода рекурсивный вызов
        return evaluate_response(response)
```

Теперь вызовем эту функцию:

```
score = evaluate_response(gpt4_response)
print("Evaluator Score:", score)
```

Сначала выводится ответ:

```
Generated Response:
### Summary:
A large language model (LLM) is a computational model…
```

Затем пользователь вводит оценку от 1 до 5, в данном случае 1:

```
Please evaluate the response based on the following criteria:
1 - Poor, 2 - Fair, 3 - Good, 4 - Very Good, 5 - Excellent Enter the relevance
and coherence score (1-5): 3
```

После этого код вычисляет статистику:

```
counter+=1
score_history+=score
mean_score=round(np.mean(score_history/counter), 2)
if counter>0:
  print("Rankings      :", counter)
  print("Score history : ", mean_score)
```

Вывод показывает относительно низкий рейтинг:

```
Evaluator Score: 3
Rankings : 21
Score history : 1.48
```

Но ведь показатель косинусного сходства был положительным. На этом будет активирован механизм привлечения экспертной оценки, так как мы установили в качестве порога его срабатывания 4:

```
threshold=4
```

Что же тут происходит? Спросим экспертов!

3.4. Экспертная оценка

Такие метрики, как косинусное сходство, измеряют лишь фактическое сходство, а не глубинную точность. Не поможет определить точность и время выполнения запроса. Но если рейтинг получился очень низкий, то почему? Потому что пользователя ответ не устроил.

Следующий код сначала загружает изображения «палец вверх» и «палец вниз» для использования экспертом:

```
from grequests import download

# Определите переменные
directory = "commons"
filename = "thumbs_up.png"
download(directory, filename, private_token)

# Определите переменные
directory = "commons"
filename = "thumbs_down.png"
download(directory, filename, private_token)
```

В качестве параметров для активации экспертной обратной связи выступают `counter_threshold` и `score_threshold`. Количество оценок, поставленных пользователями, должно превышать пороговое значение счетчика, а именно `counter_threshold=10`. Порог среднего рейтинга в нашем сценарии составляет 4 — `score_threshold=4`. Теперь симулируем срабатывание запроса оценки эксперта:

```
if counter>counter_threshold and score_history<=score_threshold:
  print("Human expert evaluation is required for the feedback loop.")
```

В этом случае вывод подтвердит необходимость привлечения экспертной обратной связи из-за низкого среднего рейтинга и общего количества пользовательских оценок ответа модели:

```
Human expert evaluation is required for the feedback loop.
```

Еще раз напомню, что в реальном проекте для определения интерфейса необходимо организовать совещание с экспертами. В нашем сценарии стандартный HTML-интерфейс на Python покажет значки «палец вверх» и «палец вниз». Если эксперт нажмет «палец вниз», он сможет ввести текст обратной связи, который будет сохранен в файле `expert_feedback.txt`. Вот соответствующий фрагмент кода:

```
import base64
from google.colab import output
from IPython.display import display, HTML

def image_to_data_uri(file_path):
    """
    Convert an image to a data URI.
    """
    with open(file_path, 'rb') as image_file:
        encoded_string = base64.b64encode(image_file.read()).decode()
        return f'{encoded_string}'

thumbs_up_data_uri = image_to_data_uri('/content/thumbs_up.png')
thumbs_down_data_uri = image_to_data_uri('/content/thumbs_down.png')
```

```
def display_icons():
    # Определяем HTML-содержимое с двумя кликабельными изображениями
…/…
def save_feedback(feedback):
    with open('/content/expert_feedback.txt', 'w') as f:
        f.write(feedback)
    print("Feedback saved successfully.")

# Регистрируем обратный вызов
output.register_callback('notebook.save_feedback', save_feedback)
print("Human Expert Adaptive RAG activated")

# Выводим значки с обработчиками щелчков
display_icons()
```

Этот код выведет значки, показанные на рис. 5.3. Если эксперт нажмет «палец вниз», ему будет предложено ввести текст обратной связи.



Рис. 5.3. Значки обратной связи

Вы можете добавить функцию для обработки нажатия значка «палец вниз», которая будет означать, что ответ некорректен и команда разработки должна либо посовещаться с советом пользователей, либо добавить в интерфейс пользовательской оценки запрос на ввод уточнения. Естественно, это решение остается за разработчиками. В нашем сценарии эксперт нажал «палец вниз», и ему было предложено ввести комментарий (рис. 5.4).



Рис. 5.4. Запрос «Введите комментарий»

Эксперт ввел комментарий, который был сохранен в файле `'/content/expert_feedback.txt'`. Таким образом мы наконец обнаружили неточность в содержимом файла, описанную в следующем фрагменте:

```
There is an inaccurate statement in the text:
"As of March 2024, the largest and most capable LLMs are built with a decoder-
only transformer-based architecture."
This statement is not accurate because the largest and most capable Large
Language Models, such as Meta's Llama models, have a transformer-based
architecture, but they are not "decoder-only." These models use the architecture
of the transformer, which includes both encoder and decoder components.
```

В дальнейшем введенный экспертом комментарий можно будет использовать для улучшения качества датасета RAG. На этом мы заканчиваем знакомство с механизмом применения обратной связи в схеме генерации, дополненной результатами поиска. Подытожим эту главу и пойдем далее.

Резюме

Подводя итоги практического исследования эффективных реализаций ИИ, стоит вспомнить весь проделанный путь знакомства с динамическим миром адаптивной RAG. Сначала мы узнали, что HF не только дополняет генеративный ИИ, но и является для него критическим улучшением, делая этот инструмент мощнее и позволяя подстраивать его под реальные задачи. Мы описали экосистему адаптивной RAG, после чего перешли к практике, реализовав этот механизм с нуля. Начиная со сбора, обработки и запроса данных, мы перешли к интеграции дополнительных элементов в систему генеративного ИИ на базе RAG. При этом наш подход состоял не только в написании кода. Он включал также добавление адаптируемости к ИИ с помощью непрерывных циклов обратной связи.

Расширяя способности GPT-4 экспертной оценкой и оценкой, данной пользователями, мы продемонстрировали практическую возможность применения HF и ее значительное влияние. Мы реализовали систему, в которой вывод не только генерируется, но и оценивается конечными пользователями. Низкий рейтинг приводит к срабатыванию механизма экспертной оценки, подчеркивая важность человеческого вмешательства для уточнения ответов ИИ. Создание адаптивной RAG с нуля дало нам глубокое понимание того, как внедрение HF способно превратить стандартную систему ИИ в такую, которая со временем развивается и улучшается.

Теперь мы готовы к масштабированию ИИ на базе RAG в сложную систему продакшен-уровня, чем и займемся в следующей главе.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Важна ли обратная связь от человека для улучшения систем генеративного ИИ на базе RAG?
2. Можно ли менять параметрические данные модели генеративного ИИ без ее переобучения?
3. Адаптивная RAG включает циклы обратной связи в реальном времени для повышения качества извлечения данных?
4. Основная цель адаптивной RAG — заменить весь ввод, выполненный пользователем, автоматизированными ответами?
5. Может ли обратная связь в адаптивной RAG вызывать появление изменений в извлекаемых документах?
6. Компания C использует адаптивную RAG только для решения проблем службы поддержки?
7. Обратная связь применяется, только когда ответы ИИ имеют высокую оценку пользователей?
8. Написанная в этой главе программа предоставляет только текстовый вывод извлеченных документов?
9. Гибридная адаптивная RAG статична, то есть не может корректироваться на основе обратной связи?
10. Оценки пользователей при определении релевантности ответов ИИ полностью игнорируются?

Ссылки

- *Kortukov E., Rubinstein A., Nguyen E., Oh S. J.* Studying Large Language Model Behaviors Under Realistic Knowledge Conflicts: https://arxiv.org/abs/2404.16032.
- Модели OpenAI: https://platform.openai.com/docs/models.

Дополнительная информация

Подробнее о векторизации и косинусном сходстве, использованных в этой главе, можно узнать из следующих источников.

- Извлечение признаков — `TfidfVectorizer`: https://scikit-learn.org/stable/modules/generated/sklearn.feature_extraction.text.TfidfVectorizer.html.
- `sklearn.metrics` — `cosine_similarity`: https://scikit-learn.org/stable/modules/generated/sklearn.metrics.pairwise.cosine_similarity.html.

6 Масштабирование банковских данных клиентов в системе RAG с помощью Pinecone

Масштабирование документов RAG, как текстовых, так и мультимодальных, строится не только на накоплении дополнительных данных — этот процесс фундаментально меняет работу приложения. Во-первых, масштабирование заключается в нахождении правильного количества данных — ни больше ни меньше. Во-вторых, по мере добавления данных могут меняться и требования к приложению — например, появится необходимость в новой функциональности для обработки повышенной нагрузки. Наконец, при масштабировании проекта мы будем упираться в определенные границы допустимых затрат и минимальной скорости выполнения. Учитывая все это, текущая глава познакомит вас с передовыми техниками использования ИИ для решения проблем масштабирования, с которыми вы можете столкнуться в своих проектах. Для этого мы с помощью Pinecone создадим рекомендательную систему на основе сопоставления с шаблоном, нацеленную на минимизацию оттока клиентов банка (закрытие ими счетов).

Начнем с пошаговой разработки первой программы пайплайна. Здесь вы научитесь скачивать датасет Kaggle, содержащий информацию о клиентах банков, и анализировать собранные данные (exploratory data analysis, EDA). Этот базовый шаг очень важен, так как ориентирует вас в процессе подготовки датасета и выработки стратегии RAG для последующих этапов обработки. Вторая программа нашего пайплайна познакомит вас с мощной комбинацией из Pinecone — векторной базы данных для обработки масштабного поиска — и модели OpenAI `text-embedding-3-small`. В этой программе вы сначала реализуете разбивку данных, после чего выполните обновление/вставку записей (операция `upsert`) в индекс Pinecone, который мы масштабируем до более миллиона векторов. Этот индекс мы подготовим под обработку сложных запросов с приемлемой скоростью. В завершение же на примере третьей программы пайплайна покажем, как создавать запросы RAG с помощью Pinecone, дополнять ввод, выполненный пользователем, и задействовать GPT-4o для генерации рекомендаций. Наша цель — сократить отток клиентов банка за счет предложения персонализированных и полезных рекомендаций. К концу главы вы научитесь уверенно применять возможности Pinecone и OpenAI в своих проектах RAG.

Масштабирование с помощью Pinecone

Далее мы будем реализовывать передовую векторную базу данных Pinecone, используя мощные возможности эмбеддинга данных с помощью OpenAI для построения систем обработки и запроса данных. Наша цель — получить рекомендательный механизм, который будет способствовать сохранению интереса клиентов к услугам банка. Усвоив описанный подход, вы сможете применять его во многих областях, требующих рекомендаций (досуг, медицина или правовая сфера). Чтобы понять и оптимизировать сложные процессы, мы с нуля создадим программы, включающие минимум компонентов. В текущей главе используем векторную базу данных Pinecone и модель LLM от OpenAI.

Выбор и проектирование архитектуры будут зависеть от задач проекта, и вы вполне сможете применить описанную методологию к другим платформам. В архитектуре, которую мы реализуем далее, комбинация векторной базы данных и модели генеративного ИИ позволит оптимизировать все процессы и упростит масштабирование. Для начала познакомимся с этой архитектурой, которую мы будем создавать на Python.

Архитектура

В этой главе мы реализуем поиск на основе сходства векторов, как делали в главах 2 и 3. Возьмем структуру из трех созданных ранее компонентов пайплайна и применим ее к нашей рекомендательной системе, как схематично показано на рис. 6.1. Если чувствуете необходимость, то еще раз просмотрите упомянутые главы, прежде чем переходить к коду в текущей.

Ключевые функции масштабированной системы рекомендаций, которую мы создадим, можно обобщить в трех компонентах пайплайна, представленных на рисунке.

- *Компонент 1: сбор и подготовка данных*. На этом этапе мы изучим и проанализируем датасет с использованием стандартных запросов и кластеризации по k-среднему.
- *Компонент 2: масштабирование индекса Pinecone (векторной базы данных)*. На этом этапе мы разобьем на чанки, преобразуем в векторы, а также обновим и вставим более миллиона документов в индекс Pinecone (векторную базу данных).
- *Компонент 3: генеративный ИИ на базе RAG*. На этом этапе перейдем к полностью масштабированной RAG, где будем выполнять запросы к базе данных с более чем миллионом векторов и дополнять полученным ответом запрос к модели GPT-4o, создавая таргетированные рекомендации.

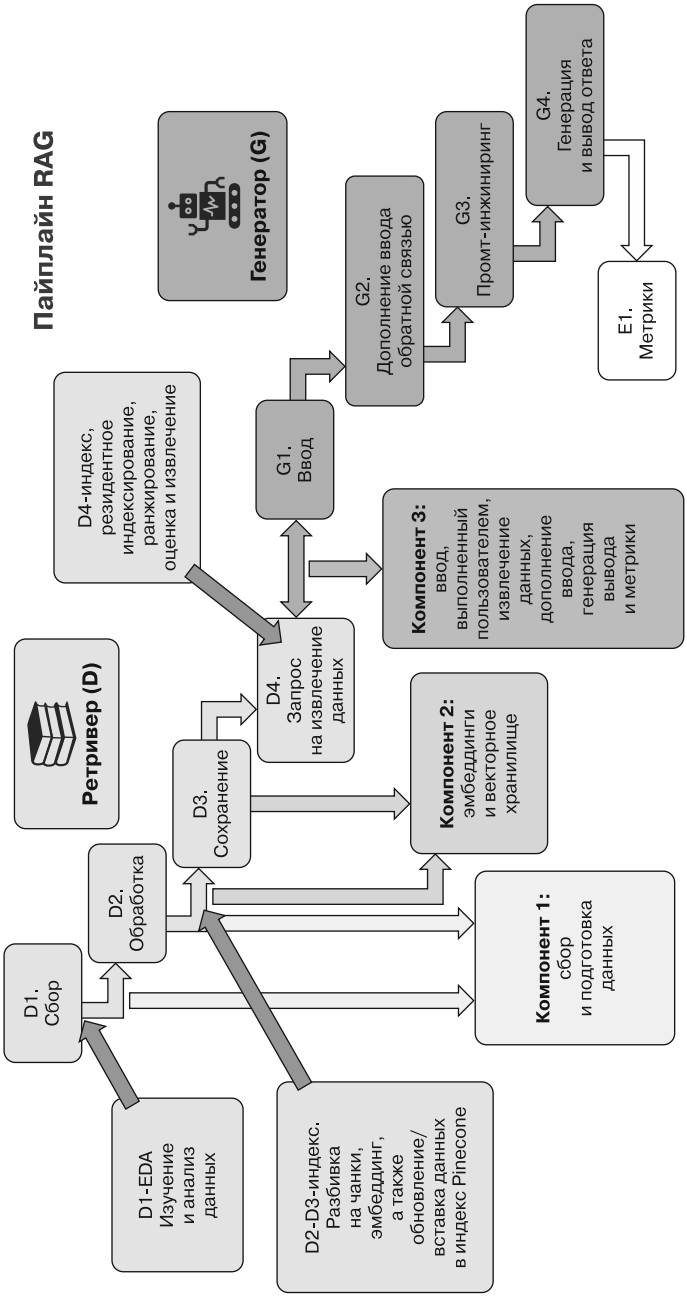


Рис. 6.1. Масштабирование пайплайна генеративного ИИ на базе RAG

Три реализованные нами программы будут характеризоваться следующими особенностями.

- *Масштабируемой и бессерверной инфраструктурой*. Мы начнем понимать бессерверную архитектуру Pinecone, которая устраняет сложности, связанные с управлением серверами и масштабированием. Нам не потребуется управлять ресурсами БД или работой машины. Это подход «плати по мере использования», основанный на бессерверных индексах, сформированных облаком и регионом, например Amazon Web Services (AWS) в `us-east-1`. Такой механизм упрощает масштабирование и процесс оплаты услуг, хотя мониторить и минимизировать расходы все равно нужно.
- *Легковесной и упрощенной средой разработки*. Наша стратегия интеграции минимизирует применение внешних библиотек за счет содержания легковесного стека разработки. Прямое использование OpenAI для генерации эмбеддингов и Pinecone для их хранения и запроса упрощает этап обработки данных и повышает эффективность системы. Но, несмотря на потенциальную эффективность такого подхода, в зависимости от контекста вашего проекта можно задействовать и иные методы, реализуемые в других главах книги.
- *Оптимизированным масштабированием и быстродействием*. Векторная база данных Pinecone создана для эффективной обработки масштабных датасетов, чтобы по мере увеличения объема данных быстродействие приложения оставалось удовлетворительным. Как и в случае любой облачной платформы или API, при реализации инструментов Pinecone и OpenAI нужно оценивать ограничения безопасности и конфиденциальности. Кроме того, как мы увидим в разделе «Компонент 2: масштабирование индекса Pinecone» текущей главы, важно непрерывно мониторить производительность и затраты системы.

Приступим к сбору и обработке датасета `Bank Customer Churn`.

Компонент 1: сбор и подготовка данных

Этот раздел будет посвящен обработке и анализу датасета `Bank Customer Churn`. Мы опишем все этапы настройки среды, управления данными и применения техник машинного обучения (ML). Прежде чем использовать алгоритмы, важно проанализировать датасет, чтобы начать его чувствовать. Из-за гибких творческих способностей человека его анализ всегда играет решающую роль. В связи с этим мы реализуем сбор и подготовку данных на Python в виде трех основных шагов.

1. Сбор и обработка датасета.
   - Настройка среды Kaggle для аутентификации и скачивания датасетов.
   - Сбор и распаковка датасета `Bank Customer Churn`.
   - Упрощение датасета за счет удаления ненужных столбцов.

2. Анализ результатов наблюдений.
   - Первичный просмотр имеющихся данных, чтобы понять их структуру и тип.
   - Изучение связей между жалобами клиентов и оттоком последних (закрытием счетов).
   - Анализ того, как возраст и уровень оплаты связаны с оттоком клиентов.
   - Генерация тепловой карты для наглядного представления корреляций между численными признаками.
3. Обучение модели ML.
   - Подготовка данных для ML.
   - Применение техник кластеризации для обнаружения паттернов в поведении клиентов.
   - Анализ эффективности различных конфигураций кластеризации.
   - Выводы и переход к созданию генеративного ИИ на базе RAG.

Наша цель — проанализировать датасет и подготовить его к реализации компонента 2 — масштабированию индекса Pinecone. Для этого нужно предварительно проанализировать результаты наблюдений (EDA) датасета. Кроме того, каждый описанный раздел выстроен в ключе практического разбора кода с нуля, чтобы вы получили реальный опыт и понимание рабочих потоков в сфере обработки данных. Начнем со сбора датасета.

Сбор и обработка датасета

Сначала скачаем с Kaggle датасет `Bank Customer Churn` и обработаем его:

`https://www.kaggle.com/datasets/radheshyamkollipara/bank-customer-churn`

Файл `Customer-Churn-Records.csv` содержит данные 10 000 записей клиентов банка, раскрывающие различные аспекты, которые могут влиять на закрытие людьми счетов. Загрузил этот датасет Редхишьям Коллипара (Radheshyam Kollipara), который справедливо отметил: «Как мы знаем, регистрация нового клиента обходится банку намного дороже, чем сохранение существующего. Поэтому банкам полезно знать, что конкретно подталкивает клиентов к отказу от их услуг. Стремление предотвратить отток подвигает компании к разработке программ лояльности и удержания, чтобы сохранить максимальное число клиентов».

Вот описания столбцов датасета, как они даны на Kaggle:

```
RowNumber – соответствует номеру записи (строки) и на вывод не влияет.

CustomerId – содержит произвольные значения и на отток клиентов не влияет.

Surname – фамилия клиента не влияет на его решение об отказе от услуг банка.
```

CreditScore – может влиять на отток клиентов, так как клиент с высоким кредитным баллом с меньшей вероятностью закроет счет.

Geography – местонахождение клиента может влиять на его решение о закрытии счета.

Gender – интересно узнать, влияет ли пол человека на вероятность того, что он откажется от дальнейшего обслуживания в банке.

Age – точно имеет значение, поскольку более возрастные клиенты закрывают счета реже молодых.

Tenure – сколько лет человек является клиентом банка. Как правило, чем дольше человек является клиентом, тем он лояльнее и вероятность его ухода ниже.

Balance – тоже хороший индикатор оттока, поскольку люди, имеющие значительные средства на счете, закрывают его реже, чем те, у кого средств меньше.

NumOfProducts – количество продуктов, приобретенных клиентом через банк.

HasCrCard – показывает, есть ли у клиента кредитная карта. Этот столбец тоже важен, так как люди с кредитными картами реже закрывают свой банковский счет.

IsActiveMember – активные клиенты реже отказываются продолжать обслуживание в банке.

EstimatedSalary – как и в случае с балансом, люди с низкими зарплатами скорее уходят из банка, чем те, у кого зарплаты выше.

Exited – закрыл клиент свой счет или нет.

Complain – жаловался клиент или нет.

Satisfaction Score – показатель удовлетворенности, который клиент присвоил решению обозначенной им проблемы.

Card Type – тип имеющейся у клиента карты.

Points Earned – очки, заработанные клиентом за использование кредитной карты.

Теперь, когда мы знаем, что содержит датасет, нужно собрать эту информацию и обработать ее для EDA. Начнем с настройки среды.

Настройка среды для Kaggle

Для автоматического скачивания датасетов с Kaggle вам потребуется зарегистрироваться на платформе www.kaggle.com/ и создать ключ API. На момент написания этого блокнота за скачивание датасета денег не берут. Следуйте инструкциям для сохранения и использования вашего ключа API. Сохраните его в безопасном месте. В нашем случае он лежит в файле на диске Google Drive, который нужно смонтировать:

```
# Ключ API
# Сохраните ключ в файле и считывайте его оттуда (можете вводить его прямо
# в блокнот, но тогда он будет виден находящимся рядом людям)
from google.colab import drive
drive.mount('/content/drive')
```

Далее программа считывает этот файл JSON и устанавливает значения переменных среды для аутентификации на Kaggle, задействуя ваше имя пользователя и ключ API:

```
import os
import json

with open(os.path.expanduser("drive/MyDrive/files/kaggle.json"), "r") as f:
  kaggle_credentials = json.load(f)
kaggle_username = kaggle_credentials["username"]
kaggle_key = kaggle_credentials["key"]

os.environ["KAGGLE_USERNAME"] = kaggle_username
os.environ["KAGGLE_KEY"] = kaggle_key
```

Теперь мы готовы к установке и аутентификации Kaggle:

```
try:
  import kaggle
except:
  !pip install kaggle
import kaggle
kaggle.api.authenticate()
```

Сделано! Теперь можно скачать датасет `Bank Customer Churn`.

Скачивание датасета

Далее мы скачаем архив датасета, извлечем из него CSV-файл, загрузим его в DataFrame, удалим лишние столбцы и выведем результат. Начнем со скачивания:

```
!kaggle datasets download -d radheshyamkollipara/bank-customer-churn
```

В выводе показан адрес источника полученных данных:

```
Dataset URL: https://www.kaggle.com/datasets/radheshyamkollipara/bank-customer-
churn
License(s): other
bank-customer-churn.zip: Skipping, found more recently modified local copy (use
--force to force download)
```

Теперь разархивируем датасет:

```
import zipfile

with zipfile.ZipFile('/content/bank-customer-churn.zip', 'r') as zip_ref:
  zip_ref.extractall('/content/')

print("File Unzipped!")
```

Вывод должен подтверждать успешную распаковку файла:

```
File Unzipped!
```

Полученный CSV-файл загружаем в DataFrame `data1`:

```
import pandas as pd
# Загрузка CSV-файла
file_path = '/content/Customer-Churn-Records.csv'
data1 = pd.read_csv(file_path)
```

Теперь удалим четыре столбца, которые в текущем сценарии не нужны.

- `RowNumber`. Это лишний столбец, так как для каждой записи мы будем создавать уникальный индекс.
- `Surname`. Наша цель — анонимизировать данные, поэтому фамилии демонстрировать не будем. Мы сосредоточимся на профилях и поведении клиентов, включая их жалобы и интенсивность использования кредитной карты (заработанные баллы).
- `Gender`. В 2020-х годах аспекты восприятия и поведение клиентов ощутимо изменились, поэтому в контексте нашего экспериментального проекта будет этичнее исключить информацию о половой принадлежности. На общую эффективность программы это никак не повлияет.
- `Geography`. Это поле в некоторых случаях может представлять интерес, но в нашем сценарии мы лучше исключим его, чтобы избежать выявления ложных взаимосвязей в результатах на основе культурных клише. Кроме того, включение этого признака потребовало бы внесения дополнительной информации, если бы мы хотели, например, вычислить расстояние для службы доставки:

```
# Удаляем столбцы и обновляем DataFrame
data1.drop(columns=['RowNumber','Surname', 'Gender','Geography'],
    inplace=True)
data1
```

В полученном выводе `data1` мы видим упрощенный, но все равно эффективный датасет (рис. 6.2).

Преимущество этого подхода в том, что он оптимизирует размер данных, которые будут вставляться в индекс Pinecone (векторную базу данных). Оптимизация размера данных до их вставки в Pinecone и уменьшение датасета за счет удаления лишних полей могут принести огромную пользу. Так мы сокращаем количество данных, которые нужно передавать, сохранять и обрабатывать в векторной БД. При масштабировании уменьшение объема данных ведет к ускорению запросов и снижению затрат, так как тарифы Pinecone могут зависеть от размера хранящихся данных и количества вычислительных ресурсов, используемых для их запроса.

Теперь сохраним обновленный DataFrame в безопасном месте:

```
data1.to_csv('data1.csv', index=False)
!cp /content/data1.csv /content/drive/MyDrive/files/rag_c6/data1.csv
```

	CustomerId	CreditScore	Age	Tenure	Balance	NumOfProducts	HasCrCard	IsActiveMember	EstimatedSalary
0	15634602	619	42	2	0.00	1	1	1	101348.88
1	15647311	608	41	1	83807.86	1	0	1	112542.58
2	15619304	502	42	8	159660.80	3	1	0	113931.57
3	15701354	699	39	1	0.00	2	0	0	93826.63
4	15737888	850	43	2	125510.82	1	1	1	79084.10
...	...	...	...	...	...	...	...	...	...
9995	15606229	771	39	5	0.00	2	1	0	96270.64
9996	15569892	516	35	10	57369.61	1	1	1	101699.77
9997	15584532	709	36	7	0.00	1	0	1	42085.58
9998	15682355	772	42	3	75075.31	2	1	0	92888.52
9999	15628319	792	28	4	130142.79	1	1	0	38190.78

0000 rows × 14 columns

Рис. 6.2. Вывод DataFrame

Можете сохранить его там, где вам удобнее, — главное сохраните, поскольку мы будем использовать этот DataFrame в разделе «Компонент 2: масштабирование индекса Pinecone». Далее нам нужно изучить итоговый оптимизированный датасет, чтобы понять, как именно превратить его в векторную базу данных.

Изучение и анализ данных (EDA)

В этом разделе мы выполним EDA, используя полученный DataFrame с информацией о клиентах банка. Очень важно делать это перед применением любых техник RAG, завязанных на векторные БД, так как это помогает понять внутренние паттерны и тренды данных.

К примеру, предварительный анализ показывает прямую связь между жалобами клиентов и показателями их оттока, говорящую о том, что наличие жалоб повышает вероятность закрытия счета клиентом. Кроме того, данные показывают, что клиенты старше 50 лет реже отказываются от услуг банка, чем более молодые. Интересно, что уровень дохода, в частности до 100 000 долларов, не оказывает ощутимого влияния на решение о закрытии счета.

Тщательно анализируя эту информацию, мы продемонстрируем, почему поспешный переход сразу к моделям ML, особенно глубокому обучению, не всегда оказывается обязательным или эффективным для составления базовой картины. В сценариях, где данные имеют очевидные взаимосвязи и паттерны, более подходящими и экономичными часто оказываются упрощенные статистические методы оценки или даже обычные техники анализа данных. Например, значимый результат может обеспечить кластеризация по методу *k*-средних, и мы реализуем ее в подразделе «Обучение модели ML» далее.

Однако это не умаляет ценности продвинутых техник RAG, которые мы изучим в разделе «Компонент 2: масштабирование индекса Pinecone». Там мы задействуем в отношении векторных баз данных глубокое обучение, чтобы обнаружить более тонкие паттерны и запутанные связи, которые не раскрываются классическими методами EDA.

Если вывести столбцы DataFrame, то мы увидим, что в них сложно найти некие паттерны:

```
#    Column            Non-Null Count    Dtype
---  ------             --------------   -----
 0   CustomerId         10000 non-null   int64
 1   CreditScore        10000 non-null   int64
 2   Age                10000 non-null   int64
 3   Tenure             10000 non-null   int64
 4   Balance            10000 non-null   float64
 5   NumOfProducts      10000 non-null   int64
 6   HasCrCard          10000 non-null   int64
 7   IsActiveMember     10000 non-null   int64
 8   EstimatedSalary    10000 non-null   float64
 9   Exited             10000 non-null   int64
 10  Complain           10000 non-null   int64
 11  Satisfaction Score 10000 non-null   int64
 12  Card Type          10000 non-null   object
 13  Point Earned       10000 non-null   int64
```

`Age`, `EstimatedSalary` и `Complain` являются возможными определяющими признаками, влияющими на показатель `Exited`. Для получения представления также можно вывести DataFrame, как показано в следующем фрагменте `data1` (рис. 6.3).

Выполнение стандартных расчетов по DataFrame показывает, что основным признаком выступает `Complain`, ведущий к `Exited`:

```
# Вычисление процента complain относительно exited
if sum_exited > 0:    # Избегаем деления на нуль
    percentage_complain_over_exited = (sum_complain/ sum_exited) * 100
else:
    percentage_complain_over_exited = 0

# Вывод результатов
print(f"Sum of Exited = {sum_exited}")
print(f"Sum of Complain = {sum_complain}")
print(f"Percentage of complain over exited = {percentage_complain_over_exited:.2f}%")
```

Вывод показывает очень высокую корреляцию жалоб и ухода клиентов из банка — 100,29 %. То есть клиенты, которые жаловались, действительно перестали пользоваться услугами банка, что является естественным трендом рынка:

```
Sum of Exited = 2038
Sum of Complain = 2044
Percentage of complain over exited = 100.29%
```

EXPERT INSIGHT

SPRINT book

RAG и генеративный ИИ

Создаем собственные RAG-пайплайны с помощью LlamaIndex, Deep Lake и Pinecon



Дэнис Ротман

packt

Теперь создадим на основе DataFrame `data1` тепловую карту:

```
import seaborn as sns
import matplotlib.pyplot as plt
# Выбираем для тепловой карты корреляций только численные столбцы
numerical_columns = data1.select_dtypes(include=['float64', 'int64']).columns
# Тепловая карта корреляций
plt.figure(figsize=(12, 8))
sns.heatmap(data1[numerical_columns].corr(), annot=True, fmt='.2f',
  cmap='coolwarm')
plt.title('Correlation Heatmap')
plt.show()
```

Максимальная корреляция наблюдается между `Complain` и `Exited` (рис. 6.4).

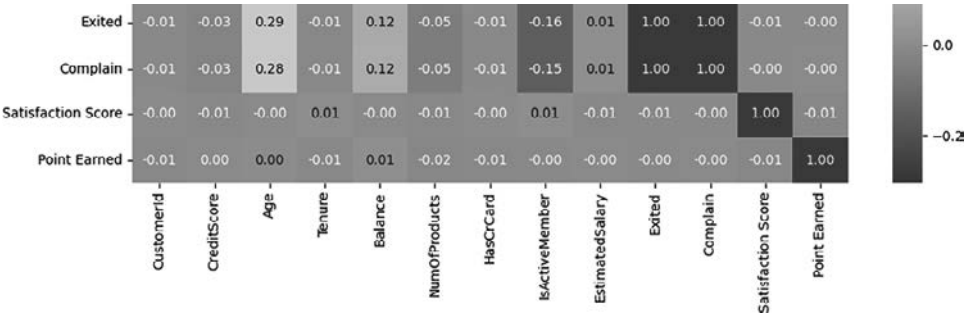


Рис. 6.4. Фрагмент тепловой карты

Представленная тепловая карта демонстрирует корреляцию между каждой парой признаков (переменных) в датасете. Она показывает коэффициент корреляции между каждой парой переменных, охватывающих диапазон от –1 (низкая корреляция) до 1 (высокая), где 0 указывает на отсутствие корреляции.

Таким образом, мы изучили несколько признаков. Теперь создадим модель ML для более детального анализа.

Обучение модели ML

Продолжим анализ и теперь более детально разберем датасет с помощью ML. В этом разделе мы обучим модель, используя технику кластеризации по методу *k*-средних, после чего применим ее для поиска паттернов в наших данных. Сначала подготовим и обработаем данные для анализа, затем применим к ним кластеризацию и в завершение оценим результаты с помощью различных метрик. Этот подход хорошо работает для получения представления без привлечения более сложных методов глубокого обучения.

ПРИМЕЧАНИЕ

Кластеризация по методу *k*-средних — это алгоритм ML для обучения модели без учителя. Он заключается в разделении датасета на *k* отдельных, не пересекающихся кластеров путем минимизации дисперсии в каждом из них. Этот алгоритм итеративно присваивает точки данных одному из *k* кластеров на основе ближайшего среднего (центроида), который повторно вычисляется после каждой итерации до момента схождения.

Теперь пошагово разберем код.

Подготовка и кластеризация данных

Сначала скопируем датасет `data1` в `data2`, чтобы иметь возможность вернуться к `data1`, если решим попробовать другие модели ML:

```
# Копирование data1 в data2
data2 = data1.copy()
```

Данные можно изучать, используя различные наборы признаков. В этом случае мы выберем `'CreditScore'`, `'Age'`, `'EstimatedSalary'`, `'Exited'`, `'Complain'` и `'Point Earned'`:

```
# Импортируем библиотеки
import pandas as pd
from sklearn.cluster import KMeans
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import silhouette_score , davies_bouldin_score
# Предполагается, что вы загрузили DataFrame data1 по инструкции
# Выбираем релевантные признаки
features = data2[['CreditScore', 'Age', 'EstimatedSalary', 'Exited',
  'Complain', 'Point Earned']]
```

Далее, прежде чем выполнять модель ML, масштабируем используемые признаки:

```
# Стандартизируем признаки
scaler = StandardScaler()
features_scaled = scaler.fit_transform(features)
```

Кредитный балл, примерная зарплата и заработанные очки, отражающие траты по кредитной карте, хорошо демонстрируют финансовое положение клиента в банке. Возраст в сочетании с перечисленными факторами показывает, что возрастные клиенты более склонны продолжать пользоваться услугами банка, чем молодые. Однако важно учесть, что жалобы могут поспособствовать уходу клиентов из любой группы, поскольку этот критерий очень ярко коррелирует с оттоком.

Теперь попробуем сформировать от двух до четырех кластеров, чтобы определить их оптимальное число при данном наборе признаков:

```
# Пробуем разное число кластеров
for n_clusters in range(2, 5):    # Пример диапазона от 2 до 5
    kmeans = KMeans(n_clusters=n_clusters, n_init=20, random_state=0)
    cluster_labels = kmeans.fit_predict(features_scaled)
    silhouette_avg = silhouette_score(features_scaled, cluster_labels)
    db_index = davies_bouldin_score(features_scaled, cluster_labels)
    print(f'For n_clusters={n_clusters}, the silhouette score is
      {silhouette_avg:.4f} and the Davies-Bouldin Index is {db_index:.4f}')
```

Вывод отражает оценку эффективности деления данных на различное число кластеров (от 2 до 4), используя две метрики — коэффициент силуэта (silhouette score) и индекс Дэвиса — Болдина:

```
For n_clusters=2, the silhouette score is 0.6129 and the Davies-Bouldin Index
is 0.6144
For n_clusters=3, the silhouette score is 0.3391 and the Davies-Bouldin Index
is 1.1511
For n_clusters=4, the silhouette score is 0.3243 and the Davies-Bouldin Index
is 1.0802
```

ПРИМЕЧАНИЕ

Коэффициент силуэта. Эта метрика оценивает качество кластеризации путем вычисления среднего расстояния внутри кластера (насколько каждая точка кластера близка к другим его точкам) и среднего расстояния до ближайшего кластера (насколько каждая точка кластера близка к точкам ближайшего кластера). Этот коэффициент может составлять от –1 до 1, где высокое значение говорит о достаточной отделенности кластера и его внутренней связности. В нашем случае максимальный показатель равен 0.6129 при двух кластерах, то есть в этом случае кластеризация получается наиболее эффективной.

Индекс Дэвиса — Болдина. Показывает качество кластеризации, сравнивая соотношение расстояний внутри кластера с расстояниями между кластерами. Более низкие значения говорят о лучшей кластеризации, так как подразумевают меньшую дисперсию внутри кластера и хорошую отдаленность кластеров друг от друга. Минимальный индекс в нашем выводе равен 0.6144 при двух кластерах. Это говорит о том, что такая конфигурация обеспечит наиболее эффективное разделение точек данных среди всех рассмотренных.

Полученные коэффициент силуэта и индекс Дэвиса — Болдина при двух кластерах предполагают относительно хорошую степень кластеризации. Причем по мере увеличения числа кластеров до трех и четырех обе метрики показывают ухудшение, то есть менее разделенные и менее связанные кластеры.

Реализация и оценка кластеризации

Поскольку оптимальным выбором для нашего датасета и набора признаков будут два кластера, выполним модель с `n_clusters=2`:

```
# Выполняем кластеризацию по методу k-средних с установленным числом кластеров
kmeans = KMeans(n_clusters=2, n_init=10, random_state=0)   # Явно устанавливая
                                                           # n_init на 10
data2['class'] = kmeans.fit_predict(features_scaled)

# Выводим первые несколько строк DataFrame для подтверждения
# того, что это столбец 'class'
data2
```

Опять же, как говорилось в подразделе «Изучение и анализ данных (EDA)» ранее в этой главе, между жалобами и оттоком клиентов наблюдается уверенная корреляция. Наглядно это подтверждает DataFrame на рис. 6.5.

Exited	Complain	Satisfaction Score	Card Type	Point Earned	class
1	1	2	DIAMOND	464	0
0	1	3	DIAMOND	456	0
1	1	3	DIAMOND	377	0
0	0	5	GOLD	350	1
0	0	5	GOLD	425	1
...	...	...	...	...	...
0	0	1	DIAMOND	300	1
0	0	5	PLATINUM	771	1
1	1	3	SILVER	564	0
1	1	2	GOLD	339	0
0	0	3	DIAMOND	911	1

Рис. 6.5. Фрагмент вывода после кластеризации по методу *k*-средних

Первый кластер — это `class=0`. Он представляет клиентов, которые подавали жалобы (`Complain`) и закрыли счета (`Exited`).

Если подсчитать строки, где `'class' == 0` и `'Exited' == 1`, то мы заметим явную связь между жалобами и уходом клиентов:

```
# 1. Суммируем строки, где 'class' == 0
sum_class_0 = (data2['class'] == 0).sum()
```

```
# 2. Суммируем строки, где 'class' == 0 и 'Complain' == 1
sum_class_0_complain_1 = data2[(data2['class'] == 0) &
  (data2['Complain'] == 1)].shape[0]
# 3. Суммируем строки, где 'class' == 0 и 'Exited' == 1
sum_class_0_exited_1 = data2[(data2['class'] == 0) &
  (data2['Exited'] == 1)]. shape[0]

# Вывод результатов
print(f"Sum of 'class' == 0: {sum_class_0}")
print(f"Sum of 'class' == 0 and 'Complain' == 1: {sum_class_0_complain_1}")
print(f"Sum of 'class' == 0 and 'Exited' == 1: {sum_class_0_exited_1}")
```

Вывод подтверждает тесную связь между жалобами клиентов и их оттоком:

```
Sum of 'class' == 0: 2039
Sum of 'class' == 0 and 'Complain' == 1: 2036
Sum of 'class' == 0 and 'Exited' == 1: 2037
```

Следующая ячейка кода посвящена второму классу, где `'class' == 1` и `'Complain' == 1`. Она также подтверждает, что клиенты с жалобами редко продолжают пользоваться услугами банка:

```
# 2. Суммируем строки, где 'class' == 1 и 'Complain' == 1
sum_class_1_complain_1 = data2[(data2['class'] == 1) &
  (data2['Complain'] == 1)].shape[0]
```

Полученный вывод согласуется с нашими наблюдениями:

```
Sum of 'class' == 1: 7961
Sum of 'class' == 1 and 'Complain' == 1: 8
Sum of 'class' == 1 and 'Exited' == 1: 1
```

Мы увидели, что довольно сложно найти признаки, которые помогут удержать клиентов классическими эффективными методами. Так что наша дальнейшая стратегия будет заключаться в преобразовании записей о клиентах в векторы с помощью OpenAI и обращении к индексу Pinecone для нахождения в датасете других паттернов с помощью запросов, выходящих за рамки датасета.

Компонент 2: масштабирование индекса Pinecone

Цель текущего раздела — создать для нашего датасета индекс Pinecone и масштабировать его с 10 000 до 1 000 000 записей. И хотя далее мы будем опираться на знания, полученные в предыдущих главах, по своей природе масштабирование отличается от управления образцами датасетов.

Каждый процесс этого компонента пайплайна может показаться простым: подготовка данных, создание эмбеддингов, загрузка в векторную БД и последующий запрос из нее документов. Все эти процессы мы уже разобрали в главах 2 и 3.

К тому же, за исключением реализации Pinecone вместо Deep Lake и несколько иного использования моделей OpenAI, в части векторной базы данных мы реализуем те же функции, что и в главах 2–4.

1. *Подготовка данных*. Начнем с подготовки датасета — его деления на чанки с помощью Python.
2. *Создание эмбеддингов*. Разбитые на чанки данные преобразуем в векторы.
3. *Создание индекса Pinecone*. Далее создадим индекс Pinecone (векторную базу данных).
4. *Загрузка (upsert)*. Загрузим эмбеддинги документов — в данном случае записи о клиентах и текст каждой записи в виде метаданных.
5. *Запрос к индексу Pinecone*. В завершение запросим релевантные документы для подготовки к следующему этапу, «Компонент 3: генеративный ИИ на базе RAG».

ПРИМЕЧАНИЕ

При необходимости еще раз пробегитесь по главам 2–4, чтобы освежить в памяти подготовку данных, разбивку на чанки, создание эмбеддингов и запрос к БД.

Мы знаем, как реализовывать каждый из перечисленных шагов, поскольку уже проделывали все это с Deep Lake. Но ведь Pinecone — такая же векторная база данных, тогда в чем подвох? Основной загвоздкой оказываются скрытые проблемы реальных проектов, которые мы будем разбирать, начав с размера, затрат и сопутствующих операций.

Сложности управления векторными базами данных

Как правило, мы начинаем очередной раздел сразу с кода. Это нормальный подход при небольших объемах данных, но при масштабировании сначала необходимо принять продуманные решения по управлению проектом. Почему? Когда программа, основанная на неудачном решении или содержащая ошибку, применяется к небольшим датасетам, последствия не особо заметны. Но в случае больших масштабов все сильно меняется. Фундаментальный принцип и риск масштабирования заключается в том, что ошибки тоже экспоненциально разрастаются.

Далее перечислим основные проблемные точки, которые вам нужно оценить, прежде чем выполнять какой-либо код. Эту методологию можно применять к любой платформе или модели. В текущей главе такими платформами будут только OpenAI и Pinecone, поскольку нас интересует сам процесс, а не управление платформами. Использование других платформ подразумевает внимательное управление рисками, что не является задачей текущей главы.

Начнем с моделей OpenAI.

- *Модели OpenAI для создания эмбеддингов*. Разработчики OpenAI постоянно вносят какие-то доработки и предлагают новые модели для эмбеддинга данных. Прежде чем делать выбор, обязательно ознакомьтесь на странице https://platform.openai.com/docs/models/embeddings с характеристиками каждой такой модели, включая ее скорость, стоимость, ограничения на ввод и частоту запросов к API.
- *Модели OpenAI для генерации*. Компания OpenAI регулярно выпускает новые модели, отказываясь от устаревших. В Google делают так же. Можно сравнить эти модели с гоночными автомобилями. Удастся ли вам победить в гонке современных автомобилей, если вы сядете за руль авто 1930 года? При масштабировании требуются самые эффективные модели, поэтому нужно оценивать их скорость, ограничения на ввод, размер вывода и допустимую частоту запросов к API. Вся эта информация доступна на сайте https://platform.openai.com/docs/models.

Это означает, что при масштабировании ради повышения скорости и уменьшения затрат нужно постоянно принимать в расчет эволюцию моделей. Кроме того, вы должны в реальном времени видеть перспективы текущих расходов и учитывать возможные технические ограничения:

- *управление тарифами* — https://platform.openai.com/settings/organization/billing/overview;
- *лимиты* — https://platform.openai.com/settings/organization/limits.

Теперь разберем ограничения Pinecone (потребуется создать учетную запись).

- *Облако и регион*. Выбор облачного сервиса (AWS, Google и пр.) и региона (места расположения бессерверной базы данных) влияет на итоговую стоимость.
- *Потребление*. Сюда входят затраты ресурсов на чтение/запись (read units/write units), а также стоимость хранения, включая облачные резервные копии. Подробнее читайте на странице https://docs.pinecone.io/guides/indexes/back-up-an-index.

Как и в любой облачной среде, вам необходимо постоянно мониторить стоимость и потребление услуг Pinecone. Делать это можно на страницах https://www.pinecone.io/pricing/ и https://docs.pinecone.io/guides/operations/monitoring.

Сценарий, который мы реализуем, — лишь один из многих путей достижения целей текущей главы с помощью других платформ и фреймворков. Однако сопутствующие ограничения, включая стоимость услуг, затраты ресурсов, скорость и лимиты, всегда будут одними и теми же.

Далее реализуем компонент 2, сосредоточившись на проблемных точках, выходящих за рамки функциональности, с которой мы познакомились ранее. Откройте блокнот `Pipeline_2_Scaling_a_Pinecone_Index.ipynb` в репозитории GitHub. Начнем с настройки среды.

Настройка среды

Как уже говорилось, у нас в программе используются только Pinecone и OpenAI, что позволяет избежать промежуточного ПО, платформ и ограничений. Сохраните ключ API в безопасном месте — в данном случае это Google Drive:

```
# Ключ API
# Сохраните ключ в файле и считывайте его оттуда (можете ввести его прямо
# в блокнот, но тогда он будет виден находящимся рядом людям)
from google.colab import drive
drive.mount('/content/drive')
```

Далее установим OpenAI и Pinecone:

```
!pip install openai==1.40.3
!pip install pinecone-client==5.0.1
```

В завершение инициализируем ключи API:

```
f = open("drive/MyDrive/files/pinecone.txt", "r")
PINECONE_API_KEY=f.readline()
f.close()
f = open("drive/MyDrive/files/api_key.txt", "r")
API_KEY=f.readline()
f.close()

# Ключ OpenAI
import os
import openai
os.environ['OPENAI_API_KEY'] =API_KEY
openai.api_key = os.getenv("OPENAI_API_KEY")
```

Теперь перейдем к обработке датасета `Bank Customer Churn`.

Обработка датасета

В этом разделе мы подготовим датасет для разбивки на оптимизированные чанки текста, которые впоследствии будут преобразованы в векторы. Первым делом извлекаем датасет `data1.csv`, который подготовили и сохранили в разделе «Компонент 1: сбор и подготовка данных»:

```
!cp /content/drive/MyDrive/files/rag_c6/data1.csv /content/data1.csv
```

Далее загрузим датасет в DataFrame:

```
import pandas as pd
# Загружаем CSV-файл
file_path = '/content/data1.csv'
data1 = pd.read_csv(file_path)
```

Следим, чтобы было загружено 10 000 строк:

```
# Подсчет чанков
number_of_lines =  len(data1)
print("Number of lines: ",number_of_lines)
```

Вывод подтверждает присутствие всех строк:

```
Number of lines: 10000
```

В этом сценарии очень важен следующий код, который преобразует каждую строку с записью о клиенте в строку списка `output_lines`:

```
import pandas as pd

# Инициализируем пустой список для сохранения строк
output_lines = []

# Перебираем каждую строку DataFrame
for index, row in data1.iterrows():
    # Создаем список "column_name: value" всех имеющихся в строке столбцов
    row_data = [f"{col}: {row[col]}" for col in data1.columns]
    # Объединяем полученный список в строку, разделяя содержимое пробелами
    line = ' '.join(row_data)
    # Добавляем строку к итоговому списку
    output_lines.append(line)

# Выводим или дополнительно обрабатываем `output_lines`
for line in output_lines[:5]:   # Выводим первые пять строк для превью
    print(line)
```

Вывод показывает, что каждая строка в списке `output_lines` содержит текст отдельной записи о клиенте:

```
CustomerId: 15634602 CreditScore: 619 Age: 42 Tenure: 2 Balance: 0.0
NumOfProducts: 1 HasCrCard: 1 IsActiveMember: 1 EstimatedSalary: 101348.88
Exited: 1 Complain: 1 Satisfaction Score: 2 Card Type: DIAMOND Point Earned: 464…
```

Здесь мы видим, что каждая строка содержит конкретную запись о клиенте. Теперь скопируем `output_lines` в `lines`, чтобы впоследствии создать из этих строк чанки:

```
lines = output_lines.copy()
```

Код перепроверяет список `lines`, убеждаясь в присутствии всех строк:

```
# Подсчет строк
number_of_lines =  len(lines)
print("Number of lines: ",number_of_lines)
```

Вывод подтверждает, что все 10 000 строк на месте:

```
Number of lines: 10000
```

Теперь данные готовы к разбивке на чанки.

Создание чанков и их преобразование в эмбеддинги

В текущем разделе мы разделим текст на предчанки и внесем их в список `lines`. Не всегда есть возможность создать список предчанков, но, если это доступно, таким образом можно повысить прозрачность модели, эффективность отслеживания данных и скорость обработки запросов. Сам же процесс создания чанков довольно прост.

Создание чанков

Практика разбивки текста на предчанки — важный элемент управления датасетами. Мы можем создать чанки из списка предчанков, сохраненных ранее в виде строк:

```
# Инициализируем пустой список для чанков
chunks = []

# Добавляем каждый чанк в виде отдельной строки
for line in lines:
    chunks.append(line)    # Каждая строка становится собственным чанком

# Теперь каждая строка рассматривается как отдельный чанк
print(f"Total number of chunks: {len(chunks)}")
```

Вывод показывает, что все данные были успешно обработаны:

```
Total number of chunks: 10000
```

Но зачем вообще заморачиваться созданием чанков? Почему бы просто не использовать сами строки? Во многих случаях строки могут потребовать дополнительного контроля и обработки, например, если есть ошибки данных, упущенные на предыдущих этапах. У нас даже может быть несколько чанков, выходящих

за текущий предел ввода (который постоянно растет) модели создания эмбеддингов.

Чтобы лучше понять структуру разбитых на фрагменты данных, оцениваем длину и содержимое полученных чанков:

```
# Выводим длину содержимого первых 10 чанков
for i in range(3):
    print(len(chunks[i]))
    print(chunks[i])
```

Вывод позволит специалисту визуально оценить полученные чанки данных:

```
224
CustomerId: 15634602 CreditScore: 619 Age: 42 Tenure: 2 Balance: 0.0
NumOfProducts: 1 HasCrCard: 1 IsActiveMember: 1 EstimatedSalary: 101348.88
Exited: 1 Complain: 1 Satisfaction Score: 2 Card Type: DIAMOND Point Earned: 464…
```

Теперь преобразуем эти чанки в векторы.

Создание эмбеддингов

В этом разделе потребуется прибегнуть к тщательному тестированию и учесть возможные проблемы. Здесь мы поймем, что *масштабирование требует скорее размышлений, чем действий*. Каждому проекту для обеспечения эффективных ответов потребуется привлечь на этапах проектирования и тестов определенный объем данных. Нужно также учесть затраты и пользу от каждого компонента пайплайна. Например, инициализация модели создания эмбеддингов — это довольно сложная задача.

На момент написания книги OpenAI предлагает три модели создания эмбеддингов, которые можно протестировать:

```
import openai
import time

embedding_model="text-embedding-3-small"
#embedding_model="text-embedding-3-large"
#embedding_model="text-embedding-ada-002"
```

Здесь мы используем `text-embedding-3-small`. Но вы можете проверить и другие, просто раскомментировав соответствующие строки. Функция `embedding` будет получать выбранную вами модель:

```
# Инициализируем клиент OpenAI
client = openai.OpenAI()

def get_embedding(text, model=embedding_model):
    text = text.replace("\n", " ")
```

```
    response = client.embeddings.create(input=[text], model=model)
    embedding = response.data[0].embedding
    return embedding
```

Прежде чем использовать какую-либо модель, обязательно проверьте сопутствующие затраты и предоставляемые ею возможности: https://platform.openai.com/docs/guides/embeddings/embedding-models.

Далее мы преобразуем чанки в эмбеддинги, но этот процесс требует продуманных решений, в частности, для эффективного управления большими датасетами с учетом допустимой частоты вызовов API. В данном случае создадим пакеты чанков, которые преобразуем в эмбеддинги:

```
import openai
import time

# Инициализация клиента OpenAI
client = openai.OpenAI()

# Инициализация переменных
start_time = time.time()    # Начало отсчета перед запросом
chunk_start = 0
chunk_end = 1000
pause_time = 3
embeddings = []
counter = 1
```

Будем преобразовывать по 1000 чанков за раз, установив `chunk_start = 0` и `chunk_end = 1000`. Чтобы избежать возможных ограничений OpenAI на частоту запросов API, мы добавили `pause_time = 3`, то есть между отправкой пакетов станем делать паузу в 3 с. Эмбеддинги будут сохраняться в `embeddings = []`, а подсчет пакетов будет вестись, начиная с `counter = 1`.

Код разделен на три основные части.

- Перебор всех пакетов чанков:

```
while chunk_end <= len(chunks):
    # Выбор текущего пакета чанков
    chunks_to_embed = chunks[chunk_start:chunk_end]…
```

- Преобразование пакета `chunks_to_embed`:

```
for chunk in chunks_to_embed:
    embedding = get_embedding(chunk, model=embedding_model)
    current_embeddings.append(embedding)…
```

- Обновление начального и конечного значений чанков в очередном пакете:

```
# Обновление индексов чанков
    chunk_start += 1000
    chunk_end += 1000
```

Кроме того, была добавлена функция для обработки последнего пакета, размер которого может не соответствовать установленному:

```
# Обработка оставшихся чанков
if chunk_end < len(chunks):
    remaining_chunks = chunks[chunk_end:]
    remaining_embeddings = [get_embedding(chunk, model=embedding_model) for
      chunk in remaining_chunks]
    embeddings.extend(remaining_embeddings)
```

В выводе отражаются счетчик и время обработки:

```
All chunks processed.
Batch 1 embedded.
...
Batch 10 embedded.
Response Time: 2689.46 seconds
```

Время выполнения получилось значительным, и для каждого раза оно будет иным, но при масштабировании это естественно. Глупо ожидать быстрой обработки больших объемов данных без просадки производительности.

При желании в качестве проверки можно вывести полученное векторное представление:

```
print("First embedding:", embeddings[0])
```

Вот первое:

```
First embedding: [-0.024449337273836136, -0.00936567410826683,…
```

Теперь проверим, совпадает ли у нас число чанков и полученных векторов:

```
# Проверяем количество чанков и эмбеддингов
num_chunks = len(chunks)
print(f"Number of chunks: {num_chunks}")
print(f"Number of embeddings: {len(embeddings)}")
```

Вывод подтверждает, что можно переходить в Pinecone:

```
Number of chunks: 10000
Number of embeddings: 10000
```

На этом разбивка и векторизация данных завершена. Далее мы продублируем эти данные, сымитировав таким образом масштабирование.

Дублирование данных

Дублирование полученных векторных данных позволяет получить большой объем, не доплачивая за услуги OpenAI. Стоимость создания эмбеддингов напрямую связана с затратами времени. Таким образом, мы можем сымитировать

масштабирование, например, до 50 000 точек данных, после чего в той же пропорции увеличить время выполнения и стоимость.

Код будет простым. Сначала определим количество повторов данных:

```
# Определяем количество дублей
dsize = 5    # Можно установить любое значение от 1 до n
total=dsize * len(chunks)
print("Total size", total)
```

Далее программа продублирует чанки и соответствующие эмбеддинги:

```
# Инициализируем новые списки для продублированных чанков и эмбеддингов
duplicated_chunks = []
duplicated_embeddings = []

# Перебираем исходные списки и дублируем каждую запись
for i in range(len(chunks)):
    for _ in range(dsize):
        duplicated_chunks.append(chunks[i])
        duplicated_embeddings.append(embeddings[i])
```

Затем проверяем, совпадает ли количество чанков с количеством эмбеддингов:

```
# Проверяем размер продублированных списков
print(f"Number of duplicated chunks: {len(duplicated_chunks)}")
print(f"Number of duplicated embeddings: {len(duplicated_embeddings)}")
```

Наконец, вывод подтверждает, что данные были продублированы пять раз:

```
Total size 50000
Number of duplicated chunks: 50000
Number of duplicated embeddings: 50000
```

Надо признать, что 50 000 точек данных — хороший стартовый объем, достаточный для заполнения векторной БД. Теперь создадим индекс Pinecone.

Создание индекса Pinecone

В первую очередь нужно инициализировать ключ API, используя выбранное имя переменной, после чего создать экземпляр Pinecone:

```
import os
from pinecone import Pinecone, ServerlessSpec
# Инициализация подключения к Pinecone (получите ключ API на app.pinecone.io)
api_key = os.environ.get('PINECONE_API_KEY') or 'PINECONE_API_KEY'
pc = Pinecone(api_key=PINECONE_API_KEY)
```

Экземпляр Pinecone `pc` создан. Теперь выберем имя индекса, облако и регион:

```
from pinecone import ServerlessSpec
index_name = [YOUR INDEX NAME]    # Например, 'bank-index-900'
cloud = os.environ.get('PINECONE_CLOUD') or 'aws'
region = os.environ.get('PINECONE_REGION') or 'us-east-1'
spec = ServerlessSpec(cloud=cloud, region=region)
```

Здесь мы указали, что будем использовать бессерверный экземпляр (`spec`) AWS в регионе `'us-east-1'`. Теперь можно создавать индекс (тип векторной базы данных) под названием `'bank-index-50000'`:

```
import time
import pinecone
# Проверяем, существует ли такой индекс
# (если он создается впервые, то его не должно быть)
if index_name not in pc.list_indexes().names():
    # Если не существует, создаем
    pc.create_index(
        index_name,
        dimension=1536,    # Размерность модели эмбеддинга
        metric='cosine',
        spec=spec
    )
    # Ожидаем инициализации индекса
    time.sleep(1)
# Подключаемся к индексу
index = pc.Index(index_name)
# Просматриваем статистику индекса
index.describe_index_stats()
```

В этом коде мы добавили в `index_name` и `spec` два параметра:

- `dimension=1536`, выражающий длину вектора эмбеддингов, которую можно подстраивать под выбранную модель;
- `metric='cosine'`, используемый для вычисления векторного сходства между эмбеддингами. Можно выбрать и другие метрики, например евклидово расстояние (https://www.pinecone.io/learn/vector-similarity/).

После создания индекса программа выводит его описание:

```
{'dimension': 1536,
'index_fullness': 0.0,
'namespaces': {},
'total_vector_count': 0}
```

Количество векторов и заполненность индекса равны 0, так как векторное хранилище мы еще не заполняли. Отлично, теперь можно приступать к загрузке!

Загрузка

В этом разделе мы внесем в векторную базу данных 50 000 векторов и связанные с ними метаданные (чанки). Наша цель — хорошо понять процесс масштабирования и использовать синтаксические данные для оперирования данными в объемах более 50 000 векторов. Вы можете вернуться к предыдущему разделу и дополнительно продублировать данные для получения любого желаемого объема. Но нужно помнить, что время загрузки (операция `upsert`) в индекс Pinecone

растет линейно. То есть для получения примерного времени выполнения вам достаточно просто увеличить его в то же количество раз, в которое увеличился объем данных. Прежде чем выполнять загрузку, ознакомьтесь с тарифами Pinecone: https://www.pinecone.io/pricing/.

Мы загрузим данные в хранилище, используя три поля:

- `ids` — содержит уникальный идентификатор для каждого чанка. Этот ID будет выступать в качестве счетчика и при загрузке данных инкрементироваться;
- `embedding` — содержит созданные эмбеддинги;
- `chunks` — содержит чанки в виде простого текста метаданных.

Теперь нужно организовать пакетную загрузку этих данных в БД. Для начала определим функцию загрузки:

```
# Функция загрузки
def upsert_to_pinecone(data,  batch_size):
    for i in range(0, len(data), batch_size):
        batch = data[i:i+batch_size]
        index.upsert(vectors=batch)
        #time.sleep(1)   # По желанию можете добавить задержку,
                         # чтобы избежать ограничения на частоту запросов
```

Будем также измерять время, необходимое для обработки корпуса:

```
import pinecone
import time
import sys
start_time = time.time()    # Начинаем отсчет перед запросом
```

Теперь создадим функцию, которая будет вычислять размер пакетов, ограничивая его до 4 Мбайт. Это примерно равно текущему лимиту Pinecone:

```
# Функция для вычисления размера пакета
# На всякий случай лимит установлен чуть ниже 4 Мбайт
def get_batch_size(data, limit=4000000):
    total_size = 0
    batch_size = 0
    for item in data:
        item_size = sum([sys.getsizeof(v) for v in item.values()])
        if total_size + item_size > limit:
            break
        total_size += item_size
        batch_size += 1
    return batch_size
```

Создадим саму функцию `upsert`:

```
def batch_upsert(data):
    total = len(data)
    i = 0
    while i < total:
```

```
        batch_size = get_batch_size(data[i:])
        batch = data[i:i + batch_size]
        if batch:
            upsert_to_pinecone(batch,batch_size)
            i += batch_size
            print(f"Upserted {i}/{total} items...")  # Выводим текущий прогресс
        else:
            break
    print("Upsert complete.")
```

Для загружаемых данных нужно сгенерировать уникальные ID:

```
# Генерируем ID для каждого элемента данных:
ids = [str(i) for i in range(1, len(duplicated_chunks) + 1)]
```

Создадим также метаданные для загрузки датасета в БД Pinecone:

```
# Подготавливаем данные к загрузке
data_for_upsert = [
    {"id": str(id), "values": emb, "metadata": {"text": chunk}}
    for id, (chunk, emb) in zip(ids, zip(duplicated_chunks, duplicated_embeddings))
]
```

Теперь у нас есть все необходимое для загрузки `data_for_upsert`:

- `"id": str(ids[i])` содержит ID, которые мы создали, оттолкнувшись от начального числа;
- `"values": emb` содержит чанки, которые были преобразованы в векторы;
- `"metadata": {"text": chunk}` содержит сами чанки.

Далее выполняем загрузку:

```
# Загрузка данных пакетами
batch_upsert(data_for_upsert)
```

Наконец, измеряем затраченное время:

```
# Измеряем время ответа
response_time = time.time() - start_time
# Выводим время ответа
print(f"Upsertion response time: {response_time:.2f} seconds")
```

Вывод содержит полезную информацию, которая показывает прогресс загрузки:

```
Upserted 316/50000 items...
Upserted 632/50000 items...
Upserted 948/50000 items...
…
Upserted 49612/50000 items...
Upserted 49928/50000 items...
Upserted 50000/50000 items...
Upsert complete.
Upsertion response time: 560.66 seconds
```

Здесь мы видим, что на загрузку 10 000 точек данных уходит около 1 мин (56 с). Попробуйте обработать корпус побольше — время должно расти линейно.

Можно также просмотреть статистику индекса, чтобы увидеть, сколько конкретно векторов было загружено:

```
print("Index stats")
print(index.describe_index_stats(include_metadata=True))
```

Вывод подтверждает успешность процесса загрузки:

```
Index stats
{'dimension': 1536,
 'index_fullness': 0.0,
 'namespaces': {'': {'vector_count': 50000}},
 'total_vector_count': 50000}
```

Запрос к индексу Pinecone

Теперь нужно проверить время обработки масштабированного индекса Pinecone. Создадим функцию, которая будет обращаться к векторной базе данных и выводить результаты:

```
# Выводим результаты запроса и метаданные
def display_results(query_results):
  for match in query_results['matches']:
    print(f"ID: {match['id']}, Score:  {match['score']}")
    if 'metadata' in match and 'text' in match['metadata']:
        print(f"Text: {match['metadata']['text']}")
    else:
        print("No metadata available.")
```

Нам нужно, чтобы функция обработки запроса использовала ту же модель создания эмбеддингов, с помощью которой мы преобразовывали чанки датасета:

```
embedding_model = "text-embedding-3-small"
def get_embedding(text, model=embedding_model):
    text = text.replace("\n", " ")
    response = client.embeddings.create(input=[text], model=model)
    embedding = response.data[0].embedding
    return embedding
```

Теперь попросим базу данных Pinecone выполнить модульный тест, после чего вывести результаты и время выполнения. Сначала инициализируем клиент OpenAI и запускаем отсчет:

```
import openai
# Инициализация клиента OpenAI
client = openai.OpenAI()
print("Querying vector store")
start_time = time.time()    # Начинаем отсчет перед запросом
```

Далее отправляем к БД запрос с профилем клиента, которого в датасете нет:

```
query_text = "Customer Robertson CreditScore 632Age 21 Tenure 2Balance
0.0NumOfProducts 1HasCrCard 1IsActiveMember 1EstimatedSalary 99000 Exited
1Complain 1Satisfaction Score 2Card Type DIAMONDPoint Earned 399"
```

Запрос преобразуется в вектор с помощью той же модели, что использовалась для преобразования датасета:

```
query_embedding = get_embedding(query_text,model=embedding_model)
```

Выполняем запрос и выводим результат:

```
query_results = index.query(vector=query_embedding, top_k=1, include_
metadata=True)    # Запрашиваем метаданные
#print("raw query_results",query_results)
print("processed query results")
display_results(query_results)    # Выводим результаты
response_time = time.time() - start_time    # Измеряем время ответа
# Выводим время ответа
print(f"Querying response time: {response_time:.2f} seconds")
```

В выводе отражаются сам ответ и время выполнения:

```
Querying vector store
Querying vector store
processed query results
ID: 46366, Score: 0.823366046
Text: CustomerId: 15740160 CreditScore: 616 Age: 31 Tenure: 1 Balance: 0.0
NumOfProducts: 2 HasCrCard: 1 IsActiveMember: 1 EstimatedSalary: 54706.75
Exited: 0 Complain: 0 Satisfaction Score: 3 Card Type: DIAMOND Point Earned: 852
Querying response time: 0.74 seconds
```

Здесь мы видим, что качество ответа удовлетворительное, так как был найден похожий профиль. Время выполнения отличное — всего 0,74 с. К слову, при достижении 1 000 000 векторов время должно оставаться постоянным в пределах 1 с. В этом заключается магия индекса Pinecone!

Если перейти на страницу организации (https://app.pinecone.io/organizations/) и щелкнуть на индексе, то можно просмотреть статистику, проанализировать потребление и не только (рис. 6.6).



Рис. 6.6. Визуализация количества векторов в индексе посредством консоли Pinecone

Теперь используем созданный индекс для дополнения ввода и генерации контента.

Компонент 3: генеративный ИИ на базе RAG

В этом разделе мы с помощью генеративного ИИ автоматически создадим персонализированное рекламное обращение к клиентам банка, нацеленное на сохранение их лояльности. При этом используем ранее созданные программы для подготовки данных и генерации индекса Pinecone. Кроме того, задействуем возможности этой БД для реализации расширенной функциональности поиска. При составлении запроса к Pinecone будем выбирать целевой вектор, представляющий определенный сегмент клиентов. В результате обработки полученного ответа будет извлечено *k* наиболее схожих векторов. Затем мы дополним ввод, выполненный пользователем, полученной информацией, попросив OpenAI создать индивидуальные рекомендации для целевого сегмента клиентов.

Откройте `Pipeline-3_RAG_Generative AI.ipynb` на GitHub. Первая секция кода в этом блокноте называется «Настройка среды» (Installing the environment) и совпадает с аналогичной секцией в блокноте `2-Pincone_vector_store-1M.ipynb`, реализованной в разделе «Компонент 2: масштабирование индекса Pinecone». Код создания индекса Pinecone во втором разделе также повторяет код в `2-Pincone_vector_store-1M.ipynb`, только на этот раз он дополнительно проверяет существование индекса и, если такой уже есть, просто к нему подключается.

Теперь выполним пример RAG, используя GPT-4o.

RAG с GPT-4o

В этой части кода мы обратимся к БД Pinecone, дополним полученной информацией ввод, выполненный пользователем, и сгенерируем ответ с помощью GPT-4o. Этот процесс будет аналогичен использованию генеративной модели на базе OpenAI и Deep Lake в главе 3. Однако природа и реализация запроса к Pinecone в этом случае будут отличаться, и вот почему.

- *Целевой вектор*. Здесь ввод, выполненный пользователем, — это не вопрос в его классической форме, а целевой вектор, представляющий профиль определенного сегмента клиентов.
- *Применение*. Суть запроса не в том, чтобы дополнить обращение к генеративному ИИ в привычном контексте диалога (вопросы, обобщения). Здесь нам нужно, чтобы модель GPT-4o составила для клиентов убедительное персонализированное письмо с предложением продуктов и услуг.
- *Время ответа*. При масштабировании приложения скорость играет решающую роль. Мы будем измерять время обработки запроса к индексу Pinecone, содержащему миллион векторов.

Запрос к датасету

Нам потребуется функция, которая будет преобразовывать ввод в эмбеддинг. В целях совместимости используем ту же модель, что и при создании эмбеддингов в разделе «Компонент 2: масштабирование индекса Pinecone»:

```
import openai
import time

embedding_model= "text-embedding-3-small"
# Инициализируем клиент OpenAI
client = openai.OpenAI()
def get_embedding(text, model=embedding_model):
    text = text.replace("\n", " ")
    response = client.embeddings.create(input=[text], model=model)
    embedding = response.data[0].embedding
    return embedding
```

Теперь можно делать запрос к индексу Pinecone.

Запрос целевого вектора

Целевой вектор представляет сегмент клиентов, для которых команда маркетологов хочет создать рекомендации с целью повышения лояльности. Здесь вас ограничивают только воображение и креативность. Как правило, маркетологи входят в состав команды проектирования этого компонента пайплайна. Вы можете провести рабочие встречи для обсуждения различных сценариев и выбора из них наиболее подходящего под задачи отдела маркетинга. Если же вы состоите в команде маркетологов, то вам нужно разработать целевые векторы. В любом случае высказанные людьми креативные идеи позволят придумать множество способов организации целевых векторов и запросов.

Здесь мы будем ориентироваться на сегмент клиентов в возрасте 42 лет (`Age 42`). Нам не нужно, чтобы их возраст был строго 42 или же ограничивался этим значением. Тут мы положимся на работу ИИ. Кроме того, нашей целью является клиент с примерным заработком больше 100 000 (`EstimatedSalary 101348.88`), потерять которого банк явно не хотел бы. При этом выбираем клиентов, которые имеют жалобы (`Complain 1`) и собираются уйти из банка (`Exited 1`). В этом случае `Exited 1` будет указывать на то, что клиент сделал запрос на закрытие счета, который еще не выполнен. Предположим также, что отдел маркетинга выбрал нужный целевой вектор.

`query_text` представляет профили клиентов, которые мы ищем:

```
import time
start_time = time.time()    # Начинаем отсчет перед запросом
# Целевой вектор
  "
# Целевой вектор
query_text = "Customer Henderson CreditScore 599 Age 37Tenure 2Balance
```

```
0.0NumOfProducts 1HasCrCard 1IsActiveMember 1EstimatedSalary 107000.88Exited
1Complain 1Satisfaction Score 2Card Type DIAMONDPoint Earned 501"

query_embedding = get_embedding(text,model=embedding_model)
```

Запрос преобразован в вектор. Теперь извлечем *k* профилей клиентов, максимально схожих с целевым вектором, и спарсим результат:

```
# Выполняем запрос, используя созданный эмбеддинг
query_results = index.query(
    vector=query_embedding,
    top_k=5,
    include_metadata=True,
)
```

Выводим ответ и сопутствующие метаданные:

```
# Выводим результат и метаданные
print("Query Results:")
for match in query_results['matches']:
    print(f"ID: {match['id']}, Score:  {match['score']}")
    if 'metadata' in match and 'text' in match['metadata']:
        print(f"Text: {match['metadata']['text']}")
    else:
        print("No metadata available.")

response_time = time.time() - start_time    # Измеряем время ответа
# Выводим время ответа
print(f"Querying response time: {response_time:.2f} seconds")
```

Полученный результат парсится в поиске *k* максимальных соответствий и их вывода вместе с показателем сходства:

```
Query Results:
ID: 46366, Score: 0.854999781
Text: CustomerId: 15740160 CreditScore: 616 Age: 31 Tenure: 1 Balance: 0.0
NumOfProducts: 2 HasCrCard: 1 IsActiveMember: 1 EstimatedSalary: 54706.75
Exited: 0 Complain: 0 Satisfaction Score: 3 Card Type: DIAMOND Point Earned: 852
Querying response time: 0.63 seconds
```

Мы получили ценную информацию:

- *ранжированные* `top-k` векторов, максимально близких к целевому. Причем в зависимости от целевого вектора модель ИИ будет каждый раз автоматически пересчитывать этот порядок;
- *показатель* соответствия в виде выбранной метрики;
- *содержимое* максимально схожих векторов и лучший показатель.

Это автоматизированный процесс под ключ! Здесь ИИ выходит на новый уровень, но все же нужен контроль специалиста, чтобы подтвердить полученный вывод.

Далее мы займемся извлечением релевантной информации для дополнения ввода.

Извлечение релевантных текстов

Код, приведенный далее, перебирает отобранные векторы, находит в них соответствующие текстовые метаданные и совмещает их, чтобы использовать для дополнения:

```
relevant_texts = [match['metadata']['text'] for match in query_
results['matches'] if 'metadata' in match and 'text' in match['metadata']]

# Объединяем все элементы списка в одну строку, разделенную заданным
# символом, например символом перевода строки или пробелом
# Для лучшей читаемости используем в качестве разделителя символ переноса строки
combined_text = '\n'.join(relevant_texts)
print(combined_text)
```

В выводе отображается `combined_text` — релевантный текст для дополнения ввода:

```
CustomerId: 15740160 CreditScore: 616 Age: 31 Tenure: 1 Balance: 0.0
NumOfProducts: 2 HasCrCard: 1 IsActiveMember: 1 EstimatedSalary: 54706.75
Exited: 0 Complain: 0 Satisfaction Score: 3 Card Type: DIAMOND Point Earned: 852
```

Теперь мы готовы дополнить промт и передать его ИИ.

Дополнение промта

Далее создадим промт, добавив в него три текста:

- `query_prompt` — инструкции для модели генеративного ИИ;
- `query_text` — целевой вектор, содержащий целевой профиль, выбранный маркетологами;
- `combined_context` — концентрированные метаданные похожих векторов, выбранных в результате запроса.

Все эти три переменные содержатся в `itext`:

```
# Совмещаем тексты в одну строку, разделенную символами перевода строки
combined_context = "\n".join(relevant_texts)
# Промт
query_prompt="I have this customer bank record with interesting information on
age, credit score and more and similar customers. What could I suggest to keep
them in my bank in an email with an url to get new advantages based on the fields
for each Customer ID:"
itext=query_prompt+ query_text+combined_context
# Дополненный ввод
print("Prompt for the Generative AI model:", itext)
```

Выводом будет базовый ввод для генеративного ИИ:

```
Prompt for GPT-4: I have this customer bank record with interesting information
on age, credit score and more and similar customers. What could I suggest to
keep them in my bank in an email with an url to get new advantages based on the
fields for each Customer ID:…
```

Теперь можно подготовить запрос для модели.

Генерация на основе дополненного запроса

В этом разделе мы отправим дополненный ввод модели OpenAI. Наша цель — получить персонализированное электронное письмо для отправки сегменту клиентов, которых мы нашли в индексе Pinecone на основе целевого вектора.

Сначала создаем клиент OpenAI и выбираем в качестве модели GPT-4o:

```
from openai import OpenAI
client = OpenAI()
gpt_model = "gpt-4o
```

Добавляем измерение времени обработки запроса:

```
import time
start_time = time.time()    # Начинаем отсчет перед запросом
```

Время ответа должно быть относительно постоянным, так как здесь мы отправляем по одному запросу за раз. Теперь приступим к созданию итогового запроса:

```
response = client.chat.completions.create(
  model=gpt_model,
  messages=[
```

В описании роли системы модели даются общие инструкции:

```
{
  "role": "system",
  "content": "You are the community manager can write engaging email based
   on the text you have. Do not use a surname but simply Dear Valued
   Customer instead."
},
```

Роль пользователя при этом содержит созданный ранее промт `itext`:

```
  {
    "role": "user",
    "content": itext
  }
],
```

Теперь установим параметры запроса:

```
  temperature=0,
  max_tokens=300,
```

```
  top_p=1,
  frequency_penalty=0,
  presence_penalty=0
)
```

Эти параметры предназначены для получения вывода с низким уровнем случайности, но все же довольно креативного:

- `temperature=0` — низкий уровень случайности ответа;
- `max_tokens=300` — ограничение длины ответа 300 токенами;
- `top_p=1` — учет всех возможных токенов, полное разнообразие;
- `frequency_penalty=0` — отсутствие штрафа за частое повторение слов, чтобы ответ оставался открытым;
- `presence_penalty=0` — отсутствие штрафа за добавление новых тем, чтобы модель могла предлагать идеи для нашего промта.

Отправляем запрос и выводим ответ:

```
print(response.choices[0].message.content)
```

Для этого сегмента клиентов вывод удовлетворительный:

```
Subject: Exclusive Benefits Await You at Our Bank!

Dear Valued Customer,

We hope this email finds you well. At our bank, we are constantly striving to
enhance your banking experience and provide you with the best possible services.
We have noticed that you are a valued customer with a DIAMOND card, and we would
like to offer you some exclusive benefits tailored just for you!

Based on your profile, we have identified several opportunities that could enhance
your banking experience:

1. **Personalized Financial Advice**: Our financial advisors are available to
help you make the most of your finances. Whether it's planning for the future or
managing your current assets, we are here to assist you.

2. **Exclusive Rewards and Offers**: As a DIAMOND cardholder, you are eligible
for special rewards and offers. Earn more points and enjoy exclusive discounts on
various products and services.

3. **Enhanced Credit Options**: With your current credit score, you may qualify
for better credit options. We can help you explore these opportunities to improve
your financial standing.

4. **Complimentary Financial Health Check**: We understand the importance of
financial well-being. Schedule a complimentary financial health check to ensure
you are on the right track.

5. **Loyalty Programs**: Participate in our loyalty programs and earn more points
for every transaction. Redeem these points for exciting rewards and benefits.
```

```
To explore these new advantages and more, please visit the following link:
[Exclusive Benefits](https://www.yourbank
```

Учитывая, что задача маркетологов — убедить клиентов продолжить сотрудничать с банком и повысить их лояльность, можно сказать, что модель сгенерировала хорошее письмо. Теперь выведем время, затраченное на получение ответа:

```
response_time = time.time() - start_time # Измеряем время ответа
print(f"Querying response time: {response_time:.2f} seconds") # Выводим время ответа
```

Вот вывод:

```
Querying response time: 2.83 seconds
```

Мы успешно создали персонализированный ответ на основе целевого вектора. Такого подхода будет достаточно для некоторых проектов в любых областях. Теперь подытожим рассмотренный процесс создания рекомендательной системы ИИ на базе RAG и пойдем дальше.

Резюме

В этой главе, используя индекс Pinecone и модели OpenAI, мы создали масштабированную систему рекомендаций на базе RAG, задача которой — снизить отток клиентов банка. На примере датасета Kaggle мы показали, как выявлять и решать проблемы, ведущие к недовольству клиентов и закрытию ими счетов. Наш подход строился на трех ключевых компонентах пайплайна.

Компонент 1 начинался с оптимизации датасета путем удаления ненужных столбцов, что уменьшило сложность обработки и затраты на хранение данных. В этом же компоненте с помощью EDA мы обнаружили уверенную корреляцию между жалобами клиентов и закрытием счетов, подтвердив это впоследствии моделью кластеризации по методу *k*-средних. Потом реализовали *компонент 2*, подготовив систему на базе RAG к генерации персонализированных рекомендаций. Здесь мы обработали чанки данных с помощью модели OpenAI, преобразовав их в векторы и поместив в индекс Pinecone. Продуманная функциональность загрузки в Pinecone обеспечила эффективную обработку данных вне зависимости от их объема. В завершение мы создали *компонент 3*, чтобы задействовать более миллиона векторов в БД Pinecone для создания персонализированных предложений, нацеленных на удержание конкретного сегмента клиентов банка и повышение их лояльности. Используя GPT-4o, мы дополнили запросы для генерации убедительных рекомендаций.

Успешное применение целевых векторов, представляющих ключевой сегмент рынка, продемонстрировало потенциал системы при выработке эффективных стратегий удержания клиентов. Однако можно дополнительно улучшить эти рекомендации, расширив индекс Pinecone до мультимодальной базы знаний, чем мы и займемся в следующей главе.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Использование датасета Kaggle обычно сопряжено со скачиванием и обработкой реальных данных для анализа?
2. Может ли Pinecone эффективно управлять масштабными векторными БД для приложений ИИ?
3. Помогает ли кластеризация методом *k*-средних установить взаимосвязи между признаками, такими как жалобы, и оттоком клиентов?
4. Станет ли сложнее персонализировать взаимодействие с клиентами, если в базе данных будет содержаться более миллиона векторов?
5. Являются ли автоматизация и повышение качества принятия решений основными причинами использования генеративного ИИ в бизнесе?
6. Выигрывают ли легковесные среды разработки в плане быстрого прототипирования и создания приложений?
7. Может ли архитектура Pinecone масштабироваться автоматически, без ручного вмешательства, для обработки увеличенного объема данных?
8. Генеративный ИИ обычно задействуется для создания динамического контента и рекомендаций на основе пользовательских данных?
9. Требует ли интеграция технологий ИИ наподобие Pinecone и OpenAI значительных ручных усилий при конфигурировании и обслуживании?
10. Должны ли проекты, задействующие векторные базы данных и ИИ, эффективно обрабатывать сложные запросы и крупные датасеты?

Ссылки

- Документация Pinecone: https://docs.pinecone.io/guides/get-started/quickstart.
- Модели OpenAI для создания эмбеддингов и генерации ответов: https://platform.openai.com/docs/models.

Дополнительная информация

- *Han Y., Liu C., Wang P.* A comprehensive survey on vector database: Storage and retrieval technique, challenge. 2023.

7 Создание масштабируемой RAG на основе графа знаний с помощью API Википедии и LlamaIndex

С увеличением датасета довольно быстро возрастает и сложность управления им. В реальных проектах управление данными озадачивает еще больше, чем ИИ. Руководители проектов, консультанты и разработчики постоянно находятся в поиске необходимых данных для успешной реализации проекта, не говоря уже о создании приложений генеративного ИИ на базе RAG. Данные зачастую приходится так или иначе структурировать, принимая сложные решения. Википедия — хороший пример того, как масштабирование данных ведет к получению преимущественно надежной, но порой все же некорректной информации. Реальные проекты часто развиваются тем же путем, что и Википедия. Данные скапливаются в компании, создавая сложности для администраторов БД, руководителей проектов и пользователей.

Одна из основных сложностей заключается в том, чтобы увидеть взаимосвязи в больших объемах разнородных данных. Хорошим средством, позволяющим визуализировать такие связи, являются *графы знаний*. Текущая глава начнется с определения архитектуры экосистемы базы знаний, созданной для генеративного ИИ на базе RAG. Эта экосистема включает пайплайн из трех компонентов, обеспечивающих сбор данных, заполнение векторной БД и выполнение программы, использующей граф знаний и индексируемую RAG. Начнем с компонента 1, представляющего собой сбор и подготовку документов, в котором создадим программу для автоматического извлечения данных через API Википедии. Мы просто выберем тему указанной страницы Википедии и позволим программе извлечь метаданные, необходимые для сбора и подготовки соответствующих документов. Это будет гибкая система, которая позволит выбирать любую тему. Первым примером ее применения станет исследование базы знаний по маркетингу, предназначенной для студентов, желающих повысить квалификацию. После этого перейдем к компоненту 2 — созданию и заполнению векторной БД Deep Lake. Здесь мы загрузим данные в векторную БД, разбив их на чанки с помощью Deep Lake и преобразовав в эмбеддинги с помощью OpenAI. Кроме того, проанализируем датасет, чтобы понять, как это чудо современных технологий справится с задачей.

Наконец, мы реализуем компонент 3 — RAG на основе индексируемого графа знаний, где LlamaIndex автоматически создаст индекс для графа знаний. Будет очень интересно понаблюдать за тем, как функция индексации перебирает наши данные и создает граф, отражающий семантические связи между ними. После этого обратимся к полученному графу, задействуя встроенную в LlamaIndex функциональность OpenAI, чтобы автоматически обработать пользовательский ввод и дать ответ на него. Мы также увидим, как выполняется повторное ранжирование, и реализуем метрики для вычисления и вывода производительности системы.

Архитектура RAG для семантического поиска на основе графа знаний

Создание графа знаний позволит нам визуально сопоставить связи между документами датасета RAG. Создать его можно автоматически с помощью LlamaIndex, чем мы займемся в разделе «Компонент 3: RAG на основе индексируемого графа знаний». В текущей главе мы напишем программу, подходящую под любую тему Википедии, что наглядно представлено на рис. 7.1.

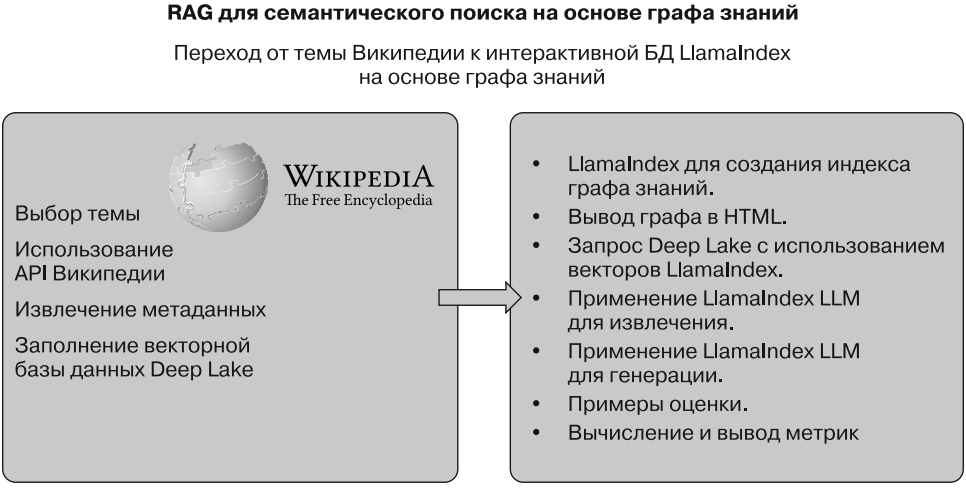


Рис. 7.1. От темы Википедии к взаимодействию с индексом векторной базы данных на основе графа знаний

Сначала мы реализуем ИИ-агент для выдачи информации по теме маркетинга, в котором граф знаний поможет визуально отобразить сложные связи между

различными принципами этой науки. После этого, разобравшись в процессе, вы сможете применить его для изучения любой темы. Мы реализуем три смежных компонента пайплайна, чтобы:

- выбрать тему Википедии, связанную с *маркетингом*. Затем вы сможете использовать этот механизм для изучения любой интересующей вас темы;
- сгенерировать корпус страниц Википедии с помощью API Википедии;
- извлечь и сохранить цитаты для каждой страницы;
- извлечь и сохранить URL каждой страницы;
- извлечь и загрузить содержимое URL в векторную БД Deep Lake;
- создать индекс графа знаний с помощью LlamaIndex;
- определить промт ввода, выполненного пользователем;
- запросить индекс базы знаний;
- применить встроенную в LlamaIndex LLM, основанную на моделях создания эмбеддингов OpenAI, создать ответ на основе векторизованных данных в графе знаний;
- оценить ответ LLM с помощью библиотеки Sentence Transformer;
- оценить ответ LLM на основе обратной связи от человека;
- вывести метрики для основных функций, которые вы сможете при желании применить и для другой функциональности;
- вычислить метрики и вывести результаты.

Для решения всех этих задач мы создадим три компонента пайплайна, используя инструменты, показанные на рис. 7.2.

- *Компонент 1: сбор и подготовка документов*. Подразумевает создание Википедии с помощью API Википедии для извлечения ссылок со страниц этого ресурса, а также метаданных всех страниц (обобщение, URL и информация об отсылках к источнику). Затем мы загрузим и спарсим эти URL, чтобы подготовить данные для загрузки.
- *Компонент 2: создание и заполнение векторной базы данных Deep Lake*. Здесь мы преобразуем в векторы и загрузим содержимое страниц Википедии, подготовленное для компонента 1, в векторную БД Deep Lake.
- *Компонент 3: RAG на основе индексируемого графа знаний*. В этом компоненте мы с помощью LlamaIndex создадим индекс графа знаний и выведем его. Затем реализуем функциональность для обращения к нему и попросим LlamaIndex LLM сгенерировать ответ на основе обновленного датасета.

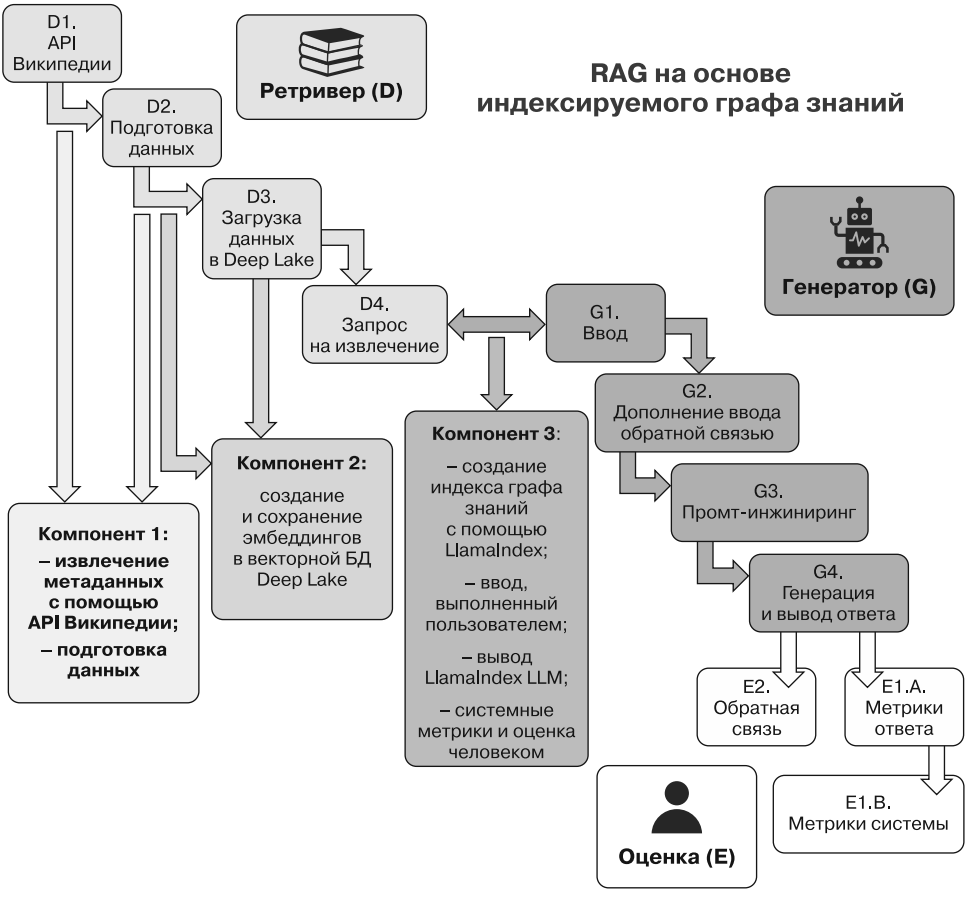


Рис. 7.2. Экосистема графа знаний для RAG

ПРИМЕЧАНИЕ

В сценарии текущей главы мы не просто дополняем ввод, а реализуем расширенную систему извлечения, использующую модели OpenAI для создания эмбеддингов. Эта реализация покажет множество способов, которыми можно улучшить процесс извлечения данных в реальном времени с помощью LLM. Здесь нет никаких правил — применяем все, что работает.

Вся эта система из трех компонентов будет использоваться в сценарии, где администратор может обращаться к векторной базе данных или добавлять новые страницы Википедии. В связи с этим ее архитектура сделает возможным безгра-

ничное масштабирование, поскольку заполняет датасет векторов, обрабатывая по одному набору страниц Википедии за раз. Эта система будет задействовать только процессор и оптимизированный объем памяти. Но у этого подхода есть ограничения, так как в индекс графа знаний LlamaIndex загружается весь датасет. По мере роста векторной базы данных мы можем загружать лишь части датасета или же создать по отдельной БД Deep Lake для каждой темы и выполнять запросы к нескольким датасетам. Все эти решения приходится реализовывать в реальных проектах, тщательно их обдумывая и планируя в зависимости от требований конкретного проекта.

Теперь перейдем к коду и начнем с построения графов из деревьев.

Создание графов из деревьев

Граф — это коллекция узлов (вершин), соединенных ребрами. Узлы представляют сущности, а ребра — связи между ними. Например, в нашем случае узлы будут представлять различные маркетинговые стратегии, а ребра — отражать их взаимосвязи. Все это помогает клиентам понять, как комбинировать различные тактики маркетинга для достижения бизнес-задач, проясняя коммуникацию и повышая эффективность стратегического планирования. Можете поиграть в этой песочнице построения графов, прежде чем приступать к созданию пайплайна.

Откройте блокнот `Tree-2-Graph.ipynb` на GitHub. Программа в нем предназначена для визуального представления связей в структуре дерева с помощью библиотек NetworkX и Matplotlib языка Python. В частности, она создает направленный граф из заданных частей, проверяет и размечает парные связи, после чего выводит полученное дерево вместе с кастомизированными визуальными атрибутами.

Сначала программа определяет основные функции:

- `build_tree_from_pairs(pairs)` — создает из списка пар узлов направленный граф (дерево), потенциально определяя корневой узел;
- `check_relationships(pairs, friends)` — проверяет наличие у каждой пары дружеской связи и выводит их;
- `draw_tree(G, layout_choice, root, friends)` — визуализирует дерево с помощью `matplotlib`, применяя к ребрам различные стили на основе дружеских связей и установленной схемы размещения узлов.

После этого дерево преобразуется в граф:

- определяются пары узлов и данные об их дружеской связи;
- из пар создается дерево;

- проверяется, какие связи являются дружескими;
- дерево отрисовывается на основе выбранной схемы с разными стилями узлов для обозначения дружеских связей.

Например, сначала определяются набор пар узлов и наличие дружеских связей между ними:

```
# Пары
pairs = [('a', 'b'), ('b', 'e'), ('e', 'm'), ('m', 'p'), ('a', 'z'), ('b', 'q')]
friends = {('a', 'b'), ('b', 'e'), ('e', 'm'), ('m', 'p')}
```

Обратите внимание, что `('a', 'z')` не являются друзьями, так как в списке `friends` их нет. То же самое касается `('b', 'q')`. Между парами можно представить любой тип отношений, например: возраст клиентов, общая сфера деятельности, одна страна проживания и пр. Как вариант, список `friends` может содержать связи между друзьями в социальных сетях, проживающими в одной стране, или любой другой интересующий вас вид связи.

После этого программа создает дерево и проверяет его внутренние связи:

```
# Создаем дерево
tree, root = build_tree_from_pairs(pairs)
# Проверяем связи
check_relationships(pairs, friends)
```

Вывод показывает, какие пары являются друзьями, а какие — нет:

```
Pair ('a', 'b'): friend
Pair ('b', 'e'): friend
Pair ('e', 'm'): friend
Pair ('m', 'p'): friend
Pair ('a', 'z'): not friend
Pair ('b', 'q'): not friend
```

Эту информацию можно использовать для поиска сходства. Далее программа рисует граф по схеме `'spring'`:

```
# Рисуем дерево
layout_choice = 'spring'    # Выбираем схему
draw_tree(tree, layout_choice=layout_choice, root=root, friends=friends)
```

Схема `'spring'` строится по принципу симуляции пружин (spring), то есть ребра в ней как бы притягивают узлы друг к другу, а сами узлы при этом отталкиваются, что позволяет избегать их столкновения. Изучить и выбрать другие схемы можно с помощью функции `draw_tree`. При этом можете изменять цвета и стили связующих линий.

В данном случае связи между дружескими узлами представлены непрерывными линиями, а между теми, которые друзьями не являются, — пунктирными (рис. 7.3).

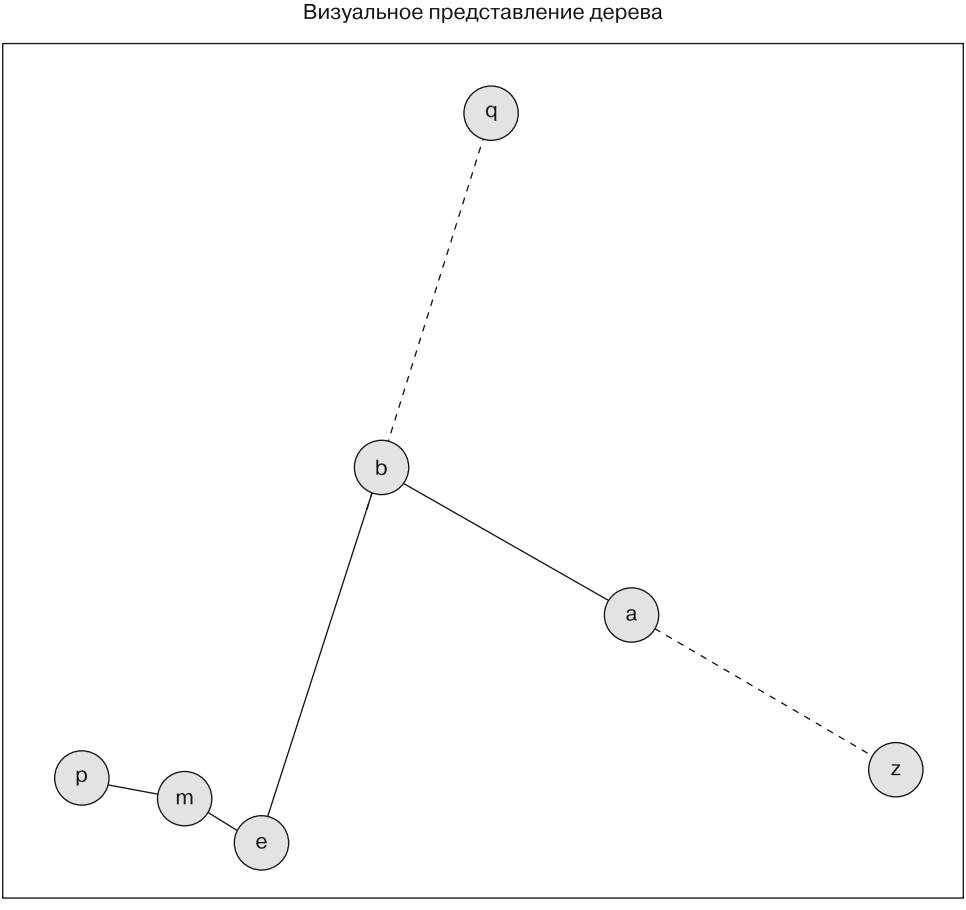


Рис. 7.3. Пример схемы «пружина»

Поэкспериментируйте с построением этого графа, используя различные пары узлов. Если вы экстраполируете этот процесс до сотен узлов, то определенно оцените возможность его автоматизации, которую мы реализуем в этой главе с помощью индекса графа знаний LlamaIndex.

Теперь перейдем от архитектуры к коду, начав со скачивания и подготовки документов.

Компонент 1: сбор и подготовка документов

Код, приводимый в этом разделе, извлекает нужные метаданные из Википедии, скачивает связанные с ними документы, очищает их и подготавливает для вставки в векторную базу данных Deep Lake. Этот процесс схематично изображен на рис. 7.4.

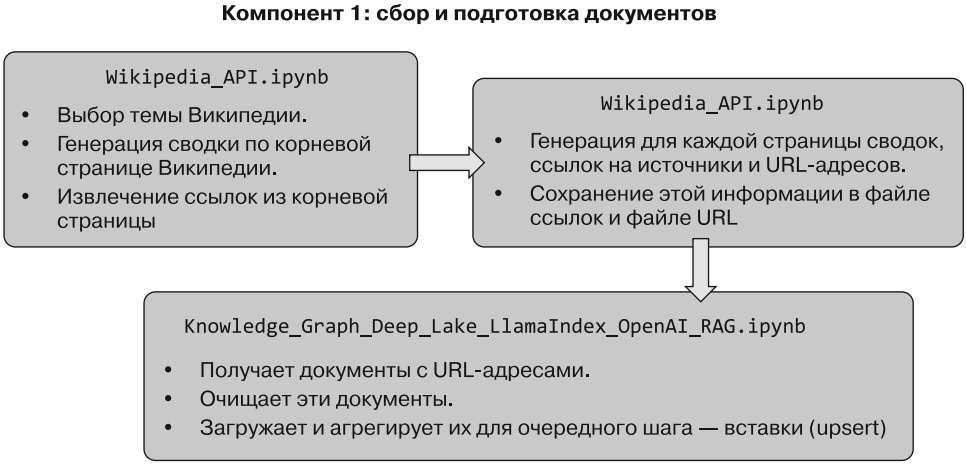


Рис. 7.4. Блок-схема компонента 1

Компонент 1 включает два блокнота кода:

- `Wikipedia_API.ipynb`, где мы реализуем API Википедии для извлечения URL страниц, связанных с корневой страницей выбранной темы, включая цитаты. Как уже говорилось, в нашем случае темой будет «маркетинг»;
- `Knowledge_Graph_Deep_Lake_LlamaIndex_OpenAI_RAG.ipynb`, где мы реализуем все три компонента. Компонент 1 будет получать URL-адреса из блокнота `Wikipedia_API`, очищать их, а также агрегировать и загружать в базу данных.

Начнем с реализации API Википедии.

Извлечение данных и метаданных из Википедии

Сначала создадим программу для взаимодействия с API Википедии с целью извлечения информации по конкретной теме, токенизации полученного текста и управления ссылками на источники из статей Википедии. Далее можете открыть блокнот `Wikipedia_API.ipynb` в репозитории GitHub и продолжать по нему.

Сначала устанавливаем необходимую библиотеку `wikipediaapi`:

```
try:
  import wikipediaapi
except:
  !pip install Wikipedia-API==0.6.0
  import wikipediaapi
```

Далее определяем функцию токенизации, которая будет вызываться для подсчета количества токенов в сводке:

```
def nb_tokens(text):
    # Более сложная токенизация, включая пунктуацию
    tokens = word_tokenize(text)
    return len(tokens)
```

Эта функция получает строку текста и возвращает количество содержащихся в ней токенов, задействуя библиотеку NLTK, позволяющую выполнить расширенную токенизацию. Далее, чтобы начать извлечение данных, нужно настроить экземпляр API Википедии, используя определенный язык и строку `user_agent`:

```
# Создаем экземпляр API Википедии с подробной строкой user_agent
wiki = wikipediaapi.Wikipedia(
    language='en',
    user_agent='Knowledge/1.0 ([USER AGENT EMAIL)'
)
```

В этом случае с помощью `'en'` мы установили английский язык, и далее вам нужно ввести информацию для `user_agent`, например адрес почты. Теперь можно определить основную тему и имя файла, связанные с интересующей нас страницей Википедии:

```
topic="Marketing"     # Тема
filename="Marketing"  # Имя файла для сохранения вывода
maxl=100
```

Здесь мы определяем три параметра:

- `topic` — тема для извлечения;
- `filename` — имя темы. Оно будет определять содержимое создаваемых нами файлов, которые от самой темы могут отличаться;
- `maxl` — максимальное количество ссылок URL на извлекаемых страницах.

Теперь нужно извлечь сводку по указанной странице Википедии, проверить существование этой страницы и вывести ее содержимое:

```
import textwrap    # Для форматирования текста вывода в виде абзацев
page=wiki.page(topic)

if page.exists()==True:
  print("Page - Exists: %s" % page.exists())
```

```
  summary=page.summary
  # Число токенов
  nbt=nb_tokens(summary)
  print("Number of tokens: ",nbt)
  # Используем textwrap для указания ширины вывода текста сводки,
  # например 70 символов
  wrapped_text = textwrap.fill(summary, width=60)
  # Выводим текст сводки
  print(wrapped_text)
else:
  print("Page does not exist")
```

В выводе получаем запрошенную информацию для управления:

```
Page - Exists: True
Number of tokens: 229
Marketing is the act of satisfying and retaining customers.
It is one of the primary components of business management and commerce.
Marketing is typically conducted by the seller, typically a retailer or
manufacturer…
```

Глядя на полученный вывод, можно оценить, на правильном ли мы пути, прежде чем выполнять полноценный поиск по основной странице темы.

- `Page - Exists: True` подтверждает, что страница существует. В противном случае отображается сообщение `print("Page does not exist")`.
- `Number of tokens: 229` сообщает размер извлекаемого содержимого. Эта информация пригодится руководителям проекта для анализа.
- `summary=page.summary` показывает сводку по странице.

В данном случае страница существует, соответствует заданной теме, и по ней дана адекватная сводка. Прежде чем продолжать, убедимся, что работаем с правильной страницей:

```
print(page.fullurl)
```

Все верно:

```
https://en.wikipedia.org/wiki/Marketing
```

Можно приступать к извлечению URL-адресов и ссылок с целевой страницы, а также сопутствующих сводок:

```
# Промт: считать программу до этой ячейки. Затем извлечь все ссылки с этой
# страницы – вывести ссылку и сводку по каждой ссылке
# Получаем все ссылки со страницы
links = page.links
# Выводим ссылку и сводку по каждой ссылке
urls = []
counter=0
```

```
for link in links:
  try:
    counter+=1
    print(f"Link {counter}: {link}")
    summary = wiki.page(link).summary
    print(f"Link: {link}")
    print(wiki.page(link).fullurl)
    urls.append(wiki.page(link).fullurl)
    print(f"Summary: {summary}")
    if counter>=maxl:
      break
  except page.exists()==False:
    # Игнорируем несуществующие страницы
    pass

print(counter)
print(urls)
```

Эта функция ограничена `maxl`, определенным в начале программы, то есть будет извлекать до `maxl` URL-адресов. Затем проверяем вывод, прежде чем переходить к следующему шагу и генерировать файлы:

```
Link 1: 24-hour news cycle
Link: 24-hour news cycle
https://en.wikipedia.org/wiki/24-hour_news_cycle
Summary: The 24-hour news cycle (or 24/7 news cycle) is 24-hour investigation
and reporting of news, concomitant with fast-paced lifestyles…
```

Здесь мы видим, что у нас есть вся необходимая информация и сводки по ней вполне приемлемы:

- `Link 1` — счетчик ссылок;
- `Link` — фактическая ссылка на страницу, извлеченную со страницы основной темы;
- `Summary` — сводка по странице, извлеченной через ссылку.

Далее применим только что созданную функцию для генерации текстового файла со ссылками, извлеченными со страницы Википедии, и их URL-адресами:

```
from datetime import datetime
# Получаем все ссылки со страницы
links = page.links

# Подготавливаем файл для сохранения ссылок
fname = filename+"_citations.txt"
with open(fname, "w") as file:
    # Записываем заголовок ссылки на источник
    file.write(f"Citation. In Wikipedia, The Free Encyclopedia. Pages retrieved
from the following Wikipedia contributors on {datetime.now()}\n")
    file.write("Root page: " + page.fullurl + "\n")
    counter = 0
    urls = []…
```

Адреса ссылок будут добавляться в `urls = []`, формируя полный список, который потребуется для заключительного шага. Итоговый файл будет содержать название темы, `datetime`, и ссылки на источники, начинающиеся с соответствующего текста:

```
Citation. In Wikipedia, The Free Encyclopedia. Pages retrieved from the following
Wikipedia contributors on {datetime.now()}\n")
```

В нашем случае результатом станет файл `Marketing_citations.txt`. Он был скачан и загружен в каталог `/citations` раздела текущей главы в репозитории GitHub.

На этом мы закончили генерацию страницы со ссылками на источники, вывели ее и сохранили в репозитории GitHub в соответствии с требованиями Википедии. Последним шагом будет генерация файла, содержащего список URL-адресов, которые мы будем использовать для получения содержимого с нужных страниц. Сначала выводим URL:

```
urls
```

Вывод подтверждает наличие необходимых адресов:

```
['https://en.wikipedia.org/wiki/Marketing',
 'https://en.wikipedia.org/wiki/24-hour_news_cycle',
 'https://en.wikipedia.org/wiki/Account-based_marketing',
…
```

Далее эти адреса записываются в файл с префиксом в виде названия темы:

```
# Записываем URL-адреса в файл
ufname = filename+"_urls.txt"
with open(ufname, 'w') as file:
    for url in urls:
        file.write(url + '\n')
print("URLs have been written to urls.txt")
```

В данном случае полученный файл называется `Marketing_urls.txt` и содержит адреса страниц, которые нам нужно получить. Этот файл также был скачан и загружен в каталог `/citations` репозитория GitHub.

Теперь можно приступать к подготовке данных для вставки в БД.

Подготовка данных к загрузке

URL-адреса, полученные через API Википедии в блокноте `Wikipedia_API.ipynb`, будут обработаны в блокноте `Knowledge_Graph_Deep_Lake_LlamaIndex_OpenAI_RAG.ipynb`, который лежит на GitHub в каталоге текущей главы. Раздел «Настройка среды» этого блокнота почти полностью совпадает с аналогичными разделами из глав 2 и 3, только в данном случае список URL был сгенерирован кодом блокнота `Wikipedia_API.ipynb` и мы его извлечем.

Для начала перейдите в раздел «Сценарий» блокнота, чтобы прописать рабочий поток:

```
# Имя файла для управления
graph_name="Marketing"

# Путь для векторной базы данных и датасета
db="hub://denis76/marketing01"
vector_store_path =  db
dataset_path = db
# Если True, то данные вставляются, если False, вставка пропускается
# и выполняется подключение
pop_vs=True
# Если pop_vs==True, то overwrite=True перепишет датасет,
# а False приведет к его дополнению:
ow=True
```

Следующие параметры определят поведение всех трех компонентов текущего блокнота.

- `graph_name="Marketing"` — префикс (тема) файлов, которые мы будем считывать и записывать.
- `db="hub://denis76/marketing01"` — название векторной базы данных Deep Lake. Можете выбрать в качестве него любое имя датасета.
- `vector_store_path = db` — путь к векторной БД.
- `dataset_path = db` — путь к датасету векторной БД.
- `pop_vs=True` — `True` активирует вставку данных, `False` — деактивирует.
- `ow=True` — если `True`, то существующий датасет переписывается, если `False` — дополняется.

Теперь можно выполнить раздел «Компонент 1: сбор и подготовка документов». Следующий код скачает список URL, сгенерированный в предыдущих разделах главы:

```
# Определите переменные
if pop_vs==True:
  directory = "Chapter07/citations"
  file_name = graph_name+"_urls.txt"
  download(directory,file_name)
```

После он считает и сохранит эти URL в списке `urls`. Остальной код этого раздела блокнота выполняет тот же процесс, что и блокнот `Deep_Lake_LlamaIndex_OpenAI_RAG.ipynb` из главы 3. Разница в том, что в главе 3 URL веб-страниц вводились в список вручную.

Этот код скачает содержимое с представленных в списке URL. После этого он очистит и подготовит полученные данные для последующего заполнения ими векторной БД Deep Lake.

Компонент 2: создание и заполнение векторной базы данных Deep Lake

Этот компонент блокнота `Deep_Lake_LlamaIndex_OpenAI_RAG.ipynb` создавался на основе кода компонента 2 из главы 3. Заметьте, что создание пайплайна в виде отдельных компонентов позволяет нам быстро корректировать их назначение и подстраивать под другие задачи. Кроме того, в Activeloop Deep Lake есть встроенная возможность разбивки данных на чанки, преобразования их в эмбеддинги и вставки в базу данных. Все это сильно упрощает интеграцию различных типов неструктурированных данных, как в случае с документами Википедии, которые мы загружаем.

Вывод функции `display_record(record_number)` демонстрирует целостность этого процесса. В нем мы видим ID и метаданные наподобие информации о файле, собранные данные, текст и эмбеддинг:

```
ID:
['a61734be-fe23-421e-9a8b-db6593c48e08']

Metadata:
file_path: /content/data/24-hour_news_cycle.txt
file_name: 24-hour_news_cycle.txt
file_type: text/plain
file_size: 2763
creation_date: 2024-07-05
last_modified_date: 2024-07-05
…
Text:
['24hour investigation and reporting of news concomitant with fastpaced
lifestyles This article is about the fastpaced cycle of news media in
technologically advanced societies.

Embedding:
[-0.00040736704249866307, 0.009565318934619427, 0.015906672924757004,
-0.009085721336305141, …]
```

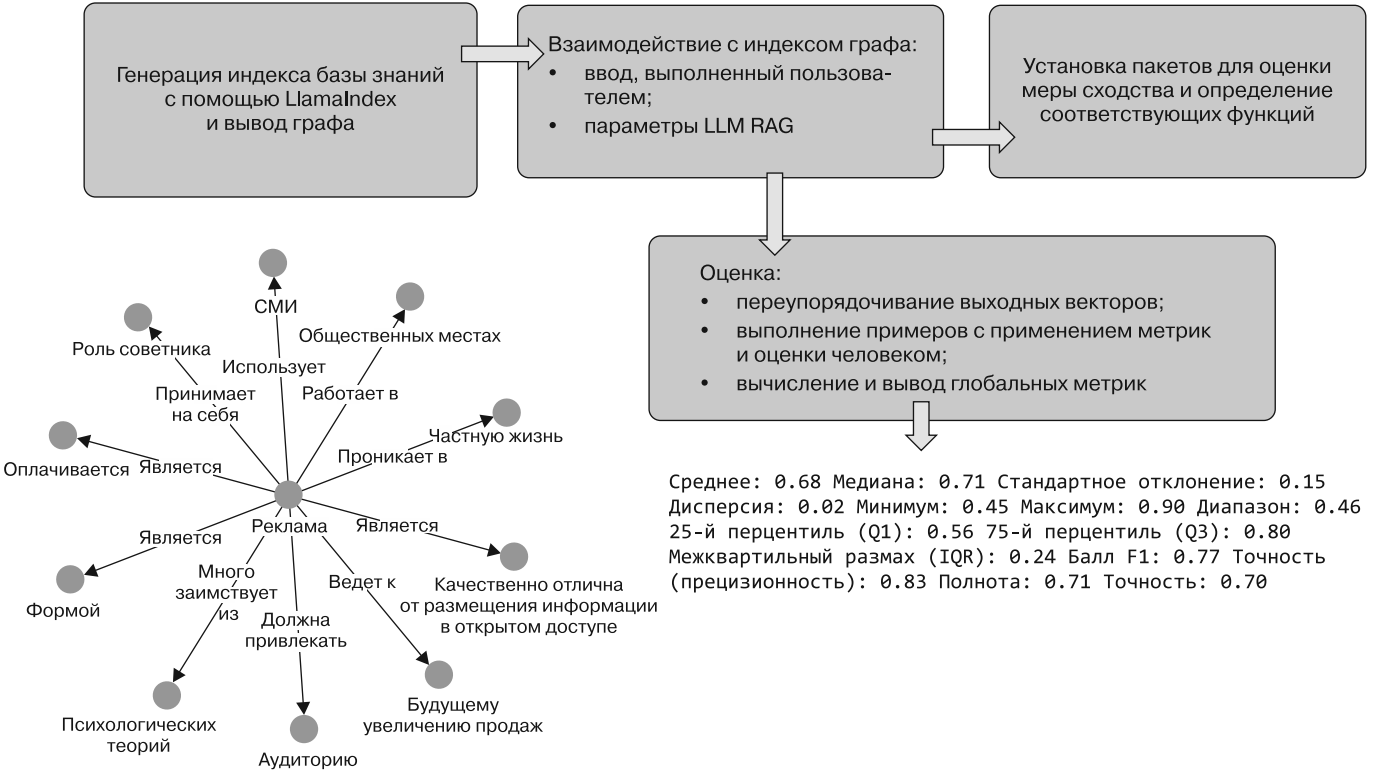
Мы успешно скорректировали компонент 2 из главы 3 и теперь можем переходить к созданию индекса графа знаний.

Компонент 3: RAG на основе индексируемого графа знаний

Далее мы создадим компонент RAG на основе индексируемого графа знаний и начнем с ним взаимодействовать. Как видно на рис. 7.5, работы предстоит много.

Компонент 3: RAG на основе индексируемого графа знаний

`Knowledge_Graph_Deep_Lake_LlamaIndex_OpenAI_RAG.ipynb`




```
Среднее: 0.68 Медиана: 0.71 Стандартное отклонение: 0.15
Дисперсия: 0.02 Минимум: 0.45 Максимум: 0.90 Диапазон: 0.46
25-й перцентиль (Q1): 0.56 75-й перцентиль (Q3): 0.80
Межквартильный размах (IQR): 0.24 Балл F1: 0.77 Точность
(прецизионность): 0.83 Полнота: 0.71 Точность: 0.70
```

Рис. 7.5. Создание RAG на основе индексируемого графа знаний с нуля

В этом разделе мы:

- сгенерируем индекс графа знаний;
- выведем граф;
- определим промт пользователя;
- определим гиперпараметры для встроенной в LlamaIndex модели LLM;
- установим пакеты для измерения сходства;
- определим функции оценки сходства;
- выполним несколько примеров оценки сходства с помощью соответствующих функций;
- переупорядочим векторы, выданные LLM;
- выполним серию примеров запросов, которые оценим с помощью метрик и обратной связи;
- вычислим метрики и выведем их.

Начнем с генерации индекса графа знаний.

Генерация индекса графа знаний

Этот индекс мы создадим на основе набора документов, используя класс `KnowledgeGraphIndex` из модуля `llama_index.core`, и попутно замерим время его создания для оценки производительности.

Соответствующая функция начинается с отсчета времени с помощью `time.time()`. В этом случае важно измерить затраченное время, так как создание индекса происходит небыстро:

```
from llama_index.core import KnowledgeGraphIndex
import time
# Запуск таймера
start_time = time.time()
```

Теперь создадим `KnowledgeGraphIndex` с эмбеддингами с помощью метода `from_documents`. В функции будут использованы следующие параметры:

- `documents` — набор документов для индексирования;
- `max_triplets_per_chunk`, установленный на 2, что ограничит количество триплетов на чанк, чтобы было оптимизировано потребление памяти и время обработки;
- `include_embeddings`, установленный на `True`, то есть эмбеддинги должны быть включены.

Таким образом, индекс графа создается всего в нескольких строках кода:

```
# Индекс графа с эмбеддингами
graph_index = KnowledgeGraphIndex.from_documents(
    documents,
    max_triplets_per_chunk=2,
    include_embeddings=True,
)
```

Таймер останавливается, и вычисляется прошедшее время:

```
# Останавливаем таймер
end_time = time.time()
# Вычисляем и выводим время выполнения
elapsed_time = end_time - start_time
print(f"Index creation time: {elapsed_time:.4f} seconds")
print(type(graph_index))
```

Вывод:

```
Index creation time: 371.9844 seconds
```

Выводим также тип графа:

```
print(type(graph_index))
```

Все верно, это граф знаний:

```
<class 'llama_index.core.indices.knowledge_graph.base.KnowledgeGraphIndex'>
```

Теперь настроим механизм обработки запросов, который будет управлять параметрами измерения сходства, температуры и размера вывода:

```
#similarity_top_k
k=3
#temperature
temp=0.1
#num_output
mt=1024
graph_query_engine = graph_index.as_query_engine(similarity_top_k=k,
  temperature=temp, num_output=mt)
```

Эти параметры определяют поведение механизма обработки запросов.

- `k=3` указывает, сколько максимально схожих результатов из общего числа будет учитываться.
- `temp=0.1` устанавливает температуру, регулирующую критерий случайности при генерации ответа механизмом обработки запросов. Чем она ниже, тем более точным должен быть вывод, и наоборот, высокие значения ведут к более креативным результатам.

- `mt=1024` устанавливает максимальное количество токенов в выводе, определяя длину генерируемых ответов.

Далее на основе заданных параметров создается механизм запросов:

```
graph_query_engine = graph_index.as_query_engine(similarity_top_k=k,
  temperature=temp, num_output=mt)
```

Проиндексированный граф и механизм обработки запросов готовы.

Вывод графа

Далее мы создадим экземпляр графа `g`, используя библиотеку Python для интерактивных визуализаций сетей `pyvis.network`. Выведенные параметры аналогичны тем, которые мы определили в подразделе «Создание графов из деревьев»:

```
# Создаем граф
from pyvis.network import Network

g = graph_index.get_networkx_graph()
net = Network(notebook=True, cdn_resources="in_line", directed=True)
net.from_nx(g)

# Устанавливаем свойства узлов и ребер: их цвет и размер
for node in net.nodes:
    node['color'] = 'lightgray'
    node['size'] = 10

for edge in net.edges:
    edge['color'] = 'black'
    edge['width'] = 1
```

Получился направленный граф, который мы сохраним в HTML-файле для дальнейшего вывода:

```
fgraph="Knowledge_graph_"+ graph_name + ".html"
net.write_html(fgraph)
print(fgraph)
```

В разделе «Сценарий» в начале блокнота мы определили `graph_name`. Теперь же выведем этот граф в виде HTML-файла:

```
from IPython.display import HTML

# Загружаем HTML-содержимое из файла и выводим его
with open(fgraph, 'r') as file:
    html_content = file.read()
# Выводим HTML в блокноте
display(HTML(html_content))
```

Можете скачать этот файл, чтобы выводить его в браузере и работать с ним. Можете также визуализировать его в блокноте, как показано на рис. 7.6.

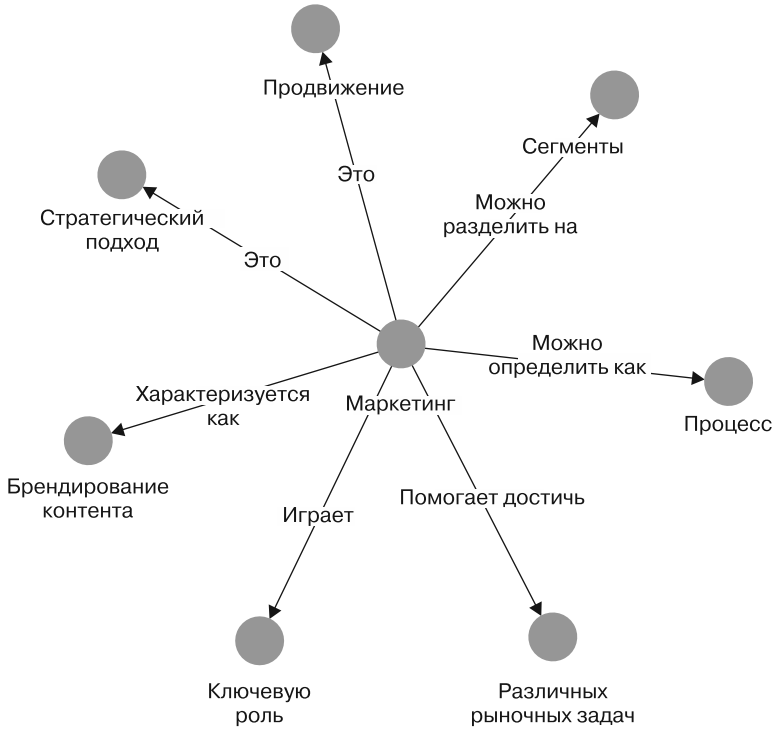


Рис. 7.6. Граф знаний

Приступим к взаимодействию с индексированным графом знаний.

Взаимодействие с индексом графа знаний

Далее мы определим функциональность, необходимую для выполнения запроса, как делали в разделе «Компонент 3: RAG с индексированием» главы 3.

- `execute_query` — это написанная нами функция, которая будет выполнять запрос: `response = graph_query_engine.query(user_input)`. Помимо этого, она измеряет затраченное время.
- `user_query="What is the primary goal of marketing for the consumer market?"` — запрос пользователя.
- `response = execute_query(user_query)` — выводимый ответ, включенный в код запроса.

В выводе отражаются время выполнения и максимально схожие векторы из тех, которые мы создали на основе данных Википедии:

```
Query execution time: 2.4789 seconds
The primary goal of marketing for the consumer market is to effectively target
consumers, understand their behavior, preferences, and needs, and ultimately
influence their purchasing decisions.
```

Теперь установим пакеты для измерения сходства и определим соответствующие функции.

Установка пакетов для измерения сходства и определение функций

Сначала извлечем токен Hugging Face с вкладки *Secrets* инструментария Google Colab, где он был сохранен в соответствии с установками блокнота:

```
from google.colab import
userdata userdata.get('HF_TOKEN')
```

В августе 2024 года этот токен является для библиотеки Hugging Face `sentence-transformers` опциональным, так что можете проигнорировать это сообщение и закомментировать код. Далее установим саму `sentence-transformers`:

```
!pip install sentence-transformers==3.0.1
```

Теперь создадим функцию оценки косинусного сходства с эмбеддингами:

```
from sklearn.metrics.pairwise import cosine_similarity
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
def calculate_cosine_similarity_with_embeddings(text1, text2):
    embeddings1 = model.encode(text1)
    embeddings2 = model.encode(text2)
    similarity = cosine_similarity([embeddings1], [embeddings2])
    return similarity[0][0]
```

Импортируем необходимые библиотеки:

```
import time
import textwrap
import sys
import io
```

Применим эту функцию для переупорядочивания.

Переупорядочивание

В этом разделе программа переупорядочит содержимое ответа, выстроив верхние результаты выдачи в ином порядке, чтобы определить, есть ли среди них более точные.

- `user_query=" Which experts are often associated with marketing theory?"` — наш запрос.
- `start_time = time.time()` регистрирует время начала выполнения запроса.
- `response = execute_query(user_query)` выполняет запрос.
- `end_time = time.time()` завершает отсчет и выводит время выполнения.
- `for idx, node_with_score in enumerate(response.source_nodes)` перебирает результаты ответа для извлечения всех содержащихся в нем узлов.
- `similarity_score3=calculate_cosine_similarity_with_embeddings(text1, text2)` вычисляет сходство между запросом пользователя и текстом узлов, извлеченных из ответа. Все результаты сравнения выводятся.
- `best_score=similarity_score3` сохраняет лучший показатель сходства.
- `print(textwrap.fill(str(best_text), 100))` выводит лучший результат после переупорядочивания.

Изначальный ответ для `user_query "Which experts are often associated with marketing theory?"`:

```
Psychologists, cultural anthropologists, and market researchers are often
associated with marketing theory.
```

Это приемлемый ответ. Однако его вариант с измененным порядком результатов оказывается более детальным — в нем упоминаются имена маркетологов-экспертов (выделены жирным):

```
Best Rank: 2
Best Score: 0.5217772722244263
[…In 1380 the German textile manufacturer Johann Fugger travelled from Augsburg
to Graben in order to gather information on the international textile industry…
During this period Daniel Defoe a London merchant published information on trade
and economic resources of England and Scotland…]
```

Переупорядоченный ответ длиннее и содержит необработанное содержимое документа вместо сводки, предоставленной механизмом обработки запросов LlamaIndex LLM. С точки зрения LLM первичный ответ лучше. Однако сложно оценить, какой вариант больше устроит конечного пользователя. Некоторые предпочитают короткие ответы, другие — более развернутые. На порядок извлеченных документов могут повлиять различные факторы, например, корректировка промта, добавление документов или их удаление. Мы можем даже решить тонко настроить LLM, как поступим в главе 9. Можно также ввести рейтинг обратной связи, как мы делали в главе 5, поскольку во многих случаях математические метрики не отразят точность ответа, например, при выдаче намеренно длинных ответов на короткие запросы. Но попробовать все равно нужно.

Давайте оценим примеры, которые собираемся выполнять, с помощью нескольких метрик.

Примеры оценки

Чтобы оценить механизм обработки запросов к индексу графа знаний, выполним десять примеров и зафиксируем полученные показатели. `rscores` отслеживает показатели обратной связи, а `scores=[]` — показатели функции оценки сходства:

```
# Создаем пустой массив для показателей обратной связи
rscores =[]
# Создаем пустой массив для показателей функции оценки сходства
scores=[]
```

В зависимости от задач проекта можно использовать любое число примеров. Структура у всех будет одинаковая:

- `user_query` — вводный текст для механизма обработки запросов;
- `elapsed_time` — результат измерения времени ответа системы;
- `response = execute_query(user_query)` выполняет запрос.

Запрос пользователя и вывод те же, что в примере с функцией переупорядочивания:

```
Query execution time: 1.9648 seconds
Psychologists, cultural anthropologists, and other experts in behavioral
sciences are often associated with marketing theory.
```

Но на этот раз мы выполним функцию оценки сходства, а также попросим обратную связь от пользователя:

```
text1=str(response)
text2=user_query
similarity_score3=calculate_cosine_similarity_with_embeddings(text1, text2)
print(f"Cosine Similarity Score with sentence transformer:
  {similarity_score3:.3f}")
scores.append(similarity_score3)
human_feedback=0.75
rscores.append(human_feedback)
```

В этой функции:

- `text1` представляет ответ механизма обработки запросов;
- `text2` — запрос пользователя;
- `similarity_score3` — показатель косинусного сходства;
- `scores.append(similarity_score3)` добавляет показатель сходства в массив;
- `human_feedback` — оценка сходства человеком. Этот показатель можно заменить документом, как мы делали в главе 5, или текстовым ответом, являющимся утвержденной истиной. В обоих случаях показатель сходства вычисляется заново с учетом содержимого обратной связи;

- `rscores.append(human_feedback)` добавляет оценку человеком в массив `rscores`.

Теперь разберем вывод нескольких из десяти примеров, сопроводив их комментариями.

> **ПРИМЕЧАНИЕ**
>
> LLM представляют собой стохастические алгоритмы, поэтому их ответы и показатели в разных случаях выполнения могут немного различаться.

- **Пример 1**
  - *Запрос пользователя*: `Which experts are often associated with marketing theory?`
  - *Ответ:* `Psychologists, cultural anthropologists, and other experts in behavioral sciences are often associated with marketing theory.`
  - *Косинусное сходство:* `0.809`.
  - *Оценка человеком:* `0.75`.
  - *Комментарий:* ответ приемлемый, но он может быть более детальным, с упоминанием имен экспертов. Но и в промте пользователь указал экспертов в обобщенном смысле.
- **Пример 3**
  - *Запрос пользователя:* `What is the difference between B2B and B2C?`
  - *Ответ:* `B2B businesses sell products and services to other companies, while B2C businesses sell directly to customers.`
  - *Косинусное сходство:* `0.760`.
  - *Оценка человеком:* `0.8`.
  - *Комментарий:* ответ довольно точный, но в некоторых случаях пользователи любят, когда приводятся примеры.
- **Пример 7**
  - *Запрос пользователя:* `What commodity programs does the Agricultural Marketing Service (AMS) maintain?`
  - *Ответ:* `The Agricultural Marketing Service (AMS) maintains programs in five commodity areas: cotton and tobacco, dairy, fruit and vegetable, livestock and seed, and poultry.`
  - *Косинусное сходство:* `0.904`.
  - *Оценка человеком:* `0.9`.

- *Комментарий*: это точный и довольно интересный ответ, поскольку приведенная в нем информация содержится на странице, связанной с основной. Можно попросить механизм Википедии найти ссылки всех страниц, связанных с основной, и углубиться на несколько уровней. Но тогда основная информация, которую мы ищем, может оказаться разбавлена менее релевантными данными. Такое решение о глубине извлекаемых данных зависит от задач конкретного проекта.

Далее мы определим показатели косинусного сходства и оценки человеком.

Вычисление и вывод метрик

Найденные показатели косинусного сходства сохраняются в массиве `scores`:

```
print(len(scores), scores)
```

И выводятся:

```
10 [0.808918, 0.720165, 0.7599532, 0.8513956, 0.5457667, 0.6963912, 0.9036964,
0.44829217, 0.59976315, 0.47448665]
```

В зависимости от задач конкретного проекта эту оценку можно расширить до любого количества примеров. Показатели точности, присвоенные человеком, сохраняются в массиве `rscores`:

```
print(len(rscores), rscores)
```

И тоже выводятся:

```
10 [0.75, 0.5, 0.8, 0.9, 0.65, 0.8, 0.9, 0.2, 0.2, 0.9]
```

Для более глубокой оценки ответов мы применим дополнительные метрики:

```
mean_score = np.mean(scores)
median_score = np.median(scores)
std_deviation = np.std(scores)
variance = np.var(scores)
min_score = np.min(scores)
max_score = np.max(scores)
range_score = max_score - min_score
percentile_25 = np.percentile(scores, 25)
percentile_75 = np.percentile(scores, 75)
iqr = percentile_75 - percentile_25
```

Каждая из них сообщит нам много полезного.

- *Мера центральной тенденции (среднее, медиана)* показывает, как выглядят типичные показатели.
- *Вариативность (стандартное отклонение, дисперсия, диапазон, IQR)* сообщает, насколько разбросаны показатели, отражая согласованность или разнородность данных.

- *Крайние значения (минимум, максимум)* показывают границы полученных показателей.
- *Распределение (в перцентилях)* показывает, как показатели распределены по диапазону значений.

Теперь разберем и выведем эти метрики, вычисленные на основе показателей косинусного сходства и обратной связи.

1. *Среднее.*
   - *Определение:* среднее — это сумма всех показателей, разделенная на их количество.
   - *Смысл:* показывает центральное значение данных, давая представление о типичной оценке.
   - *Вычисление:*

$$\text{среднее} = \frac{\sum \text{показатель}}{\text{число показателей}}.$$

   - *Вывод:* `Mean: 0.68`.
2. *Медиана.*
   - *Определение:* медиана — это среднее значение в списке показателей, упорядоченном от наименьшего к наибольшему.
   - *Смысл:* показывает центральную точку набора показателей и меньше подвержено влиянию крайних значений (выбросов) по сравнению со средним.
   - *Вывод:* `Median: 0.71`.
3. *Стандартное отклонение.*
   - *Определение:* стандартное отклонение измеряет усредненное значение, на которое каждый показатель отличается от среднего.
   - *Смысл:* показывает, насколько сильно разбросаны показатели относительно среднего. Более высокое отклонение говорит о большей вариативности.
   - *Вычисление:*

$$\text{стандартное отклонение} = \sqrt{\frac{\sum (\text{показатель} - \text{среднее})^2}{\text{число показателей}}}.$$

   - *Вывод:* `Standard Deviation: 0.15`.
4. *Дисперсия.*
   - *Определение:* дисперсия — это квадрат стандартного отклонения.
   - *Смысл:* также измеряет разброс показателей, отражая, насколько они отличаются от среднего.
   - *Вывод:* `Variance: 0.02`.

5. *Минимум.*
   - *Определение:* минимум — это наименьший показатель среди всех.
   - *Смысл:* показывает минимальное значение.
   - *Вывод:* `Minimum: 0.45`.
6. *Максимум.*
   - *Определение:* максимум — это наибольший показатель из всех.
   - *Смысл:* показывает максимальное значение.
   - *Вывод:* `Maximum: 0.90`.
7. *Диапазон.*
   - *Определение:* диапазон — это разница между максимальным и минимальным показателями.
   - *Смысл:* показывает охват полученных результатов от наименьшего до наибольшего значения.
   - *Вычисление:*

$$\text{диапазон} = \text{максимум} - \text{минимум}.$$

   - *Вывод*: `Range: 0.46`.
8. *25-й перцентиль (Q1).*
   - *Определение:* 25-й перцентиль — это значение, ниже которого находятся 25 % показателей.
   - *Смысл:* показывает точку, ниже которой расположена четверть показателей.
   - *Вывод:* `25th Percentile (Q1): 0.56`.
9. *75-й перцентиль (Q3).*
   - *Определение:* 75-й перцентиль — это значение, ниже которого находятся 75 % показателей.
   - *Смысл:* показывает точку, ниже которой расположено три четверти всех данных.
   - *Вывод:* `75th Percentile (Q3): 0.80`.
10. *Межквартильный размах (IQR).*
    - *Определение:* IQR — это диапазон между 25-м (Q1) и 75-м (Q3) перцентилями.
    - *Смысл*: измеряет промежуточные 50 % данных, демонстрируя их разброс без влияния крайних значений.
    - *Вычисление:*

$$\text{IQR} = \text{Q3} - \text{Q1}.$$

    - *Вывод:* `Interquartile Range (IQR): 0.24`.

Итак, мы создали RAG на основе индексируемого графа знаний, поработали с ним и оценили с помощью серии примеров и метрик. Резюмируем этот процесс.

Резюме

В этой главе мы занимались созданием масштабируемой системы RAG на основе графа знаний с помощью API Википедии и LlamaIndex. Разработанные нами техники и инструменты вполне применимы в различных областях, включая управление данными, маркетинг и любую сферу, где требуется налаженный и доступный механизм извлечения данных.

Начали мы с выполнения компонента 1 — сбора данных, где реализовали автоматизированный процесс извлечения содержимого статей Википедии. Используя API Википедии, создали программу для сбора метаданных и URL-адресов со страниц Википедии на основе выбранной темы — в данном случае «Маркетинг». Далее в рамках реализации компонента 2 мы создали и заполнили векторную базу данных Deep Lake. Здесь данные, собранные при выполнении компонента 1, были преобразованы в эмбеддинги и загружены в БД Deep Lake. Мы подчеркнули простоту интеграции обширных объемов данных в структурированное векторное хранилище, готовое для последующей обработки и выполнения запросов к нему. Наконец, реализуя компонент 3, ввели RAG на основе индексируемого графа знаний. Здесь с помощью LlamaIndex мы автоматически создали из векторизованных данных индекс графа знаний. Он визуально отобразил связи между различными частями информации, обеспечив семантическое представление данных. После этого мы обратились к созданному графу с запросом, используя встроенную в LlamaIndex языковую модель для генерации оптимальных ответов. Помимо этого, реализовали метрики для оценки качества созданной системы, обеспечив точное и эффективное извлечение данных.

К концу главы мы создали обширную автоматизированную систему RAG на основе графа знаний, способную собирать, преобразовывать в эмбеддинги и затем запрашивать обширные объемы данных Википедии при минимальном участии человека. Весь этот процесс показал нам широкий потенциал совмещения нескольких ИИ-инструментов и моделей для создания эффективного пайплайна обработки и извлечения данных. Теперь вы можете самостоятельно реализовывать системы RAG на основе графов знаний в реальных проектах. А в следующей главе научитесь создавать динамическую RAG для краткосрочного применения.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Эта глава была посвящена созданию масштабируемой системы RAG на основе графа знаний с помощью API Википедии и LlamaIndex?

2. Основной пример использования, рассмотренный в этой главе, касался здравоохранения и управления данными?
3. Компонент 1 включает сбор и подготовку документов Википедии с помощью API?
4. В компоненте 2 платформа Deep Lake использовалась для создания реляционной базы данных?
5. В компоненте 3 фреймворк LlamaIndex применялся для создания индекса графа знаний?
6. Реализованная система предназначена для обработки только одной конкретной темы — маркетинга?
7. Описывается ли в этой главе, как извлекать URL-адреса и метаданные со страниц Википедии?
8. Нужен ли для выполнения описанных в этой главе компонентов GPU?
9. Отображает ли индекс графа знаний связи между элементами данных?
10. Требуется ли участие человека на каждом этапе процесса для выполнения запроса к индексу графа знаний?

Ссылки

- Репозиторий GitHub API Википедии: https://github.com/martin-majlis/Wikipedia-API.
- PyVis Network: *Interactive Network Visualization in Python*.

Дополнительная информация

- *Hogan A., Blomqvist E., Cochez M. et al.* Knowledge Graphs. arXiv:2003.02320.

8 Создание динамических RAG с помощью Chroma и Hugging Face Llama

В этой главе вы узнаете, в чем прагматизм динамических RAG. В современном активно развивающемся ландшафте мира ПО чрезвычайно ценна возможность принимать оперативные решения на основе актуальных данных. Ответственные лица во всевозможных сферах, от здравоохранения и научных исследований до управления обслуживанием клиентов, все чаще нуждаются в текущей информации, которая остается актуальной очень недолго. К примеру, на совещании могут требоваться хоть и временные, но тщательно подготовленные данные. Все это ведет к перестройке концепции неизменности данных. Не всю информацию нужно хранить бесконечно. Напротив, во многих случаях акцент смещается на использование точной, релевантной информации, нужной для конкретных задач в конкретное время, например, на ежедневных брифингах или критически важных совещаниях.

В этой главе представлен инновационный и эффективный подход к обработке подобных данных посредством создания эмбеддингов и временных коллекций Chroma. Его суть в том, что каждое утро собирается новая коллекция, содержащая только данные, которые нужны для совещаний этого дня. Это исключает лишние издержки, связанные с долгосрочным хранением данных и управлением ими. В качестве такой информации могут выступать медицинские отчеты для команды медиков, обсуждающих варианты лечения пациентов, отчет о работе с клиентами для сервисных бригад, планирующих решение срочных вопросов, или последние данные научных исследований для ученых, ежедневно принимающих решения в рамках экспериментов. Ознакомившись с теоретической частью, мы создадим программу Python, которая позволит динамически принимать решения на ежедневных совещаниях, опираясь на датасет естественных и физических наук. Такой подход подчеркнет гибкость и эффективность современных тенденций управления данными. В нашем случае сценарий будет следующим. Команде нужно получать актуальную научную информацию, не прибегая к поиску в Интернете или взаимодействию с ИИ-ассистентами онлайн. Наша задача — разработать бесплатного, с открытым исходным кодом, общедоступного ассистента, для чего использовать ресурсы Chroma и Hugging Face.

В первую очередь нужно создать временную коллекцию Chroma. Здесь мы симулируем обработку свежего, ежедневно компилируемого датасета, связанного с конкретной темой предстоящих совещаний, чтобы обеспечить релевантность и согласованность его данных. Для этого скачаем с платформы Hugging Face датасет SciQ, содержащий тысячи научных вопросов по физике, химии и биологии. Оперируя этим датасетом, наша программа будет преобразовывать актуальные на конкретный день данные в эмбеддинги, гарантируя использование в дискуссии последней, наиболее релевантной информации.

Это позволит перед совещанием выполнять проверочные запросы, убеждаясь в соответствии подготовленного материала задачам текущего дня. Наконец, вопросы, возникающие непосредственно на совещании, будут вести к извлечению реальных данных, дополненных с помощью технологии *LLM от Meta AI* (Llama) для генерации динамических дидактических карточек. Эти карточки будут предоставлять быстрые и точные ответы, сохраняя продуктивность дискуссии на основе актуальной информации. К концу главы вы выработаете навыки реализации RAG с открытым исходным кодом в широком спектре областей.

Архитектура динамической RAG

Представьте, что вы работаете в динамической среде, где информация меняется ежедневно. Каждое утро вы собираете свежий пакет из более чем 10 000 вопросов и проверенных ответов по всему миру. Проблема в том, чтобы иметь возможность быстро и эффективно обращаться к этой информации во время совещаний, не прибегая к использованию долгосрочного хранилища или сложной инфраструктуры.

Динамическая RAG позволяет поддерживать компактную отзывчивую систему, предоставляющую актуальную информацию без лишних издержек на хранение данных. Такой механизм отлично подойдет для сред, в которых данные остаются актуальными недолго, но при этом важны для принятия решений.

Применим этот механизм к датасету точных наук. Но такой динамический подход не ограничивается конкретно нашим примером. Он может пригодиться для решения широкого спектра задач в разных областях, например, следующих.

- *Поддержка клиентов.* Реализация ежедневно обновляемых FAQ с возможностью доступа в реальном времени для предоставления быстрых ответов на запросы клиентов.
- *Здравоохранение.* Во время совещаний команды медиков могут использовать последние данные об исследованиях или пациентах, чтобы отвечать на сложные вопросы, связанные с оказанием медицинской помощи.
- *Финансы.* Финансовые аналитики могут запрашивать последние рыночные данные для принятия взвешенных инвестиционных решений и выбора правильных стратегий.

- *Образование*. Преподаватели могут использовать данные образовательных ресурсов и исследований, чтобы отвечать на текущие вопросы и оптимизировать процесс обучения.
- *Техническая поддержка*. IT-команды могут применять наиболее актуальную техническую документацию для решения проблем и эффективного консультирования пользователей.
- *Продажи и маркетинг*. Команды могут быстро анализировать последнюю информацию о продукте и трендах рынка, чтобы отвечать на запросы клиентов и планировать торговые стратегии.

В этой главе мы реализуем один из типов динамической системы RAG. Ограничением здесь станет только ваше воображение, так что можете смело применять эту систему в собственных проектах разными способами. Начнем мы с того, что рассмотрим, как компоненты динамической RAG вписываются в систему, представленную в разделе «Экосистема RAG» главы 1.

С помощью Python упростим эту интеграцию и будем использовать динамическую информацию в контексте принятия решений в реальном времени, например, на ежедневных совещаниях. Вот схематичное представление принципа работы этой инновационной стратегии (рис. 8.1).

- *Создание временной коллекции Chroma (D1, D2, D3, E2)*. Каждое утро под конкретное совещание создается коллекция Chroma. Поскольку такие коллекции служат только для решения задач текущего дня, после совещания они не сохраняются, чтобы не загромождать систему в долгосрочной перспективе.
- *Создание эмбеддингов из релевантных данных (D1, D2, D3, E2)*. На основе созданной коллекции генерируются эмбеддинги необходимых данных, таких как отчеты о работе с клиентами, медицинские отчеты или научные факты. В ходе этого процесса обрабатываются данные конкретно под тему совещания, чтобы у его участников под рукой была вся актуальная информация. Собранные данные могут включать обратную связь от людей в отношении документов и даже другие системы генеративного ИИ.
- *Проверка данных перед совещанием (D4)*. Перед началом совещания временная коллекция Chroma проверяется серией запросов, которые позволяют убедиться в точности содержащейся в ней информации и ее соответствии задачам встречи.
- *Обработка запросов в реальном времени (G1, G2, G3, G4)*. В процессе совещания система способна обрабатывать вопросы, возникающие у его участников. На основе этих вопросов будет извлекаться конкретная информация, используемая для дополнения запроса к Llama и динамической генерации дидактических карточек. Полученные карточки позволяют давать сжатые и точные ответы во время совещания, повышая его продуктивность.

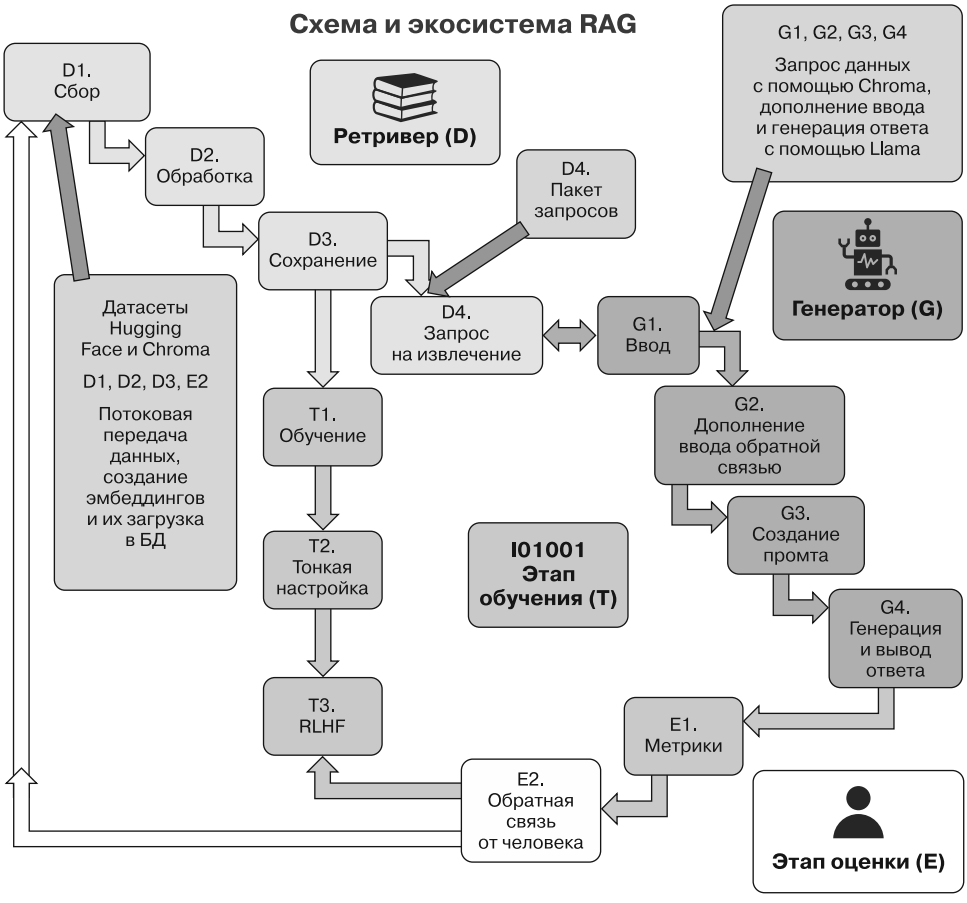


Рис. 8.1. Система динамической RAG

В этом процессе мы будем использовать Chroma — мощную векторную базу данных с открытым исходным кодом и возможностями ИИ, предназначенную для хранения и поиска эмбеддингов в коллекциях, а также управления ими. Chroma содержит все необходимое для начала работы, и ее можно запустить на локальной машине. Она также прекрасно подходит для создания приложений с функциями LLM. В общем, коллекции Chroma оказываются удачным выбором для временной малозатратной системы RAG, работающей в реальном времени. Архитектура динамической RAG с применением Chroma, которую мы реализуем в текущей главе, инновационная и практичная. Вот несколько преимуществ, которые делают ее актуальной в современной быстро меняющейся действительности.

- *Эффективность и экономичность.* Использование Chroma для временного хранилища и Llama для генерации ответов обеспечивает легковесность системы, лишенной текущих затрат на хранение данных. В результате такая систе-

ма идеально подходит для условий работы, в которых данные обновляются часто и долгосрочное хранилище не требуется. Она очень привлекательна для принимающих важные решения людей, для которых имеют значение ее экономичность и эффективность.

- *Гибкость*. Динамичная природа позволяет ежедневно интегрировать в систему новые данные, делая доступной самую актуальную информацию. Особенно ценна такая возможность в оживленных рабочих средах, где информация изменяется часто.
- *Масштабируемость*. Этот подход легко масштабировать на другие аналогичные датасеты при условии, что их можно преобразовать в векторы и эффективно обращаться к ним посредством запросов. Такая возможность делает систему адаптируемой под различные сферы деятельности, выходящие за рамки приведенного примера. Причем масштабирование не только увеличивает объемы данных, но и расширяет возможность применения системы в различных областях и ситуациях.
- *Удобство для пользователя*. Система имеет простой и понятный дизайн, что позволяет получать от нее быстрые ответы даже тем пользователям, которые не обладают глубокими техническими знаниями. Такая простота дополнительно привлекает людей и повышает удовольствие от работы. Удовлетворение запросов пользователей с помощью экономичного, прозрачного и легковесного ИИ однозначно повысит их интерес к генеративным моделям на базе RAG.

Ну а теперь перейдем к созданию динамической программы с применением RAG.

Настройка среды

Нужная нам среда включает ресурсы с открытым кодом, которые можно запускать как на своей машине, так и на бесплатном аккаунте Google Colab. Мы будем работать на Google Colab, используя ресурсы Hugging Face и Chroma.

Начнем с установки Hugging Face.

Hugging Face

Далее мы с помощью открытых ресурсов Hugging Face скачаем датасет для модели Llama. Зарегистрируйтесь по ссылке https://huggingface.co/ и получите токен API. Если вы используете Google Colab, можете с помощью панели инструментов создать Google Secret и активировать его. В таком случае закомментируйте следующую ячейку блокнота: `# Сохраните токен Hugging Face в безопасном месте`:

```
# 1. Раскомментируйте следующие строки, если хотите извлекать токен с Google Drive

from google.colab import drive
drive.mount('/content/drive')
```

```
f = open("drive/MyDrive/files/hf_token.txt", "r")
access_token=f.readline().strip()
f.close()

# 2. Раскомментируйте следующую строку, если хотите вводить токен вручную
#access_token =[YOUR HF_TOKEN]

import os
os.environ['HF_TOKEN'] = access_token
```

Первым делом код извлекает токен API Hugging Face. Сохраните его в надежном месте. Здесь можно выбрать между использованием Google Drive и вводом токена вручную. До этого момента процесс установки шел плавно. Теперь установим `datasets`:

```
!pip install datasets==2.20.0
```

Тем не менее могут возникнуть конфликты: например, на момент написания книги версия `pyarrow` в Hugging Face конфликтует с ее более новой версией в Google Colab. В случае применения быстро развивающихся пакетов такие конфликты не редкость. Когда в Hugging Face обновят свои библиотеки, проблема исчезнет. Но могут возникнуть другие. Этот конфликт не помешает нам скачать датасеты. Если бы помешал, то пришлось бы деинсталлировать пакеты Google Colab и заново установить `pyarrow`, но проблемы могут создать и другие зависимости. К этому нужно быть готовым, о чем говорилось еще в подразделе «Настройка среды» главы 2.

Далее установим пакет Hugging Face `transformers`:

```
!pip install transformers==4.41.2
```

Установим также `accelerate`, чтобы пакеты PyTorch оперировали ресурсами GPU. Для этого блокнота такая оптимизация настоятельно рекомендуется, как и настройка других функций наподобие вычислений со смешанной точностью и ускорения обработки:

```
!pip install accelerate==0.31.0
```

Наконец, мы инициализируем в качестве токенизатора и модели чата `meta-llama/Llama-2-7b-chat-hf`. Llama — это серия языковых моделей от Meta AI (бывшая Facebook AI) на основе трансформеров, которые можно использовать с помощью Hugging Face:

```
from transformers import AutoTokenizer
import tranformers
import torch
model = "meta-llama/Llama-2-7b-chat-hf"
tokenizer = AutoTokenizer.from_pretrained(model)
```

Это делается через пайплайн Hugging Face:

```
pipeline = transformers.pipeline(
    "text-generation",
    model=model,
    torch_dtype=torch.float16,
    device_map="auto",
)
```

Разберем его.

- `transformers.pipeline` — функция для создания пайплайна генерации текста. Он абстрагирует значительную часть сложности, которой в системе динамической RAG желательно избегать.
- `text-generation` указывает тип задачи, для которой настроен пайплайн. В нашем случае это генерация текста.
- `model` указывает выбранную модель.
- `torch_dtype=torch.float16` устанавливает тип данных тензоров PyTorch на `float16`. Это ключевой момент в динамических RAG, который сокращает потребление памяти и может ускорять вычисления, в частности, на GPU, поддерживающих вычисления с использованием чисел половинной точности. В таких вычислениях для ускорения обработки задействуется 16 бит, то есть половина стандартной 32-битной точности. Как раз то, что нам нужно.
- `device_map="auto"` просит пайплайн автоматически определять наилучшее устройство для выполнения модели (CPU, GPU, мультиGPU и т. д.). Этот параметр особенно важен для оптимизации производительности и динамического распределения слоев модели по доступным устройствам (например, GPU) максимально эффективным способом. Если системе доступны несколько GPU, нагрузка будет распределяться по всем параллельно. Если у вас есть доступ к GPU, активируйте эту функцию для ускорения пайплайна.

Пайплайн Hugging Face готов. Теперь займемся Chroma.

Chroma

Следующий код установит векторную базу данных Chroma с открытым кодом:

```
!pip install chromadb==0.5.3
```

Взглянем на следующий фрагмент вывода, который показывает установленные пакеты, в частности *Open Neural Network Exchange (ONNX)*:

```
Successfully installed asgiref-3…onnxruntime-1.18.0…
```

ONNX (https://onnxruntime.ai/) — ключевой компонент текущего сценария, так как он полностью интегрирован с Chroma. Это стандартный формат представления

моделей ML, позволяющий применять их в разных фреймворках и на разном оборудовании, не ограничиваясь одной экосистемой.

Мы будем использовать ONNX Runtime — заточенный под быстродействие механизм для выполнения моделей ONNX. Он выступает в качестве кроссплатформенного ускорителя моделей, обеспечивающего гибкий интерфейс, который позволяет интегрировать их с аппаратными библиотеками. Это дает возможность оптимизировать модели под различные аппаратные конфигурации (CPU, GPU и другие ускорители). В случае Hugging Face рекомендуется активировать GPU, если у вас есть такая возможность. Кроме того, мы выберем модель, присутствующую в установочных пакетах ONNX Runtime.

На этом установка Hugging Face и Chroma, включая ONNX Runtime, закончена. Фреймворк Hugging Face будет использоваться в течение всего жизненного цикла модели, начиная с доступа и развертывания предварительно обученных моделей и заканчивая их обучением и тонкой настройкой внутри его экосистемы. ONNX вдобавок к многим своим возможностям может включаться после обучения модели для обеспечения ее совместимости и эффективного выполнения на разном оборудовании и в разных программных конфигурациях. Модели можно разрабатывать и подстраивать с помощью инструментов Hugging Face, после чего преобразовывать в формат ONNX для широкого и оптимизированного развертывания в рамках ONNX Runtime.

Далее мы используем spaCy для вычисления точности соответствия ответа, полученного на запрос к векторной базе данных, исходному завершенному тексту. Следующая команда установит из spaCy английскую языковую модель среднего размера, предназначенную для общих задач NLP:

```
!python -m spacy download en_core_web_md
```

Это модель `en_core_web_md`. Она разработана на основе взятого из Сети английского текста и сбалансирована по скорости и точности, что важно для нашей динамической RAG. Эта модель хорошо справляется с вычислением сходства текста. Имейте в виду, что после установки ее пакета потребуется перезапустить сеанс.

На этом мы успешно установили оптимизированные и экономичные ресурсы с открытым кодом, которые нужны нам для динамической RAG, и теперь готовы перейти к выполнению самой программы.

Запуск отсчета времени сеанса

В реальных проектах динамической RAG, как и в нашем сценарии, время играет очень важную роль. Например, если ежедневное совещание начинается в 10:00, то ответственная за RAG команда может начать готовиться к нему в 08:00, чтобы собрать в Сети необходимые данные и представить их в формате

обработанных пакетов информации или другим подходящим для задач совещания способом.

Первым делом активируйте использование GPU, если такое возможно. Например, на Google Colab для этого нужно перейти в раздел Runtime ▸ Change runtime type и выбрать GPU. Если такая опция недоступна, код блокнота будет выполняться на центральном процессоре и чуть дольше. Далее мы разберем и выполним каждый раздел блокнота, чтобы хорошо понять весь процесс.

Следующий код активирует измерение времени сеанса, начиная с завершения настройки среды и вплоть до конца блокнота:

```
# Запуск отсчета перед запросом
session_start_time = time.time()
```

Наконец, перезапустите сеанс, снова перейдите в Runtime и щелкните на Run all. Когда программа завершится, перейдите к последнему разделу блокнота, «Общее время сеанса». Здесь вы узнаете, сколько примерно длится подготовка данных. Оставшееся до совещания время можно посвятить корректировке под свои задачи этих данных, а также запросов и параметров модели.

Умение применять динамический подход подготовки RAG на лету кратно повысит ценность команды в современном меняющемся мире. Далее мы приступим к реализации основной части программы и начнем с подготовки датасета.

Скачивание и подготовка датасета

Мы будем использовать датасет SciQ, который в 2017 году собрали Йоханнес Вельбль (Welbl), Нельсон Ф. Лю (Liu) и Мэтт Гарднер (Gardner). Сделали они это путем *краудсорсинга*, собрав от людей со всего мира тематические научные вопросы с многовариантными ответами. Датасет содержит 13 679 таких вопросов, придуманных специально для обучения NLP-моделей, применяющихся в экзаменах по естественным наукам. Процесс создания датасета состоит из двух основных этапов: выбора релевантных абзацев и генерации вопросов с правдоподобными ложными вариантами ответов.

В контексте использования этого набора данных для дополнения запросов посредством применения коллекции Chroma мы реализуем только столбцы `question`, `correct_answer` и `support`. Датасет содержит также столбцы `distractor` с ложными ответами, но их мы отбросим.

Подготовленный датасет интегрируем в систему извлечения данных, чтобы повысить релевантность извлекаемых вопросов на основе конкретной научной темы или формата вопроса, задаваемого модели Hugging Face Llama. Это позволит реализовать в программе текущей главы динамическую генерацию

дополненного завершенного текста. Сначала программа загружает обучающие данные из датасета `sciq`:

```
# Импорт необходимых библиотек
from datasets import load_dataset
import pandas as pd

# Загрузка датасета SciQ с HuggingFace
dataset = load_dataset("sciq", split="train")
```

Далее датасет очищается так, чтобы сохранились только непустые столбцы `support` и `correct_answer`:

```
# Очистка датасета, чтобы остались только вопросы с "support" и "correct answer"
filtered_dataset = dataset.filter(lambda x: x["support"] != "" and
  x["correct_answer"] != "")
```

Выведем количество сохраненных строк:

```
# Вывод количества вопросов с "support"
print("Number of questions with support: ", len(filtered_dataset))
```

Вывод показывает, что всего получился 10 481 документ:

```
Number of questions with support: 10481
```

Нам нужно очистить полученный DataFrame, чтобы остались только нужные столбцы. Отбросим ложные варианты:

```
# Преобразуем очищенный датасет в DataFrame
df = pd.DataFrame(filtered_dataset)

# Какие столбцы удалить
columns_to_drop = ['distractor3', 'distractor1', 'distractor2']

# Удаление столбцов из DataFrame
df.drop(columns=columns_to_drop, inplace=True)
```

Теперь у нас остались только верные ответы и пояснительная информация, которые мы совместим:

```
# Создаем столбец 'completion', совмещая 'correct_answer' и 'support'
df['completion'] = df['correct_answer'] + " because " + df['support']

# Делаем так, чтобы в столбце 'completion' не было значений NaN
df.dropna(subset=['completion'], inplace=True)
df
```

В приведенном далее выводе показаны столбцы, необходимые для подготовки данных к извлечению в столбце `completion`. Здесь мы видим фрагмент DataFrame из поля завершенного текста (completion), в котором правильный ответ aerobic, а остальная часть текста выступает пояснением к нему:

```
aerobic because "Cardio" has become slang for aerobic exercise that raises your
heart rate for an extended amount of time. Cardio can include biking, running,
or swimming. Can you guess one of the main organs of the cardiovascular system?
Yes, your heart.
```

Далее программа выводит форму DataFrame:

```
df.shape
```

Вывод подтверждает, что у нас по-прежнему есть все изначальные строки и четыре столбца:

```
(10481, 4)
```

Этот код выведет названия столбцов:

```
# Предполагается, что ваш DataFrame называется 'df'
print(df.columns)
```

В выводе видим четыре нужных нам столбца:

```
Index(['question', 'correct_answer', 'support', 'completion'], dtype='object')
```

Теперь эти данные можно преобразовывать в эмбеддинги и загружать в БД.

Создание эмбеддингов и их загрузка в коллекцию Chroma

Начнем с создания клиента Chroma и определения коллекции:

```
# Импортируем Chroma и инстанцируем клиента. По умолчанию клиент Chroma
# создается временно и на диск ничего не сохраняет
import chromadb
client = chromadb.Client()
collection_name="sciq_supports6"
```

Прежде чем создавать коллекцию и загружать в нее данные, нужно проверить, не существует ли уже такая коллекция:

```
# Перебираем все коллекции
collections = client.list_collections()

# Проверяем, есть ли среди них конкретная коллекция
collection_exists = any(collection.name == collection_name for collection
  in collections)
print("Collection exists:", collection_exists)
```

Если коллекция существует, вывод будет `True`, если нет — `False`:

```
Collection exists: False
```

Если такой коллекции не существует, мы создаем ее с заданным ранее именем `collection_name`:

```
# Создаем коллекцию Chroma для хранения пояснительной информации. Функцию
# создания эмбеддингов указывать не нужно, будет использована штатная
if collection_exists!=True:
  collection = client.create_collection(collection_name)
else:
  print("Collection ", collection_name," exists:", collection_exists)
```

Взглянем на структуру словаря созданной коллекции:

```
# Вывод словаря
results = collection.get()
for result in results:
    print(result)    # Вывод словаря для каждого элемента
```

В выводе показан словарь каждого элемента коллекции:

```
ids
embeddings
metadatas
documents
uris
data
included
```

Вкратце рассмотрим три ключевых поля для нашего сценария:

- `ids` представляет уникальные идентификаторы для каждого элемента коллекции;
- `embeddings` — эмбеддинги документов;
- `documents` относится к столбцу `completion`, в котором мы совместили ответ и пояснительный текст.

Теперь нашему механизму RAG нужна легковесная и быстрая модель LLM.

Выбор модели

Chroma инициализирует предустановленную модель, которой может оказаться `all-MiniLM-L6-v2`. Но для полной уверенности лучше сделать это самим:

```
model_name = "all-MiniLM-L6-v2"  # Имя модели, которая будет использоваться
                                 # для создания эмбеддингов и их запроса
```

Модель `all-MiniLM-L6-v2` разрабатывалась с применением оптимального, доработанного группой исследователей (Wang et al., 2021) метода сжатия с акцентом на взаимодействии между компонентами трансформеров посредством механизма внимания. Этот подход позволяет настраивать количество голов внимания (attention heads) между обучающей и обучаемой моделями,

повышая эффективность сжатия. Итоговая модель полностью интегрируется с Chroma с помощью ONNX, о чем говорится в разделе «Настройка среды» текущей главы.

Вся магия MiniLM заключается в сжатии и дистилляции знаний между обучающей и обучаемой моделями.

- *Обучающая модель*. Исходная модель, обычно более крупная и сложная, наподобие BERT, RoBERTa и XLM-R. В нашем случае она была предварительно обучена на масштабном датасете. Обучающая модель обладает точным и глубоким пониманием задач, на которых обучалась. Она выступает источником знаний, которые мы хотим передать.
- *Обучаемая модель*. Это наша более мелкая и менее сложная модель `all-MiniLM-L6-v2`, обученная имитировать поведение учителя, что будет очень кстати для архитектуры динамической RAG. Нам нужно, чтобы обучаемая модель смогла максимально точно воспроизвести поведение обучающей, используя при этом намного меньше параметров и вычислительных ресурсов.

В нашем случае `all-MiniLM-L6-v2` ускорит процесс создания и запроса эмбеддингов. Мы увидим, что сейчас, в эпоху продвинутых моделей LLM, таких как GPT-4o, есть возможность выполнять повседневные задачи с помощью более компактных моделей, оперирующих «сжатыми» знаниями. Перейдем к созданию эмбеддингов.

Создание эмбеддингов и сохранение завершенного текста

Процесс создания эмбеддингов и их загрузки в коллекцию Chroma протекает легко и быстро. В этом сценарии мы преобразуем в векторы и загрузим все завершенные тексты ответов датасета `df` в `completion_list`:

```
ldf=len(df)
nb=ldf    # Сколько вопросов нужно преобразовать в эмбеддинги и сохранить

import time
start_time = time.time()    # Начинаем отсчет перед запросом

# Делаем из последовательности список строк
completion_list = df["completion"][:nb].astype(str).tolist()
```

Чтобы избежать повторной загрузки данных, мы используем `collection_exists`, которую определили при создании коллекции. Здесь у нас временная коллекция, которую нужно загрузить и использовать всего раз. При попытке повторной загрузки данных будут выводиться предупреждения. Но вы вполне можете изменить код, если хотите попробовать другие датасеты и методы, например, быстро подготовить прототип для другого проекта.

В текущем сценарии мы сначала проверяем, существует ли коллекция, после чего загружаем `id` и `documents` в `complete_list`, сохраняя тип (`type`) данных, а именно `completion`, в поле `metadatas`:

```
# Избежание повторной загрузки данных в этом блокноте создания динамической RAG
if collection_exists!=True:
  # Преобразуем в эмбеддинги и сохраняем первые nb пояснительных текстов
  # для этой демонстрации
  collection.add(
      ids=[str(i) for i in range(0, nb)],    # ID – это просто строки
      documents=completion_list,
      metadatas=[{"type": "completion"} for _ in range(0, nb)],
  )
```

В завершение также измеряем время ответа:

```
response_time = time.time() - start_time    # Измеряем время ответа
print(f"Response Time: {response_time:.2f} seconds")  # Выводим время ответа
```

Вывод показывает, что в данном случае Chroma активировала предустановленную модель через onnx, о чем говорилось в начале текущего раздела и в разделе «Настройка среды»:

```
/root/.cache/chroma/onnx_models/all-MiniLM-L6-v2/onnx.tar.gz: 100%|
79.3M/79.3M [00:02<00:00, 31.7MiB/s]
```

В выводе также отражается время обработки более чем 10 000 документов, в данном случае удовлетворительное:

```
Response Time: 234.25 seconds
```

Время ответа может варьироваться и зависит от того, задействуете ли вы для обработки GPU. При использовании GPU время выполнения вписывается в требования для сценариев динамических RAG.

На этом заполнение базы данных Chroma завершено. Теперь взглянем на полученные эмбеддинги.

Вывод эмбеддингов

Далее программа получает эмбеддинги и выводит первый:

```
# Получение эмбеддингов из коллекции
result = collection.get(include=['embeddings'])

# Извлечение из результата первого эмбеддинга
first_embedding = result['embeddings'][0]

# Если вам нужно поработать с длиной первого эмбеддинга:
embedding_length = len(first_embedding)

print("First embedding:", first_embedding)
print("Embedding length:", embedding_length)
```

Вывод первого эмбеддинга показывает, что завершенные тексты были векторизованы:

```
First embedding: [0.03689068928360939, -0.05881563201546669,
-0.04818134009838104,…
```

В нем также присутствует длина эмбеддинга, кстати, заслуживающая особого внимания:

```
Embedding length: 384
```

Модель `all-MiniLM-L6-v2` уменьшает сложность текстовых данных за счет отображения предложений и абзацев в пространстве размерностью 384. Такой объем значительно меньше стандартной размерности векторов в унитарной кодировке (one-hot encoding), например 1526 измерений в модели OpenAI `text-embedding-ada-002`. Это показывает, что в `all-MiniLM-L6-v2` используются плотные векторы, задействующие для кодирования информации все векторное пространство для создания более детальных семантических связей между разными документами, чем в случае разреженных векторов.

Иногда модели разреженных векторов, такие как *мешок слов* (bag-of-words, BoW), тоже вполне эффективны. Однако их основное ограничение в том, что они не отражают порядок слов окружающего контекста, что может критически сказаться на понимании смысла текста при обучении LLM.

На этом мы закончили преобразование векторов в плотные векторы, которые используют пространство меньшей размерности, чем полномасштабные LLM, но обеспечивают удовлетворительные результаты.

Обращение к коллекции

Код текущего раздела выполняет запрос к базе данных Chroma с помощью встроенной в нее функции семантического поиска. Он запрашивает все векторы вопросов коллекции Chroma исходного датасета:

```
dataset["question"][:nbq].
```

Мы устанавливаем `n_results=1`, чтобы находить по одному наиболее релевантному документу для каждого вопроса, но этот параметр можно изменить.

Текст каждого вопроса преобразуется в эмбеддинг. Затем Chroma выполняет поиск на основе векторного сходства, сопоставляя полученные эмбеддинги с нашей коллекцией векторов документов, чтобы выявить максимальное совпадение:

```
import time
start_time = time.time()    # Начинаем отсчет перед запросом

# Количество извлекаемых записей
results = collection.query(
```

```
    query_texts=df["question"][:nb],
    n_results=1)

response_time = time.time() - start_time    # Измеряем время ответа
print(f"Response Time: {response_time:.2f} seconds")  # Выводим время ответа
```

Для более чем 10 000 запросов получено удовлетворительное время выполнения:

```
Response Time: 199.34 seconds
```

Теперь проанализируем эти более чем 10 000 запросов. Мы будем с помощью spaCy оценивать результаты запроса и сравнивать их с исходным завершенным текстом. Сначала загрузим модель spaCy, которую установили в разделе «Настройка среды» текущей главы:

```
import spacy
import numpy as np

# Загружаем обученную языковую модель spaCy
nlp = spacy.load('en_core_web_md')
# Модель должна быть установлена с помощью инструкции
# 'python -m spacy download en_core_web_md'
```

Далее программа создает функцию оценки сходства, которая получает два аргумента, изначальный завершенный `text1` и извлеченный `text2`, и возвращает показатель их сходства:

```
def simple_text_similarity(text1, text2):
    # Преобразуем тексты в объекты документов spaCy
    doc1 = nlp(text1)
    doc2 = nlp(text2)

    # Получаем векторы для каждого документа
    vector1 = doc1.vector
    vector2 = doc2.vector

    # Вычисляем косинусное сходство между этими векторами
    # Проверяем наличие векторов, чтобы избежать деления на нуль
    if np.linalg.norm(vector1) == 0 or np.linalg.norm(vector2) == 0:
        return 0.0    # Возвращаем нуль, если один из текстов
                      # не имеет векторного представления
    else:
        similarity = np.dot(vector1, vector2) / (np.linalg.norm(vector1) *
          np.linalg.norm(vector2))
        return similarity
```

Теперь проверим все 10 000 запросов. В следующем блоке кода видно, что проверка начинается с определения необходимых переменных:

- `Nbqd` — для вывода только первых и последних 100 результатов;
- `acc_counter` — для подсчета результатов с показателем сходства выше 0,5. Этот критерий можно изменить;

- `display_counter` — для подсчета количества выведенных результатов:

```
nbqd = 100  # Количество ответов, которые нужно вывести,
            # когда записей более 100

# Выводим вопрос, исходный завершенный текст и извлеченный документ,
# сравнивая два последних
acc_counter=0
display_counter=0
```

Код перебирает `nb` результатов, число которых равно всей длине нашего датасета:

```
for i, q in enumerate(df['question'][:nb]):
    # Обращение к исходному завершенному тексту, связанному с вопросом
    original_completion = df['completion'][i]
    # Извлечение соответствующего документа
    retrieved_document = results['documents'][i][0]
    similarity_score = simple_text_similarity(original_completion,
      retrieved_document)
```

Код находит исходный завершенный текст и сохраняет его в `original_completion`. Затем он извлекает результат и сохраняет его в `retrieved_document`. Наконец, он вызывает определенную ранее функцию оценки сходства `simple_text_similarity`. Результат сравнения сходства извлеченного документа и завершенного текста сохраняется в `similarity_score`.

Далее добавим метрику оценки точности. В этом сценарии порог сходства будет установлен на 0,7, что вполне разумно:

```
if similarity_score > 0.7:
  acc_counter+=1
```

Если `similarity_score > 0.7`, счетчик `acc_counter` инкрементируется. Второй счетчик, `display_counter`, тоже инкрементируется, но только до установленного в начале функции числа первых и последних `nbqd` (максимума выводимых результатов):

```
display_counter+=1
if display_counter<=nbqd or display_counter>nb-nbqd:
```

Представленная информация отражает эффективность работы системы:

```
print(i," ", f"Question: {q}")
print(f"Retrieved document: {retrieved_document}")
print(f"Original completion: {original_completion}")
print(f"Similarity Score: {similarity_score:.2f}")
print()    # Пустая строка между записями для лучшей читаемости
```

В выводе отражаются четыре ключевые переменные:

- `{q}` — заданный вопрос, то есть запрос;
- `{retrieved_document}` — извлеченный документ;

- `{original_completion}` — исходный документ датасета;
- `{similarity_score:.2f}` — показатель сходства между исходным и извлеченным документами, позволяющий оценить качество каждого ответа.

В начале вывода показывается информация, запрошенная экспертом для контроля ответа на запрос и его отслеживания вплоть до источника.

Первым идет вопрос, он же запрос:

```
Question: What type of organism is commonly used in preparation of foods such as
cheese and yogurt?
```

Вторая часть — это извлеченный документ:

```
Retrieved document: lactic acid because Bacteria can be used to make cheese
from milk. The bacteria turn the milk sugars into lactic acid. The acid is
what causes the milk to curdle to form cheese. Bacteria are also involved in
producing other foods. Yogurt is made by using bacteria to ferment milk ( Figure
below ). Fermenting cabbage with bacteria produces sauerkraut.
```

Третья часть — это исходный завершенный текст. В данном случае видно, что извлеченный текст несет релевантную информацию, но не точный исходный текст:

```
Original completion: mesophilic organisms because Mesophiles grow best in
moderate temperature, typically between 25°C and 40°C (77°F and 104°F).
Mesophiles are often found living in or on the bodies of humans or other
animals. The optimal growth temperature of many pathogenic mesophiles is 37°C
(98°F), the normal human body temperature. Mesophilic organisms have important
uses in food preparation, including cheese, yogurt, beer and wine.
```

Наконец, вывод показывает вычисленное spaCy сходство:

```
Similarity Score: 0.73
```

Этот показатель говорит, что хоть выбранный ответ и отличался от исходного завершенного текста, этот выбор оказался релевантным.

После анализа всех результатов программа вычисляет среднюю точность для более чем 10 000 запросов:

```
if nb>0:
  acc=acc_counter/nb
```

В основе вычисления лежит следующее:

- `Acc` — общая полученная точность;
- `acc_counter` — общее количество результатов с `Similarity score > 0.7`;
- `nb` — количество запросов, в нашем случае `nb=len(df)`;
- `acc=acc_counter/nb` — вычисляет общую точность для всех результатов.

Затем код выводит количество оцененных документов и общий показатель сходства:

```
print(f"Number of documents: {nb:.2f}")
print(f"Overall similarity score: {acc:.2f}")
```

Вывод показывает, что на все вопросы были возвращены релевантные ответы:

```
Number of documents: 10481.00
Overall similarity score: 1.00
```

Этот удовлетворительный общий показатель демонстрирует, что система работает в закрытой среде. Нам же нужно расширить контекст и посмотреть, как она себя покажет в открытом контексте активного обсуждения темы на совещании.

Создание промта и извлечение ответа

Код данного раздела относится к обработке запросов в ходе совещания. Интерфейс можете настроить под свои задачи. Мы же сосредоточимся на функциональности.

Разберем первый промт:

```
# Изначальный вопрос
prompt = "Millions of years ago, plants used energy from the sun to form what?"
# Вариант 1 – похожий
#prompt = "Eons ago, plants used energy from the sun to form what?"
# Вариант 2 – измененный
#prompt = "Eons ago, plants used sun energy to form what?"
```

Здесь у нас два закомментированных варианта запроса. Давайте их проясним.

- Изначальный вопрос — это точный текст, который присутствует в исходном датасете. Маловероятно, что участник совещания или пользователь зададут его именно в таком виде. Но его можно использовать для проверки работоспособности системы.
- Вариант 1 похож на изначальный вопрос и вполне может прозвучать на совещании.
- Вариант 2 несколько изменен и может создать для модели сложности.

В этом разделе выберем вариант 1 и получим удовлетворительный результат.

Здесь мы в очередной раз убедимся, что для всех программ ИИ необходим контроль со стороны человека. Чем более видоизмененным относительно исходного вопроса становится вариант 2, тем сложнее системе сохранять стабильность и давать ожидаемые ответы. И это ограничение объясняет, почему, несмотря на быструю адаптируемость динамической RAG, проектирование надежной системы требует внимательной и непрерывной доработки.

Если обратиться к коллекции так, как мы делали в предыдущем разделе, используя только один промт, то ответ вернется быстро:

```
import time
import textwrap

# Начинаем отсчет перед запросом
start_time = time.time()

# Обращаемся к коллекции, применяя промт
results = collection.query(
    # Используем промт в списке, как того ожидает метод выполнения запроса
    query_texts=[prompt],
    n_results=1    # Сколько результатов извлечь
)

# Измеряем время ответа
response_time = time.time() - start_time

# Выводим время ответа
print(f"Response Time: {response_time:.2f} seconds\n")

# Проверяем, были ли извлечены документы
if results['documents'] and len(results['documents'][0]) > 0:
    # Форматируем вывод для лучшей читаемости
    # Сокращаем длину строки вывода до 70 символов
    wrapped_question = textwrap.fill(prompt, width=70)
    wrapped_document = textwrap.fill(results['documents'][0][0], width=70)

    # Выводим отформатированные ответы
    print(f"Question: {wrapped_question}")
    print("\n")
    print(f"Retrieved document: {wrapped_document}")
    print()
else:
    print("No documents retrieved."
```

Ответ получен очень быстро:

```
Response Time: 0.03 seconds
```

Вывод показывает, что извлеченный документ релевантен запросу:

```
Response Time: 0.03 seconds

Question: Millions of years ago, plants used energy from the sun to form what?

Retrieved document: chloroplasts because When ancient plants underwent
photosynthesis, they changed energy in sunlight to stored chemical energy in
food. The plants used the food and so did the organisms that ate the plants.
After the plants and other organisms died, their remains gradually changed
to fossil fuels as they were covered and compressed by layers of sediments.
Petroleum and natural gas formed from ocean organisms and are found together.
Coal formed from giant tree ferns and other swamp plants.
```

Мы успешно извлекли результат запроса. Такой семантический векторный поиск иногда может вполне удовлетворить участников совещания. В случае чего всегда можно доработать механизм, дополнительно задействовав Llama.

Далее мы передадим ответ модели Llama, которая напишет на него короткую сводку, используя NLP.

RAG с Llama

В разделе «Настройка среды» мы инициализировали `meta-llama/Llama-2-7b-chat-hf`. Теперь нужно создать функцию, которая будет регулировать поведение Llama 2:

```
def LLaMA2(prompt):
    sequences = pipeline(
        prompt,
        do_sample=True,
        top_k=10,
        num_return_sequences=1,
        eos_token_id=tokenizer.eos_token_id,
        max_new_tokens=100,    # Более детальный контроль длины вывода
        temperature=0.5,    # Слегка повышена для большего разнообразия
        repetition_penalty=2.0,    # Корректируйте, исходя из результатов
        truncation=True
    )
    return sequences
```

Каждый из этих параметров можно настраивать в соответствии с ожиданиями.

- `prompt` — входной текст, который модель использует для генерации вывода. Это стартовая точка для ее ответа.
- `do_sample` — логическое значение (`True` либо `False`). При установке `True` включает стохастическую выборку, то есть модель будет выбирать токены случайно на основе их вероятностного распределения. Позволяет получать более разнообразные результаты.
- `top_k` — ограничивает число наиболее вероятных токенов словаря, которые будут рассматриваться в процессе выборки. Установка этого параметра равным `10` означает, что модель будет выбирать из десяти наиболее вероятных следующих токенов.
- `num_return_sequences` — указывает количество независимо сгенерированных ответов. Здесь мы установили значение `1`, то есть функция будет возвращать для каждого промта одну последовательность.
- `eos_token_id` — этот токен отмечает конец последовательности в токенизированной форме. На нем модель завершает генерацию. Завершающий токен представлен в виде `id`, указывающего на Llama `eos_token`.

- `max_new_tokens` — ограничивает число новых токенов, которые может сгенерировать модель. Здесь установлено значение `100`, то есть длина вывода может превышать размер промта на 100 токенов.
- `temperature` — регулирует случайность выборки. Температура `0.5` ведет к получению достаточно акцентированных ответов по сравнению с более высокими ее значениями, но в то же время допускает некоторую вариативность.
- `repetition_penalty` — модификатор, уменьшающий вероятность повторения одного и того же токена. Штраф `2.0` означает низкую вероятность повторного выбора любого уже используемого токена, что способствует генерации более разнообразного текста с меньшим числом повторов.
- `truncation` — в активном состоянии обеспечивает соблюдение максимальной длины, указанной в `max_new_tokens`, отсекая лишние токены.

Промт будет содержать инструкцию для Llama из `ipromt` и результат, полученный в разделе блокнота «Промт и извлечение данных». Этот результат добавляется к `ipromt`:

```
iprompt='Read the following input and write a summary for beginners.'
lprompt=iprompt + " " + results['documents'][0][0]
```

Дополненный ввод, используемый для вызова Llama (завершенный текст), называется `lprompt`. Следующий код будет измерять время его создания и выполнять запрос:

```
import time
start_time = time.time()    # Начинаем отсчет перед запросом

response=LLaMA2(lprompt)
```

Теперь извлекаем из ответа сгенерированный текст и выводим время, которое потребовалось Llama для его генерации:

```
for seq in response:
    # Удаляем из вывода часть, представляющую ввод
    generated_part = seq['generated_text'].replace(iprompt, '')

response_time = time.time() - start_time    # Измеряем время ответа
print(f"Response Time: {response_time:.2f} seconds")  # Выводим время ответа
```

Результат показывает, что Llama вернула завершенный текст за вполне адекватный промежуток времени:

```
Response Time: 5.91 seconds
```

Теперь оформим ответ в более читабельный формат для вывода:

```
wrapped_response = textwrap.fill(response[0]['generated_text'], width=70)
print(wrapped_response)
```

В техническом смысле текст вывода верен:

```
chloroplasts because When ancient plants underwent photosynthesis, they changed
energy in sunlight to stored chemical energy in food. The plants used the
food and so did the organisms that ate the plants. After the plants and other
organisms died, their remains gradually changed to fossil fuels as they were
covered and compressed by layers of sediments. Petroleum and natural gas formed
from ocean organisms and are found together. Coal formed from giant tree ferns
and other swamp plants. Natural Gas: 10% methane (CH4) - mostly derived from
anaerobic decomposition or fermentation processes involving microorganism such
As those present In wetlands; also contains smaller amounts Of ethene(C2H6),
propiene/propadiene/( C3 H5-7). This is where most petrol comes frm! But
there're more complex hydrocarbons like pentanes & hexans too which can come
```

С технической точки зрения сгенерированный Llama ответ вполне приемлем. Чтобы получить другой, возможно, более качественный результат в рамках того же сеанса, пользователь может несколько раз выполнить запрос и дополненную генерацию с другими параметрами Llama.

Можно даже попробовать другую LLM. У динамических RAG исходный код не всегда открыт на 100 %. Если того требуют обстоятельства, нужно быть прагматичными и задействовать все необходимое. Например, приведенный далее промт, который мы использовали для запроса Llama, был передан ChatGPT на базе GPT-4o:

```
Write a nice summary with this text: Question: Millions of years ago, plants used
energy from the sun to form what?

Retrieved document: chloroplasts because When ancient plants underwent
photosynthesis, they changed energy in sunlight to stored chemical energy in
food. The plants used the food and so did the organisms that ate the plants.
After the plants and other organisms died, their remains gradually changed
to fossil fuels as they were covered and compressed by layers of sediments.
Petroleum and natural gas formed from ocean organisms and are found together.
Coal formed from giant tree ferns and other swamp plants.
```

В данном случае GPT-4o превосходит Llama 2, давая более качественный ответ:

```
Millions of years ago, plants harnessed energy from the sun through
photosynthesis to produce food, storing chemical energy. This energy was vital
for the plants themselves and for the organisms that consumed them. Over time,
the remains of these plants and animals, buried under sediment, transformed into
fossil fuels. Ocean organisms' remains contributed to the formation of petroleum
and natural gas, often found together, while the remains of giant tree ferns and
swamp plants formed coal.
```

Если потребуется, вы можете заменить `meta-llama/Llama-2-7b-chat-hf` на GPT-4o, как мы делали в главе 4, и настроить эту модель на получение ответов такого уровня. Единственным правилом в динамических RAG является эффективность. Теперь мы увидели, что реализовать подобную систему можно разными способами. Как только сеанс будет завершен, коллекцию можно удалить.

Удаление коллекции

Вручную коллекция удаляется следующим кодом:

```
#client.delete_collection(collection_name)
```

Это также можно сделать путем закрытия сеанса. Теперь проверим, существует ли созданная нами коллекция `collection_name`:

```
# Перебираем все коллекции
collections = client.list_collections()

# Проверяем, существует ли конкретная коллекция
collection_exists = any(collection.name == collection_name for collection
  in collections)
print("Collection exists:", collection_exists)
```

Если мы все еще работаем с коллекцией в сеансе, то ответ будет `True`:

```
Collection exists: True
```

Если удалить коллекцию с помощью кода или закрытия сеанса, ответ будет `False`. Теперь посмотрим, какой была продолжительность сеанса.

Общая продолжительность сеанса

Код, приведенный далее, измеряет время от начала сеанса до момента, следующего за разделом «Настройка среды»:

```
end_time = time.time() - session_start_time    # Измеряем время ответа
# Выводим время ответа
print(f"Session preparation time: {response_time:.2f} seconds")
```

Вывод может показывать:

- время, в течение которого мы готовили сценарий динамической RAG, включая ежедневную генерацию коллекции Chroma, запрос из нее данных и дополнение запроса с помощью Llama;
- время, затраченное на выполнение всего блокнота без вмешательства человека.

В нашем случае время сеанса — это полное выполнение программы без участия человека.

```
Session preparation time: 780.35 seconds
```

Весь процесс занимает меньше 15 мин, что укладывается в ограничение по времени подготовки в сценарии с динамической RAG. Так что здесь можно до-

полнительно прогнать программу несколько раз, чтобы лучше подготовить ее к совещанию.

Итак, мы успешно разобрали процесс построения динамической RAG и теперь подведем итоги.

Резюме

В нашем быстро развивающемся мире возможность оперативно собрать информацию для принятия решений обеспечивает конкурентное преимущество. Динамическая RAG позволяет задействовать потенциал ИИ на совещаниях благодаря быстрым и экономичным моделям. Мы создали систему, которая симулировала потребность в получении ответов на вопросы в сфере точных наук, возникающие в рамках ежедневных совещаний. После настройки всех компонентов среды мы скачали и подготовили датасет научных вопросов и ответов SciQ. Участники совещания не хотят тратить время на поиск ответов в Интернете, чтобы не затягивать принятие важных решений. Этот механизм может эффективно применяться для маркетинговой кампании, фактчекинга материалов статьи и в любой ситуации, где требуются знания из области точных наук.

Мы создали векторную БД Chroma. Затем преобразовали в эмбеддинги и загрузили в нее более 10 000 документов, используя модель `all-MiniLM-L6-v2`. Весь этот процесс потребовал минимум средств и оказался довольно быстрым. Коллекцию мы создали локально, поэтому платить за ее хранение не потребовалось. При этом она была временной, а значит, никаких лишней траты пространства и его захламления. После этого мы обратились к созданной коллекции с запросом, чтобы оценить точность системы. Результаты оказались удовлетворительными, и мы обработали весь датасет для их подтверждения. Наконец, мы реализовали функциональность для создания промтов и запросов к модели в реальном времени в процессе совещания. Результат запроса дополнял ввод, выполненный пользователем и переданный модели `meta-llama/Llama-2-7b-chat-hf`, которая на его основе давала краткие обобщенные сведения.

Для окончательного релиза нашего примера динамической RAG потребовались бы некоторые доработки. Тем не менее он уже является отличной базой, позволяющей создавать легковесные генеративные ИИ с открытым кодом на базе RAG для быстрого сбора данных, преобразования их в эмбеддинги и выполнения запросов. Если мы хотим сохранить данные для извлечения, не создавая большие векторные БД, можно интегрировать наши датасеты, например, в модель GPT-4o-mini от OpenAI посредством тонкой настройки. Об этом речь пойдет в следующей главе.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Обеспечивает ли наш скрипт, чтобы токен Hugging Face API из соображений безопасности никогда не прописывался в самом блокноте?
2. Библиотека `accelerate` в программе текущей главы использовалась для упрощения развертывания моделей ML на облачных платформах?
3. Помимо токена API, в этом скрипте для доступа к БД Chroma необходима также аутентификация пользователя?
4. Chroma применяется в блокноте для временного хранения векторов в процессе динамического извлечения данных?
5. Блокнот настроен на использование механизма ускорения запросов в реальном времени за счет GPU?
6. Можно ли использовать измеряемое в этом блокноте время сеанса для оптимизации процесса динамической RAG?
7. Демонстрирует ли этот скрипт возможность совмещения Chroma с моделями ML для повышения эффективности извлечения данных?
8. Включает ли этот скрипт возможность корректировки параметров БД Chroma на основе показателей эффективности выполнения сеанса?

Ссылки

- *Welbl J., Liu N. F., Gardner M.* Crowdsourcing Multiple Choice Science Questions: http://arxiv.org/abs/1707.06209.
- *Wang W., Bao H., Huang S., Dong L., Wei F.* MiniLMv2: Multi-Head Self-Attention Relation Distillation for Compressing Pretrained Transformers: https://arxiv.org/abs/2012.15828.
- Документация модели Hugging Face Llama: https://huggingface.co/docs/transformers/main/en/model_doc/llama.
- ONNX: https://onnxruntime.ai/.

Дополнительная информация

- *Wang W., Wei F., Dong L., Bao H., Yang N., Zhou M.* MiniLM: Deep Self-Attention Distillation for Task-Agnostic Compression of Pre-Trained Transformers: https://arxiv.org/abs/2002.10957.
- *Touvron H., Lavril T., Lzacard G. et al.* LLaMA: Open and Efficient Foundation Language Models: https://arxiv.org/abs/2302.13971.
- Создание пакета ONNX Runtime: https://onnxruntime.ai/docs/build/custom.html#custom-build-packages.

9 Улучшение моделей ИИ: тонкая настройка данных RAG и обратная связь

Организация, которая постоянно увеличивает объем данных, используемых для RAG, однажды достигнет порога, когда потребуется перевести непараметрические данные (не преобразованные в веса LLM) в параметрические. Это объясняется тем, что управлять массивом собранных данных RAG становится очень трудно. Начинают возникать сложности, связанные с затратами на хранение, необходимыми для извлечения ресурсами и возможностями самих моделей генеративного ИИ. Более того, предварительно обученная модель оперирует данными, актуальными до определенной даты, то есть не учитывает новые знания, начиная буквально со следующего дня. Это означает, что пользователь не сможет вести диалог с чат-ботом по теме газетной статьи, вышедшей после конечной даты обучения. В таких ситуациях механизм извлечения дополнительной информации начинает играть ключевую роль в предоставлении актуального содержимого посредством RAG.

Крупным компаниям, таким как Google, Microsoft, Amazon, требуется экспоненциально наращивать количество данных и доступных ресурсов. Определенные сферы, например законодательная система США, требуют огромных объемов данных. Однако список таких сфер весьма ограничен. У многих корпораций нет необходимости поддерживать столь масштабные датасеты, и в некоторых случаях, например в сфере точных наук, большие объемы статических данных могут оставаться неизменными довольно долго. Такие данные можно тонко настроить для уменьшения их объема, необходимого механизму RAG.

В текущей главе мы начнем с изучения схемы уменьшения объема данных RAG посредством тонкой настройки. Для наглядной демонстрации этой техники применяем датасет, содержащий готовые к использованию документы, а также подчеркивающий важность обратной связи от человека. Мы покажем, как преобразовывать непараметрические данные в параметрический, тонко настроенный вариант в модели OpenAI. Для этого скачаем и подготовим датасет из предыдущей главы, преобразовав его данные в структурированные пары «промт — ответ» для последующей тонкой настройки в формате JSONL. Мы настроим малозатратную модель GPT-4o-mini, которой будет достаточно для выдачи ответов. После настройки модели протестируем ее на нашем датасете,

чтобы убедиться в успешном усвоении ею необходимых данных. В завершение познакомимся с интерфейсом метрик OpenAI, который позволяет мониторить технические показатели, такие как точность и объем потребления ресурсов, для анализа экономической целесообразности нашего подхода.

Схема тонкой настройки статических данных RAG

В этом разделе мы поставим под сомнение использование непараметрических данных, когда их объем начинает выходить за определенный порог, о чем говорилось в разделе «RAG или тонкая настройка?» главы 1. Там описывался принцип определения этого порога, и на рис. 9.1 он скорректирован для темы текущего раздела.

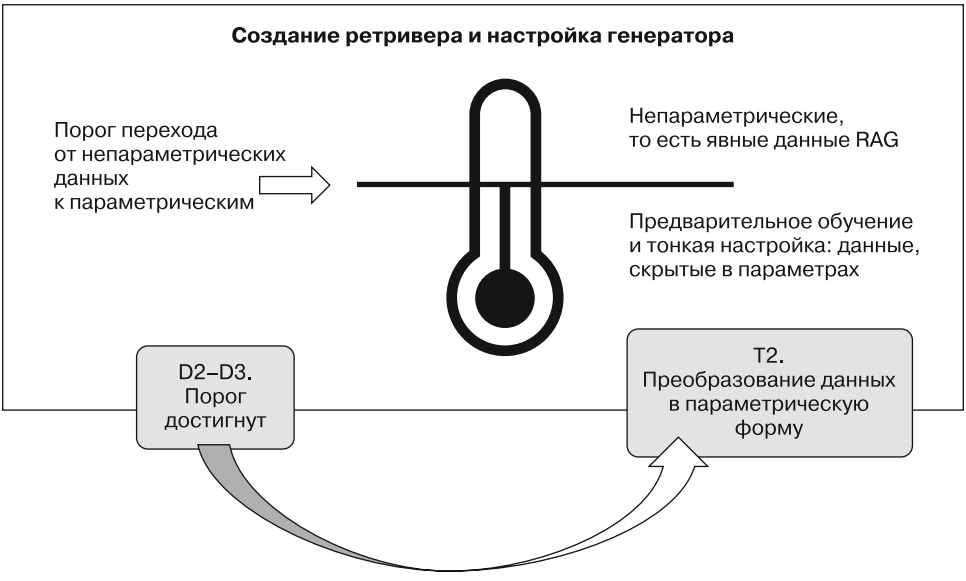


Рис. 9.1. Достижение порога для тонкой настройки данных RAG

Обратите внимание на то, что пороги обработки *(D2)* и хранения *(D3)* были достигнуты в среде RAG для статических, а не динамических данных. Этот порог зависит от особенностей конкретного проекта и его параметров, таких как следующие.

- *Объем обрабатываемых данных RAG*. Преобразование данных в эмбеддинги требует человеческих и машинных ресурсов. Даже если этого не делать, нагромождение статических данных (сохраняющих стабильность длительное время) не имеет смысла.

- *Объем данных RAG, которые нужно хранить и извлекать.* Если продолжать накапливать данные, то в определенный момент они могут начать пересекаться друг с другом.
- *Ресурсы, требующиеся для извлечения.* Даже если система с открытым кодом, число ресурсов, требующих управления, все равно растет.

При этом в каждом отдельном проекте могут фигурировать и прочие факторы. Какой бы ни была причина, при достижении определенного порога объема данных тонкая настройка может стать удачным решением.

Экосистема RAG

В этом разделе мы вернемся к экосистеме RAG, описанной в главе 1. Сосредоточимся на конкретных компонентах, которые нужны для данной главы. На рис. 9.2 компоненты тонкой настройки представлены серыми блоками, а побочные — белыми.

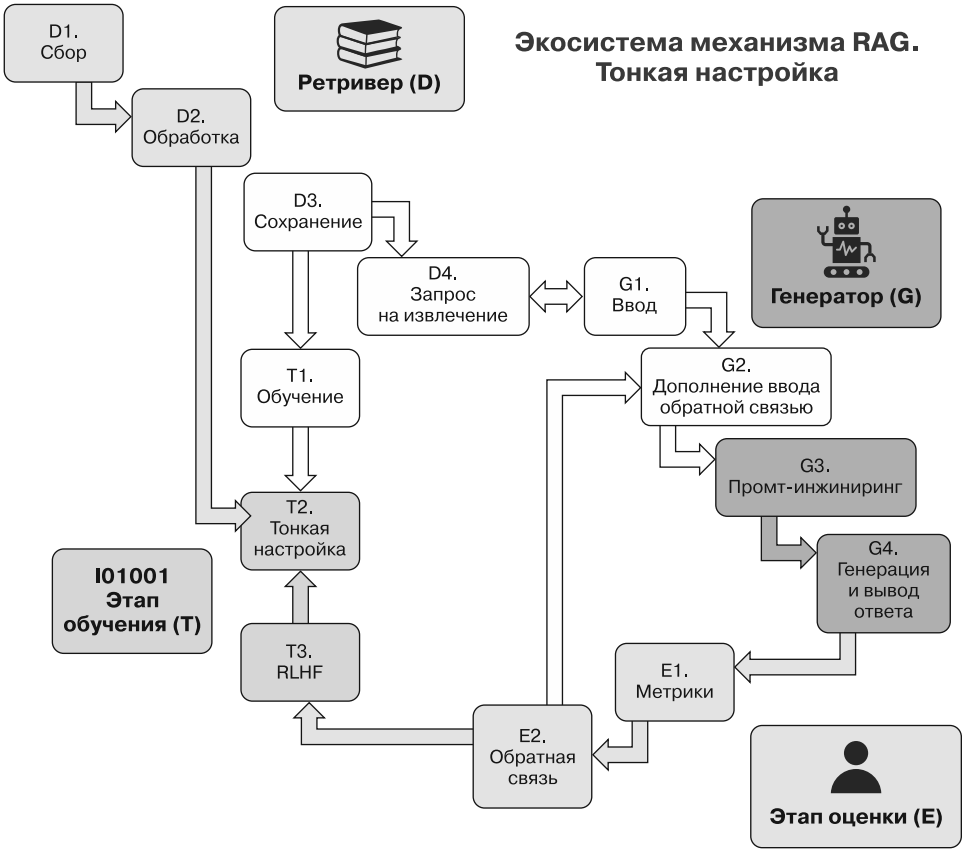


Рис. 9.2. Компоненты тонкой настройки в экосистеме RAG

Вот ключевые компоненты тонкой настройки, которую будем выполнять.

- *Скачивание (D1) и подготовка (D2) датасета*. Мы скачаем и обработаем собранный посредством краудсорсинга датасет точных наук SciQ, с которым познакомились в предыдущей главе (https://huggingface.co/datasets/sciq).
- *Обратная связь (E2)*. Можно предположить, что в создании датасета SciQ важную роль играла обратная связь от человека. Он формировался и обновлялся под управлением людей, поэтому является демонстрацией того, как преобразовывать собранные таким образом данные для уменьшения объема датасетов RAG. Можно пойти еще дальше и добавить, что в реальных проектах представленные в SciQ объяснения могут быть следствием оценки человеком результатов моделей, о чем шла речь в главе 5.
- *Тонкая настройка (T2)*. Мы настроим недорогую модель `GPT-4o-mini` от OpenAI.
- *Промт-инжиниринг (G3), генерация и вывод ответа (G4)*. Создадим промты в соответствии с рекомендациями OpenAI и выведем ответ модели.
- *Метрики (E1)*. Разберем основные возможности интерфейса метрик.

Теперь можно приступать к скачиванию и подготовке датасета SciQ.

Настройка среды

В свете стремительной эволюции ИИ и появления кросс-платформенных конфликтов зависимостей настройка среды стала сложным процессом, как было показано в одноименном разделе главы 2. В связи с этим мы по возможности будем фиксировать версии пакетов.

Для текущей программы откройте блокнот `Fine_tuning_OpenAI_GPT_4o_mini.ipynb` в каталоге `Chapter09` на GitHub. Начинается он с извлечения ключа OpenAI API:

```
# Можете извлечь ключ API из файла (1)
# либо ввести его вручную (2)

# Если планируете вводить ключ вручную, закомментируйте эту ячейку
# (1) Извлеките ключ API из файла
# Сохраните ключ в файле и считайте его (можете вводить ключ прямо
# в блокнот, но тогда его могут увидеть находящиеся рядом люди)
from google.colab import
drive drive.mount('/content/drive')
f = open("drive/MyDrive/files/api_key.txt", "r")
API_KEY=f.readline()
f.close()
```

Далее установим `openai` и настроим ключ API:

```
try:
  import openai
except:
  !pip install openai==1.42.0
  import openai
```

```
# (2) Введите ключ вручную, заменив API_KEY своим значением
# Ключ OpenAI
import os
os.environ['OPENAI_API_KEY'] =API_KEY
openai.api_key = os.getenv("OPENAI_API_KEY")
```

Теперь установим `jsonlines` для генерации данных JSONL:

```
!pip install jsonlines==4.0.0
```

Далее установим `datasets`:

```
!pip install datasets==2.20.0
```

В разделе «Настройка среды» главы 8 даны пояснения по поводу возможных конфликтов зависимостей при установке `datasets`.

В ходе установки могут возникнуть некоторые проблемы, но скачиванию датасета это не помешает. Необходимо быть готовым к подобным сложностям, поскольку ведущие платформы постоянно обновляют используемые пакеты, что приводит к конфликтам с предустановленными средами наподобие Google Colab. Для этого скрипта можете создать отдельную среду, но нужно помнить, что другие программы вполне могут сталкиваться с проблемами из-за ограничений пакетов.

Теперь мы готовы заняться датасетом.

Подготовка датасета к тонкой настройке

Если мы не хотим, чтобы процесс тонкой настройки провалился, необходимо все тщательно подготовить. В текущем разделе мы выполним следующие шаги.

1. Скачаем датасет с Hugging Face и подготовим его, обработав столбцы.
2. Перенесем датасет в файл JSON в формате JSONL. Начинается программа со скачивания датасета.

Скачивание и визуализация датасета

Скачаем датасет, который преобразовали в эмбеддинги в главе 8. Как мы видели, для создания тысяч эмбеддингов требуются деньги и ресурсы. В текущем разделе скачаем этот датасет, но на сей раз *векторизовать его не станем*. Поручим этот процесс модели OpenAI вместе с тонкой настройкой.

Программа скачивает датасет с Hugging Face и отфильтровывает из него обучающую часть, оставляя только непустые записи с верным ответом и пояснительным текстом для ответа на вопросы:

```
# Импортируем необходимые библиотеки
from datasets import load_dataset
import pandas as pd
```

```
# Загружаем датасет SciQ с HuggingFace
dataset_view = load_dataset("sciq", split="train")

# Очищаем датасет, оставляя только вопросы с ответами и пояснениями
filtered_dataset = dataset_view.filter(lambda x: x["support"] != "" and
  x["correct_answer"] != "")

# Выводим количество вопросов с пояснениями
print("Number of questions with support: ", len(filtered_dataset))
```

В завершение этот код выводит количество оставшихся вопросов с пояснениями. Судя по выводу, получилась 10 481 запись:

```
Number of questions with support: 10481
```

Теперь загрузим датасет в DataFrame и удалим отвлекающие столбцы (с неверными ответами на вопросы):

```
# Преобразуем очищенный датасет в DataFrame
df_view = pd.DataFrame(filtered_dataset)

# Удаляемые столбцы
columns_to_drop = ['distractor3', 'distractor1', 'distractor2']

# Удаление столбцов из DataFrame
df_view = df.drop(columns=columns_to_drop)

# Вывод DataFrame
df_view.head()
```

В выводе наблюдаем три нужных столбца (рис. 9.3).

	question	correct_answer	support
0	What type of organism is commonly used in prep...	mesophilic organisms	Mesophiles grow best in moderate temperature, ...
1	What phenomenon makes global winds blow northe...	coriolis effect	Without Coriolis Effect the global winds would...
2	Changes from a less-ordered state to a more-or...	exothermic	Summary Changes of state are examples of phase...
3	What is the least dangerous radioactive decay?	alpha decay	All radioactive decay is dangerous to living t...
4	Kilauea in hawaii is the world's most continuo...	smoke and ash	Example 3.5 Calculating Projectile Motion: Hot...

Рис. 9.3. Вывод, демонстрирующий три оставшихся столбца

Далее нам потребуется вопрос, который станет промтом. Столбцы `correct_answer` и `support` будут использоваться для составления развернутого текста ответа. Теперь, когда мы проверили датасет, можно передать его непосредственно в файл JSON.

Подготовка датасета к тонкой настройке

Для обучения нужной модели завершения текста запишем файл JSON в точном формате JSONL.

Датасет мы скачаем и обработаем так же, как делали для его визуализации в подразделе «Скачивание и визуализация датасета» выше, где рекомендовалось перед тонкой настройкой проверять данные.

Теперь напишем сообщения для `GPT-4o-mini` в виде JSONL:

```
# Подготавливаем элементы данных для файла JSON Lines
items = []
for idx, row in df.iterrows():
    detailed_answer = row['correct_answer'] + " Explanation: " + row['support']
    items.append({
        "messages": [
            {"role": "system", "content": "Given a science question, provide
              the correct answer with a detailed explanation."},
            {"role": "user", "content": row['question']},
            {"role": "assistant", "content": detailed_answer}
        ]
    })
```

Сначала определяем подробный ответ (`detailed_answer`), включающий корректный ответ (`'correct_answer'`) и сопутствующее пояснение (`support`).

Затем определяем сообщения (`messages`) для модели GPT-4o-mini.

- `{"role": "system", "content": ...}` устанавливает изначальную инструкцию для языковой модели, прося ее предоставлять подробные ответы на научные вопросы.
- `{"role": "user", "content": row['question']}` выступает в роли пользователя, задающего вопрос. Берется из столбца `question` в DataFrame.
- `{"role": "assistant", "content": detailed_answer}` представляет ответ ассистента, содержащий подробную информацию, собранную ранее.

Теперь запишем наш датасет JSONL в файл:

```
# Записываем датасет в файл JSON lines
with jsonlines.open('/content/QA_prompts_and_completions.json', 'w') as writer:
    writer.write_all(items)
```

Мы дали модели OpenAI ожидаемую ею структуру, которую она была обучена понимать. Для проверки содержимого этой структуры можно загрузить только что созданный файл JSON в DataFrame:

```
dfile="/content/QA_prompts_and_completions.json"
import pandas as pd
# Загружаем данные
df = pd.read_json(dfile, lines=True)
df
```

Приведенный на рис. 9.4 фрагмент показывает, что мы успешно подготовили файл JSON.

index	messages
0	{'role': 'system', 'content': 'Given a science question, provide the correct answer with a detailed explanation.'},{'role': 'user', 'content': 'What type of organism is commonly used in preparation of foods such as cheese and yogurt?'}, {'role': 'assistant', 'content': 'mesophilic organisms Explanation: Mesophiles grow best in moderate temperature, typically between 25°C and 40°C (77°F and 104°F). Mesophiles are often found living in or on the bodies of humans or other animals. The optimal growth temperature of many pathogenic mesophiles is 37°C (98°F), the normal human body temperature. Mesophilic organisms have important uses in food preparation, including cheese, yogurt, beer and wine.'}
1	{'role': 'system', 'content': 'Given a science question, provide the correct answer with a detailed explanation.'},{'role': 'user', 'content': 'What phenomenon makes global winds blow northeast to southwest or the reverse in the northern hemisphere and northwest to southeast or the reverse in the southern hemisphere?'},{'role': 'assistant', 'content': 'coriolis effect Explanation: Without Coriolis Effect the global winds would blow north to south or south to north. But Coriolis makes them blow northeast to southwest or the reverse in the Northern Hemisphere. The winds blow northwest to southeast or the reverse in the southern hemisphere.'}

Рис. 9.4. Фрагмент файла

Вот и все. Можно приступать к тонкой настройке.

Тонкая настройка модели

Для обучения модели мы извлечем созданный файл и создадим задачу по тонкой настройке. Начнем с инициализации клиента OpenAI:

```
from openai import OpenAI
import jsonlines
client = OpenAI()
```

Затем используем сгенерированный файл для создания другого обучающего файла, который будет загружен в OpenAI:

```
# Загрузка обучающего файла

result_file = client.files.create(
  file=open("QA_prompts_and_completions.json", "rb"),
  purpose="fine-tune"
)
```

Выводим информацию, содержащуюся в файле, который будем использовать для тонкой настройки:

```
print(result_file)
param_training_file_name = result_file.id
print(param_training_file_name)
```

Теперь создадим и выведем задачу по тонкой настройке:

```
# Создаем задачу тонкой настройки
ft_job = client.fine_tuning.jobs.create(
  training_file=param_training_file_name,
```

```
  model="gpt-4o-mini-2024-07-18"
)

# Выводим задачу тонкой настройки
print(ft_job)
```

В выводе сначала указываются имя файла, его назначение, статус и имя OpenAI в качестве ID:

```
FileObject(id='file-EUPGmm1yAd3axrQ0pyoeAKuE', bytes=8062970, created_
at=1725289249, filename='QA_prompts_and_completions.json', object='file',
purpose='fine-tune', status='processed', status_details=None) file-
EUPGmm1yAd3axrQ0pyoeAKuE
```

Выводятся также детали задачи:

```
FineTuningJob(id='ftjob-O1OEE7eEyFNJsO2Eu5otzWA8', created_at=1725289250,
error=Error(code=None, message=None, param=None), fine_tuned_model=None,
finished_at=None, hyperparameters=Hyperparameters(n_epochs='auto', batch_
size='auto', learning_rate_multiplier='auto'), model='gpt-4o-mini-2024-07-18',
object='fine_tuning.job', organization_id='org-h2Kjmcir4wyGtqq1mJALLGIb',
result_files=[], seed=1103096818, status='validating_files', trained_
tokens=None, training_file='file-EUPGmm1yAd3axrQ0pyoeAKuE', validation_
file=None, estimated_finish=None, integrations=[], user_provided_suffix=None)
```

В выводе содержатся подробности, необходимые для отслеживания задачи. Вот краткое описание некоторых пар «ключ — значение» из вывода:

- `Job ID` — `ftjob-O1OEE7eEyFNJsO2Eu5otzWA8`;
- `Status` — `validating_files`. Означает, что OpenAI сейчас проверяет обучающий файл, чтобы убедиться в его пригодности для тонкой настройки;
- `Model` — `gpt-4o-mini-2024-07-18`. Мы используем уменьшенную, более дешевую версию GPT-4;
- `Training File` — `file-EUPGmm1yAd3axrQ0pyoeAKuE`. Предоставленный нами файл с примерами для обучения модели.

Вот некоторые ключевые параметры:

- `n_epochs` — `'auto'`. OpenAI будет автоматически определять оптимальное количество циклов обучения;
- `batch_size` — `'auto'`. OpenAI будет автоматически выбирать наиболее подходящий для обучения размер пакета;
- `learning_rate_multiplier` — `'auto'`. OpenAI в процессе обучения будет автоматически корректировать его скорость;
- `Created at` — `2024-06-30 08:20:50`.

Эта информация окажется полезной для углубленного изучения тонкой настройки моделей OpenAI. Ее можно использовать и для отслеживания процесса тонкой настройки и управления им.

Отслеживание тонкой настройки

В этом разделе мы извлечем минимум информации, необходимой для мониторинга задач всех операций тонкой настройки. Сначала обратимся к OpenAI для получения трех последних задач:

```
import pandas as pd
from openai import OpenAI
client = OpenAI()
# Предполагается, что клиент настроен и аутентифицирован
# Увеличьте это значение, чтобы включить вашу историю тонкой настройки
response = client.fine_tuning.jobs.list(limit=3)
```

Далее инициализируем список информации, которую хотим визуализировать:

```
# Инициализируем списки для хранения извлеченных данных
job_ids = []
created_ats = []
statuses = []
models = []
training_files = []
error_messages = []
fine_tuned_models = []  # Список для хранения имен тонко настроенных моделей
```

Затем перебираем содержимое `response` для извлечения нужной информации:

```
# Перебираем возвращенные в ответе задачи
for job in response.data:
    job_ids.append(job.id)
    created_ats.append(job.created_at)
    statuses.append(job.status)
    models.append(job.model)
    training_files.append(job.training_file)
    error_message = job.error.message if job.error else None
    error_messages.append(error_message)

# Добавляем имя тонко настроенной модели
    fine_tuned_model = job.fine_tuned_model if hasattr(job, 'fine_tuned_model')
    else None
    fine_tuned_models.append(fine_tuned_model)
```

Теперь создаем DataFrame с извлеченной информацией:

```
import pandas as pd
# Предполагается, что клиент настроен и аутентифицирован
response = client.fine_tuning.jobs.list(limit=3)
# Создаем DataFrame
df = pd.DataFrame({
    'Job ID': job_ids,
    'Created At': created_ats,
    'Status': statuses,
    'Model': models,
    'Training File': training_files,
    'Error Message': error_messages,
```

```
    # Добавляем названия тонко настроенных моделей
    'Fine-Tuned Model': fine_tuned_models
})
```

Наконец, преобразуем временные метки в читабельный вид и выводим список задач тонкой настройки вместе с их статусом:

```
# Преобразуем временные метки в читабельный формат
df['Created At'] = pd.to_datetime(df['Created At'], unit='s')
df = df.sort_values(by='Created At', ascending=False)

# Выводим DataFrame
df
```

Как показано на рис. 9.5, вывод демонстрирует данные мониторинга списка наших задач.

	Job ID	Created At	Status	Model	Training File	Error Message	Fine-Tuned Model
0	ftjob-O1OEE7eEyFNJsO2Eu5otzWA8	2024-09-02 15:00:50	running	gpt-4o-mini-2024-07-18	file-EUPGmm1yAd3axrQ0pyoeAKuE	None	None
1	ftjob-gQGiuvPMvSop0tzGaDn1NMql	2024-09-02 14:26:35	succeeded	gpt-4o-mini-2024-07-18	file-1OyEhi0D2b1kcL54JbQ3P1Pa	None	ft:gpt-4o-mini-2024-07-18:personal::A32qtJOo
2	ftjob-oVB0RAcwn3NEi4u0qOMeMZUF	2024-09-02 14:11:28	succeeded	gpt-4o-mini-2024-07-18	file-0dxQmL84uLME7ehnGljQAxit	None	ft:gpt-4o-mini-2024-07-18:personal::A32VfYIz

Рис. 9.5. Список задач в DataFrame

Здесь мы видим, что задача `0` имеет статус `running`. Статус информирует вас о различных этапах процесса, таких как `validating the files` (проверка файлов), `running` (выполнение), `failure` (сбой) или `succeeded` (завершено). Если регулярно обновлять эту ячейку, то вы увидите изменение статуса.

Теперь извлечем последнюю обученную модель из столбца `Fine-Tuned Model`. Если обучение провалится, он окажется пустым. В противном случае можно будет ее извлечь:

```
import pandas as pd

# Пока текущая модель проходит тонкую настройку,
# ищем первую непустую настроенную модель
generation=False
non_empty_models = df[df['Fine-Tuned Model'].notna() & (df['Fine-Tuned Model'] != '')]

if not non_empty_models.empty:
    first_non_empty_model = non_empty_models['Fine-Tuned Model'].iloc[0]
    print("The latest fine-tuned model is:", first_non_empty_model)
    generation=True
```

```
else:
    first_non_empty_model='None'
    print("No fine-tuned models found.")
# Выводим первую непустую настроенную модель в DataFrame
first_non_empty_model = df[df['Fine-Tuned Model'].notna() & (df['Fine-Tuned
  Model'] != '')]['Fine-Tuned Model'].iloc[0]
print("The lastest fine-tuned model is:", first_non_empty_model)
```

В выводе отобразится имя последней настроенной модели, если таковая есть, либо сообщение о том, что она отсутствует. В нашем случае `GPT-4o-mini` была успешно обучена:

```
The latest fine-tuned model is: ft:gpt-4o-mini-2024-07-18:personal::A32VfYIz
```

Если настроенная модель будет найдена, то есть `generation=True`, то в следующих ячейках будут активированы вызовы текста развернутого ответа. Если модель не обнаружена, то есть `generation=False`, OpenAI API в остальной части блокнота выполняться не будет, чтобы не задействовать модели, которые вы не обучаете. Можете установить генерацию на `True` в новой ячейке и затем выбрать любую настроенную модель.

Мы понимаем, что задача обучения требует времени, поэтому рекомендуем периодически обновлять DataFrame. Вы можете написать код, который будет проверять статус очередной задачи и ожидать появления ее имени или сообщения об ошибке. Можете также подождать, пока OpenAI не уведомит вас о завершении задачи по электронной почте. Если обучение провалится, необходимо проверить обучающие данные на любые несоответствия, отсутствие значений или ошибочные метки. Кроме того, проследите, чтобы формат файла JSON следовал указанной схеме OpenAI, включая имена полей, типы данных и структуру.

После завершения обучения можно переходить к выполнению заданий по выдаче развернутых ответов.

Использование настроенной модели OpenAI

Теперь мы готовы применить настроенную модель `GPT-4o-mini`. Начнем с определения промта на основе вопроса, взятого из исходного датасета:

```
# Определяем промт
prompt = "What phenomenon makes global winds blow northeast to southwest or the
reverse in the northern hemisphere and northwest to southeast or the reverse in
the southern hemisphere?"
```

Наша цель — проверить, успешно ли был усвоен датасет и будет ли модель давать результаты, аналогичные определенным нами развернутым ответам. Выполним настроенную модель:

```
# Предполагается, что first_non_empty_model была определена ранее
if generation==True:
    response = client.chat.completions.create(
        model=first_non_empty_model,
        temperature=0.0,  # Корректируйте для повышения вариативности ответа
        messages=[
            {"role": "system", "content": "Given a question, reply with
              a complete explanation for students."},
            {"role": "user", "content": prompt}
        ]
    )
else:
    print("Error: Model is None, cannot proceed with the API request.")
```

Параметры запроса должны вписываться в наш сценарий:

- `model=first_non_empty_model` — предварительно обученная модель;
- `prompt=prompt` — предопределенный промт;
- `temperature=0.0` — установлена на минимальное значение, так как в этой задаче по выдаче ответов в сфере точных наук креативность не требуется.

После выполнения запроса отформатируем и выведем текст ответа. Код, приведенный далее, содержит две ячейки для его вывода и извлечения содержимого.

Сначала выведем необработанный ответ:

```
if generation==True:
  print(response)
```

В выводе содержатся ответ и информация о самом процессе:

```
ChatCompletion(id='chatcmpl-A32pvH9wLvNsSRmB1sUjxOW4Z6Xr6',…
```

Далее извлечем текст ответа:

```
if (generation==True):
  # Обращаемся к содержимому первого варианта ответа
  response_text = response.choices[0].message.content
  # Выводим этот ответ
  print(response_text)
```

Вывод будет представлен в виде строки:

```
Coriolis effect Explanation: The Coriolis effect is…
```

Теперь форматируем эту строку в читабельный абзац с помощью `textwrap`:

```
import textwrap
if generation==True:
wrapped_text = textwrap.fill(response_text.strip(), 60)
print(wrapped_text)
```

По выводу видно, что наши данные были учтены:

```
Coriolis effect Explanation: The Coriolis effect is a phenomenon that causes
moving objects, such as air and water, to turn and twist in response to the
rotation of the Earth. It is responsible for the rotation of large weather
systems, such as hurricanes, and the direction of trade winds and ocean
currents. In the Northern Hemisphere, the effect causes moving objects to
turn to the right, while in the Southern Hemisphere, objects turn to the left.
The Coriolis effect is proportional to the speed of the moving object and
the strength of the Earth's rotation, and it is negligible for small-scale
movements, such as water flowing in a sink.
```

Взглянем на изначальный развернутый ответ для нашего промта (рис. 9.6).

coriolis effect	Without Coriolis Effect the global winds would blow north to south or south to north. But Coriolis makes them blow northeast to southwest or the reverse in the Northern Hemisphere. The winds blow northwest to southeast or the reverse in the southern hemisphere.

Рис. 9.6. Изначальный развернутый ответ

Иными словами, ответ получился удовлетворительным. Но так бывает не всегда, и порой требуется дополнительно поработать с датасетом (оптимизировать данные, изменить их объем и т. д.), пока не будет достигнут нужный результат.

Сохраните имя модели в текстовом файле или другом месте. Теперь можно в любое время задействовать ее в другой программе по имени или заново загрузить этот блокнот.

1. Выполнить код настройки среды.
2. Определить желаемый промт по теме созданного датасета.
3. Ввести имя модели в запрос.
4. Выполнить запрос и проанализировать ответ.

Дополнительную информацию по тонкой настройке можно найти в документации: https://platform.openai.com/docs/guides/fine-tuning/fine-tuning.

Метрики

В OpenAI есть пользовательский интерфейс для анализа метрик процесса обучения и самой модели. Метрики, связанные с вашими настроенными моделями, доступны по ссылке https://platform.openai.com/finetune/.

В интерфейсе отображается список задач тонкой настройки (рис. 9.7).

Здесь можно выбирать между просмотром всех задач, только успешно завершенных или только проваленных. Если выбрать, например, успешную задачу, то мы увидим ее детали (рис. 9.8).



Рис. 9.7. Список задач тонкой настройки



Рис. 9.8. Пример просмотра деталей задачи

Разберем представленную на этом скриншоте информацию.

- Status — указывает статус процесса тонкой настройки. В данном случае процесс был завершен успешно.
- Job ID — уникальный идентификатор задачи. Его можно использовать для обращения к задаче в запросах или сопутствующих целях.
- Base model — указывает предварительно обученную модель, использованную в качестве стартовой точки для тонкой настройки. В нашем случае это `GPT-4o-mini` от OpenAI.
- Output model — идентификатор модели, получившейся в результате тонкой настройки. Она включает в себя изменения и оптимизации на базе предоставленных обучающих данных.

- Created at — дата и время инициализации задачи тонкой настройки.
- Trained tokens — общее количество токенов (фрагментов текста наподобие слов и знаков препинания), обработанных в ходе обучения. Эта метрика помогает оценить масштаб обучения.
- Epochs — количество полных проходов по обучающим данным в процессе тонкой настройки. Большее число эпох может повышать эффективность настройки, но излишнее их количество приведет к переобучению.
- Batch size — количество обучающих примеров, задействованных в одной итерации обучения модели. Чем меньше размер пакетов, тем больше обновлений весов и тем детальнее обучение, но при этом растет его длительность.
- LR multiplier — множитель, корректирующий скорость обучения базовой модели в процессе тонкой настройки. Чем меньше множитель, тем реже обновляются веса модели.
- Seed — начальное значение для случайного генератора чисел, применяемого в обучении. Его указание позволяет воссоздавать процесс настройки, то есть получать одни и те же результаты при использовании одного значения.

Эта информация поможет проследить задачи тонкой настройки, чтобы вписаться в конкретные требования проекта, а также изучить возможные альтернативные решения по оптимизации и кастомизации. Кроме того, интерфейс содержит дополнительную информацию, с которой можно ознакомиться, чтобы лучше понять процесс тонкой настройки. Если промотать вниз вкладку Information нашей модели, то мы увидим такие метрики (рис. 9.9).

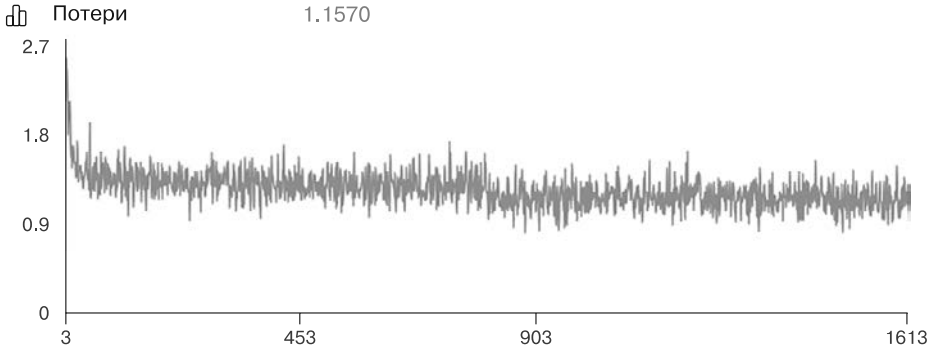


Рис. 9.9. Метрики модели после тонкой настройки

Показатель потерь (training loss) и прочая информация помогут скорректировать стратегии обучения — данные, файлы и параметры.

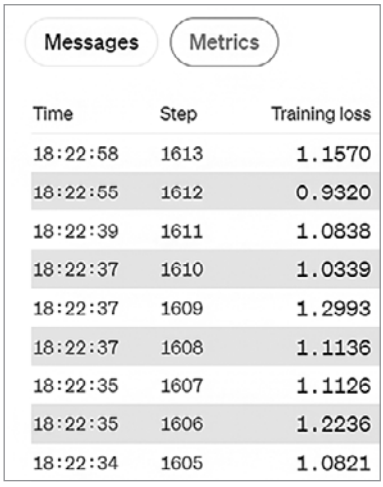
Training loss — это надежная метрика, используемая для оценки эффективности модели ML в процессе обучения. В данном случае показатель `Training loss (1.1570)` отражает среднюю ошибку модели в отношении обучающего датасета. Чем меньше значение потерь, тем лучше модель работает с обучающими данными. Показатель

`1.1570` говорит о том, что модель в процессе тонкой настройки научилась хорошо прогнозировать или классифицировать обучающие данные.

Дополнительно можно сопоставить этот показатель с информацией `Time` и `Step` (рис. 9.10).

Желательно также оценивать объем потребления платных ресурсов, чтобы отслеживать затраты на использование модели и каждый цикл обучения. OpenAI предоставляет для этого информативный интерфейс, доступный по ссылке https://platform.openai.com/usage.

Тонкая настройка может стать очень эффективным способом оптимизации данных RAG, если обучать модель на качественных данных и с правильными параметрами. Ну а теперь пора подвести итоги главы и переходить к следующей реализации генеративного ИИ на базе RAG.



Time	Step	Training loss
18:22:58	1613	1.1570
18:22:55	1612	0.9320
18:22:39	1611	1.0838
18:22:37	1610	1.0339
18:22:37	1609	1.2993
18:22:37	1608	1.1136
18:22:35	1607	1.1126
18:22:35	1606	1.2236
18:22:34	1605	1.0821

Рис. 9.10. Потери в процессе обучения

Резюме

Эта глава показала, что по мере накопления данных в системе RAG некоторые из них оказываются динамическими и требуют постоянного обновления, создавая сложности с тонкой настройкой. Тем не менее некоторые данные являются статическими, то есть остаются стабильными длительное время. Такие данные можно делать параметрическими, то есть сохранять в виде весов обученной LLM.

Начали мы главу со скачивания и обработки датасета SciQ, содержащего вопросы из сферы точных наук. Эти стабильные данные идеально подходят для тонкой настройки. Они были разбиты на вопросы, ответы и пояснения, что сделало их очень удобными для этого процесса. Кроме того, предполагается, что при их сборе требовалась обратная связь от экспертов. Можно даже допустить, что эта обратная связь была дана на основе анализа ответов модели генеративного ИИ.

Следуя рекомендациям от инструмента OpenAI, мы преобразовали подготовленные данные в промты и развернутые ответы в файле JSONL. Структура JSONL была выбрана в целях совместимости с языковой моделью, такой как `GPT-4o-mini`. Затем мы тонко настроили эту недорогую мини-модель и выполнили ее, получив удовлетворительный ответ. Наконец, мы познакомились с метриками настроенной модели в специальном пользовательском интерфейсе OpenAI.

Можно сделать вывод, что тонкая настройка в определенных случаях способна оптимизировать данные системы RAG. Тем не менее в следующей главе мы дополнительно развернем этот процесс — создадим и применим полномасштабную экосистему генеративного ИИ на базе RAG.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Всем ли организациям приходится управлять большими объемами данных в системах RAG?
2. Модель GPT-4o-mini описывается в главе как недостаточная для задач тонкой настройки?
3. Может ли обученная модель обновлять свою базу знаний новой информацией, относящейся к периоду после крайней даты обучающего датасета, без помощи систем извлечения дополнительных данных?
4. Верно ли, что статические данные никогда не меняются, а значит, обновления не требуют?
5. Является ли скачивание данных с Hugging Face единственным ресурсом для подготовки датасетов?
6. Все ли данные RAG в конечном итоге преобразуются в параметры обученной модели?
7. Рекомендуется ли в этой главе использовать для тонкой настройки моделей ИИ только новые данные?
8. Позволяет ли интерфейс OpenAI Metrics корректировать скорость обучения модели в ходе этого процесса?
9. Можно ли эффективно отслеживать процесс тонкой настройки с помощью информационной панели OpenAI?
10. При подготовке датасета точных наук наподобие SciQ обратная связь от человека считается необязательной?

Ссылки

- Документация OpenAI по тонкой настройке: https://platform.openai.com/docs/guides/fine-tuning/.
- Тарифы OpenAI: https://openai.com/api/pricing/.

Дополнительная информация

- *Wang Y. et al.* Test of Fine-Tuning GPT by Astrophysical Data. Интересная статья по теме тонкой настройки модели на данных точных наук, требующей тщательной подготовки: https://arxiv.org/pdf/2404.10019.

10 RAG для создания стоковых видео с помощью Pinecone и OpenAI

Человеческая креативность выходит за рамки привычных паттернов, так как мы обладаем уникальной способностью менять привычки и находить новые способы действия в любых сферах. А вот генеративный ИИ, напротив, опирается на хорошо известные устоявшиеся паттерны, не создавая что-то новое, а воссоздавая наши привычки. Поэтому в текущей главе мы будем использовать термин «создавать» в смысле «генерировать». Генеративный ИИ, отличаясь высокой эффективностью в области автоматизации, будет расширять свои возможности, пока не научится воспроизводить все задачи человека, на какие потенциально способен. Поэтому нам нужно разобраться в принципах работы таких автоматизированных систем, чтобы полноценно задействовать их в своих проектах. Эта глава познакомит вас с архитектурой RAG в эпоху передовых достижений в сфере интеграции человека и искусственного интеллекта. Здесь мы примерим на себя роль участника стартапа по созданию стоковых онлайн-видео с помощью ИИ. Для этого организуем команду ИИ-агентов, которые будут работать сообща над генерацией размеченных видеороликов с использованием технических комментариев.

Начнем мы с агента «Генератор» из раздела «Компонент 1: генератор и комментатор». Он будет создавать симуляции с помощью модели генерации видео из текста Sora от OpenAI. На этом этапе вы увидите, как ИИ-приложение inVideo на базе Sora, используя свое «воображение», преобразует полученную идею в видео. После этого агент «Комментатор» будет разделять сгенерированные ролики на кадры и с помощью модели компьютерного зрения генерировать технические комментарии. Затем в разделе «Компонент 2: администратор векторной базы данных» мы перейдем к созданию «Администратора», который станет управлять хранилищем Pinecone. Он будет преобразовывать сгенерированные ранее комментарии в эмбеддинги, загружать их в БД Pinecone и обращаться к ней, проверяя функциональность системы. Наконец, мы создадим «Видеоэксперта», который будет обрабатывать ввод, выполненный пользователем, делать на его основе запрос к векторной БД и извлекать соответствующие кадры видео. На этом этапе в разделе «Компонент 3: видеоэксперт» одноименный агент будет дополнять ввод, сделанный пользователем, необработанным ответом из БД и активировать

модель GPT-4o. Она, в свою очередь, будет анализировать комментарий, находить его неточности, переформулировать и присваивать ролику метку.

К концу главы вы научитесь автоматически генерировать коллекцию коротких роликов путем автоматизации перехода от сырых видеозаготовок к роликам с описаниями и метками. Этот навык позволит вам реализовать сервис, в котором пользователи смогут вводить буквально несколько слов и получать видео с кастомным описанием и меткой.

Архитектура RAG для создания видео

Автоматизация генерации видео, его комментирования и разметки чрезвычайно актуальна для различных индустрий, таких как медиа, маркетинг, сфера развлечений и образование. Владельцы бизнесов и разработчики постоянно ищут эффективные способы создания контента и управления им, которые можно успешно масштабировать под растущий спрос. В текущей главе вы получите практические навыки, которые сможете применить для достижения этой цели.

Здесь наша задача — наладить генерацию и обработку видео с помощью ИИ-агентов для создания коллекции размеченных роликов, которые можно идентифицировать. Эта система также будет динамически генерировать кастомные описания, ориентируясь на сгенерированные ИИ технические комментарии к конкретным кадрам видео, соответствующим вводу, сделанному пользователем. На рис. 10.1 показана команда ИИ-агентов, выполняющих обработку RAG при создании видео.

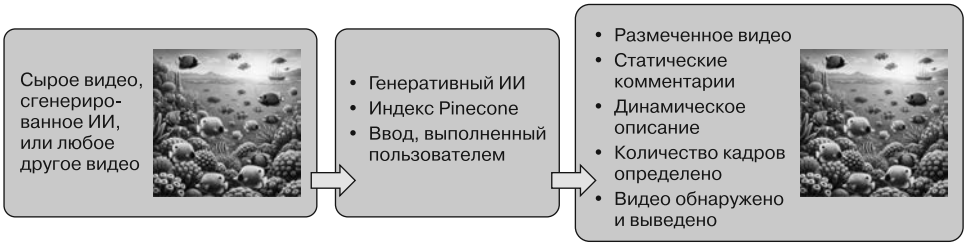


Рис. 10.1. Переход от необработанных видео к размеченным и откомментированным роликам

Мы реализуем для нашего пайплайна производства видео ИИ-агенты, которые будут:

- автоматически генерировать видеоролики и скачивать их;
- разбивать видео на кадры;
- анализировать выборку кадров;

- активировать LLM от OpenAI для генерации технических комментариев;
- сохранять технические комментарии, включая их уникальный индекс, количество проанализированных кадров и имя видеофайла;
- загружать данные в индекс векторной базы данных Pinecone;
- обращаться к БД Pinecone, применяя ввод, выполненный пользователем;
- извлекать конкретный кадр видео, наиболее похожий на его технический комментарий;
- дополнять ввод, сделанный пользователем, техническим комментарием к извлеченному кадру;
- поручать LLM анализ логики технического комментария, который может указывать на противоречия и неточности в видео, после чего выдавать динамическое описание этого видео, содержащее количество кадров и имя файла;
- показывать выбранное видео;
- оценивать полученные результаты и применять к ним метрики.

Таким образом, мы будем на основе пользовательского ввода переходить от базовых видео к размеченным роликам с грамотными описаниями. Например, сможем задавать конкретные вопросы наподобие следующего:

```
"Find a basketball player that is scoring with a dunk."
```

Это означает, что система сможет находить кадр (изображение) внутри изначально не размеченного видео, выбирать это видео, показывать его и динамически генерировать сопутствующий комментарий. Для достижения своей цели мы реализуем ИИ-агентов в трех компонентах пайплайна (рис. 10.2).

На этом рисунке изображено следующее.

- *Компонент 1: генератор и комментатор*. Генератор создает с помощью ИИ видео, используя модель Sora от OpenAI. Комментатор разбивает эти видео на кадры, которые поочередно комментирует модель компьютерного зрения от OpenAI. Затем комментатор сохраняет созданные комментарии.
- *Компонент 2: администратор векторной базы данных*. На этом этапе созданные ранее комментарии преобразуются в векторы и загружаются в индекс Pinecone.
- *Компонент 3: видеоэксперт*. В этом компоненте на основе пользовательского ввода выполняется запрос к векторной БД. В ответ на него механизм будет возвращать наиболее подходящий кадр, дополнять ввод техническим комментарием и просить GPT-4o найти в видео логические несоответствия, указать на них, а затем создать сопутствующий комментарий для пользователя и метку. Здесь же будут происходить оценка вывода и вычисление метрик.

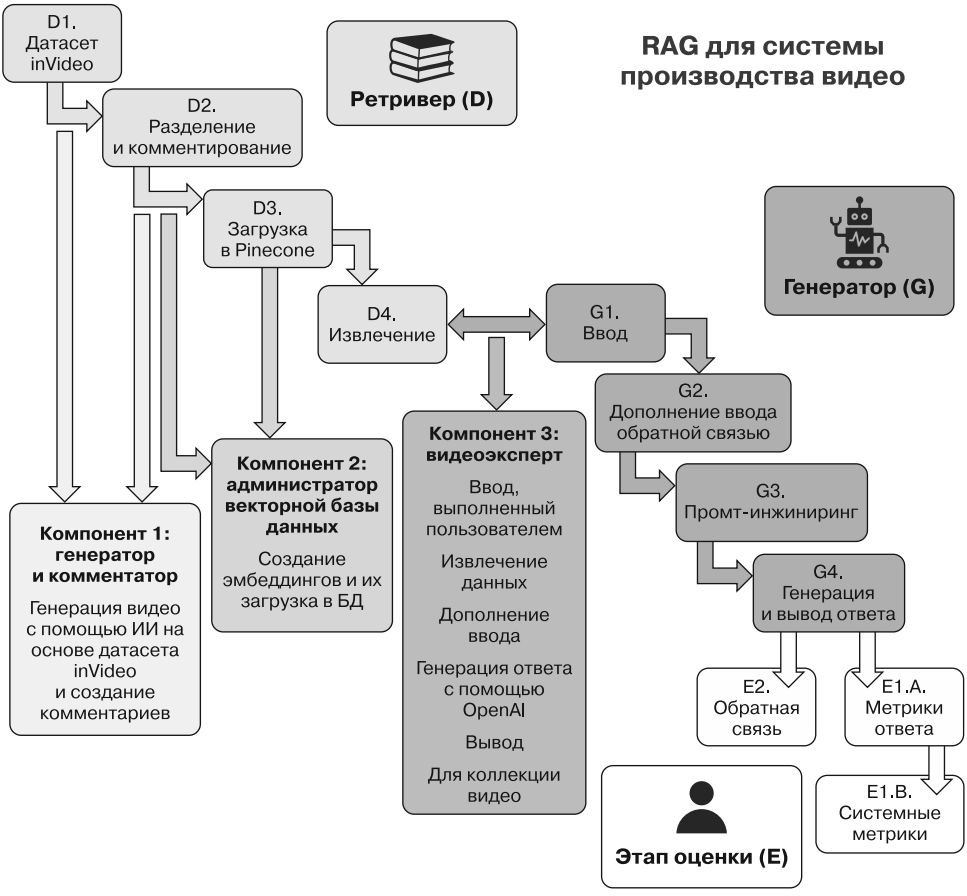


Рис. 10.2. RAG для системы производства видео с использованием агентов генеративного ИИ

ПРИМЕЧАНИЕ

Функции измерения времени заключены в нескольких ключевых функциях предшествующей экосистемы.

Систему производства видео с помощью RAG, которую мы создадим, можно будет бесконечно масштабировать путем поштучной обработки видео, используя только CPU и небольшой объем памяти, а в остальном опираясь на возможности хранилища Pinecone. Это решение демонстрирует принцип автоматизированного производства видео, но реализация подобной системы в реальном проекте потребует серьезных усилий. Тем не менее все необходимые технологии доступны, и будущее производства видео претерпевает историческую эволюцию. Ну а далее мы перейдем к коду и начнем с настройки среды.

Среда для производства видео

В каталоге `Chapter10` репозитория GitHub присутствует среда для всех блокнотов текущей главы:

- `Videos_dataset_visualization.ipynb`;
- `Pipeline_1_The_Generator_and_the_Commentator.ipynb`;
- `Pipeline_2_The_Vector_Store_Administrator.ipynb`;
- `Pipeline_3_The_Video_Expert.ipynb`.

Все блокноты содержат раздел «Настройка среды», включающий набор следующих подразделов:

- «Импорт модулей и библиотек»;
- GitHub;
- «Функции скачивания и вывода видео»;
- OpenAI;
- Pinecone.

В этой главе мы будем опираться на общую политику настройки компонентов пайплайна для написания кода RAG, используемого при производстве видео. Она ограничивается сценарием, рассматриваемым в текущей главе, и будет иной для каждой реальной производственной среды.

> **ПРИМЕЧАНИЕ**
>
> Для выполнения кода блокнотов этой главы потребуется только CPU, а также небольшой объем памяти и дискового пространства. Это позволит бесконечно расширять процесс в оптимизированной и масштабируемой среде, добавляя по одному видео за раз.

Начнем с импорта необходимых модулей и библиотек.

Импорт модулей и библиотек

Наша цель — подготовить общую для всех блокнотов глобальную среду. Поэтому указанные модули и библиотеки присутствуют во всех блокнотах независимо от того, используются они в конкретной программе или нет:

```
from IPython.display import HTML  # Для показа видео
import base64                     # Для кодирования видео в base64
from base64 import b64encode      # Для кодирования видео в base64
import os                         # Для взаимодействия с операционной системой
import subprocess                 # Для выполнения команд
import time                       # Для измерения времени выполнения
import csv                        # Для сохранения комментариев
```

```
import uuid             # Для генерации уникальных ID
import cv2              # Для разбивки видео на кадры
from PIL import Image   # Для вывода видео
import pandas as pd     # Для вывода комментариев
import numpy as np      # Для использования Numerical Python
from io import BytesIO  # Для управления потоком двоичных данных в памяти
```

Каждый из четырех блокнотов проекта содержит модули и библиотеки, приведенные в табл. 10.1.

Таблица 10.1. Модули и библиотеки для системы производства видео

Код	Комментарий
`from IPython.display import HTML`	Для показа видео
`import base64`	Для кодирования видео в base64
`from base64 import b64encode`	Для кодирования видео в base64
`import os`	Для взаимодействия с операционной системой
`import subprocess`	Для выполнения команд
`import time`	Для измерения времени выполнения
`import csv`	Для сохранения комментариев
`import uuid`	Для генерации уникальных ID
`import cv2`	Для разбивки видео (библиотека компьютерного зрения с открытым исходным кодом)
`from PIL import Image`	Для вывода видео
`import pandas as pd`	Для вывода комментариев
`import numpy as np`	Для использования Numerical Python
`from io import BytesIO`	Для потока двоичных данных в памяти

В графе Код указано имя модуля или библиотеки, а в графе Комментарий — краткое описание их назначения. Теперь перейдем к командам GitHub.

GitHub

`download(directory, filename)` присутствует во всех четырех блокнотах. Ее основная задача — скачивать нужные файлы из репозитория GitHub:

```
def download(directory, filename):
    # Базовый URL-адрес файлов образов в репозитории GitHub
    base_url = 'https://raw.githubusercontent.com/Denis2054/
      RAG-Driven-Generative-AI/main/'

    # Полный URL-адрес файла
    file_url = f"{base_url}{directory}/{filename}"
```

```
    # Использование curl для скачивания файла
    try:
        # Подготовка команды curl
        curl_command = f'curl -o {filename} {file_url}'

        # Выполнение команды curl
        subprocess.run(curl_command, check=True, shell=True)
        print(f"Downloaded '{filename}' successfully.")
    except subprocess.CalledProcessError:
        print(f"Failed to download '{filename}'. Check the URL, your
              internet connection, and if the token is correct and has
              appropriate permissions.")
```

Показанная функция получает два аргумента:

- `directory` — каталог GitHub, где расположен файл, который нужно скачать;
- `filename` — имя скачиваемого файла.

OpenAI

Пакет OpenAI устанавливается в трех блокнотах пайплайна, но не в `Video_dataset_visualization.ipynb`, которому LLM не нужна. Вы можете извлекать ключ API платформы из файла либо вводить его вручную (но в последнем случае он будет видим другим):

```
# Можете извлекать ключ API из файла (1)
# или вводить его вручную (2)
# Закомментируйте эту ячейку, если хотите вводить ключ вручную

# (1) Извлечение ключа API из файла
# Сохраните ключ в файле и считывайте его оттуда (можете вводить его
# непосредственно в блокнот, но тогда он будет виден людям рядом с вами)
from google.colab import drive
drive.mount('/content/drive')
f = open("drive/MyDrive/files/api_key.txt", "r")
API_KEY=f.readline()o
Nf.close()
```

Прежде чем выполнять этот код и получать ключ API, вам потребуется зарегистрироваться на www.openai.com. Вот код для установки пакета `openai`:

```
try:
  import openai
except:
  #!pip install openai==1.45.0
  import openai
```

В завершение устанавливаем переменную среды для ключа API:

```
#(2) Введите ключ вручную, заменив
# API_KEY своим значением
# Ключ OpenAI
os.environ['OPENAI_API_KEY'] =API_KEY
openai.api_key = os.getenv("OPENAI_API_KEY")
```

Pinecone

Раздел Pinecone присутствует только в блокнотах `Pipeline_2_The_Vector_Store_Administrator.ipynb` и `Pipeline_3_The_Video_Expert.ipynb`, где нам потребуется векторное хранилище Pinecone. Следующая команда устанавливает Pinecone, после чего эта БД импортируется:

```
!pip install pinecone-client==4.1.1
import pinecone
```

Далее мы извлекаем ключ из файла либо вводим его вручную:

```
f = open("drive/MyDrive/files/pinecone.txt", "r")
PINECONE_API_KEY=f.readline()
f.close()
```

В продакшене можете установить переменную среды или реализовать наиболее подходящий для вашего проекта метод, при котором ключ API никогда не будет видим.

> **ПРИМЕЧАНИЕ**
>
> Раздел «Этап оценки» блокнота Pipeline_3_The_Video_Expert.ipynb содержит собственные требования и устанавливаемые компоненты.

На этом мы закончили настройку среды для всех четырех блокнотов, которые содержат одинаковые подразделы в разделе «Настройка среды». Теперь можно полностью сфокусироваться на процессах, задействуемых в программах по производству видео. Начнем с генератора и комментатора.

Компонент 1: генератор и комментатор

В сфере компьютерного зрения, автоматической генерации видео и их анализа происходят революционные изменения. В разделе «Генерация датасета видео с помощью ИИ» введем агент «Генератор». В этом разделе мы узнаем, как модель Sora на базе диффузионного трансформера станет использоваться для генерации видео в текущей главе. Сама эта технология была в определенной степени ожидаемой и частично применялась в профессиональной среде съемки видео. Новшество же заключается в том, что она стала очень популярной. Сейчас с ее помощью можно получать результаты буквально после нескольких щелчков мышью, и одним из примеров этого служит сервис inVideo.

В разделе «Компонент 1: генератор и комментатор» мы расширим область генератора на сбор и обработку видео, сгенерированных ИИ. Он будет разбивать

видео на кадры и сотрудничать с комментатором — LLM от OpenAI, создавая комментарии для полученных кадров.

Задача генератора начинается с создания датасета видео.

Генерация датасета с помощью ИИ

Первым ИИ-агентом в этом проекте будет диффузионная модель трансформера, отвечающая за генерацию нужного нам датасета видео. Для этой главы ролики генерировались с помощью Sora — модели преобразования текста в видео, выпущенной OpenAI в феврале 2024 года. Вы можете посмотреть сгенерированные Sora общедоступные видео и создавать собственные на платформе https://ai.invideo.io/. Применение ИИ позволяет генерировать бесплатные ролики с гибкими условиями лицензирования, описанными на странице https://invideo.io/terms-and-conditions/.

ПРИМЕЧАНИЕ

Освоив материал текущей главы, вы сможете создать собственный датасет видео, используя для этого любой источник, будь то смартфон, стоковое видео или социальные сети.

Применение ИИ позволяет также повысить скорость создания датасетов видео. Разработчикам больше не нужно тратить драгоценное время на поиск подходящих роликов. Они могут получать их очень быстро, просто передавая системе промт, который может выражать некую идею буквально в нескольких словах. Генерация видео с помощью ИИ стала огромным прорывом в направлении будущих ИИ-приложений. Потенциал Sora может быть реализован во множестве отраслей, включая киносъемку, образование и маркетинг. Ее способность генерировать детальный видеоконтент на основе простых текстовых промтов открывает новые горизонты для творчества и образования.

Но, несмотря на то что генерация видео с помощью ИИ, в частности диффузионных трансформеров, изменила механизм создания симуляций окружающего мира, это создает риск для специалистов в различных областях, например в киноиндустрии. Угроза создания дипфейков и дезинформации здесь вполне реальна. Поэтому при реализации генеративного ИИ в проекте нам необходимо учитывать этические аспекты, создавая этичный и реалистичный контент.

Итак, пришло время увидеть, как именно диффузионный трансформер может создавать такой реалистичный контент.

Как работает диффузионный трансформер

В основе Sora, как пишет в своей работе 2024 года Исинь Лю (Yixin Liu) (см. в разделе «Ссылки» в конце главы), лежит диффузионная модель-трансформер, работающая между энкодером и декодером. Для генерации контента она использует текстовый ввод, ассоциируя его с фрагментами, полученными из энкодера. Модель итеративно очищает эти шумные скрытые представления, повышая их ясность и согласованность. Далее очищенные данные передаются в декодер для воссоздания высокоточных видеокадров. Применяемая технология включает визуальные трансформеры, такие как CLIP, LLM, подобные GPT-4o, а также другие компоненты, которые OpenAI постоянно включает в релизы моделей компьютерного зрения.

Как показано на рис. 10.3, энкодер и декодер являются неотъемлемыми элементами диффузионной модели и играют критически важную роль в ее рабочем потоке.

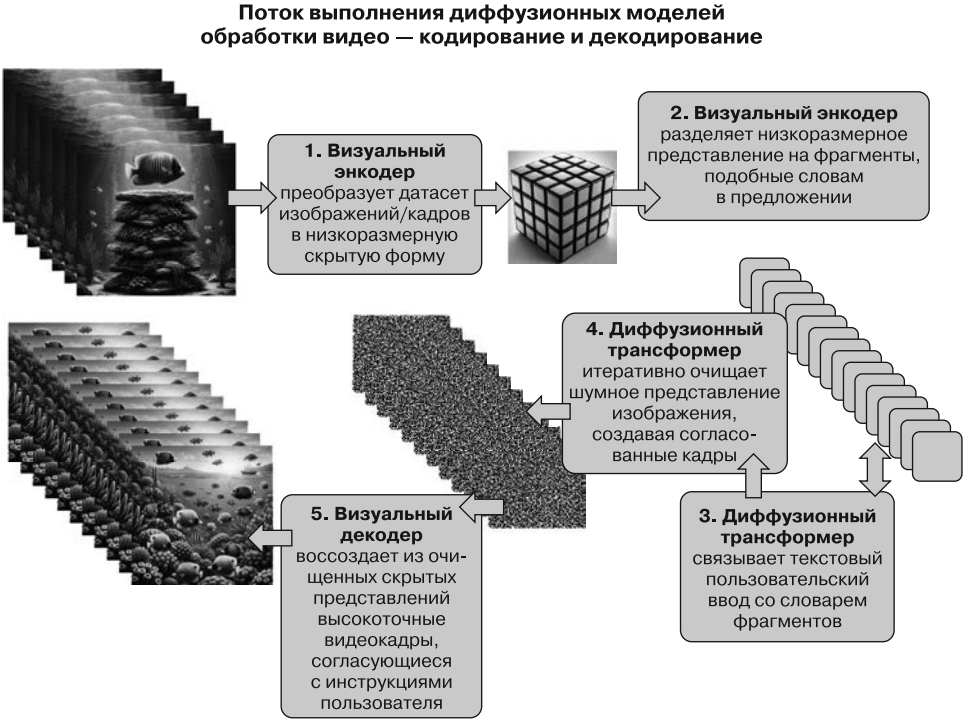


Рис. 10.3. Кодирование и декодирование видео с помощью диффузионных моделей

- *Энкодер.* Первичная функция энкодера состоит в сжатии входных данных, таких как изображения или видео, в скрытую форму меньшей размерности. То есть энкодер трансформирует высокоразмерные визуальные данные в ком-

пактное представление с сохранением важной информации. Полученная форма меньшей размерности снижает сложность, сохраняя при этом наиболее важные признаки. Например, высокоразмерное изображение (1024 × 1024 пиксела, три цветовых канала) энкодер может сжать в вектор из 1000 значений, отражающий такие детали, как форма и текстура. Этот механизм повышает эффективность обработки изображений и их использования.

- *Декодер*. Воссоздает исходные данные из созданного энкодером неявного представления. То есть он выполняет обратную операцию, преобразуя данные из скрытой низкоразмерной формы в высокоразмерное пиксельное представление, генерируя итоговый вывод — изображения или видео.

Как видно на рисунке, обработка диффузионным трансформером происходит за пять основных шагов.

1. Визуальный энкодер преобразует датасет изображений в низкоразмерное скрытое представление.
2. Энкодер разделяет полученное представление на фрагменты, подобные словам в предложении.
3. Диффузионный трансформер связывает текстовый ввод, выполненный пользователем, с полученным словарем фрагментов.
4. Трансформер итеративно очищает шумные представления изображений, создавая согласованные кадры.
5. Декодер воссоздает из очищенных скрытых представлений высокоточные видеокадры, согласующиеся с инструкциями пользователя.

Затем полученные видеокадры можно воспроизводить последовательно. Каждая секунда видео содержит определенный набор кадров. Чуть позже мы будем аналогичным образом разбивать сгенерированные ИИ видео на кадры и комментировать их.

Сейчас же займемся анализом датасета, созданного диффузионным трансформером.

Анализ датасета, созданного диффузионной моделью

Откройте блокнот `Videos_dataset_visualization.ipynb` на GitHub. Для продолжения работы среда у вас должна быть настроена в соответствии с инструкциями этой главы. Далее мы перейдем к написанию функции для скачивания и вывода видео.

Функции для скачивания и показа видео

Каждая из трех основных функций будет использовать в качестве аргумента `filename` (имя видеофайла). Они будут отвечать за скачивание видео, его вывод, а также вывод его кадров.

`download_video` скачивает видео по одному из датасета на GitHub, вызывая функцию `download`, которую мы написали в разделе «GitHub»:

```
# Скачивание файла с GitHub
def download_video(filename):
  # Определите свои переменные
  directory = "Chapter10/videos"
  filename = file_name
  download(directory, filename)
```

`display_video(file_name)` выводит скачанное видео, сначала преобразуя его в `base64` — схему кодирования двоичных данных в текстовом формате ASCII. Далее кодированное видео выводится в виде HTML:

```
# Открываем файл в двоичной форме
def display_video(file_name):
    with open(file_name, 'rb') as file:
      video_data = file.read()

  # Кодируем видео в формат base64
  video_url = b64encode(video_data).decode()

  # Создаем HTML-строку, встраивая в нее видео
  html = f'''
  <video width="640" height="480" controls>
    <source src="data:video/mp4; base64,{video_url}" type="video/mp4">
  Your browser does not support the video tag.
  </video>
  '''
  # Выводим видео
  HTML(html)
  # Возвращаем HTML-объект
  return HTML(html)
```

`display_video_frame` получает в качестве аргументов `file_name`, `frame_number` и `size` (размер изображения для вывода), показывая кадр из видео. Эта функция сначала открывает видеофайл, после чего определяет количество кадров в нем, указываемое параметром `frame_number`:

```
def display_video_frame(file_name, frame_number, size):
    # Открываем видеофайл
    cap = cv2.VideoCapture(file_name)

    # Переходим к frame_number
    cap.set(cv2.CAP_PROP_POS_FRAMES, frame_number)

    # Считываем кадр
    success, frame = cap.read()
    if not success:
      return "Failed to grab frame"
```

Эта функция изменяет цветовую схему файла с BGR (синий, зеленый, красный) на RGB (красный, зеленый, синий), преобразует его в PIL — массив изображе-

ний (которые обрабатывает OpenCV) и корректирует его размер с помощью параметров `size`:

```
# Изменяем цветовую схему с BGR на RGB
    frame = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)

    # Преобразуем в формат PIL и меняем размер
    img = Image.fromarray(frame)
    img = img.resize(size, Image.LANCZOS)   # Изменяем размер изображения
```

Наконец, функция кодирует изображение в строку `base64` и выводит ее в виде HTML:

```
# Преобразуем изображение PIL в строку base64 для встраивания в HTML
buffered = BytesIO()
img.save(buffered, format="JPEG")
img_str = base64.b64encode(buffered.getvalue()).decode()
# Создаем HTML-строку со встроенным изображением
html_str = f'''
<img src="{img_str}" width="{size[0]}"
  height="{size[1]}">
'''
# Выводим изображение
display(HTML(html_str))
# Возвращаем HTML-объект для последующего использования
return HTML(html_str)
```

После настройки среды и подготовки функций обработки видео можно вывести вступительное видео.

Вступительное видео (со звуком)

Код в следующих ячейках скачивает и выводит вступительное видео, используя созданные ранее функции. Выбирает видеофайл и скачивает его функция `download_video`:

```
# Выбор файла
print("Collecting video")
file_name="AI_Professor_Introduces_New_Course.mp4"
# Введите имя файла для обработки
#file_name = "AI_Professor_Introduces_New_Course.mp4"
print(f"Video: {file_name}")

# Скачивание видео
print("Downloading video: downloading from GitHub")
download_video(file_name)
```

Вывод подтверждает выбор и успешность скачивания:

```
Collecting video
Video: AI_Professor_Introduces_New_Course.mp4
Downloading video: downloading from GitHub
Downloaded 'AI_Professor_Introduces_New_Course.mp4' successfully.
```

Мы можем предпочесть выводить только один кадр в качестве заставки с помощью функции `display_video_frame`, указав имя файла, номер кадра и нужный размер изображения для вывода. Программа сначала вычисляет `frame_count` (количество кадров в видео), `frame_rate` (количество кадров в секунду) и `video_duration` (продолжительность видео). Затем проверяет, чтобы `frame_number` (кадр, который мы хотим вывести) не превышал `frame_count`, и, наконец, выводит картинку в виде заставки:

```
print("Displaying a frame of video: ",file_name)

video_capture = cv2.VideoCapture(file_name)
frame_count = int(video_capture.get(cv2.CAP_PROP_FRAME_COUNT))
print(f'Total number of frames: {frame_count}')

frame_rate = video_capture.get(cv2.CAP_PROP_FPS)
print(f"Frame rate: {frame_rate}")

video_duration = frame_count / frame_rate
print(f"Video duration: {video_duration:.2f} seconds")

video_capture.release()
print(f'Total number of frames: {frame_count}')

frame_number=5
if frame_number > frame_count and frame_count>0:
  frame_number = 1

display_video_frame(file_name, frame_number, size=(135, 90));
```

Здесь `frame_number` установлено на 5, но вы можете выбрать и другое значение. В выводе показана информация о видео и заставке:

```
Displaying a frame of video: /content/AI_Professor_Introduces_New_Course.mp4
Total number of frames: 340
```



При желании можно вывести видео целиком:

```
#print("Displaying video: ",file_name)
display_video(file_name)
```

Представленное видео можно будет воспроизвести вместе со звуком (рис. 10.4).



Рис. 10.4. Видео, сгенерированное ИИ

Теперь опишем и выведем сгенерированные ИИ видео из каталога `/videos` с GitHub. Вы можете разместить этот датасет в другом месте и масштабировать его до размеров, отвечающих требованиям вашего проекта. Содержимое нашего образовательного датасета видео перечислено в `lfiles`:

```
lfiles = [
    "jogging1.mp4",
    "jogging2.mp4",
    "skiing1.mp4",
    …
    "female_player_after_scoring.mp4",
    "football1.mp4",
    "football2.mp4",
    "hockey1.mp4"
]
```

Теперь можно переходить к выводу любого нужного нам видео.

Вывод заставок и видео из сгенерированного ИИ датасета

Этот раздел станет расширенной версией раздела «Вступительное видео (со звуком)». Здесь мы вместо скачивания одного ролика будем скачивать их все и выводить для них заставки. После этого можно будет выбирать видео из списка и просматривать его.

Первым делом программа скачивает датасет:

```
for i in range(lf):
  file_name=lfiles[i]
  print("Collecting video",file_name)
  print("Downloading video",file_name)
  download_video(file_name)
```

Вывод показывает имена скачанных видеофайлов:

```
Collecting video jogging1.mp4
Downloading video jogging1.mp4
Downloaded 'jogging1.mp4' successfully.
Collecting video jogging2.mp4…
```

Программа вычисляет количество видео в списке:

```
lf=len(lfiles)
```

Код перебирает список и выводит информацию по каждому видео, а также его заставку:

```
for i in range(lf):
  file_name=lfiles[i]

  video_capture.release()
  display_video_frame(file_name, frame_number=5, size=(100, 110))
```

Выводится информация о видео и его заставка:

```
Displaying a frame of video: skiing1.mp4 Total number of frames: 58
Frame rate: 30.0
Video duration: 1.93 seconds
```



Можно выбирать видео из списка и выводить его:

```
file_name="football1.mp4"    # Вводим имя видеофайла для обработки
#print("Displaying video: ",file_name)
display_video(file_name)
```

Щелкнув на видео, вы сможете его просмотреть (рис. 10.5).

На этом знакомство с созданием видео с помощью ИИ и механизмами его визуализации закончено. Теперь мы готовы к созданию генератора и комментатора.

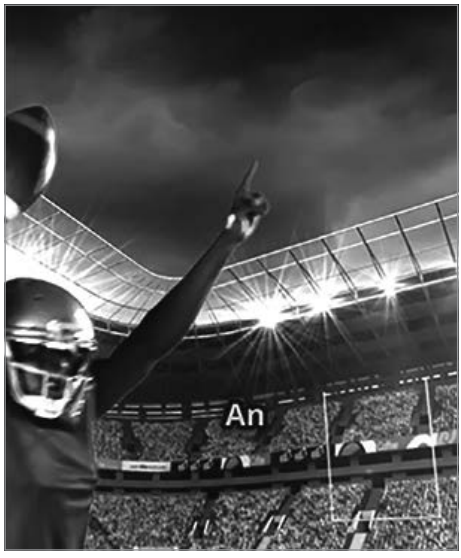


Рис. 10.5. Кадр из видео про футболиста

Генератор и комментатор

Датасет видео готов. Пора переходить к созданию генератора и комментатора, которые будут поочередно обрабатывать видео, делая масштабирование бесшовным. Поочередно можно обрабатывать неограниченное число видео, для чего потребуются лишь CPU и немного пространства на диске. Генератор и комментатор работают сообща, что схематично отражено на рис. 10.6. Эти ИИ-агенты будут создавать базовые видео из текста, после чего разбивать их на кадры и комментировать.

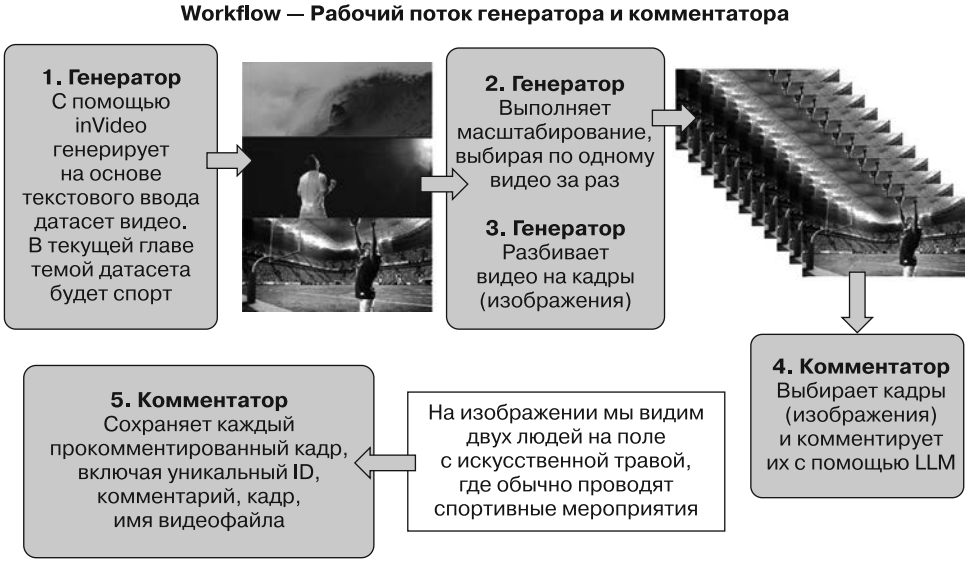


Рис. 10.6. Генератор и комментатор работают сообща, создавая комментарии к видеокадрам

Генератор и комментатор создают прокомментированные кадры, необходимые для четырех основных шагов, которые мы реализуем на Python.

1. *Генератор* на основе текстового ввода генерирует датасет видео, используя inVideo. В этой главе темой датасета будет спорт.
2. *Генератор* выполняет масштабирование, выбирая по одному видео за раз.
3. *Генератор* разбивает видео на кадры (изображения).
4. *Комментатор* выбирает кадры (изображения) и комментирует их с помощью LLM. Он сохраняет каждый прокомментированный кадр, включая:
   - уникальный ID;
   - комментарий;
   - сам кадр;
   - имя видеофайла.

Далее мы создадим генератор и комментатор на Python, начав со сгенерированных с помощью ИИ видео. Откройте блокнот `Pipeline_1_The_Generator_and_the_Commentator.ipynb` из каталога этой главы на GitHub. Описание настройки среды для блокнота приводится в разделе «Настройка среды». Переход от видео к готовым комментариям выборки кадров с помощью Python происходит в три простых шага.

1. Вывод видео.
2. Разделение видео на кадры.
3. Комментирование кадров.

Для каждого из этих этапов мы определим функции и будем вызывать их в разделе `Pipeline-1 Controller` программы. В первую очередь определяется функция для вывода видео.

Шаг 1. Вывод видео

Функция `download` была написана в подразделе «GitHub» раздела «Настройка среды» текущего блокнота. Она будет вызываться в рамках «Компонента 1: администратор векторной базы данных» в разделе «Администратор: компонент 1» этого блокнота на GitHub.

`display_video(file_name)` будет тем же, который мы определили в предыдущем разделе, «Датасет сгенерированного ИИ видео»:

```
# Открываем файл в двоичном представлении
def display_video(file_name):
```

```
  with open(file_name, 'rb') as file:
      video_data = file.read()
…
  # Возвращаем HTML-объект
  return HTML(html)
```

Далее разобьем скачанное видео на кадры.

Шаг 2. Разделение видео на кадры

Как и в предыдущем разделе, функция `split_file(file_name)` извлекает из видео кадры, только на этот раз расширим эту функцию для их сохранения в виде файлов JPG:

```
def split_file(file_name):
  video_path = file_name
  cap = cv2.VideoCapture(video_path)

  frame_number = 0
  while cap.isOpened():
      ret, frame = cap.read()
      if not ret:
          break

      cv2.imwrite(f"frame_{frame_number}.jpg", frame)
      frame_number += 1
      print(f"Frame {frame_number} saved.")

  cap.release()
```

Мы разбили видео на кадры и сохранили их в виде изображений JPG вместе с соответствующими номерами кадров, обозначенных `frame_number`. На этом работа генератора заканчивается и эстафету принимает комментатор.

Шаг 3. Комментирование кадров

Генератор начал с генерации видео на основе текста и закончил сохранением полученных из этих видео кадров в виде изображений JPG. Теперь комментатор займется комментированием этих кадров, используя три функции:

- `generate_openai_comments(filename)` просит модель компьютерного зрения GPT-4 проанализировать ответ, который содержит комментарий к кадру;
- `generate_comment(response_data)` извлекает из ответа комментарий;
- `save_comment(comment, frame_number, file_name)` сохраняет комментарий.

Первым делом нужно создать функцию извлечения:

```
def generate_comment(response_data):
    """Extract relevant information from GPT-4 Vision response."""
    try:
        caption = response_data.choices[0].message.content
        return caption
    except (KeyError, AttributeError):
        print("Error extracting caption from response.")
        return "No caption available."
```

Далее мы напишем функцию для сохранения извлеченного комментария в CSV-файле с тем же именем, что и у файла видео:

```
def save_comment(comment, frame_number, file_name):
    """Save the comment to a text file formatted for seamless loading
       into a pandas DataFrame."""
    # Добавляем .csv к указанному имени файла, создавая полное имя файла
    path = f"{file_name}.csv"
    # Проверяем существование файла, чтобы понять, нужно ли добавлять заголовки
    write_header = not os.path.exists(path)

    with open(path, 'a', newline='') as f:
        writer = csv.writer(f, delimiter=',', quotechar='"', quoting=csv.QUOTE_MINIMAL)
        if write_header:
            # Если файл создается, добавляем заголовок
            writer.writerow(['ID', 'FrameNumber', 'Comment', 'FileName'])
            # Генерируем для каждого комментария уникальный UUID
            unique_id = str(uuid.uuid4())
            # Записываем данные
            writer.writerow([unique_id, frame_number, comment, file_name])
```

Задача — сохранить комментарий в формате, который мы сможем загрузить напрямую в Pinecone:

- `ID` — уникальный строковый ID, генерируемый с помощью `str(uuid.uuid4())`;
- `FrameNumber` — номер кадра комментируемого изображения JPEG;
- `Comment` — комментарий, сгенерированный моделью компьютерного зрения от OpenAI;
- `FileName` — имя видеофайла.

Основная задача комментатора — генерировать комментарии с помощью модели компьютерного зрения от OpenAI. Тем не менее в этом сценарии мы будем сохранять не все кадры, а лишь их выборку. Сначала программа определяет количество кадров для обработки:

```
def generate_openai_comments(filename):
  video_folder = "/content"    # Каталог с кадрами
  total_frames = len([file for file in os.listdir(video_folder)
    if file.endswith('.jpg')]
```

Далее устанавливается частота выборки, которую можно изменять с помощью счетчика:

```
nb=3        # Частота выборки
counter=0   # Счетчик частоты выборки
```

После этого комментатор перебирает выбранные кадры и запрашивает по ним комментарий:

```
for frame_number in range(total_frames):
    counter+=1    # Выборка
    if counter==nb and counter<total_frames:
      counter=0
      print(f"Analyzing frame {frame_number}...")
      image_path = os.path.join(video_folder, f"frame_{frame_number}.jpg")
      try:
          with open(image_path, "rb") as image_file:
              image_data = image_file.read()

              response = openai.ChatCompletion.create(
                 model="gpt-4-vision-preview",
```

Его вопрос очень сжат: `"What is happening in this image?"` («Что происходит на этом изображении?») и включает также само изображение кадра:

```
              messages=[
                  {
                      "role": "user",
                      "content": [
                        {"type": "text", "text": "What is happening in this image?"},
                        {
                            "type": "image",
                            "image_url": f"{base64.
                               b64encode(image_data).decode('utf-8')}"
                        },
                    ],
                }
            ],
            max_tokens=150,
        )
```

После получения ответа для извлечения и сохранения комментария вызываются функции `generate_comment` и `save_comment` соответственно:

```
    comment = generate_comment(response)
    save_comment(comment, frame_number,file_name)

except FileNotFoundError:
    print(f"Error: Frame {frame_number} not found.")
except Exception as e:
    print(f"Unexpected error: {e}")
```

Последней нужной нам функцией комментатора будет вывод комментариев путем загрузки созданного CSV-файла в DataFrame:

```
# Считываем файл комментариев к видео в DataFrame библиотеки Pandas
def display_comments(file_name):
  # Добавляем .csv к указанному имени файла, создавая полное имя файла
  path = f"{file_name}.csv"
  df = pd.read_csv(path)
  return df
```

Эта функция возвращает DataFrame с комментариями. За выполнение компонента 1, включая генератор и комментатор, отвечает контроллер.

Контроллер компонента 1 пайплайна

Контроллер запускает задачи всех трех предыдущих шагов. Он начинает с шага 1, который включает выбор видео, его скачивание и вывод. В автоматизированном пайплайне эти функции можно разделить. Например, скрипт будет перебирать список видео, автоматически выбирать каждое и инкапсулировать функции контроллера. В нашем же случае контекст образовательный, и здесь мы лишь подготавливаемся к производству видео, так что будем собирать, скачивать и выводить ролики по одному:

```
session_time = time.time()    # Запуск отсчета перед запросом

# Шаг 1: вывод видео
# Выбираем файл
print("Step 1: Collecting video")
file_name = "skiing1.mp4"    # Вводим имя видеофайла для обработки
print(f"Video: {file_name}")

# Скачиваем видео
print("Step 1:downloading from GitHub")
directory = "Chapter10/videos"
download(directory,file_name)

# Выводим видео
print("Step 1:displaying video")
display_video(file_name)
```

Затем контроллер разбивает видео на кадры и комментирует их:

```
# Шаг 2. Разбивка видео
print("Step 2: Splitting the video into frames")
split_file(file_name)
```

Контроллер активирует генератор для создания комментариев к кадрам:

```
# Шаг 3. Комментирование видеокадров
print("Step 3: Commenting on the frames")
start_time = time.time()    # Начинаем отсчет перед запросом
generate_openai_comments(file_name)
response_time = time.time() - session_time    # Измеряем время ответа
```

Кроме того, измеряется время ответа. После этого контроллер добавляет дополнительные инструкции для вывода количества кадров, комментариев, времени генерации контента и общего времени обработки:

```
# Количество кадров
video_folder = "/content"    # Каталог с кадрами
total_frames = len([file for file in os.listdir(video_folder)
  if file.endswith('.jpg')])
print(total_frames)
# Выводим комментарии
print("Commenting video: displaying comments")
display_comments(file_name)

total_time = time.time() - start_time    # Начинаем отсчет перед запросом

print(f"Response Time: {response_time:.2f} seconds")  # Выводим время ответа
print(f"Total Time: {total_time:.2f} seconds")    # Выводим общее время
```

На этом контроллер завершает задачу по производству контента. Однако в зависимости от проекта вы можете подключить динамическую RAG для обработки некоторых или всех видео. Если вам нужна эта функциональность, можете применить к выдаче комментатора процесс, описанный в главе 5, включая оценку косинусного сходства, как мы и поступим в разделе «Компонент 3: видеоэксперт».

Контроллер также может сохранять комментарии и кадры.

Сохранение комментариев

Для сохранения комментариев установите `save=True`, а для сохранения кадров — `save_frames=True`. Если хотите просто выполнить программу и увидеть результат, установите эти параметры на `False`. В нашем случае используем `True`:

```
# Проверяем, существует ли файл, и еще раз перепроверяем
# перед сохранением комментариев
save=True    # Перепроверяем перед сохранением комментариев
save_frames=True    # Перепроверяем перед сохранением кадров
```

Комментарий сохраняется в формате CSV в `cpath` и содержит имя файла с расширением `.csv`. Его можно сохранить и в любом другом месте. В нашем случае файлы будут сохраняться на платформе Google Drive (убедитесь, что этот путь существует):

```
# Сохраняем комментарии
if save==True:    # Перепроверяем перед сохранением комментариев
  # Добавляем .csv к указанному имени файла, создавая полное имя файла
  cpath = f"{file_name}.csv"
  if os.path.exists(cpath):
```

```
      # Корректно используем переменную Python 'path' в команде оболочки
      !cp {cpath} /content/drive/MyDrive/files/comments/{cpath}
      print(f"File {cpath} copied successfully.")
  else:
      print(f"No such file: {cpath}")
```

Вывод подтверждает успешное сохранение файла:

```
File alpinist1.mp4.csv copied successfully.
```

Кадры сохраняются в виде имени файла без расширения, которое мы удаляем с помощью `root_name = root_name + extension.strip('.')`:

```
# Сохраняем кадры
import shutil
if save_frames==True:
  # Извлекаем основное имя файла, удаляя его расширение
  root_name, extension = os.path.splitext(file_name)
  # Этот код удаляет из расширения точку
  root_name = root_name + extension.strip('.')
  # Путь к тому месту, куда вы хотите скопировать файлы jpg
  target_directory = f'/content/drive/MyDrive/files/comments/{root_name}'
  # Проверяем существование каталога
  os.makedirs(target_directory, exist_ok=True)
  # Предполагается, что ваши файлы jpg находятся в текущем каталоге.
  # При необходимости измените этот параметр
  source_directory = os.getcwd()    # Или укажите другой каталог
  # Выводим все файлы jpg в исходном каталоге
  for file in os.listdir(source_directory):
      if file.endswith('.jpg'):
        shutil.copy(os.path.join(source_directory, file), target_directory)
```

В выводе мы получим каталог, содержащий все сгенерированные кадры. Если контроллер будет циклически перебирать все видео в одном сеансе, полученные файлы перед каждым новым видео нужно будет удалять.

Удаление файлов

Для удаления просто установите `delf=True`:

```
delf=False        # Перепроверяем, прежде чем удалять файлы сеанса
if delf==True:
  !rm -f *.mp4    # Видеофайлы
  !rm -f *.jpg    # Кадры
  !rm -f *.csv    # Комментарии
```

Теперь вы можете поочередно обрабатывать неограниченное число видео и масштабировать датасет до любого размера при наличии достаточного пространства на диске и CPU.

Компонент 2: администратор векторной базы данных

Администратор векторной базы данных выполняет задачи, реализованные нами в главе 6, за тем отличием, что все данные, которые мы загружаем для RAG, на этот раз будут сгенерированы ИИ. Откройте блокнот `Pipeline_2_The_Vector_Store_Administrator.ipynb` в репозитории GitHub. Мы создадим администратора поверх генератора и комментатора за четыре шага (рис. 10.7).

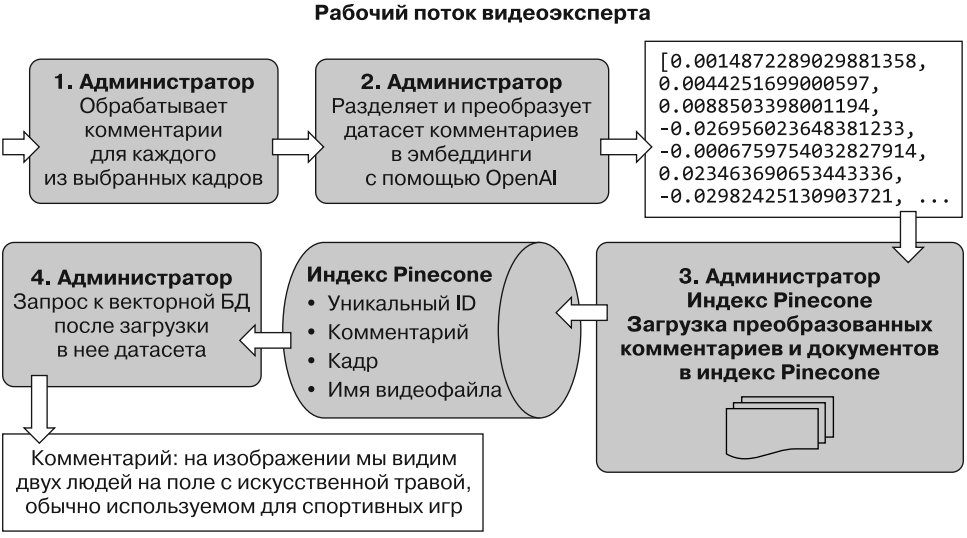


Рис. 10.7. Рабочий поток администратора векторной базы данных от обработки комментариев до их запроса из БД

1. *Обработка комментариев к видео.* Администратор векторной БД будет загружать и подготавливать комментарии к разделению так же, как мы делали в разделе «Компонент 2: масштабирование индекса Pinecone» главы 6. Поскольку в пайплайне мы обрабатываем по одному видео за раз, система удаляет обработанные файлы, чтобы не занимать лишнего пространства на диске. Вы можете доработать эту функциональность и масштабировать процесс до бесконечности.
2. *Разбивка и преобразование датасета в эмбеддинги.* Столбец имен `('ID', 'FrameNumber', 'Comment', 'FileName')` датасета уже был подготовлен комментатором в компоненте 1. Программа разбивает и преобразует датасет в векторы, используя ту же функциональность, что и в подразделе «Создание чанков и их преобразование в эмбеддинги» главы 6.

3. *Индекс Pinecone*. Здесь создается индекс Pinecone и загружаются данные точно так же, как в подразделах «Создание индекса Pinecone» и «Загрузка» главы 6.
4. *Запрос к векторной базе данных после загрузки датасета*. Здесь происходит тот же процесс, что и в главе 6, только на этот раз извлечение смешанное, так как для хранения видео и кадров используется и векторная БД, и отдельная файловая система.

Просмотрите шаги 1–3, чтобы понять функции администратора. После шага 3 индекс Pinecone будет готов к выполнению гибридных запросов.

Запрос к индексу Pinecone

В этом блокноте на GitHub раздел «Этап 4: запрос к индексу Pinecone» реализует функции для поиска комментария, соответствующего пользовательскому вводу, и его отслеживания вплоть до связанного с ним видеокадра. Отслеживание приведет нас к исходному видео и кадру, которые мы выведем. Мы можем выводить видео и кадры из любого удобного расположения. Такой гибридный подход включает в себя извлечение информации с помощью запроса к индексу Pinecone, а также получение медиафайлов из любого другого места.

В главе 4 мы видели, что векторная БД может содержать запрашиваемые изображения. В текущей же главе видео и кадры сохраняются отдельно — в репозитории GitHub. В продакшене файлы видео и кадров можно извлекать из любой системы хранения, возможно, более экономичной по сравнению с Pinecone. Решение о хранении изображений в векторной БД или отдельном месте будет зависеть от задач проекта.

Начнем с определения количества максимально схожих результатов (`k`), которые мы хотим обработать:

```
k=1    # Количество результатов
```

Теперь придумаем довольно сложный промт:

```
query_text = "Find a basketball player that is scoring with a dunk."
```

Лишь несколько кадров во всем датасете содержат изображение баскетболиста, забивающего слэм-данк. Сможет ли наша система его найти? Сейчас выясним.

Сначала преобразуем запрос в вектор, чтобы он соответствовал формату данных в БД:

```
import time
# Начинаем отсчет перед запросом
start_time = time.time()

# Целевой вектор
#query_text = "Find a basketball player."
query_embedding = get_embedding(query_text, model=embedding_model)
```

Теперь найдем в датасете векторы, схожие с вектором запроса:

```
# Выполняем запрос, используя его векторизованную форму
query_results = index.query(vector=query_embedding, top_k=k,
  include_metadata=True)    # Запрашиваем метаданные

# Выводим результаты запроса вместе с метаданными
print("Query Results:")
for match in query_results['matches']:
    print(f"ID: {match['id']}, Score: {match['score']}")

    # Проверяем, доступны ли метаданные
    if 'metadata' in  match:
        metadata = match['metadata']
        text = metadata.get('text', "No text metadata available.")
        frame_number = metadata.get('frame_number',
          "No frame number available.")
        file_name = metadata.get('file_name', "No file name available.")
```

Наконец, выводим содержимое ответа и время, затраченное на его получение:

```
        print(f"Text: {text}")
        print(f"Frame Number: {frame_number}")
        print(f"File Name: {file_name}")
    else:
        print("No metadata available.")

# Измеряем продолжительность выполнения запроса
response_time = time.time() - start_time
print(f"Querying response time: {response_time:.2f} seconds")  # Выводим ответ
```

Вывод содержит ID извлеченного комментария и показатель его сходства:

```
Query Results:
ID: f104138b-0be8-4f4c-bf99-86d0eb34f7ee, Score: 0.866656184
```

В выводе также содержится комментарий, сгенерированный LLM (комментатором):

```
Text: In this image, there is a person who appears to be in the process of
executing a dunk in basketball. The individual is airborne, with one arm
extended upwards towards the basketball hoop, holding a basketball in hand,
preparing to slam it through the hoop. The word "dunk" is superimposed on the
image, confirming the action taking place. The background shows clear skies
and a modern building, suggesting this might be an outdoor basketball court in
an urban setting. The player is wearing athletic wear and a pair of basketball
shoes, suitable for the sport. The dynamic posture and the context indicate an
athletic and powerful movement, typical of a basketball dunk.
```

Итоговый вывод содержит номер прокомментированного кадра, соответствующий видеофайл и время, затраченное на извлечение:

```
Frame Number: 191
File Name: basketball3.mp4
Querying response time: 0.57 seconds
```

Мы можем вывести видео, скачав его по имени файла:

```
print(file_name)
# Скачивание файла с GitHub
directory = "Chapter10/videos"
filename = file_name
download(directory,file_name)
```

Затем используем стандартную функцию Python для его вывода:

```
# Открываем файл в двоичном представлении
def display_video(file_name):
  with open(file_name, 'rb') as file:
      video_data = file.read()
  # Кодируем файл в формат base64
  video_url = b64encode(video_data).decode()
  # Создаем HTML-строку со встроенным в нее видео
  html = f'''
  <video width="640" height="480" controls>
    <source src="data:video/mp4; base64,{video_url}" type="video/mp4">
  Your browser does not support the video tag.
  </video>
  '''
  # Выводим видео
  HTML(html)
  # Возвращаем HTML-объект
  return HTML(html)
display_video(file_name)
```

Вот это видео, на котором баскетболист забивает слэм-данк (рис. 10.8).

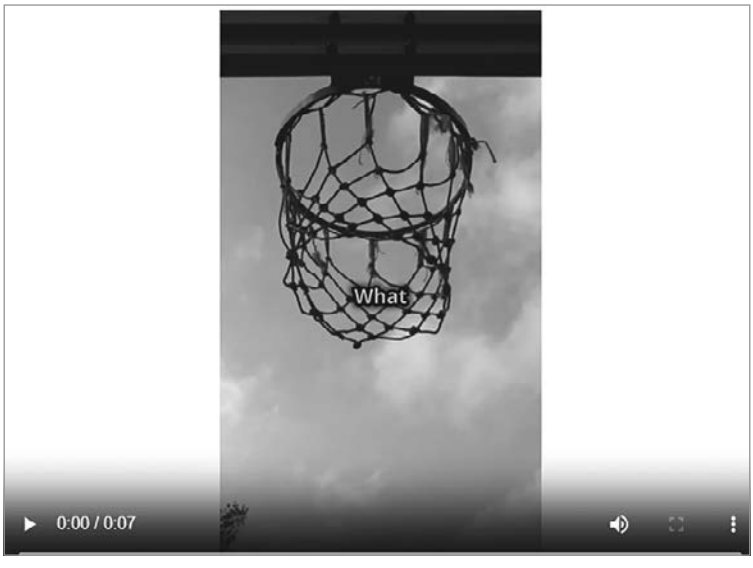


Рис. 10.8. Кадр видео

Можно пойти еще дальше и вывести прокомментированный кадр, который был извлечен:

```
file_name_root = file_name.split('.')[0]
…
from IPython.display import Image, display
# Указываем каталог и имя файла
directory = '/content/'    # При необходимости измените каталог
file_path = os.path.join(directory, frame)

# Проверяем существование файла и его размер
if os.path.exists(file_path):
    file_size = os.path.getsize(file_path)
    print(f"File '{frame}' exists. Size: {file_size} bytes.")

    # Устанавливаем логический размер файла в байтах, например 1000 байтов
    logical_size = 1000    # При необходимости скорректируйте этот порог

    if file_size > logical_size:
        print("The file size is greater than the logical value.")
        display(Image(filename=file_path))
    else:
        print("The file size is less than or equal to the logical value.")
else:
    print(f"File '{frame}' does not exist in the specified directory.")
```

Вывод показывает точный кадр, соответствующий запросу пользователя (рис. 10.9).



Рис. 10.9. Кадр, соответствующий пользовательскому вводу

ПРИМЕЧАНИЕ

Для этой программы в целях экономии пространства диска на GitHub были сохранены только кадры видео basketball3.mp4. В продакшене же вы сможете сохранять и извлекать все кадры, какие посчитаете нужными.

Команда ИИ-агентов этой главы работала сообща над генерацией видео (генератор), комментированием кадров этих видео (комментатор), загрузкой преобразованных в векторы комментариев в БД (администратор векторной БД) и подготовкой процесса извлечения (администратор векторной БД). Мы также увидели, что процесс извлечения уже содержал в себе дополненный ввод и вывод благодаря LLM от OpenAI (комментатору), которая сгенерировала комментарии на естественном языке. Процесс, который позволил достичь этой точки, определенно окажется полезным во множестве областей, будь то пожаротушение, медицинские снимки, маркетинг или др.

Чего еще можно ожидать от этой системы? Ответ даст ИИ-агент «Видеоэксперт».

Компонент 3: видеоэксперт

Роль видеоэксперта, в качестве которого выступает GPT-4o, состоит в анализе сделанных LLM комментариев, указании на логические противоречия в описании, переписывании комментария и создании метки. Как видно на рис. 10.10, рабочий поток видеоэксперта включает также код из подраздела «Вычисление и вывод метрик» главы 7.



Рис. 10.10. Рабочий поток видеоэксперта для динамического создания описаний и меток

Задачей комментатора было просто описать то, что он увидел. Видеоэксперт же нужен, чтобы осмыслить описание к кадру и сопроводить видео метками, позволяющими классифицировать их в датасете для дальнейшего использования.

1. *Индекс Pinecone*. Подключение к индексу Pinecone будет происходить так же, как описывалось в разделе «Компонент 2: администратор векторной базы данных», только на сей раз мы не будем загружать данные, а просто подключимся к БД.
2. *Определение функций RAG*. Здесь используем простые функции, написанные в компонентах 1 и 2 текущей главы.
3. *Запрос к векторной базе данных*. Выполняем запрос к индексу Pinecone так, как описывалось в компоненте 2.
4. *Генерация с дополненной выборкой*. На этом этапе проявляется основная роль видеоэксперта GPT-4o — анализ и улучшение ответов от векторной базы данных. Он также будет включать функции оценки и вычисления метрик.

Реализовать описанный в текущей главе процесс производства видео в каждом проекте можно по-разному, но видеоэксперт играет важную роль. Откройте на GitHub блокнот `Pipeline_3_The_Video_Expert.ipynb` и перейдите в раздел «Этап 2: определение функций RAG» раздела «Генерация с дополненной выборкой».

Эта функция, как и комментатор в компоненте 1, вызывает GPT-4o, но на сей раз роль LLM несколько иная:

```
          "role": "system",
            "content": "You will be provided with comments of an image frame
taken from a video. Analyze the text and 1. Point out the cognitive dissonances
2. Rewrite the comment in a logical engaging style. 3. Provide a label for this
image such as Label: basketball, football, soccer or other label."
```

Разберем инструкцию для GPT-4o.

- `You will be provided with comments of an image frame taken from a video` («Ты получишь комментарии к кадрам, взятым из видео»). Здесь мы просим LLM проанализировать сгенерированные ИИ комментарии. Комментатор должен был сохранять нейтралитет и описывать кадры такими, какими их видел. У видеоэксперта же несколько иная роль — ему нужно анализировать и улучшать эти комментарии.
- `1. Point out the cognitive dissonances` («Укажи на логические несоответствия»). Мы просим модель найти в комментарии противоречия или несостыковки, которые могут возникнуть и в результате генерации самого видео (отсутствие в нем логики).
- `2. Rewrite the comment in a logical engaging style` («Перепиши комментарий в логичной и интересной форме»). Мы просим видеоэксперта переписать

комментарий, преобразовав техническую информацию в осмысленное описание.

- `3. Provide a label for this image such as Label: basketball, football, soccer or other label` («Присвой этому изображению метку, например: баскетбол, футбол или иную»). Здесь мы просим модель сопроводить видео меткой для использования в дальнейшем. На GitHub код раздела «Этап 3: запрос к векторной базе данных» воссоздает запрос и вывод, описанный в Pipeline 2 в отношении баскетболиста, забивающего слэм-данк, вместе с соответствующим видео и кадром. Вот этот вывод:

```
ID=f104138b-0be8-4f4c-bf99-86d0eb34f7ee
score=0.866193652
text=In this image, there is a person who appears to be in the process of
executing a dunk in basketball. The individual is airborne, with one arm
extended upwards towards the basketball hoop, holding a basketball in hand,
preparing to slam it through the hoop. The word "dunk" is superimposed on the
image, confirming the action taking place. The background shows clear skies
and a modern building, suggesting this might be an outdoor basketball court in
an urban setting. The player is wearing athletic wear and a pair of basketball
shoes, suitable for the sport. The dynamic posture and the context indicate an
athletic and powerful movement, typical of a basketball dunk.
frame_number=191
file_name=basketball3.mp4
```

Данное комментатором описание выглядит приемлемым. Но давайте посмотрим, что по этому поводу думает GPT-4o. Код из раздела «Этап 4: генерация с дополненной выборкой» на GitHub получает этот вывод и отправляет его в виде промта пользователя видеоэксперту:

```
prompt=text
```

После этого мы вызываем видеоэксперта для получения его варианта ответа:

```
response_content = get_openai_response(prompt)
print(response_content)
```

Вот что думает видеоэксперт:

```
1. Cognitive Dissonances:
   - The comment redundantly describes the action of dunking multiple times.
   - The mention of "the word 'dunk' is superimposed on the image" is
unnecessary and breaks the flow of the description.
   - The background details about clear skies and a modern building are somewhat
extraneous and could be more succinctly integrated.

2. Rewritten Comment:
   In this image, a basketball player is captured mid-air, executing a powerful
dunk. With one arm extended towards the hoop and the basketball firmly in hand,
the athlete's dynamic posture showcases the intensity of the moment. The outdoor
```

```
court, set against a backdrop of clear skies and a modern building, highlights
the urban setting. Dressed in athletic wear and basketball shoes, the player
epitomizes the energy and skill of the sport.

3. Label: Basketball
```

Ответ хорошо структурирован и весьма неплох. Как уже неоднократно говорилось, ввиду креативной природы агентов генеративного ИИ вывод от случая к случаю может слегка различаться.

Код раздела Evaluator, следующего за Step 4, выполняет десять примеров, используя тот же процесс, что и в случае запроса про баскетбол, который мы сделали. То есть каждый пример содержит:

- промт пользователя;
- комментарий, возвращенный в ответ на запрос к векторной БД;
- улучшенный комментарий, сделанный моделью GPT-4o.

Кроме того, каждый пример включает тот же процесс оценки, что и в подразделе «Примеры оценки» главы 7. Однако в этом случае оценка человеком предполагает не баллы (0 или 1), а текстовое описание. Оно выступает истинным, ожидаемым выводом.

Перед началом оценки программа создает показатели для отслеживания исходного ответа на запрос.

Человек, выполняющий анализ, переписывает вывод, предоставленный видеоэкспертом:

```
# Комментарий обратной связи в виде дидактической карточки
text1 = "This image shows soccer players on a field dribbling and passing the ball."
```

Сначала из ответа извлекается переписанный видеоэкспертом комментарий:

```
# Извлекаем переписанный комментарий
text2 = extract_rewritten_comment(response_content)
```

Затем комментарий человека (истинный ориентир) и комментарий LLM выводятся:

```
print(f"Human Feedback Comment: {text1}")
print(f"Rewritten Comment: {text2}")
```

Далее косинусное сходство между этими комментариями вычисляется и добавляется к `scores`:

```
similarity_score3=calculate_cosine_similarity_with_embeddings(text1, text2)
print(f"Cosine Similarity Score with sentence transformer:
  {similarity_score3:.3f}")
scores.append(similarity_score3)
```

Изначальный показатель, возвращенный вместе с ответом на запрос, присоединяется к показателю качества извлеченного ответа, `rscores`:

```
rscores.append(score)
```

В выводе показаны обратная связь от человека, переписанный GPT-4o комментарий и показатель косинусного сходства:

```
Human Feedback Comment: This image shows soccer players on a field dribbling and
passing the ball.
Rewritten Comment: "A group of people are engaged in a casual game of soccer on a
grassy field. One player is dribbling the ball while others are either defending
or waiting for a pass. They are dressed in athletic attire, indicating this is a
recreational game among friends or acquaintances.
Interestingly, there is a superimposed text 'female' that seems unrelated to the
activity shown in the image."
Cosine Similarity Score with sentence transformer: 0.621
```

Эта программа содержит десять примеров, но мы можем ввести корпус, содержащий любое их количество, которое сочтем необходимым для оценки системы. При оценке каждого примера используется тот же набор метрик, что и в главе 7. После анализа всех примеров код из подраздела «Вычисление и вывод метрик» также вычисляет метрики, определенные в одноименном разделе главы 7.

Для полноценного анализа качества созданной системы можно задействовать все метрики. Измерение времени выполнения запросов в ходе программы тоже дает полезную информацию. В качестве первой метрики хорошо использовать точность (accuracy). В нашем случае она показывает, что есть потенциал для улучшений:

```
Mean: 0.65
```

Некоторые запросы и ответы были сложными и требовали доработки для оптимизации системы:

- проверка качества видеороликов и их содержания;
- проверка комментариев и, возможно, их изменение на основе обратной связи от человека, как мы делали в главе 5;
- тонкая настройка модели с помощью изображений и текста, как в главе 9;
- разработка любой другой конструктивной идеи, которая возникнет у команды, занятой производством видео.

Мы видим, что основанные на RAG системы генеративного ИИ очень эффективны при производстве контента. Тем не менее путь от проекта до реального производства требует усердия. Несмотря на то что технология ИИ невероятно прогрессировала, ей все еще нужен человек для проектирования, разработки и реализации в продакшен-среде.

Резюме

В этой главе мы изучили современные тенденции взаимодействия человека и ИИ-агентов, сосредоточившись на создании цельного процесса генерации, комментирования и разметки видео. Интегрировав передовые модели генеративного ИИ, мы показали, как создавать автоматизированный пайплайн, который преобразует сырые видео в структурированный, информативный и доступный видеоконтент.

Начали мы с агента «Генератор» (см. раздел «Компонент 1: генератор и комментатор»), который отвечал за создание видео на основе текстовых промтов. Очевидно, что процессы генерации видео продолжат развиваться за счет использования «воображения» и описывающих контент генеративных агентов. В разделе «Компонент 2: администратор векторной базы данных» организовали и преобразовали в эмбеддинги сгенерированные комментарии и метаданные, загрузив их в векторную базу данных. Здесь мы подчеркнули процесс оптимизации, связанный с созданием масштабируемой коллекции видео при минимуме машинных ресурсов, применяя только CPU без участия GPU. Наконец, в разделе «Компонент 3: видеоэксперт» мы ввели экспертный ИИ-агент, выполняющий задачу улучшения и разметки видеоконтента на основе ввода, сделанного пользователем. Здесь же реализовали функции оценки и вычисления метрик.

К концу главы мы выстроили полноценную автоматизированную систему генеративного ИИ на базе RAG, способную генерировать, комментировать и размечать видео с минимальным участием человека. Весь этот процесс продемонстрировал мощь и потенциал совмещения нескольких агентов и моделей ИИ для создания эффективного пайплайна производства видеоконтента.

Изученные нами техники и инструменты способны революционизировать различные отрасли жизнедеятельности, автоматизируя повторяющиеся задачи, повышая качество контента и эффективность извлечения информации. Эта глава не просто раскрыла подробную техническую перспективу технологии ИИ, но подчеркнула ее трансформирующее влияние в сфере современного создания контента и управления им. Теперь вы полностью готовы к реализации генеративных ИИ на базе RAG в реальных проектах.

Вопросы

Ответьте «да» или «нет» на следующие вопросы.

1. Может ли современный ИИ автоматически комментировать и размечать видео?
2. Подразумевает ли обработка видеороликов их разбивку на кадры?
3. Могут ли программы из этой главы создать 200-минутный фильм?

4. Требуется ли GPU для работы программ, рассмотренных в текущей главе?
5. Хранится ли преобразованный в эмбеддинги видеоконтент на диске?
6. Подразумевают ли используемые скрипты запрос к БД для извлечения данных?
7. Есть ли в скриптах функциональность для вывода изображений?
8. Полезно ли использовать в каком-либо из скриптов функции, которые будут проверять существование файла и его размер?
9. Оперируют ли эти скрипты мультимодальными данными?
10. Упоминается ли в каком-либо из скриптов применение ИИ в реальных сценариях?

Ссылки

- Информация о модели для генерации видео Sora и возможность ее использования:
  - Sora ▸ OpenAI: https://ai.invideo.io/;
  - https://openai.com/index/video-generation-models-as-world-simulators/.
- *Liu Y., Zhang K., Li Y. et al.* Sora: A Review on Background, Technology, Limitations, and Opportunities of Large Vision Models: https://arxiv.org/pdf/2402.17177.

Дополнительная информация

- OpenAI, ChatGPT: https://openai.com/chatgpt/.
- OpenAI, Research: https://openai.com/research/.
- Pinecone: https://docs.pinecone.io/home.

Приложение

В этом приложении даются ответы на все вопросы, приведенные в конце каждой главы, чтобы вы могли проверить себя и убедиться в правильном понимании ключевых концепций.

Глава 1. Почему именно генерация с дополненной выборкой (RAG)

1. RAG создана для повышения точности моделей генеративного ИИ?

   Да, RAG извлекает релевантные данные для повышения качества ответов генеративного ИИ.

2. Простейшая конфигурация RAG опирается на сложные эмбеддинги данных?

   Нет, в простейшей RAG используется поиск по ключевым словам без продвинутых техник создания эмбеддингов.

3. Всегда ли тонкая настройка станет лучшим решением, нежели использование RAG?

   Нет, RAG лучше подходит для обработки динамических данных в реальном времени.

4. RAG для повышения качества ответов применяет данные из внешних источников в реальном времени?

   Да, RAG при обработке запроса подтягивает данные из внешних источников.

5. RAG можно применить только к текстовым данным?

   Нет, RAG может работать с текстом, изображениями и аудиоданными.

6. Процесс извлечения данных в RAG запускается пользователем или автоматизированным вводом?

   Обычно процесс извлечения в RAG запускается запросом, который может поступить как от пользователя, так и от автоматизированной системы.

7. В продвинутых конфигурациях RAG используются и метрики косинусного сходства, и TF-IDF?

   Да, оба этих метода оценки применяются для анализа сходства между запросами и извлеченными документами.

8. Экосистема RAG в качестве компонентов включает только сбор данных и генерацию?

   Нет, она включает также хранение, извлечение, оценку и обучение.

9. Могут ли продвинутые конфигурации RAG обрабатывать мультимодальные данные, такие как изображения и аудио?

   Да, продвинутая RAG поддерживает обработку структурированных и неструктурированных мультимодальных данных.

10. Обратная связь от человека при оценке RAG не имеет значения?

    Нет, обратная связь важна для повышения точности ответов RAG и их релевантности.

Глава 2. Хранение эмбеддингов RAG с помощью Deep Lake и OpenAI

1. В механизме RAG текст преобразуется в эмбеддинги для ускорения извлечения информации?

   Да, эмбеддинги — это векторы, отражающие семантическое значение текста.

2. Оказывается ли поиск по ключевым словам эффективнее использования эмбеддингов при извлечении подробного семантически схожего содержимого?

   Нет, эмбеддинги лучше отражают содержание текста, чем ригидный поиск по ключевым словам.

3. Рекомендуется ли разделять пайплайны RAG на отдельные компоненты?

   Да, это позволяет вести параллельную разработку и упрощает обслуживание.

4. Пайплайн RAG состоит из двух основных компонентов?

   Нет, пайплайн состоит из трех компонентов — сбора данных, создания эмбеддингов и генерации.

5. База данных Activeloop Deep Lake может обрабатывать и создание эмбеддингов, и их хранение?

   Да, она эффективно сохраняет эмбеддинги для их быстрого извлечения.

6. В этой главе модель `text-embedding-3-small` от OpenAI использовалась для генерации эмбеддингов?

   Да, эта модель была выбрана за ее баланс между детализированностью и вычислительной эффективностью.

7. В системе на базе RAG эмбеддинги видимы и их создание можно легко проследить вплоть до исходных данных?

Да, в отличие от параметрических моделей, эмбеддинги в RAG позволяют отслеживать данные вплоть до их источника.

8. Пайплайн RAG выполняется непрерывно без разделения на отдельные компоненты?

   Разделение пайплайна RAG на компоненты позволяет разделить обязанности разработчиков, а также повышает масштабируемость и безопасность, обеспечивая более плавную работу системы. Более простые механизмы RAG могут продолжать функционировать эффективно без явного разделения компонентов, хотя такая конфигурация может оказаться неоптимальной.

9. Нужно ли для создания и сохранения эмбеддингов разделять большие тексты на меньшие части?

   Да, разбивка текста помогает оптимизировать процесс генерации эмбеддингов и повышает эффективность обработки запросов.

10. Метрики косинусного сходства используются для оценки релевантности извлеченной информации?

    Да, метрика косинусного сходства помогает оценить, насколько извлеченные данные соответствуют запросу.

Глава 3. Реализация индексированной RAG с помощью LlamaIndex, Deep Lake и OpenAI

1. Повышают ли индексы точность и скорость работы генеративного ИИ на базе генерации с дополненной выборкой?

   Да, индексы ускоряют извлечение и повышают его точность.

2. Могут ли индексы обеспечить возможность отслеживания источника вывода RAG?

   Да, индексы позволяют отслеживать результат вывода вплоть до точных исходных данных.

3. В больших датасетах поиск по индексу медленнее векторного поиска?

   Нет, поиск по индексу быстрее и оптимизирован для применения в больших датасетах.

4. Легко ли LlamaIndex интегрируется с Deep Lake и OpenAI?

   Да, LlamaIndex, Deep Lake и OpenAI прекрасно взаимодействуют.

5. Индексы на основе дерева, списка, векторов и ключевых слов — это единственные виды индексов?

   Нет, эти виды наиболее широко распространены, но существуют и другие.

6. Опирается ли механизм извлечения данных по индексу ключевых слов на понимание семантики?

   Нет, извлечение происходит на основе ключевых слов, а не семантики.

7. Может ли LlamaIndex автоматически обрабатывать разбивку текста на чанки и создание эмбеддингов?

   Да, LlamaIndex автоматизирует эти процессы, повышая удобство управления данными.

8. Необходимо ли добавление метаданных для обеспечения возможности отслеживания источника сгенерированных RAG результатов?

   Да, метаданные помогают отслеживать источник сгенерированного контента.

9. Легко ли в механизме поиска по индексу применять обновления в реальном времени?

   Проиндексированные данные зачастую требуют обновления индексов в случае изменений. Однако некоторые современные системы индексирования созданы с учетом более эффективной обработки обновлений в реальном времени или близком к нему.

10. Метрика косинусного сходства в этой главе использовалась для оценки точности запроса?

    Да, метрика косинусного сходства помогает оценить релевантность ответа на запрос.

Глава 4. Мультимодальная RAG

1. Мультимодальная RAG обрабатывает разные типы данных, такие как текст и изображения?

   Да, она обрабатывает различные типы данных, включая текст и изображения.

2. Беспилотники используются только для мониторинга в сельском хозяйстве и аэрофотосъемки?

   Нет, беспилотники используются также для спасательных операций, контроля дорожного движения и инспектирования инфраструктуры.

3. Датасет VisDrone применялся в этой главе только для получения текстовых данных?

   Нет, он содержит размеченные снимки с дронов, а не только текст.

4. Можно ли добавлять на сделанные беспилотниками снимки ограничивающие рамки для обозначения объектов, например, грузовиков и пешеходов?

   Да, ограничивающие рамки используются для обозначения объектов на изображениях.

5. Модульная система в ответ на запросы извлекает текстовые данные и изображения?

   Да, она извлекает данные и генерирует ответы на основе текстового датасета и датасета изображений.

6. Необходимо ли создать векторный индекс для запросов к мультимодальному датасету VisDrone?

   Да, векторный индекс создается для эффективного извлечения мультимодальных данных.

7. Извлеченные изображения обрабатываются без добавления меток и ограничивающих рамок?

   Нет, обработка изображений подразумевает добавление меток и ограничивающих рамок.

8. Метрики оценки эффективности мультимодальных RAG основываются только на анализе текстовых ответов?

   Нет, они также позволяют оценить точность анализа изображений.

9. Мультимодальная система, подобная описанной в этой главе, может обрабатывать только данные, связанные с технологией БПЛА?

   Нет, ее можно адаптировать и под другие сферы.

10. В мультимодальной RAG изображения оценивать так же легко, как текст?

    Нет, оценивать изображения сложнее, и для этого требуются специальные метрики.

Глава 5. Повышение эффективности RAG за счет обратной связи от эксперта

1. Важна ли обратная связь от человека для улучшения систем генеративного ИИ на базе RAG?

   Да, обратная связь непосредственно повышает качество ответов ИИ.

2. Можно ли менять параметрические данные модели генеративного ИИ без ее переобучения?

   Нет, параметрические данные модели остаются фиксированными, пока она не будет переобучена.

3. Адаптивная RAG включает циклы обратной связи в реальном времени для повышения качества извлечения данных?

   Да, в адаптивной RAG обратная связь используется для уточнения полученных ответов.

4. Основная цель адаптивной RAG — заменить весь ввод, выполненный пользователем, автоматизированными ответами?

   Нет, ее задача — совместить автоматическую обработку с обратной связью от человека.

5. Может ли обратная связь в адаптивной RAG вызывать появление изменений в извлекаемых документах?

   Да, обратная связь может привести к обновлению извлеченных документов для улучшения качества дальнейших ответов.

6. Компания C использует адаптивную RAG только для решения проблем службы поддержки?

   Нет, адаптивная RAG в ней применяется также для объяснения сотрудникам принципов работы ИИ.

7. Обратная связь применяется, только когда ответы ИИ имеют высокую оценку пользователей?

   Нет, обратная связь часто применяется при низкой оценке ответов модели.

8. Написанная в этой главе программа предоставляет только текстовый вывод извлеченных документов?

   Нет, в ней для ответов используется и текст, и обратная связь от эксперта.

9. Гибридная адаптивная RAG статична, то есть не может корректироваться на основе обратной связи?

   Нет, она динамически корректируется в соответствии с обратной связью и оценками пользователей.

10. Оценки пользователей при определении релевантности ответа ИИ полностью игнорируются?

    Нет, оценки пользователей напрямую влияют на корректировку системы.

Глава 6. Масштабирование банковских данных клиентов в системе RAG с помощью Pinecone

1. Использование датасета Kaggle обычно сопряжено со скачиванием и обработкой реальных данных для анализа?

   Да, датасеты Kaggle используются для практического анализа и моделирования реальных данных.

2. Может ли Pinecone эффективно управлять масштабными векторными БД для приложений ИИ?

Да, Pinecone предназначена для создания больших векторных баз данных, что делает ее подходящей для реализации сложных задач с применением ИИ.

3. Помогает ли кластеризация методом k-средних установить взаимосвязи между признаками, такими как жалобы, и оттоком клиентов?

   Да, этот метод кластеризации помогает выявлять и проверять паттерны в датасетах.

4. Станет ли сложнее персонализировать взаимодействие с клиентами, если в базе данных будет содержаться более миллиона векторов?

   Нет, обработка больших количеств векторов делает возможным более персонализированное и таргетированное взаимодействие с клиентами.

5. Являются ли автоматизация и повышение качества принятия решений основными причинами использования генеративного ИИ в бизнесе?

   Да, генеративный ИИ предназначен для автоматизации и повышения качества принятия решений в различных бизнес-приложениях.

6. Выигрывают ли легковесные среды разработки в плане быстрого прототипирования и создания приложений?

   Да, они упрощают процесс разработки, в том числе ускоряя тестирование и развертывание приложений.

7. Может ли архитектура Pinecone масштабироваться автоматически, без ручного вмешательства, для обработки увеличенного объема данных?

   Да, бессерверная архитектура Pinecone поддерживает автоматическое масштабирование для эффективной обработки больших объемов данных.

8. Генеративный ИИ обычно задействуется для создания динамического контента и рекомендаций на основе пользовательских данных?

   Да, генеративный ИИ часто используется для динамической генерации кастомизированного контента и рекомендаций.

9. Требует ли интеграция технологий ИИ наподобие Pinecone и OpenAI значительных ручных усилий при конфигурировании и обслуживании?

   Нет, эти технологии созданы с целью минимизировать ручные усилия при настройке и обслуживании проекта за счет автоматизации.

10. Должны ли проекты, задействующие векторные базы данных и ИИ, эффективно обрабатывать сложные запросы и большие датасеты?

    Да, векторные базы данных в тандеме с ИИ очень хорошо подходят для обработки сложных запросов и управления крупными датасетами.

Глава 7. Создание масштабируемой RAG на основе графа знаний с помощью API Википедии и LlamaIndex

1. Эта глава была посвящена созданию масштабируемой системы RAG на основе графа знаний с помощью API Википедии и LlamaIndex?

   Да, в ней описывается создание RAG на основе графа знаний с помощью этих инструментов.

2. Основной пример использования, рассмотренный в этой главе, касался здравоохранения и управления данными?

   Нет, основной пример относился к сфере маркетинга и пр.

3. Компонент 1 включает сбор и подготовку документов Википедии с помощью API?

   Да, компонент 1 автоматизирует сбор и подготовку документов, используя API Википедии.

4. В компоненте 2 платформа Deep Lake использовалась для создания реляционной базы данных?

   Нет, Deep Lake используется для создания и заполнения векторной базы данных, а не реляционной.

5. В компоненте 3 фреймворк LlamaIndex применялся для создания индекса графа знаний?

   Да, в компоненте 3 LlamaIndex используется для автоматического создания индекса графа знаний.

6. Реализованная система предназначена для обработки только одной конкретной темы — маркетинга?

   Нет, это гибкая система, которая может обрабатывать различные темы помимо маркетинга.

7. Описывается ли в этой главе, как извлекать URL-адреса и метаданные со страниц Википедии?

   Да, в ней объясняется процесс извлечения URL-адресов и метаданных с помощью API Википедии.

8. Нужен ли для выполнения описанных в этой главе компонентов GPU?

   Нет, пайплайн спроектирован для эффективного выполнения с помощью одного только CPU.

9. Отображает ли индекс графа знаний связи между элементами данных?

   Да, индекс графа знаний наглядно демонстрирует семантические связи между данными.

10. Требуется ли участие человека на каждом этапе процесса для выполнения запроса к индексу графа знаний?

    Нет, запрос к индексу графа знаний делается автоматически, с минимальным участием человека.

Глава 8. Создание динамических RAG с помощью Chroma и Hugging Face Llama

1. Обеспечивает ли наш скрипт, чтобы токен Hugging Face API из соображений безопасности никогда не прописывался в самом блокноте?

   Да, этот скрипт предоставляет методы, позволяющие использовать для обработки токенов API либо Google Drive, либо ручной ввод, избегая жесткого прописывания их значений в коде.

2. Библиотека `accelerate` в программе текущей главы использовалась для упрощения развертывания моделей ML на облачных платформах?

   Нет, `accelerate` используется для выполнения моделей с помощью локальных ресурсов, таких как несколько GPU, TPU и CPU, а не конкретно облачных платформ.

3. Помимо токена API в этом скрипте для доступа к БД Chroma необходима также аутентификация пользователя?

   Нет, в скрипте не прописаны дополнительные механизмы аутентификации помимо использования для доступа к Chroma токена API.

4. Chroma применяется в блокноте для временного хранения векторов в процессе динамического извлечения данных?

   Да, скрипт задействует Chroma для временного хранения векторов с целью более эффективного извлечения данных.

5. Блокнот настроен на использование механизма ускорения запросов в реальном времени за счет GPU?

   Да, `accelerate` позволяет задействовать ресурсы GPU для оптимизации запросов, что особенно полезно в контексте динамического извлечения данных.

6. Можно ли использовать измеряемое в этом блокноте время сеанса для оптимизации процесса динамической RAG?

   Да, за счет измерения времени сеанса код блокнота позволяет понять, как можно оптимизировать процесс динамической RAG для достижения высокой эффективности выполнения.

7. Демонстрирует ли этот скрипт возможность совмещения Chroma с моделями ML для повышения эффективности извлечения данных?

   Да, интеграция Chroma с моделью Llama подчеркивает ее способность повышать эффективность извлечения данных за счет использования продвинутых техник машинного обучения.

8. Включает ли этот скрипт возможность корректировки параметров БД Chroma на основе показателей эффективности выполнения сеанса?

   Да, код блокнота позволяет вносить корректировки на основе показателей эффективности, таких как продолжительность сеанса, что в зависимости от проекта может влиять на структуру этого блокнота и самого процесса.

Глава 9. Улучшение моделей ИИ: тонкая настройка данных RAG и обратная связь

1. Всем ли организациям приходится управлять большими объемами данных в системах RAG?

   Нет, многим корпорациям достаточно их небольшого объема.

2. Модель `GPT-4o-mini` описывается в главе как недостаточная для задач тонкой настройки?

   Нет, `GPT-4o-mini` описывается как экономичное решение для задач тонкой настройки.

3. Может ли обученная модель обновлять свою базу знаний новой информацией, относящейся к периоду после крайней даты обучающего датасета, без помощи систем извлечения дополнительных данных?

   Нет, модели статичны, и для получения новой информации им требуется ее дополнительно извлекать.

4. Верно ли, что статические данные никогда не меняются, а значит, обновления не требуют?

   Нет, верно только то, что они сохраняют стабильность длительное время, но не вечно.

5. Является ли скачивание данных с Hugging Face единственным ресурсом для подготовки датасетов?

   Да, Hugging Face конкретно обозначается как источник данных.

6. Все ли данные RAG в конечном итоге преобразуются в параметры обученной модели?

   Нет, непараметрические данные остаются внешними.

7. Рекомендуется ли в этой главе использовать для тонкой настройки моделей ИИ только новые данные?

   Нет, она предлагает применять тонкую настройку с использованием релевантных зачастую стабильных данных.

8. Позволяет ли интерфейс OpenAI Metrics корректировать скорость обучения модели в ходе этого процесса?

   Нет, он отслеживает эффективность модели и затраты на ее использование после обучения.

9. Можно ли эффективно отслеживать процесс тонкой настройки с помощью информационной панели OpenAI?

   Да, информационная панель предоставляет обновленную информацию по задачам тонкой настройки в реальном времени.

10. При подготовке датасета точных наук наподобие SciQ обратная связь от человека считается необязательной?

    Нет, обратная связь очень важна для обеспечения точности и релевантности извлекаемых данных.

Глава 10. RAG для создания стоковых видео с помощью Pinecone и OpenAI

1. Может ли современный ИИ автоматически комментировать и размечать видео?

   Да, теперь мы можем до определенной степени создавать стоковые видео автоматически.

2. Подразумевает ли обработка видеороликов их разбивку на кадры?

   Да, мы можем разбивать видео на кадры перед анализом этих кадров.

3. Могут ли программы из этой главы создать 200-минутный фильм?

   Нет, сейчас сделать это напрямую нельзя. Нам придется создать множество видео и затем склеить их с помощью видеоредактора.

4. Требуется ли GPU для работы программ, рассмотренных в текущей главе?

   Нет, нужен только CPU, что существенно экономит ресурсы, так как время обработки получается вполне адекватным и программы преимущественно опираются на вызовы API.

5. Хранится ли преобразованный в эмбеддинги видеоконтент на диске?

   Нет, эмбеддинги загружаются в векторную базу данных Pinecone.

6. Подразумевают ли используемые скрипты запрос к БД для извлечения данных?

   Да, скрипты для извлечения данных отправляют запрос к векторной базе данных.

7. Есть ли в скриптах функциональность для вывода изображений?

   Да, программы включают код для вывода изображений после их скачивания.

8. Полезно ли использовать в каком-либо из скриптов функции, которые будут проверять существование файла и его размер?

   Да, это исключает попытку вывести файлы, которые не существуют или пусты.

9. Оперируют ли эти скрипты мультимодальными данными?

   Да, все скрипты ориентированы на обработку мультимодальных данных (текст, изображения и видео).

10. Упоминается ли в каком-либо из скриптов применение ИИ в реальных сценариях?

    Да, эти скрипты оперируют мультимодальными данными, что делает их пригодными для использования в системах управления контентом, поиске и механизмах извлечения на базе ИИ.

Read IT Club

**Комьюнити рецензентов
и переводчиков ИТ-литературы**

Миссия участников клуба — обеспечить высокое качество профессиональной переводной литературы в России. «Книжные дебагеры» проверяют корректность терминологии и подписей на схемах и иллюстрациях, чтобы сделать книги более понятными русскоязычному читателю. Стать участником Read IT Club может любой ИТ-специалист, готовый поделиться опытом с сообществом.





Брендан Бёрнс

РАСПРЕДЕЛЕННЫЕ СИСТЕМЫ. ПАТТЕРНЫ И ПАРАДИГМЫ ДЛЯ МАСШТАБИРУЕМЫХ И НАДЕЖНЫХ СИСТЕМ НА ОСНОВЕ KUBERNETES

2-е издание

Разработчики распределенных систем стремятся создать их надежными, производительными и качественными, но добиться этого непросто. Набор паттернов проектирования позволяет разработчикам ПО и системным архитекторам говорить на одном языке при описании своих систем и учиться на примерах и практиках, разработанных другими.

Популярность контейнеров и Kubernetes открывает путь к применению основных паттернов распределенных систем и многократно используемых контейнеризированных компонентов. В этом практическом руководстве представлена коллекция паттернов, которые помогут вам создавать свои системы с использованием устоявшихся практик, заимствованных из некоторых наиболее эффективных современных распределенных систем. Применяя эти паттерны, вы сможете сделать свои системы более доступными и эффективными, даже если никогда раньше не создавали распределенных систем.

Автор книги Брендан Бёрнс демонстрирует, как адаптировать существующие паттерны проектирования для создания надежных распределенных приложений. Системные инженеры и разработчики узнают, как эти давно известные паттерны позволяют значительно усовершенствовать и повысить качество их систем.

Полностью обновленное второе издание также включает новые главы, посвященные искусственному интеллекту, его обучению и применению для создания надежных систем.

SPRINT book